

Contenuti

Contenuti

Gli asterischi (*) indicano le competenze minime.

- Caratteristiche real-time dei sistemi di automazione industriale.
- Vincoli temporali e scelte progettuali. Obiettivi prestazionali, metriche di valutazione. Modelli time-driven ed event-driven. Caratterizzazione dei flussi di traffico nei diversi tipi di sistemi di automazione.(*)
- Schedulazione delle trasmissioni nelle reti real-time. Non adeguatezza delle soluzioni FIFO, Shortest Job First, Round Robin. Vincoli di precedenza e sulle risorse. Scheduling di flussi real-time periodici e aperiodici: Timeline scheduling, Rate Monotonic, Deadline monotonic, Earliest Deadline First. Test di schedulabilità.(*). Cenni su schedulazione non preemptive/con vincoli di precedenza/ibrida.
- Il **Data Link layer** delle reti per automazione industriale. Indirizzamento. Panoramica sugli aspetti distintivi dei MAC protocol Master/Slave, Token passing, TDMA, CSMA, CSMA/CD, CSMA/BA.(*)
- Reti per applicazioni automotive. Domini funzionali e loro caratterizzazione. Tecnologie in uso: LIN, CAN, CAN FD, CAN XL, CAN FD light, TTCAN, FlexRAY, MOST.
- Automotive Ethernet, 10BASE-T1S, IEEE Audio Video Bridging. Middleware per automotive: overview su SOME/IP. Cenni su AUTOSAR.
- Time-Sensitive Networking. (*)
- Architettura di comunicazione nei sistemi di automazione. Fieldbus. PROFIBUS. (*)
- Industrial Ethernet: lo Standard IEC 61784. Esempi di Communication Profile: MODBUS, POWERLINK, PROFINET, ETHERCAT. (*)
- Sistemi wireless per applicazioni di automazione. Vantaggi e problemi aperti. Industrial wireless sensor networks.
- Tecnologie abilitanti per l'Industrial IoT: Bluetooth Low Energy e LoRa nelle applicazioni di automazione industriale real-time.(*)
- Il simulatore ad eventi discreti OMNeT++: installazione, configurazione, uso e sviluppo di modelli di reti di automazione. (*)

Scheduling

Definizioni

- **Task:** Sequenza di istruzioni che, in assenza di altre attività concorrenti, viene eseguita in maniera continuativa dal processore fino a completamento.
- **Arrival time (a_i)** : arrivo in coda pronti;
- **Start time (s_i)** : istante in cui il processo va in esecuzione ($s_i = a_i$, se ha priorità alta);
- **Computation time (c_i)** : tempo di esecuzione,
- **Finishing time (f_i)** : quando termina la sua esecuzione;
- **Response time ($f_i - a_i$)** : intervallo tra l'arrivo in coda pronti e la fine dell'esecuzione.

I response time dipendono dai processi con priorità più alta che con preemption vengono eseguiti, che fanno **interferenza di scheduling**, ritardo di scheduling dovuto alla preemption di un processo con priorità più alta. Il tempo di esecuzione è il response time minimo.

- *Il caso peggiore dei processi a priorità a bassa priorità si fa considerando tutti i processi sincroni, tutti in coda pronti.*

In un sistema concorrente, più task possono essere simultaneamente attivi, ma solo uno può essere in esecuzione (**running**).

- Un task attivo che non è in esecuzione si dice **pronto (ready)**.
- I task pronti sono mantenuti in una **coda dei pronti (ready queue)**, gestita da una politica di scheduling (**scheduling policy**).
- Il processore viene assegnato al primo task della coda attraverso un'operazione di (**dispatching**).

Activation → Ready queue → Dispatching → CPU → Termination

Preemption: consiste in un meccanismo **kernel** che permette di sospendere l'esecuzione del task corrente, che ritorna in coda pronti, e sostituirlo con uno più importante.

Il **dispatcher** si occupa di leggere la coda pronti e mandare il processo alla cpu, attuando il **context switching**. Risulta fondamentale per ridurre il response time di task ad alta priorità.

La **schedulazione** può essere definita come una funzione continua a tratti che definisce la sequenza di esecuzione dei task. Dato un set di task, la schedulazione è una funzione (da $R^+ \rightarrow N$) che associa un intero K ad ogni intervallo di tempo $[t, t+1)$ in questo modo:

- Se $k=0$, processore idle,
- Se $k>0$, il processore esegue p_k .

Una **configurazione di scheduling** si dice **fattibile** se il response time di tutti i singoli task rientra nel vincolo temporale.

Si fa quindi uno **studio di fattibilità**: si studia il response time con tutti i processi e si calcola se rientra nel vincolo temporale, deadline, tempo entro il quale la lezione deve essere conclusa.

Un set di task si dice **schedulabile** se esiste almeno un algoritmo che produce una schedulazione fattibile.

Un task è caratterizzato da un vincolo temporale chiamato **deadline**, il tempo entro il quale deve completare la sua esecuzione. La deadline può essere relativa o assoluta:

- **Deadline assoluta**: vincolo temporale assoluto.
- **Deadline relativa**: vincolo temporale relativo all'arrival time.

La **deadline relativa** si confronta con il response time, la deadline assoluta con il finishing time. Un set di task real time si dice fattibile se:

- Avendo una **deadline assoluta**: $f_i < d_i$, finishing time minore della deadline.

La deadline assoluta rappresenta l'istante di tempo effettivo (calcolato a partire dal tempo zero) entro il quale un task deve obbligatoriamente completare la sua esecuzione per evitare danni al sistema. Per verificare se un task ha rispettato il suo vincolo temporale assoluto, è necessario confrontare la deadline con il **finishing time**, ovvero l'istante di tempo esatto in cui il task termina l'esecuzione.

La relazione matematica fondamentale che esprime il ritardo di un task rispetto alla deadline, la **Lateness**, è infatti definita come: $L_i = f_i - d_i$.

Se la lateness è negativa o nulla il task è schedulabile, se la lateness è positiva, il task ha fallito il suo vincolo. Se vale per tutti i set il task set è fattibile, schedulabile (relativo all'algoritmo di scheduling), magari non schedulabile per un altro algoritmo.

- Avendo una **deadline relativa**: $R_i < D_i$, response time minore della deadline relativa.

La **deadline relativa** è un parametro che specifica l'intervallo di tempo massimo che può intercorrere tra *l'istante di arrivo/attivazione del task e il suo completamento*. Il **response time**, invece, è il tempo totale effettivamente impiegato dal task per completarsi, misurato proprio a partire dal momento in cui diventa pronto per l'esecuzione. Poiché sia la deadline relativa sia il response time sono grandezze relative misurate a partire dallo **stesso punto di riferimento comune**, cioè l'istante di attivazione del task, esse sono direttamente confrontabili.

Modalità di attivazione

- **Time driven**: il tempo attiva i task *periodici*, ciclici (es. dati dal sensore al controller)
- **Event driven**: non predicibile, *aperiodico* (es. superamento di una soglia di un valore limite).

Entrambi i tipi di eventi sono real time, entrambi possono avere un lifetime. Nelle reti real time meglio UDP di TCP (si accanisce a far ricevere un messaggio anche se scaduto, ma non ha valore se è scaduto e occupa banda inutilmente).

Il **job** o istanza è un piccolo figlio di un task periodico. I task periodico (ma anche aperiodici) consistono di una sequenza di attività, chiamate job o istanze. Ogni job deve essere identificato rispetto al task. Tutti i job di uno stesso task hanno tutto o lo stesso **tempo di esecuzione** o lo stesso caso peggiore. Ma può anche essere variabile, perché possono subire preemption. Viene introdotto un **timer** per fare in modo che ogni job sia sincronizzato.

Lo studio di fattibilità deve essere fatto anche sui job.

Task Periodici

Un task periodico genera una sequenza infinita di job, utilizzando praticamente lo stesso codice su dati diversi,

Con i task periodici ragioniamo in termini di carico alla CPU con il termine **utilization factor**

- **$U_i = C_i / T_i$** , tempo di computazione su periodo.

Tutti i job di uno stesso task hanno stesso C_i . Inoltre hanno tutti la stessa deadline relativa che coincide con il periodo, ma non hanno in comune la deadline assoluta. Ogni job ha il proprio istante di arrivo e la propria deadline assoluta.

Differenza tra task aperiodici e sporadici.

- Task Aperiodici: gli istanti di arrivo sono **completamente casuali**, non è possibile conoscere a priori l'intervallo minimo che intercorre tra due arrivi successivi.
- Task Sporadici: hanno un **tempo minimo di interarrivo**, quindi se una istanza arriva al tempo t_k , la prossima istanza non potrà arrivare prima di **$t_k + MIT$** (minimum interarrival time).

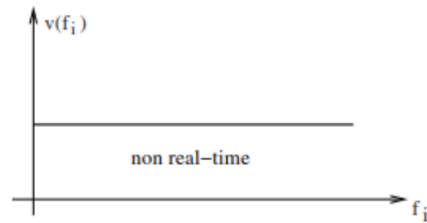
Stimare il computation time non è facile perché ogni job opera su dati diversi e può cambiare la sua esecuzione in modi diversi. Anche utilizzando gli stessi dati, il tempo di computazione dipende dal processore (cache, code, preemption).

Criticità dei task real time:

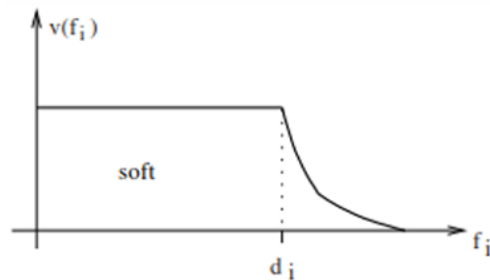
- **Soft real-time**: non rispettare una deadline causa una degradazione del servizio, ma il task ha comunque un valore per il servizio ma impatta sulla *qualità* del servizio percepito (chiamate audio, video ecc),
- **Firm real-time**: perdere una deadline produce output senza valore, non causa danni al sistema (bottiglia che non viene chiusa da un braccio meccanico).
- **Hard real-time**: mancare una deadline potrebbe avere conseguenze gravi sul sistema;

Funzione di valore o funzione di utilità: funzione che rappresenta l'andamento del beneficio del sistema rispetto al completamento del task all'interno della deadline.

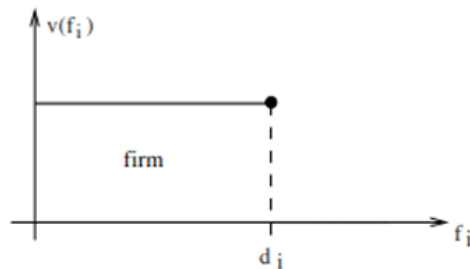
Non real time: Il messaggio ha sempre valore.



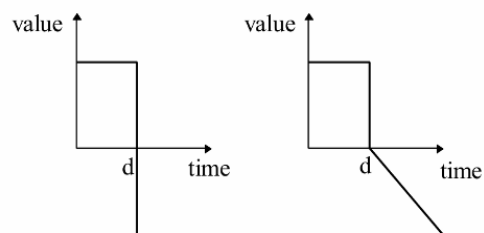
Soft real time: Non vi sono danni se il messaggio arriva dopo la deadline, ma le prestazioni peggiorano. Il valore del messaggio continua ad avere valore dopo la deadline ma decresce fino ad arrivare a zero.



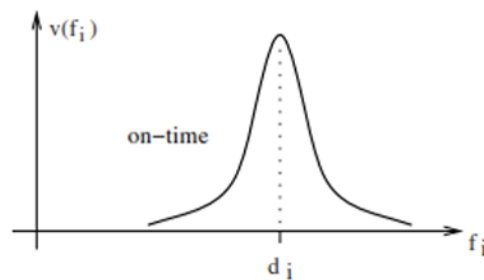
Firm real time: l'output non ha valore dopo la deadline.



Hard real time: il beneficio diventa un danno aver mancato la deadline.



Sistemi on time: beneficio massimo in corrispondenza della deadline, se un risultato arrivasse troppo presto potrebbe avere un valore basso.



Predicibilità e efficienza non vanno d'accordo: la predicibilità va a scapito dell'efficienza e viceversa.

Jitter

Jitter: grandezza che ha senso solo se si parla di qualcosa di periodico.

Misura della variazione temporale di un evento periodico.

Jitter assoluto: differenza tra scostamento massimo e scostamento minimo:

$$- \max_k (t_k - a_k) - \min_k (t_k - a_k)$$

Jitter relativo: massima deviazione tra due task consecutivi. Si considerano a coppie tra due istanze consecutive, tra tutte queste differenze si prende quella con il valore assoluto massimo.

$$- \max_k |(t_k - a_k) - (t_{k-1} - a_{k-1})|$$

Start time jitter: jitter del tempo di inizio, variabilità o scostamento dallo start time.

Finishing time jitter: jitter del tempo di fine, scostamento dall'istante in cui viene completata l'esecuzione.

Tipo di vincoli

- Vincoli **temporali**:
- Vincoli di **precedenza**: i task vanno in pipeline, che sono propedeutici o necessari per altre operazioni e che quindi devono essere schedulati prima, altrimenti mancherebbe l'input ad altri task. Impongono un ordine di esecuzione.
- Vincoli sulle **risorse**: concorrenza o mutua esclusione rispetto ad una sezione critica, occorre serializzare perché due processi non possono accedere alla stessa zona. La mutua esclusione produce ritardi.
- Vincoli specifici di sistema:

Nell'analisi di schedulabilità bisogna considerare il tempo massimo in cui un processo può attendere al semaforo per il suo turno in zona critica, garantendo correttezza logica ma anche

rispettare le deadline (nel primo grafico non c'è correttezza logica ma si rispettano le deadline nel secondo viceversa).

Un computer più veloce non garantisce il rispetto delle deadline: ciò che conta è lo scheduling. Soprattutto se vi è un accesso in sezione critica. Bisogna riuscire a utilizzare i semafori e garantire predicibilità in ogni caso. La predicibilità si ottiene attraverso un appropriato **algoritmo di scheduling**, appropriata **sincronizzazione tra i protocolli** e efficienti **meccanismi di comunicazione**.

Algoritmi di Scheduling

- **Preemptive**: il processo in esecuzione può essere sospeso per assegnare il processore ad un altro task attivo.
- **Non-preemptive**: il task schedulato procede sempre fino al suo completamento. Sulla rete è solitamente non preemptive;
- **Statico**: le decisioni di scheduling sono prese in base a parametri fissi, assegnati ai task in maniera statica prima delle attivazioni.
- **Dinamico**: i parametri che definiscono la priorità possono variare nel tempo se lo scheduling è dinamico.

Task(C_i , T_i , D_i): tempo di computazione, periodo, deadline relativa;

$d_{i,k} = a_{i,k} + D_i$;

Deadline assoluta: schedulazione che dipende dal tempo, **dinamica**;

Deadline relativa: fissa, periodica, schedulazione **statica**;

- **Online**: **schedulazione online** significa a **runtime**, lo scheduler a runtime riceve la richiesta di esecuzione e prende la decisione di scheduling a runtime. Quindi confronta la priorità con il job in esecuzione o decide che se quello appena arrivato ha priorità maggiore lo toglie dalla coda pronti, fa il context switching. La cpu è impegnata per eseguire l'algoritmo di scheduling, a runtime voglio degli algoritmi di scheduling molto veloci ed efficienti.
- **Offline**: Nella **schedulazione offline** lo scheduler conosce in anticipo tutti i processi che deve schedulare, quando arrivano, quanto tempo di computazione e la deadline che hanno. Posso definire una lista di esecuzione offline, che non impatta sulle prestazioni. Tutto è stabilito in anticipo: lo scheduler elabora lo scheduling, la lista di attivazione, il dispatcher legge la lista di esecuzione ed esegue. Quindi a runtime il dispatcher fa il lavoro minimo. Nei sistemi di automazione, tutto ciò che è periodico e che prevede rispetto delle deadline, quindi tutti i processi sono già noti. Quando tutto è noto a priori (sistema time driven, tutte le attività sono attivate dal passaggio del tempo), lo scheduling può essere fatto offline, e posso usare tecniche sofisticate con zero impatto sulle performance a runtime del sistema. Se il sistema è un po' event un po' time driven dovrò schedulare online, allora non uso tecniche di schedulazione pesanti.

- **Ottimo:** gli algoritmi ottimi sono quelli che cercano di massimizzare un certo valore (tipo *value function*, massimizzare il *total value*, minimizzare la *lateness massima*).
- **Euristico:** Un approccio euristico cerca di fare meglio possibile attraverso una funzione euristica che cerca di soddisfare un criterio di ottimalità. Non garantisce ottimalità, ci mette meno tempo anche se la soluzione è migliore possibile.

Criteri di ottimalità

- **Fattibile:** trovare una configurazione, se esiste, che permette a tutti i task di rispettare la deadline;
- Minimizzare la *lateness*,
- Minimizzare le *deadline miss*;

Gli algoritmi di scheduling tradizionali non sono predicibili e non tengono conto della deadline.

FIFO (First In First Out, First Come First Served)

- La CPU viene assegnata in base ai tempi di arrivo.
- Non preemptive, dinamico.
- I *response time* dipendono dall'istante di arrivo dei task.
- I tempi di completamento non sono predicibili, non vanno bene in real-time.

SJF (Shorted Job First)

- Vengono eseguiti prima i task con il tempo di computazione più basso.
- Non preemptive.
- Statico (Ci è un parametro costante).
- Può essere usato online o offline.
- *Minimizza il response time medio.*
- Esiste una versione preemptive chiamata Shortest Remaining Time Next (SRTN) che utilizza una assegnazione dinamica della priorità basata sul tempo di computazione rimanente.
- Non è ottimale in termini di fattibilità perché non tiene conto delle deadline.
- C'è anche il rischio di **starvation**.

Round Robin

- La coda pronti viene gestita con un FCFS ma ogni task ha a disposizione lo stesso quanto di tempo di CPU, quando il tempo finisce tornano in coda pronti.
- Ma in ambito real time non ci interessa se minimizza il tempo di risposta medio e soprattutto non ci interessa della **fairness**, i task non sono tutti uguali, perché hanno priorità diverse. In macchina ci sono diversi controlli che non sono paritetici. Task di allarmi e download di statistiche che hanno criticità diverse che richiedono un trattamento diversificato.
- Non considera le deadline.
- Sia n il numero di task del sistema. Ogni processo è N volte più lento.

La schedulazione multilivello permette di schedulare con algoritmi diversi i task in base alla tipologia e alla criticità.

- High priority: task di Sistema, vengono fatti con priority scheduling;
- Medium priority: task interattivi, round robin;
- Low Priority: batch task, FCFS

Algoritmi di schedulazione Real Time

La schedulazione avviene in base a: deadline relativa, deadline assoluta.

- **Notazione di Graham** : $\alpha \mid \beta \mid \gamma$
(numero di processori) | (vincoli sui task) | (criterio da ottimizzare)

EDD: Earliest Due Date : 1 | sync | Lmax

Detto anche algoritmo di **Jackson** :

- Algoritmo uniprocessore per task sincroni, cioè che arrivano tutti simultaneamente, che minimizza la lateness massima ($L_i = f_i - d_i$).
- L'algoritmo ordina i task in base alla *deadline relativa crescente*.
- Se la lateness massima è negativa allora nessun task ha mancato la sua deadline.
- Questo algoritmo seleziona i task con la deadline più vicina,
- È un algoritmo **statico** a priorità fissa perché tutti i task sono noti a priori e sono presenti in coda nello stesso momento.
- La **preemption non** è un requisito necessario perché i task arrivano simultaneamente, allo stesso istante, vengono quindi ordinati in ordine rispetto alle deadline crescenti.
- I task sono **indipendenti**: non ci sono vincoli di precedenza o risorse condivise.

Guarantee Test (OFFLINE)

Il **test di garanzia** per lo studio della fattibilità viene fatto *offline* e deve dimostrare che, nel caso peggiore, tutti i task completino la loro esecuzione prima della deadline.

Consideriamo che i task siano in ordine di deadline crescente; quindi, il caso peggiore del task può essere calcolato come la somma dei tempi di computazione di tutti i task precedenti.

- Per ogni processo il finishing time è : $f_i = \sum_{k=1}^i C_k$.
- Per ogni processo deve quindi valere che: $f_i \leq d_i$.
- Il test di garanzia è il seguente: $\sum_{k=1}^i C_k \leq d_i$.

EDF: Earliest Deadline First: 1 | preem. | Lmax

Detto anche algoritmo di Horn:

- Algoritmo uniprocessore per task asincroni che minimizza la lateness massima.
- I task vengono schedulati per deadline assoluta crescente.
- I task non sono sincroni e possono essere attivati dinamicamente durante l'esecuzione.
- La priorità è **dinamica**, poiché può arrivare un processo con una deadline relativa più vicina rispetto a quello in esecuzione.
- La **preemption** è fondamentale: **EDF è ottimo con esecuzione preemptive**.
- L'inserimento in coda è ordinato così l'estrazione ha complessità costante.

Guarantee Test (ONLINE)

Quando i task hanno attivazioni dinamiche e i tempi di arrivo non sono noti a priori, il test di garanzia deve essere fatto dinamicamente, ogni volta che un task entra nel sistema, praticamente online.

Per effettuare il test di garanzia possiamo assumere che tutti i task siano ordinati in ordine di deadline **crescenti**, ma quando arriva un task con priorità più alta potrebbe bloccarne l'esecuzione e quindi abbiamo già una parte di task eseguito. Quindi consideriamo il tempo rimanente dell'esecuzione del caso peggiore del task iesimo.

Il finishing time può essere calcolato in questo modo:

- $\forall i \ f_i \leq d_i$
- $f_i = \sum_{k=1}^i c_k(t)$: del tempo di esecuzione rimanente del caso peggiore del task iesimo.
- $\sum_{k=1}^i c_k(t) \leq d_i$

Potremmo ridefinire il response time come segue: **Ri = Ci + li + Bi**

- Ci: computation time,
- li: interferenza di scheduling,
- Bi: blocco a risorsa condivisa.

Quello a priorità più bassa si prende l'interferenza massima perché si becca l'esecuzione di tutti gli altri task. Invece un task a priorità più alta può essere bloccato da un task a priorità bassa ma che è entrato in una sezione critica perché il semaforo lo blocca fino a quando non ha finito, Bi, blocco da task a priorità minore su risorsa condivisa.

Proprietà dell'ottimalità:

- Un algoritmo si dice ottimo se genera una schedulazione fattibile, se ne esiste una.
- Se un algoritmo ottimale produce uno scheduling non fattibile, allora nessun algoritmo può farlo, a parità di condizioni.

- **EDF è ottimo se la preemption è consentita.**

Alcuni algoritmi lasciano la cpu idle anche se ci sono task in coda pronti. Non è un caso completamente impossibile perché spesso in automazione i task sono noti, quindi gli scheduler conoscono a priori tempi di arrivo e deadline. **Esistono Idle scheduler e Non-idle scheduler. NP-EDF è ottimo tra gli algoritmi di schedulazione non-idle.**

- **Non idle Scheduler:** scheduler che cerca di tenere la CPU sempre occupata, se ci sono task in coda pronti.
- **Idle Scheduler:** scheduler che in alcuni casi può lasciare la CPU inutilizzata, idle, nonostante ci siano dei task in coda pronti che devono essere schedulati.

Gli algoritmi di schedulazione non preemptive per trovare uno schedule fattibile lavorano offline con una struttura ad albero, costruendo dei cammini usando il tempo di arrivo e il finishing time.

Algoritmo di Bratley : 1 | no-preem | Lmax

- Può essere usato solo offline, quando tutti i parametri dei task (inclusi gli arrival times) sono noti.
- Si basa sugli alberi di ricerca, con tecniche di **pruning**, riduce la complessità media.
- Non si espande fino a quando la schedulazione parziale è fortemente fattibile, cioè se anche aggiungendo i nodi rimanenti lo scheduling rimane fattibile.
- Questo potrebbe essere un sistema time-triggered, dove i task sono attivati da un timer, la lista che viene data al dispatcher viene chiamata **Task Activation List**.

LDF : Latest Deadline First : 1 | prec, sync | Lmax

Dato un **grafo** delle precedenze, LDF costruisce una lista di compiti partendo dalla coda: tra i nodi che non hanno successori, LDF seleziona ogni volta il compito con la scadenza (*deadline*) più lontana nel tempo, fino a includere tutti i compiti. Lo schedule effettivo è la lista risultante letta in ordine inverso.

Per tutti i nodi che non hanno successore si sceglie quello con la deadline più lontana e si esegue in ordine inverso, (cioè si mette alla fine della lista di schedulazione) i task vengono estratti dalla testa della coda. La struttura dati consiste in un grafo diretto aciclico.

La lista verrà poi letta al contrario, cioè partendo da quello con la deadline più vicina verso quello con la deadline più lontana.

Schedulazione di task periodici

Formulazione del problema:

- Tutti i job di uno stesso task hanno $T_i(C_i, T_i, D_i)$ comuni.

Per ogni task periodico dobbiamo garantire:

- L'attivazione periodica di questi job : $r_{ik} = (k - 1)T_i$
- Il rispetto della deadline relativa: $d_{ik} = r_{ik} + D_i$

Ogni job può avere una esecuzione diversa che dipende dalla interferenza di processi a priorità più alta o da job a priorità più bassa che bloccano l'accesso in memoria critica.

Timeline scheduling

Algoritmi di scheduling che si usano per i task periodici: **timeline scheduling**, uno degli algoritmi più semplici. Si usa con i task time-driven, periodici, avviata dal semplice passare del tempo. In questo caso, negli eventi periodici, dato che li conosciamo, possiamo fare una schedulazione offline, con tutte le ottimizzazioni del caso, a runtime il dispatcher deve solo eseguire la lista (*zero overhead di schedulazione*).

L'**obiettivo** è che ogni task periodico sia attivato con la frequenza di attivazione richiesta.

Il **metodo** consiste nel dividere l'asse temporale in time slot, ogni task viene allocato in uno slot per la sua attivazione che viene generata da un timer.

Questo algoritmo va bene con il TDMA.

L'asse del tempo viene suddiviso in intervalli regolari di lunghezza fissa, chiamati **slot temporali** o **cicli minori** (*minor cycles / frames*).

La lunghezza di questo intervallo (f) viene solitamente scelta in base al Massimo Comune Divisore (MCD) dei periodi dei vari task presenti nel sistema.

L'intera sequenza di scheduling si ripete in modo identico dopo un intervallo di tempo chiamato **Hyper Period**, che corrisponde al Minimo Comune Multiplo (mcm) dei periodi di tutti i task.

Una volta pianificato lo schedule per la durata di un ciclo maggiore, questo viene ripetuto ciclicamente all'infinito.

Hyper Period: intervallo minimo di tempo dopo il quale la schedulazione si ripete.

Ogni macro ciclo viene diviso in microcicli, i microcicli vengono scelti considerando il massimo comune denominatore, meglio sovracampionare e ottenere. Dobbiamo verificare che la somma dei task di ogni microciclo non superi la lunghezza di ogni microciclo.

I tempi di context switching li inglobiamo nei tempi di esecuzione del task.

Per l'implementazione serve un timer che scandisca i microcicli. Tutti i nodi devono essere sincronizzati.

Vantaggi:

- Facilità di implementazione (timer),
- Bassissimo overhead a runtime (scheduling offline),
- controllo del jitter molto accurato.

Svantaggi:

- Se il sistema è event-driven non possiamo inserire niente nello scheduling (neanche nei tempi morti).
- Non è espandibile: per inserire un nuovo task bisogna riprogettare tutto.
- Se usiamo il caso tipico e non il worst case qualche task potrebbe andare in overrun e sballare tutto. Non è robusto nel caso **dell'overrun**.
- Se il task va in overrun anche nel worst case è un disastro (potrebbe essere stato attaccato da un virus).

Cosa si fa se un task va in overrun?

1. Si lascia continuare il task: possiamo avere un effetto domino su tutti gli altri.
2. Abortire il task, ma il sistema può restare in stati inconsistenti.
3. Potremmo dividere il task in dei sotto task e riprogettare lo scheduling.

Priority Scheduling

Ad ogni task viene assegnata una priorità , i task con priorità maggiori hanno la precedenza rispetto a quelli con priorità più bassa, anche con preemption.

Se vogliamo flessibilità e abbiamo task event-driven utilizziamo il **priority scheduling**.

Il problema però è come assegnare le priorità. Prevedere che tutti i task siano sincroni (arrivino insieme) è una scelta *conservativa*, letteralmente il worst case.

La priorità deve tenere conto della **deadline** , non solo dell'importante.

Ogni task periodico insiste sulla cpu con il fattore di utilizzo ($U_i = \frac{C_i}{T_i}$).

$U_{tot} = C_1/T_1 + C_2/T_2$. Però anche con un fattore di utilizzo molto basso (anche prossimo a zero), può accadere di non rispettare una deadline.

Come assegnare le priorità? La priorità viene assegnata in base a vincoli temporali.

La fattibilità viene poi verificata attraverso dei metodi analitici.

PREMESSA:

- I task periodici con periodo fisso,
- Tutti i job hanno lo stesso worst case execution time.
- Tutti i job hanno la stessa deadline relativa, che è uguale al periodo.
- La schedulazione è preemptive,
- I task sono indipendenti tra loro e non hanno vincoli su risorse o di precedenza.

Rate Monotonic : $P_i \propto \frac{1}{T_i}$

La priorità è inversamente proporzionale al periodo (periodo più breve priorità maggiore).

È *statico*, poiché dipende da una grandezza che non cambia.

Tutti i job del task X hanno una priorità maggiore degli altri job del task Y.

È un algoritmo ottimo rispetto a quelli con priorità statica

- **Se esiste un'assegnazione di priorità che produce uno scheduling fattibile allora l'assegnazione RM è fattibile. Se non è schedulabile da Rate Monotonic**

Si basa sul fatto che la deadline relativa sia uguale al periodo: $D_i = T_i$. Di conseguenza, ordinare i task per periodo equivale a ordinarli per la loro scadenza. Sotto questa specifica condizione, Liu e Layland hanno dimostrato matematicamente che RM è l'algoritmo a priorità fissa ottimale.

Deadline Monotonic: $P_i \propto \frac{1}{D_i}$

Se la deadline è minore del periodo, la priorità è inversamente proporzionale alla deadline relativa.

È statico, la deadline relativa non dipende dal tempo.

Earliest Deadline First: $P_i \propto \frac{1}{d}$

La priorità è inversamente proporzionale alla deadline assoluta, la priorità maggiore va alla deadline assoluta più vicina.

Ma siccome la deadline assoluta è relativa all'istante di arrivo (mentre quella relativa all'istante di attivazione), in questo caso vengono scelti i job con priorità maggiore volta per volta (non funziona come prima che il job di un task vince sempre su un job di un altro task). Quindi è un algoritmo di scheduling **dinamico**.

Ha uno spazio di schedulabilità maggiore del rate monotonic, riesce a schedulare più task di rate monotonic.

EDF calcola di volta in volta la deadline dei task che arrivano: se il nuovo task ha una deadline più vicina toglie quello corrente, se quello corrente ha una deadline più vicina lo lascia.

- $d_{i,k} = r_{i,k} + D_i$ (Istante di attivazione + Deadline relativa)

EDF con preemption è ottimale tra tutti gli algoritmi: se esiste uno scheduling fattibile lo trova, se non riesce a schedulare lui non lo può fare nessun altro algoritmo.

Come identificare il worst case?

La fattibilità potrebbe dipendere dall'istante di attivazione.

Istante critico: istante di arrivo che genera il response time più lungo per un certo task.

Per un certo task questo istante accade quando arriva sincrono a tutti i task che hanno priorità più alta della sua, si becca la massima interferenza possibile.

Se ho una schedulazione statica è molto semplice calcolare il worst case , ma se ho una schedulazione dinamica risulta molto più complesso.

Come si verifica la fattibilità?

Condizione sufficiente per la non-schedulabilità o necessaria per la schedulabilità

Ogni task periodico utilizza il processore per una frazione di tempo: $U_i = \frac{C_i}{T_i}$.

Quindi l'utilizzo totale del processore sarà la somma dei fattori di utilizzo: $U_p = \sum_{i=1}^n \frac{C_i}{T_i}$.

U_p è il carico del processore.

- Se $U_p > 1$ il processore è in overload, sovraccarico, il task non può essere schedolato, non esiste una schedulazione fattibile per quel task set.

Essendo condizione necessaria ma non sufficiente per la schedulabilità ci sono casi in cui $U_p < 1$ ma un task non è comunque schedulabile.

Task set armonico: i periodi dei task sono multipli tra loro, permettono di ottenere la più alta utilizzazione del sistema.

Least Upper Bound

Il least upper bound di un algoritmo è il minimo di tutti i limiti superiori (Uub) calcolato considerando tutti i possibili insiemi di task immaginabili che saturano il processore.

$$U_{lub}(A) = \min U_{ub}(\Gamma, A)$$

Prendo tutti i task set che utilizzano pienamente il processore, calcolo tutti gli upper bound e cerco il valore più piccolo, al di sotto di questo è fattibile.

Condizione sufficiente di schedulabilità

Se $U_p \leq U_{lub}$ il task set è sicuramente schedulabile con Rate Monotonic.

È una condizione sufficiente di schedulabilità con rate monotonic

U_{lub} per Rate Monotonic

Liu e Layland hanno provato che per un set di task periodici : $U_{lub}^{RM} = n(2^{\frac{1}{n}} - 1)$

Inoltre con se $n \rightarrow \infty$, $U_{lub} \rightarrow \ln 2$: 0.69

Test di garanzia per RM (condizione sufficiente) $Up \leq n(2^{\frac{1}{n}} - 1)$

Se si utilizza l'algoritmo Rate Monotonic (RM) per schedulare un insieme di task periodici e calcoli che il fattore di utilizzo totale della CPU (Up) è inferiore o uguale a 0.69 (o più precisamente, al valore esatto calcolato con la formula di Liu e Layland per quel numero di task), il sistema è sicuramente schedulabile.

Questa formula vale per un numero di task che tende a infinito, se i task sono pochi il limite è più alto.

Schedulabilità EDF

Condizione necessaria e sufficiente affinché un task sia schedulabile con EDF è che $Up \leq 1$. Con EDF con preemption, un qualsiasi task set può utilizzare fino al 100% del processore.

EDF vs RM

EDF:

- Efficiente,
- Meno context switches,
- Più fairness negli overloads.

RM

- Più semplice da implementare
- Più predicibile.

Schedulazione di task periodici e aperiodici

I task aperiodici sono solitamente attivati da eventi esterni. Da un lato si vuole ridurre il response time dei task aperiodici ma dall'altro non mettere a rischio la schedulabilità dei task periodici.

Task aperiodici con hard deadline

Se i task aperiodici hanno delle deadline HARD, devono essere garantiti anche nel caso worst case. Possiamo dare delle garanzie se abbiamo un bound del tempo di interarrivo, come avviene nei task sporadici. Quindi i task sporadici possono essere garantiti come task periodici con **$C_i = WCET$** , **$T_i = MIT$ (minimum interval time)**.

Task aperiodici con soft deadline

Devono essere eseguiti senza mettere a rischio di task real time. Potremmo essere interessanti nel: minimizzare il response time medio,

Servire i task in maniera **immediata** potrebbe ridurre il loro response time medio ma potrebbe causare deadline miss. Potrebbero essere serviti in **background**: la richiesta aperiodica viene eseguita nei tempi morti della cpu, in questo caso aumenta il response time e non interferiscono

con la schedulazione di altri task, ma non sempre potrebbe andargli bene, potrebbe rischiare la *starvation*, se arrivano sempre task periodici quelli aperiodici non vengono eseguiti mai e anch'essi potrebbero mancare delle deadline.

Server Aperiodici

Un approccio potrebbe essere quello di gestire l'esecuzione dei task aperiodici con un processo server. Normalmente un server è un'attività kernel, **task periodico fittizio**, che si presenta come un processo periodico: quando viene il suo turno usa la sua capacità computazionale per eseguire i task aperiodici arrivati fino a quel momento.

Un task è un processo periodico con due parametri:

- **Cs (capacità o budget)**
- **Ts (periodo).**
- Il server è schedulato con un qualsiasi processo periodico;
- I task aperiodici da eseguire vengono scelti in base ad una coda, che deve essere gestita in maniera opportuna.
- I server a priorità fisse possono essere usati con RM e DM.

Polling server

- All'inizio di ogni periodo, il budget è carico al massimo.
- Il budget si consuma durante l'esecuzione dei vari job.
- Quando il server diventa attivo e ci sono processi in coda, essi vengono eseguiti fino a quando $C_s > 0$.
- Se il server è attivo ma non ci sono task in coda, il budget viene scaricato e viene eseguito un task periodico.
- I task aperiodici devono aspettare che il budget si ricarichi completamente per essere eseguiti.

Test di schedulabilità per task periodici con polling server

Per garantire un set di task periodici hard real time in presenza di task aperiodici soft real time gestiti da un polling server dobbiamo valutare l'interferenza prodotta dal polling server sulle esecuzioni periodiche.

Nel caso peggiore, tale interferenza è la stessa introdotta da un task periodico che ha tempo di computazione C_s e periodo T_s .

Quindi la schedulabilità di un set di task periodici con n task e fattore di utilizzo U_p può essere garantita se:

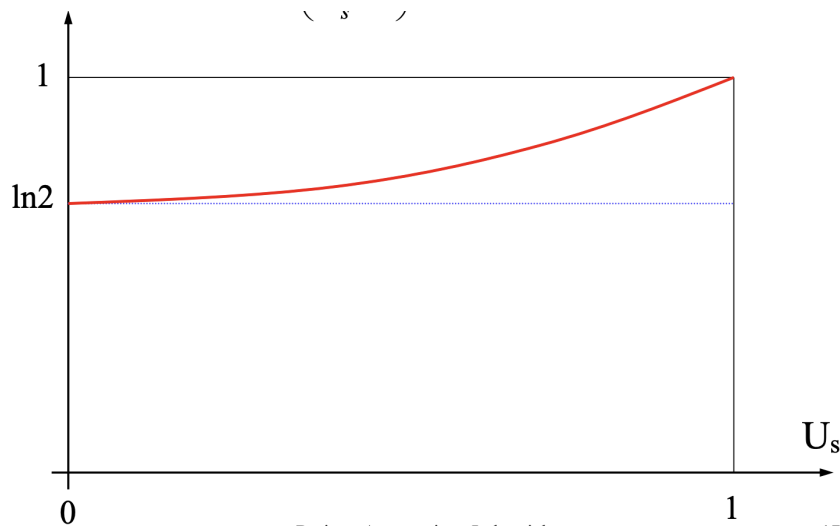
$$U_p + U_s \leq U_{lub}(n + 1)$$

Se sono schedulati con il RM : $U_p + U_s \leq (n + 1)[2^{\frac{1}{n+1}} - 1]$

Potrebbe essere necessario avere diversi polling server per gestire diversi task set.

Magari un server ad alta priorità per i set con priorità maggiore, o un server con priorità bassa.

$$U_p + \sum_{i=1}^m U_{si} \leq U_{lub}(n + m) \quad , \text{ con } m \text{ numero di servers.}$$



La curva rossa rappresenta la funzione limite che definisce la condizione sufficiente di schedulabilità per i task periodici in presenza del server.

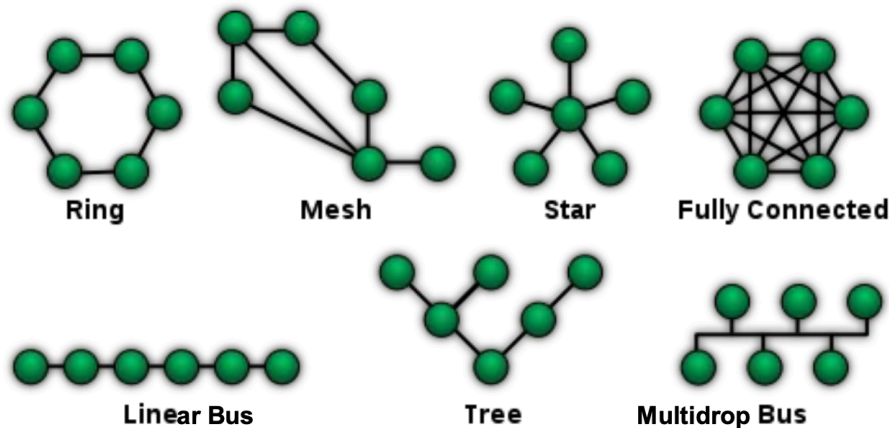
Se non c'è alcun server per i task aperiodici **$U_s = 0$ (Estremo sinistro)**, il limite per i task periodici è esattamente il classico valore limite 0.693.

$U_s = 1$ (Estremo destro): Quando il server tende a occupare l'intera CPU, il limite per i restanti task periodici tocca quota 1.

Network basics

Una rete è un sistema di dispositivi informatici interconnessi che comunicano e condividono risorse tra loro.

Le reti locali (**LAN**) sono reti che collegano tra loro i dispositivi all'interno di un'area limitata, come una fabbrica, un edificio o un'automobile. Esse possono avere diverse **topologie**:



- **Ad Anello (Ring Topology)**: Ogni dispositivo è collegato esattamente ad altri due, formando un percorso circolare continuo per i segnali.
- **A Maglia (Mesh Topology)**: I dispositivi sono collegati tra loro in modo ridondante. È estremamente affidabile perché i dati possono prendere percorsi alternativi se un collegamento fallisce.
- **A Stella (Star Topology)**: Ogni dispositivo è collegato a un hub o switch centrale. È la più comune oggi; se un cavo si rompe, solo quel dispositivo perde la connessione.
- **Fully Connected (Maglia Completa)**: In questa topologia, **ogni singolo nodo è collegato direttamente a tutti gli altri nodi** della rete. Massima affidabilità (tolleranza ai guasti totale) e prestazioni elevate, poiché non c'è condivisione di banda, ma costi proibitivi e complessità di cablaggio estrema man mano che il numero di nodi cresce. Viene usata quasi esclusivamente per dorsali critiche (backbone) o reti militari.
- **Bus Topology**: Tutti i nodi sono collegati a un unico cavo centrale. È semplice ma vulnerabile: se il cavo principale si rompe, l'intera rete si interrompe.
 - **Linear Bus**: È la forma più pura della topologia a bus, dove tutti i nodi sono disposti in fila lungo un unico mezzo di trasmissione. I dati viaggiano lungo il cavo e vengono letti solo dal destinatario. Ai due estremi del cavo sono necessari dei **terminatori** per evitare la riflessione del segnale (eco). Molto economica e facile da installare in piccoli uffici. Se il cavo principale si danneggia, l'intera rete cade. Inoltre, con molti dispositivi, le collisioni dei dati rallentano drasticamente le prestazioni.
 - **Bus Multidrop**: il termine "multidrop" si riferisce specificamente a una configurazione dove un **singolo collegamento principale** ha diversi punti di accesso (drop) per i dispositivi. È un'architettura tipica dei sistemi industriali o dei

vecchi sistemi mainframe. Un unico bus "scende" (drops) verso i vari terminali. Mentre il bus lineare è spesso peer-to-peer, il multidrop è frequentemente basato su una gerarchia **Master-Slave**, dove un dispositivo centrale controlla chi può parlare sul bus per evitare il caos.

- **Tree Topology** (Gerarchica o ad Albero): Questa topologia combina le caratteristiche della stella e del bus. Vi è una serie di reti a stella collegate a un cavo bus principale o a un nodo radice (root). I nodi sono disposti in modo gerarchico. Un nodo "padre" può avere più nodi "figli", creando dei livelli. È altamente scalabile e permette di isolare e dare priorità a determinati segmenti della rete. Se il nodo radice o il segmento di bus principale (trunk) fallisce, l'intera sezione sottostante viene isolata.

Network protocols

I **protocolli di rete** sono documenti che definiscono un insieme di regole, procedure e formati che consentono la comunicazione tra due o più dispositivi su una rete.

- Possono essere **standard proprietari** o standard rilasciati da organismi internazionali come l'**IEC**, l'**IEEE**, ecc.

I protocolli di rete definiscono:

- tutte le operazioni all'interno di una rete e come le varie operazioni devono essere eseguite.
- tutto, dalle strutture dati di rete di base (**frame**) fino ai servizi.

In termini semplici: "I protocolli di rete gestiscono le comunicazioni sulle reti".

Invece di progettare un unico protocollo monolitico e complesso, l'approccio comune consiste nel progettare diversi protocolli più semplici e cooperanti che formano uno **stack di protocolli di comunicazione**.

- Lo stack di protocolli segue un **modello di riferimento**, ovvero un'architettura a livelli (layer) in cui ogni livello:
 - è responsabile di una parte diversa della comunicazione;
 - offre un servizio (cioè una funzionalità di comunicazione) al livello superiore, utilizzando i servizi forniti dal livello inferiore e nascondendo come tale servizio venga effettivamente realizzato.
- Questo concetto di stratificazione è stato sviluppato per adattarsi ai cambiamenti tecnologici.
- I livelli sono disposti dai livelli più bassi, a partire da quello fisico (prevalentemente hardware), fino ai livelli più alti (software).

Overview reti industriali

Le reti industriali collegano dispositivi, macchinari e sistemi di controllo all'interno di un ambiente industriale per lo scambio di comandi e dati.

- A differenza delle tradizionali reti domestiche o da ufficio, le reti industriali sono progettate per operare in **ambienti ostili**, soggetti ad esempio a temperature estreme, polvere e interferenze elettromagnetiche, ecc.
- Le reti industriali supportano lo **scambio di dati critici in tempo reale**, elemento essenziale per il controllo dei processi e l'automazione di fabbrica.

Caratteristiche chiave delle reti industriali:

1. **Comportamento in tempo reale (Real-time)**: ovvero la prevedibilità, per garantire ritardi di comunicazione limitati (bounded delays) per le operazioni critiche dal punto di vista temporale.
2. **Determinismo**: La garanzia che un messaggio arrivi a destinazione entro un intervallo di tempo certo e prevedibile.
3. **Robustezza**: I componenti (cavi, switch, connettori) hanno protezioni speciali (IP67) per resistere a vibrazioni, umidità e agenti chimici.
4. **Scalabilità**: per connettere un numero elevato di dispositivi.
5. **Ridondanza**: Poiché il fermo macchina ha costi elevatissimi, queste reti utilizzano spesso topologie ad anello o a maglia per garantire la continuità operativa anche in caso di guasto.
6. **Dispositivi e protocolli specializzati**: i dispositivi sono costruiti per scopi specifici, pertanto possiedono generalmente software dedicati ed eseguono protocolli progettati appositamente per ottenere ritardi garantiti e soddisfare le esigenze dell'applicazione.
Protocolli Specifici: Spesso utilizzano varianti "industrializzate" di standard comuni, come **Profinet**, **EtherNet/IP** o **Modbus TCP**.

I ritardi tipici (latenza) possono essere nell'ordine di:

- **Centinaia di millisecondi**: per applicazioni con l'operatore umano nel ciclo di controllo (*human-in-the-loop*).
- **Decine di millisecondi**: per il controllo di processo (es. monitoraggio di temperature o flussi).
- **1 millisecondo o meno**: per il controllo del movimento (*motion control*, come i bracci robotici o le macchine a controllo numerico), a seconda delle applicazioni industriali supportate.

Alcune applicazioni (ad esempio il controllo del movimento) richiedono **comunicazioni deterministiche**, ovvero comunicazioni che seguono una pianificazione precisa (*schedule* o cronotabella).

Indicatori Chiave di Prestazione (KPI) per le reti Real-Time (RT)

Tempo di consegna (Delivery time): il tempo necessario per trasportare una *Service Data Unit* (**SDU**, il carico utile o *payload* del messaggio) da un nodo (sorgente) a un altro nodo (destinazione).

- Il tempo di consegna viene misurato all'interfaccia del **livello Applicazione**.
- Deve essere dichiarato il **tempo di consegna massimo** (tipicamente per due casi: assenza di errori di trasmissione e perdita di un singolo *frame* con relativo recupero).

Misurare il tempo tra le interfacce applicative è fondamentale perché include non solo il tempo di viaggio sul cavo, ma anche il tempo di elaborazione dello **stack di protocolli** (incapsulamento e decapsulamento) su entrambi i nodi. In ambito industriale, è l'unico valore che conta davvero per garantire che il comando arrivi al macchinario in tempo.

I due scenari di calcolo:

1. **Caso nominale (No errors):** Rappresenta la latenza standard della rete in condizioni ottimali. È il valore di riferimento per il determinismo.
2. **Caso degradato (Recovery):** Poiché le reti industriali operano in ambienti rumorosi, è essenziale sapere quanto tempo impiega la rete a rilevare un errore e a trasmettere nuovamente il dato. Se anche con un errore il tempo totale rimane sotto la soglia critica del processo, la rete è considerata affidabile.

L'obiettivo di una rete real-time è garantire la consegna tempestiva di **ogni singolo frame**. L'obiettivo di una rete veloce, invece, è minimizzare il **tempo di consegna medio** di un insieme di frame. Ma **non fidarti della media quando devi garantire la prestazione del singolo!**

È necessaria l'analisi dei ritardi (**Delay analysis**).

Nelle reti real-time, è necessario considerare il tempo di consegna massimo per valutare se i vincoli temporali dell'applicazione siano soddisfatti o meno. Il ritardo medio non è utile.

Esperimenti e simulazioni da soli potrebbero non essere sufficienti a verificare il rispetto dei vincoli temporali dell'applicazione:

- Gli esperimenti potrebbero non coprire l'intero spettro delle condizioni operative reali della rete.
- Le simulazioni potrebbero non modellare accuratamente il sistema reale.

Si utilizzano tecniche analitiche per calcolare il tempo di consegna ottenuto nella condizione peggiore (worst-case condition). Se non è possibile calcolare esattamente il ritardo nel caso peggiore, deve essere almeno possibile ottenere un limite superiore sicuro (safe upper bound).

Fonti di imprevedibilità nelle comunicazioni

Ogni fonte di imprevedibilità deve essere presa in considerazione:

- **Controllo dell'accesso al mezzo:** Il modo in cui i dispositivi si contendono l'uso del canale fisico.
- **Collisioni:** Quando due o più dispositivi trasmettono contemporaneamente, corrompendo i dati.
- **Gestione delle code** (Queue management): Il modo in cui i pacchetti vengono ordinati e prioritizzati all'interno dei nodi.
- **Sovraccarico delle code** (Queue overflow): Quando la memoria temporanea (buffer) è piena e i nuovi dati vengono scartati.

In questo contesto, definiamo la **tempestività** (timeliness) come la capacità della rete di consegnare i dati al momento giusto per l'uso previsto. Nelle reti real-time, la tempestività deve essere garantita già in fase di progettazione (by design) e il livello Data Link svolge un ruolo cruciale in questo.

Data Link Layer

Il Data Link Layer, detto anche **livello di collegamento**, è il secondo livello del modello OSI a 5 livelli della rete. Ogni dispositivo che opera a livello di collegamento prende il nome di **nodo**.

Questo è il livello dei protocolli che trasferiscono i dati tra nodi di rete adiacenti appartenenti ad una WAN (wide area network) o ad una LAN (local area network). I canali di comunicazione, che collegano nodi adiacenti lungo un cammino prendo il nome di **link**. I datagrammi del livello di rete che devono essere trasferiti da un host sorgente ad uno di destinazione, devono essere trasportati lungo ciascun link da un estremo all'altro.

Su ogni collegamento, un nodo trasmittente incapsula il datagramma in un **frame** e lo trasmette lungo il link stesso. Un datagramma può essere gestito da differenti protocolli a livello di collegamento lungo le diverse tratte che costituiscono il suo percorso.

Il data link layer fornisce le seguenti **funzioni**:

- Inserisce i dati all'interno delle **frame** e si assicura che vi sia una trasmissione error-free su ciascuno dei link (non tra sorgente e destinazione),
- Controlla la **temporizzazione** della trasmissione,
- Aggiunge alla frame il tipo, l'indirizzo e le informazioni necessarie per il controllo degli errori.

L'IEEE ha suddiviso questo livello in due sottolivelli:

- **Logical Link Control (LLC):** gestisce il collegamento tra due nodi occupandosi dei servizi di trasferimento delle informazioni, come l'invio con o senza conferma (acknowledgement), il rilevamento degli errori e il recupero, ecc.

- **Media Access Control (MAC):** determina l'ordine di trasmissione dei nodi che competono per l'accesso alla rete. Il MAC è fondamentale per il comportamento in **tempo reale** delle reti che utilizzano un mezzo condiviso.

Protocolli di Controllo dell'Accesso al Mezzo (MAC)

A livello MAC, sono possibili diversi approcci per gestire la trasmissione dei frame:

- **Master/Slave:** Un nodo centrale (Master) controlla e autorizza le trasmissioni di tutti gli altri nodi (Slave).
- **Token Passing:** Un "gettone" virtuale viene passato da un nodo all'altro; solo chi lo possiede può trasmettere.
- **Time Division Multiple Access (TDMA):** Il tempo viene diviso in slot fissi assegnati a ciascun dispositivo.
- **Carrier Sense Multiple Access (CSMA):** I nodi "ascoltano" il canale prima di trasmettere (disponibile in molte varianti).
- **Switched (es. Switched Ethernet):** L'uso di switch per creare connessioni punto-punto dedicate, eliminando i domini di collisione.

Ogni soluzione ha pregi e difetti. La misura in cui una soluzione superi le altre dipende fortemente dai parametri di prestazione desiderati. Per scegliere il protocollo adatto, devono essere considerate almeno le seguenti caratteristiche:

- **Prevedibilità:** Latenza limitata (*bounded*) e basso **jitter** (ovvero una variazione minima nei tempi di consegna dei frame).
- **Larghezza di banda (Bandwidth):** La capacità effettiva di trasmissione dati del canale.
- **Overhead del protocollo:** La quantità di dati di controllo necessari rispetto ai dati utili.
- **Necessità di sincronizzazione:** Se i nodi debbano o meno condividere un orologio comune preciso.
- **Flessibilità:** Facilità nel modificare la configurazione o il carico della rete.
- **Scalabilità:** Capacità della rete di mantenere le prestazioni all'aumentare del numero di nodi.
- **Tolleranza ai guasti (Fault-tolerance):** Capacità del sistema di continuare a funzionare in caso di errore o rottura di un componente.

Nelle reti industriali, la scelta ricade spesso su **TDMA** o **Master/Slave** quando il determinismo è la priorità assoluta (basso jitter). Al contrario, il **CSMA** (tipico dell'Ethernet commerciale) è molto flessibile e facile da scalare, ma soffre di imprevedibilità a causa delle possibili collisioni.

L'**Ethernet Commutato (Switched)** rappresenta oggi il compromesso standard, poiché l'uso degli switch elimina le collisioni pur mantenendo l'alta velocità dell'Ethernet.

Problematiche relative al data link layer:

- **Indirizzamento (Addressing)**
- **Controllo del collegamento logico: LLC (Logical Link Control)**
 - Controllo degli errori di trasmissione
- **Controllo dell'accesso al mezzo - MAC (Medium Access Control)** (*per mezzi condivisi*)

Indirizzamento (Addressing)

Consiste nell'identificazione delle parti coinvolte in una transazione di rete. Può essere di tre tipi:

- **Indirizzamento diretto (Direct addressing):** Il mittente e il destinatario (o i destinatari) sono identificati esplicitamente in ogni transazione, utilizzando **indirizzi fisici** (come avviene in Ethernet).
- **Indirizzamento indiretto basato sulla sorgente (source addressing):** Vengono identificati esplicitamente i **contenuti** del messaggio. I ricevitori che necessitano di quel messaggio lo prelevano dalla rete (come avviene in CAN e WorldFIP). *Source inteso come produttore del dato da inserire nel contesto dell'architettura produttore consumatore.*
- **Indirizzamento indiretto basato sul tempo (time-based addressing)** Il messaggio è identificato dall'**istante temporale** in cui viene trasmesso (come avviene in TTP). Necessita di una tabella di schedulazione pronta e di un controllo rigido della rete, i nodi sono programmati per conoscere i messaggi inviati in un certo momento.

Logical Link Control

Si occupa del trasferimento delle informazioni a questo livello. I servizi tipici sono:

- **Invio con conferma immediata (Send with immediate acknowledge)** Il mittente attende una conferma (*acknowledge*) da parte del ricevitore. È fondamentale quando la perdita di un dato non è tollerabile. Il mittente sa con certezza se il messaggio è arrivato. Tuttavia, l'attesa della conferma introduce un ritardo (latenza) che va calcolato nell'analisi del caso peggiore.
- **Invio senza conferma (Send without acknowledge)** Non vi è alcuna sincronizzazione tra mittente e ricevitore. Molto più veloce, usato spesso per dati ciclici (come la lettura continua di un sensore) dove, se un pacchetto va perso, verrà semplicemente sovrascritto da quello successivo pochi millisecondi dopo.
- **Servizi orientati alla connessione (Connection-oriented services)** Deve essere stabilita una connessione tra le parti prima di qualsiasi comunicazione. Garantisce che i

dati arrivino nell'ordine corretto e gestisce il controllo di flusso, evitando che un mittente troppo veloce "affoghi" un ricevitore lento.

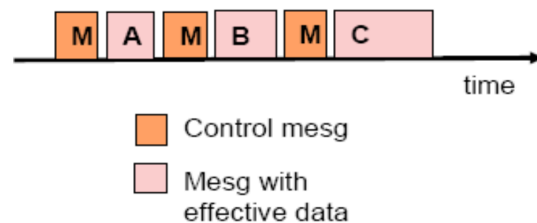
Livello Data Link

Controllo dell'accesso al mezzo (MAC)

- Determina l'ordine di accesso alla rete da parte dei nodi che competono tra loro, e di conseguenza il **ritardo di accesso alla rete**.
- È di fondamentale importanza per il **comportamento in tempo reale delle reti** che utilizzano un mezzo condiviso.

Master-slave

- **L'accesso viene concesso dal Master:** Solo l'unità centrale decide chi può trasmettere sul bus.
- **Nodi sincronizzati con il master:** Tutti i nodi Slave seguono il ritmo e le tempistiche dettate dal Master.
- **Richiede un messaggio di controllo per ogni messaggio di dati:** Per ogni invio di informazioni utili, è necessario un comando preventivo del Master (interrogazione o *polling*).



Il nodo master è l'unico che può trasmettere direttamente sulla rete. Attraverso un meccanismo di interrogazione sequenziale il master invia un pacchetto di comando (chiamato *Poll Request*) al primo Slave della lista. Solo dopo aver ricevuto questo comando, lo Slave può trasmettere il suo messaggio di dati (il *payload*) sul bus. Una volta che lo Slave ha finito (o dopo un tempo di *timeout* se lo Slave non risponde), il Master passa all'interrogazione dello Slave successivo. Completato il giro di tutti gli N Slave, il ciclo ricomincia da capo.

Il messaggio di polling non porta informazioni di conseguenza produce **overhead** di protocollo: il protocollo ne ha bisogno ma non hanno contenuti informativi.

- Permette di evitare le **collisioni**, perché i nodi trasmettono uno alla volta.
- Inoltre è **predicibile** perché schedula i vari slave in un certo ordine, con un algoritmo di scheduling real-time, tenendo conto dei periodi, iper periodo e deadline ma **non è preemptive**.
- Non ha un gran controllo del jitter, soprattutto se gli slave hanno periodi diversi.
- Tuttavia non scala assolutamente bene perché è centralizzato: troppi slave, troppi torni, periodi troppo lunghi.
- Però è **flessibile** perché si rifanno i calcoli se arriva un processo event driven.
- Per quanto riguarda la robustezza, è centralizzato quindi se il master avesse problemi l'intera comunicazione ne risentirebbe. Per risolvere questo problema i master sono sempre **ridondanti**, vi sono dei **master shadow** che sono sempre pronti a subentrare in sue veci.

Esistono soluzioni per questo e si chiamano: **hot swap**, **warm swap** e **cold swap**, ovvero per avere una maggiore affidabilità il master viene replicato.

- **Hot swap**: È la capacità di sostituire un componente mentre il sistema è **acceso e pienamente operativo**. Il sistema è acceso e in funzione. Non vi è nessuna interruzione del servizio. Il sistema operativo e il bus di comunicazione riconoscono automaticamente la rimozione e l'inserimento del nuovo hardware.
- **Warm swap**: Il componente può essere sostituito a sistema acceso, ma è necessario **sospendere temporaneamente le attività** o riavviare il driver/bus. Il sistema è acceso, ma in uno stato di "pausa" o con il bus disabilitato. Produce una breve interruzione dell'attività. Non è necessario spegnere completamente l'alimentazione, ma il componente non è "plug-and-play" immediato.
- **Cold swap**: Il sistema deve essere **completamente spento** e scollegato dall'alimentazione prima di procedere. Il sistema è spento (Off). Interruzione totale del servizio fino al riavvio completo. È la modalità standard per componenti non ridondanti o non progettati per la resilienza.

Token-passing

- **L'accesso è concesso dal possesso del token**: Solo il nodo che detiene il "gettone" (token), una frame di controllo, ha il diritto di trasmettere.
- **Ordine di accesso imposto dalla circolazione del token**: Il token si muove lungo un anello logico, definendo chi sarà il prossimo a parlare.
- **Accesso al bus asincrono**: È impossibile determinare a priori gli istanti esatti di accesso (poiché dipendono da quanto tempo ogni nodo trattiene il token).
- **Il funzionamento in tempo reale richiede un tempo limitato di detenzione del token**: Per garantire il determinismo, deve essere fissato un tempo massimo (**Token Holding Time**) entro il quale un nodo deve rilasciare il token. Il token può arrivare sia prima che dopo, magari se un nodo rinuncia ad una trasmissione troppo lunga.
- Il token è overhead, ma è più piccolo, quindi un po' meglio del master slave.

- **Robustezza:** Intrinseca nel meccanismo perché poi il token se uno non lo passa poi lo passa al prossimo. Se un nodo si guasta devono essere messi in atto meccanismi di autoriparazione.

Nelle reti industriali reali, i protocolli Token-passing includono procedure di **gestione dei guasti**:

- **Timeout:** Se un nodo (es. Nodo 1) passa il token e non vede attività sul bus entro un tempo limite, capisce che il successore è "morto". Il nodo 1 invia un messaggio speciale ("Who is next?") per trovare il primo nodo attivo dopo quello guasto (es. il Nodo 3). Una volta trovato, il Nodo 1 aggiorna la sua tabella interna e passerà il token direttamente al Nodo 3, saltando il nodo guasto.
- **Perdita del Token:** Se il nodo che si guasta è proprio quello che **possedeva** il token in quel momento, il token scompare dalla rete. In questo caso la rete rimane in silenzio. Dopo un tempo di inattività prestabilito, un nodo designato (o quello con indirizzo più alto) capisce che il token è andato perduto e ne **genera uno nuovo** per riavviare la circolazione.

Presenta una buona scalabilità, perché la rete è distribuita. Però se un nuovo nodo entra tutti gli altri devono saperlo.

TDMA (Time-Division Multiple Access)

- **L'accesso è concesso in slot temporali dedicati:** Ogni nodo ha un tempo riservato esclusivamente a lui per trasmettere.
 - **Gli slot temporali sono predefiniti in una struttura ciclica:** La sequenza temporale è stabilita a priori e si ripete costantemente.
 - **Sincronizzazione stretta con il tempo del bus:** Gli istanti di accesso al bus sono predeterminati, eliminando l'incertezza.
 - **Richiede una sincronizzazione globale (clock):** Tutti i nodi devono condividere lo stesso orologio con estrema precisione.
-
- L'asse temporale viene diviso in time slot, ogni processo trasmette solo quando arriva il suo slot.
 - La schedulazione viene fatta **offline**, ho controllo del jitter, ma l'overhead di protocollo sta nella **sincronizzazione**.
 - Risulta molto predicibile e annulla le collisioni.
 - Ma se ho un processo che segnala un allarme gli devo sempre riservare uno slot, anche se potrebbe non trasmettere mai, sprecando banda e tempo.
 - Zero flessibilità tutta la schedulazione da capo, robustezza anche se un nodo si guasta non succede niente se la sincronizzazione continua per gli altri, però se salta il protocollo di sincronizzazione salta tutto.
 - A volte i nodi mandano **heart beat** per dire che va tutto bene.

- Però se il nodo trasmette, guasto, al di fuori del suo time slot, allora salta tutta la comunicazione. Per questo esiste il **bus guardian** che controlla se i nodi trasmettono nel proprio time slot, impedisce che i nodi trasmettono al di fuori del proprio slot.
- **Determinismo totale:** Il ritardo massimo è noto e fisso.
- **Jitter quasi nullo:** I messaggi arrivano sempre con la stessa cadenza temporale.
- **È poco flessibile:** se aggiungi un nodo, devi programmare l'intera cronotabella (schedule) della rete.

CSMA (Carrier Sense Multiple Access)

- I protocolli si basano sul monitorare l'inattività del bus prima di trasmettere, totalmente asincrono.
- Ci potrebbero essere collisioni.
- Se ci sono collisioni i nodi interrompono la trasmissione e riprovano più avanti, in base a regole specifiche.
- La soluzione alle collisioni è assolutamente non deterministica. Non so a priori quando riuscirà a trasmettere con successo. Dal punto di vista del consumo di banda TDMA non era proprio il massimo: si spreca banda, invece con il csma se c'è un solo nodo che deve trasmettere si prende tutta la banda. La banda se la contendono solo i nodi che devono effettivamente trasmettere, cosa che non avviene in TDMA.
- Non è predicibile ma è efficiente. Nelle reti di real time si paga il prezzo dell'efficienza per avere predicibilità.

CSMA/CD (Collision Detection)

- **Utilizzato nell'Ethernet condiviso (basato su hub):** Le collisioni sono distruttive e vengono rilevate all'interno di finestre di collisione (slot).
- **In caso di collisione, l'intervallo di tentativo è casuale** e la finestra di randomizzazione viene raddoppiata per ogni tentativo successivo fino a 1024 slot (algoritmo BEB - *Binary Exponential Back-off*).
- **Non-deterministico** (particolarmente in presenza di collisioni a catena).

In una rete Ethernet condivisa, prima di trasmettere un pacchetto (frame), un nodo "ascolta" il canale (Carrier-Sense). Se il canale è libero, trasmette. Tuttavia, poiché i segnali impiegano del tempo per propagarsi lungo il cavo, due nodi distanti potrebbero sentire il canale libero nello stesso istante e iniziare a trasmettere contemporaneamente, causando una collisione. Nelle vecchie reti con hub, queste collisioni sono "distruttive", ovvero sovrappongono i segnali elettrici rendendo i dati illeggibili. I nodi riescono a accorgersi del problema solo all'interno di una specifica finestra temporale iniziale chiamata slot.

Quando viene rilevata una collisione, i nodi interrompono la trasmissione, attendono un tempo casuale e poi riprovano. Per evitare che i due nodi ricollidano immediatamente riprovando nello stesso istante, viene usato l'algoritmo **BEB (Binary Exponential Back-off)**.

Questo algoritmo funziona così: al primo fallimento il nodo sceglie un tempo di ritrasmissione in una piccola random. Se collide di nuovo, la finestra temporale in cui scegliere il numero casuale

raddoppia (crescita esponenziale), riducendo la probabilità di un ulteriore scontro, fino a un massimo di 1024 slot temporali. **Non è deterministico**: l'Ethernet classico basato su CSMA/CD non è adatto per contesti in tempo reale.

CSMA/BA (Bit-wise arbitration)

- **Arbitrato bit a bit con collisioni non distruttive.**
- **In caso di collisione, il nodo a priorità più alta non subisce conseguenze.** I nodi con priorità inferiori riprovano subito dopo.
- **La risoluzione della collisione è deterministica** e dipende dalla priorità dei nodi.

Il nodo con priorità più alta non si accorge quasi della competizione: continua la sua trasmissione come se nulla fosse accaduto e il suo messaggio arriva a destinazione senza alcun ritardo causato dalla collisione.

I nodi a priorità inferiore smettono di trasmettere non appena si accorgono di aver perso l'arbitrato e attendono che il bus torni libero, riprovando la trasmissione **subito dopo**, senza dover aspettare tempi casuali di back-off.

Ad esempio il CAN (controller area network) utilizza questo meccanismo:

- Ha un canale condiviso sul quale si possono verificare collisioni, che però non sono distruttive per il messaggio che ha priorità più alta.
- La frame can parte con un **identificatore** con n bit, che servono per identificare il messaggio, avviene quindi un **indirizzamento data based**, basato sul messaggio trasportato (Non vengono identificati i nodi ma le informazioni i dati trasmessi).
- Gli identificatori colgono la priorità del messaggio: i dati con priorità maggiori hanno identificatori con valori più piccoli.
- Il canale fa un **end logico** (zero vince sempre), basta che un bit di collisione è zero si sente zero. Se rileva un bit diverso da zero, un 1, percepisce che c'è stata una collisione e va in modalità listening. Tutti i messaggi pensano di essere da soli fin quando non sentono una collisione. Quello a priorità più alta neanche lo sa che ci sono state collisioni.
- Se l'id si basa sul periodo o sulla deadline allora la priorità varia in base a queste cose (si sta creando una sorta di rate monotonic). Si può calcolare caso peggiore e tutto. Rispetto alle analisi di Liu e Layland non c'è preemption. Potrebbe diventare prevedibile applicando delle analisi, possiamo trovare un upper bound. Flessibile, si deve conoscere il messaggio per definire una priorità. Però se un nodo ha il sincronizzatore guasto sono problemi.
- **Il bit dominante corrisponde al valore logico 0.** Il nodo con il bit recessivo, 1, perde l'arbitration.

CSMA/CA (Collision Avoidance, ASINCRONO)

- **Accesso basato sul rilevamento dell'inattività del bus** durante un intervallo di sincronizzazione più uno specifico intervallo di accesso casuale (*p-persistent*).
- **Non è esente da collisioni** (Es: IEEE 802.11 / Wi-Fi).

Il meccanismo di Back-off casuale:

Quando un nodo vuole trasmettere, ascolta il canale. Se lo trova libero, non parte subito: aspetta un **tempo fisso** più un **tempo casuale** (chiamato Back-off, indicato nel grafico come T1 e T2). Questo serve a evitare che più nodi, rimasti in attesa mentre il canale era occupato, trasmettano tutti nello stesso istante esatto non appena si libera.

Se un nodo sente che il canale è tornato occupato, congela il proprio timer (T2 deferred). Appena il canale si libererà di nuovo, riprenderà a far scorrere il timer dal punto in cui si era interrotto, per poi trasmettere finalmente il suo frame C.

Sebbene riduca drasticamente le collisioni, questo metodo non è privo di collisioni al 100% (due nodi potrebbero estrarre lo stesso identico valore casuale) ed è intrinsecamente non-deterministico a causa della componente casuale.

C'è un tempo di ascolto minimo, più un tempo di ascolto aggiuntivo per ogni nodo.

Non evita le collisioni by design: se si aspettano tempi diversi da tempi diversi.

Si evitano le collision by design se si evitano le collisioni aspettando tempi diversi ma dallo stesso istante di partenza (solo se è sincrono).

CSMA/BA (*Collision Avoidance, SINCRONO*)

- **Accesso basato sul rilevamento dell'inattività del bus** durante un numero di intervalli di tempo predefiniti (*mini-slot*) dopo la ricezione di un messaggio di riferimento sincrono.
- **Il mini-slot corrispondente determina la priorità.**
- **Esente da collisioni e deterministico.**
- **Il tempo è partizionato in una sequenza di minislot.** A ogni nodo è assegnato un numero unico di minislot da attendere quando il canale viene trovato libero, prima di poter iniziare l'invio.
- **I mini-slot temporali indicano una priorità più alta.**

Questa variante modifica radicalmente la filosofia del CSMA/CA per renderla adatta ai sistemi industriali in tempo reale, garantendo il **determinismo**.

1. **Il messaggio di riferimento (R):** All'inizio di ogni ciclo, viene inviato un messaggio di riferimento periodico (R) che sincronizza gli orologi di tutti i nodi della rete.
2. **I Mini-slot come priorità fissa:** Subito dopo il messaggio R, il tempo viene diviso in "mini-slot" numerati (1, 2, 3, 4...). A ogni nodo della rete viene assegnato un mini-slot fisso ed esclusivo. Più il numero del mini-slot è basso, più il nodo ha priorità alta.
3. **Funzionamento senza collisioni:** Quando il canale diventa libero, ogni nodo inizia a contare i mini-slot. Se un nodo ha assegnato il mini-slot 4, aspetta che passino i mini-slot 1, 2 e 3. Se in quel lasso di tempo nessun nodo a priorità più alta ha iniziato a trasmettere, il nodo occupa il canale inviando, mentre se il nodo successivo, che magari era associato al mini-slot 5, vede il canale occupato e aspetta il turno successivo.

Grazie a questa struttura rigidamente pianificata e priva di estrazioni casuali, questo protocollo diventa **totalmente esente da collisioni** (*collision-free*) e perfettamente **deterministico**, permettendo di calcolare con assoluta certezza il tempo massimo di trasmissione.

Mentre nel **TDMA**, la slot temporale (*time-slot*) definisce sia la **priorità** sia l'**intera durata** della trasmissione, nel **CSMA/CA sincrono**, invece, il mini-slot serve **solo a stabilire chi ha il diritto di parlare per primo** (la priorità). Una volta che il nodo con priorità maggiore inizia a trasmettere, "allunga" il suo tempo sul canale per trasmettere l'intero messaggio, e gli altri nodi scalano la propria attesa di conseguenza.

Mantiene il **determinismo** del TDMA (azzerando le collisioni) ma elimina la rigidità e lo spreco di tempo tipici degli slot prefissati.

Switched Ethernet

Gli Switch Ethernet possono supportare il traffico real time? Spoiler: NO

- **I nodi sono connessi tramite collegamenti dedicati point-to-point** (punto-a-punto) a un dispositivo centrale chiamato **Switch** (o Bridge).
- **I frame vengono inoltrati** dallo switch verso la porta di destinazione corretta basandosi sull'indirizzo MAC di destinazione (*Destination MAC address*), non lavorano in broadcast.
- **Nessuna collisione** sui collegamenti point-to-point (se operano in modalità *Full-Duplex*).
- **Più trasmissioni parallele** possono avvenire simultaneamente attraverso lo switch.
- Operano tradizionalmente nel data link layer, ma possono anche implementare funzionalità del livello 3, come il routing tra sottoreti.
- **Rigenerano le frame**,
- Applicano filtering e forwarding:
 - **Filtering**: capacità di determinare se una frame debba essere inoltrata o scartata.
 - **Forwarding**: capacità di determinare la porta alla quale la frame deve essere mandata.

Vantaggi degli Switch Ethernet

- Con un hub, l'intera rete costituisce un unico dominio di collisione. Al contrario, gli switch creano un dominio di collisione separato per ciascuna porta dello switch.
- **Segmentazione**: La sezione di rete in cui i frame possono collidere durante la loro trasmissione è limitata al singolo segmento connesso alla porta dello switch.
- **Microsegmentazione**: Poiché gli switch dispongono di un elevato numero di porte, è possibile fornire un accesso dedicato anche tramite il collegamento diretto di un nodo allo switch, assegnare ovvero una porta dello switch separata per ciascun nodo.
- Alcuni switch possono operare in modalità **full-duplex**: sono in grado di inviare e ricevere frame contemporaneamente sulla stessa porta.
- La combinazione di micro-segmentazione e funzionamento in full-duplex migliora significativamente il comportamento dell'Ethernet: la contesa del mezzo e le collisioni

vengono eliminate (collision free), rimuovendo la necessità di utilizzare il CSMA/CD e l'algoritmo Binary Exponential Back-off (BEB).

Gli switch comunque non forniscono predicibilità.

Gli switch però non permettono di riservare la banda ai nodi real time. Di solito in un'unica infrastruttura viaggiano frame best effort e frame real time, questi tipi di traffico sono comunque interdipendenti.

I frame vengono memorizzati in code di ricezione/trasmissione (*Rx/Tx queues*) all'interno dello switch. Se più nodi trasmettono contemporaneamente verso la stessa destinazione, i frame vengono accodati nella porta di uscita dello switch. Se l'arrivo dei frame supera la capacità della coda, si verifica una perdita di frame (dovuta a *buffer overflow* / traboccamento del buffer).

Ma al traffico real time devo riservare banda, che di solito hanno payload piccoli ma urgenti, mentre il traffico best effort ha frame grandi. Quindi quelle best effort potrebbero bloccare quelle real time. Le risorse possono saturarsi (**queue overflow**).

Anche se abbiamo una connessione praticamente punto-punto senza collisioni, il problema è quando queste frame si materializzano nella rispettiva code, anche se abbiamo una sorta di differenziazione accade che una frame non real-time ritarda quella real time.

In sintesi:

Uno switch commerciale che non include meccanismi specifici real time non garantisce il rispetto dei vincoli real time perchè:

1. **Non riserva banda al traffico real time** (il traffico best effort può ritardarlo in maniera imprevedibile).
2. **Potenziale queue overflow.**
3. **Fifo scheduling: non sono previsti algoritmi di scheduling real time. Non ci sono garanzie per il traffico real time.**

Reti a bordo auto

Time triggered / event triggered communication

Nei sistemi **time-triggered**, le frame sono trasmesse in momenti ben precisi e sono schedulate in time slot predefiniti con una schedulazione che si ripete in TDMA.

- Azionati dal tempo che passa.
- Usato in FlexRay.

Vantaggi:

- Totalmente predicibile.
- Facile da analizzare.
- Veloce nell'identificare messaggi mancanti o verificare che un nodo si è guastato (missing heartbeat).
- Scheduling diversi possono essere usati per applicazioni diverse.
- Garantisce **dependability**: provvede diversi livelli di fault tolerance.

Limiti:

- Occupazione inefficiente della banda.
- Non adatta a traffico asincrono.
- Flessibilità limitata: per introdurre un nuovo nodo bisogna rifare tutto lo scheduling.

Nei sistemi event triggered i messaggi sono trasmessi se si verificano determinati eventi.

Il protocollo di comunicazione deve evitare collisioni o gestirle in maniera adeguata (come fa il CAN con la CSMA/BA). Usato in CAN.

Vantaggi:

- Supporto al traffico asincrono.
- Occupazione efficiente della banda: solo i messaggi necessari sono trasmessi.
- Flessibilità: è possibile effettuare cambiamenti senza ridisegnare tutti i nodi esistenti.
Caratteristica fondamentale nell'industria automotive dato che il **design incrementale** è una pratica comune.

Limiti:

- Verificare che i vincoli temporali siano rispettati o capire che un nodo ha dei problemi è più difficile.

Domini funzionali: applicazioni differenti, vincoli diversi

Domini tradizionali

- **Powertrain (Gruppo motopropulsore):** controllo del motore e della trasmissione.
- **Chassis (Telaio):** controllo della stabilità e della dinamica del veicolo in base alle sollecitazioni di sterzata/frenata e alle condizioni di guida (es. superficie stradale, vento, ecc.).
- **Body & Comfort (Carrozzeria e Comfort):** controllo di porte, finestrini, tettuccio e sedili, climatizzazione, ecc.

Nuovi domini:

- **Multimedia/Infotainment (Multimedia e Intrattenimento):** lettori CD audio, DVD, lettori MP3, TV, intrattenimento per i sedili posteriori, servizi di informazione per la navigazione, ecc.
- **Human Machine Interface (Interfaccia Uomo-Macchina - HMI):** tecnologie di visualizzazione avanzate (display)...
- **Camera-based Advanced Driver Assistance Systems (Sistemi avanzati di assistenza alla guida basati su telecamera - ADAS):** avviso di deviazione dalla corsia (Lane Departure Warning), riconoscimento dei segnali stradali, visione notturna, rilevamento dei pedoni, assistente al parcheggio, visuale a 360° (Bird's-eye view), ecc.
- **Guida automatizzata (Automated driving):** dal livello 1 al livello 5.

Livelli di guida automatizzata: Concetti base

I cinque livelli di automazione della guida definiti dallo standard **SAE J3016** incrementano progressivamente il grado di automazione del sistema, riducendo parallelamente il coinvolgimento umano nell'attività di guida.

I livelli di automazione della guida categorizzano e classificano le funzionalità automobilistiche in base alla loro capacità di eseguire il **Dynamic Driving Task (DDT - Compito di Guida Dinamica)**, il quale consiste ampiamente in due sotto-attività:

1. **Comportamenti operativi (Operational behaviors):** che comprendono il controllo longitudinale (accelerazione/frenata), il controllo laterale (sterzata), nonché il rilevamento e la risposta a oggetti ed eventi (OEDR);
2. **Comportamenti tattici (Tactical behaviors):** come la selezione della velocità e della corsia, e la pianificazione delle manovre.

Si noti che i comportamenti strategici, come la pianificazione del percorso e della destinazione, sono esclusi dalla definizione dello standard SAE J3016.

Un concetto molto utile è quello di **Operational Design Domain (ODD - Dominio di Progettazione Operativa)**, che indica il dominio (o contesto) per il quale il sistema automatizzato è stato progettato.

Un ODD include restrizioni ambientali, geografiche e/o fasce orarie della giornata, e/o la necessaria presenza o assenza di determinate caratteristiche del traffico o della carreggiata (es. "autostrade ad accesso controllato negli Stati Uniti o in Canada", ecc.).

- Powertrain (Gruppo motopropulsore)
 - La funzione principale del dominio powertrain è il controllo del motore.
 - Presenta leggi di **controllo complesse**.
 - Periodi di campionamento nell'ordine di alcuni millisecondi (a causa della velocità di rotazione del motore).
 - Implementato in **microcontrollori ad elevata potenza di calcolo**.
 - Vincoli **temporali stringenti**.
 - Scambi di dati frequenti con altri domini dell'auto, ad esempio lo chassis (ESP, ABS) e il body (climatizzazione, ecc.).
 - Le funzioni del powertrain:
 - operano principalmente come sistemi di controllo a ciclo chiuso (closed-loop);
 - sono attualmente supportate per lo più da reti **CAN**, ma le comunicazioni del powertrain includono anche alcune funzioni critiche dal punto di vista temporale che potrebbero, in linea di principio, migrare verso comunicazioni di tipo *time-triggered* (attivate dal tempo) per ragioni di determinismo.

- Chassis (Telaio)
 - La funzione principale del dominio chassis è il controllo dello sterzo, della frenata e delle condizioni di guida.
 - I sistemi chassis fanno parte dei sistemi di sicurezza attiva del veicolo, che includono:
 - Sistema antibloccaggio delle ruote (ABS)
 - Controllo della dinamica del veicolo (Vehicle Dynamics Control)
 - Regolatore del livello di velocità adattativo (Adaptive Cruise Control - ACC)
 - e molti altri...
 - Requisiti di comunicazione simili a quelli del powertrain.
 - c'è spazio per le comunicazioni di tipo *time-triggered* (attivate dal tempo).
 - maggiore enfasi sulla sicurezza funzionale (*safety*), a causa dell'impatto sulla stabilità, sull'agilità e sulla dinamica del veicolo.

X-by-Wire

- **X-by-Wire**: sostituzione dei sistemi meccanici o idraulici con sistemi completamente elettrici/elettronici.
- I principali problemi di progettazione sono legati alla sicurezza funzionale (*safety*).
- Esempi di sistemi x-by-wire:
 - *Steer-by-wire* (sterzo elettronico),
 - *Shift-by-wire* (cambio elettronico),
 - *Brake-by-wire* (frenata elettronica).

I sistemi x-by-wire si basano principalmente sul controllo in retroazione (*feedback control*), pertanto richiedono una bassa latenza e devono essere affidabili almeno quanto i sistemi che vanno a sostituire.

L'implementazione delle funzioni X-by-Wire richiede inoltre un approccio di tipo **time-triggered**, al fine di garantire il comportamento temporale deterministico del sistema. La progettazione di tipo TT consente la verifica formale dei vincoli temporali end-to-end, un aspetto che è recentemente diventato una parte cruciale nel processo di progettazione dei veicoli.

Body & Comfort

Il dominio **Body & Comfort** (Carrozzeria e Comfort) è costituito da una serie di sistemi basati su software che includono funzioni per il controllo del cruscotto, dei tergicristalli, delle luci, delle portiere, dei finestrini, dei sedili, degli specchietti, della climatizzazione, ecc.

Queste funzioni software si scambiano reciprocamente numerose piccole informazioni.

L'attivazione delle funzioni di carrozzeria e comfort è innescata principalmente dalle azioni del conducente e dei passeggeri (es. l'apertura di un finestrino, il bloccaggio delle portiere, ecc.): una comunicazione di tipo **event-triggered** (attivata dagli eventi) risulta quindi appropriata.

Non tutti i nodi richiedono un'elevata larghezza di banda.

A causa delle diverse esigenze di comunicazione, si utilizzano architetture di rete gerarchiche, basate su reti a basso costo interconnesse attraverso una dorsale (*backbone*).

Multimedia/Infotainment (Multimedia e Intrattenimento)

Insieme di dispositivi hardware installati nelle automobili che forniscono intrattenimento audio e/o visivo e altre funzionalità.

- **include:**
 - Riproduzione di contenuti multimediali
 - CD, DVD, MP3
 - Sistemi di intrattenimento per i sedili posteriori (Rear Seat Entertainment - RSE)
 - TV
 - Console per videogiochi, ecc.
- **Requisiti:**
 - Elevata larghezza di banda (High Bandwidth)
 - Qualità del servizio (QoS) per i contenuti multimediali, latenza e jitter limitati (soft real-time)

ADAS

Telecamere (Camera)

- **Vista posteriore (Rear View):** Assistenza al parcheggio (Park assist), Cambio di corsia (Lane change).
- **Vista anteriore (Front View):** Evitamento delle collisioni (Collision Avoidance), Rilevamento dei pedoni (Pedestrian Detection), Assistenza per corsia e traffico trasversale (Lane / Cross traffic assist).
- **Vista laterale (Side View):** Specchietto elettronico (e-Mirror), Monitoraggio dell'angolo cieco (Blind spot monitoring).
- **Cruscotto (Dashboard):** Monitoraggio del conducente (Driver Monitoring), Registro di guida (Drive log).
- **All'interno dell'abitacolo (In cabin):** Videoconferenza (Video conferencing).
- **Vista panoramica (Surround View)**

Unità di Elaborazione (Processing units)

- Centralina di fusione dati e immagini (Image and Data Fusion ECU)
- Processore ADAS (ADAS processor)
- Registratore video digitale (Digital Video Recorder)
- Registratore dati (Data logger)
- Sistema telematico (Telematics system)

Requisiti di Comunicazione (Communication Requirements)

- Capacità di trasferire grandi quantità di dati con vincoli in tempo reale (real-time).
- Larghezza di banda elevata e garantita.

Il Sistema di Assistenza al Traffico Trasversale (Cross-Traffic Assist)

L'assistenza al traffico trasversale evita le collisioni con i veicoli che incrociano la traiettoria in corrispondenza delle intersezioni.

Il sistema:

- È in grado di rilevare una collisione imminente con il traffico trasversale e di **avvisare il conducente** (tramite un segnale visivo e acustico). Il rilevamento si basa sui dati forniti dalla telecamera stereo e dai sensori radar che monitorano le condizioni del traffico.
- **Supporta attivamente** il conducente nella frenata.

Il sistema può ridurre significativamente la probabilità di un incidente agli incroci o la gravità dell'impatto qualora la collisione non possa essere evitata.

Diagnostica di Bordo (OBD)

La diagnostica di bordo (OBD) è richiesta da molteplici funzioni del veicolo, tra cui:

- **Monitoraggio delle emissioni**
- **Diagnostica di componenti e proprietà**
- **Assistenza e manutenzione**, con la possibilità di scaricare e aggiornare il software

I sistemi OBD consentono al tecnico riparatore di accedere alle informazioni sullo stato di salute dei vari sottosistemi del veicolo.

- La quantità di informazioni diagnostiche disponibili tramite OBD è variata notevolmente fin dall'introduzione, nei primi anni '80, dei computer di bordo sui veicoli.
- Le prime versioni di OBD si limitavano ad accendere una **spia di malfunzionamento** (MIL) in caso di rilevamento di un problema, senza però fornire alcuna informazione sulla natura del problema stesso.

Le Moderne Implementazioni OBD Utilizzano:

- Una **porta di comunicazione digitale standardizzata** per fornire dati in tempo reale (run-time data).
- Una **serie standardizzata di Codici di Guasto Diagnostici** (DTC - *Diagnostic Trouble Codes*), che consentono di identificare e porre rimedio rapidamente ai malfunzionamenti all'interno del veicolo.

Le reti di diagnostica sono inattive finché non vi viene collegato uno strumento esterno; pertanto, non devono soddisfare i requisiti applicati alle reti che sono attive durante il normale funzionamento del veicolo (ad esempio, la compatibilità elettromagnetica - EMC, o le modalità di sospensione e risparmio energetico per l'efficienza energetica).

Requisiti delle reti automobilistiche

*La misura in cui un aspetto prevale sugli altri dipende fortemente dallo specifico dominio funzionale.

- **La predicibilità** è la proprietà che consente al progettista della rete di valutare, prima di avviare il funzionamento della rete stessa, se tutti i frame in tempo reale saranno consegnati a destinazione entro le scadenze (*deadline*) previste. Ciò significa che tutte le componenti del ritardo end-to-end del frame possono essere calcolate in modo esatto o stocastico.
- **La tolleranza ai guasti (Fault-tolerance)** è la capacità della rete di erogare un servizio anche a seguito del verificarsi di un guasto (ad esempio, errori nei messaggi, circuiti difettosi, guasti di linea). La ridondanza, sia spaziale che temporale, e i bus guardian sono esempi di misure utilizzate per ottenere la tolleranza ai guasti.
- **I requisiti di larghezza di banda (Bandwidth)** dipendono fortemente dai flussi di dati. Alcuni flussi, come quelli dei video grezzi provenienti dalle telecamere utilizzate per i

sistemi ADAS o per la guida automatizzata, richiedono un'elevata larghezza di banda, mentre i flussi dei tipici cicli di controllo nel dominio del gruppo motopropulsore (*powertrain*) non ne hanno bisogno.

- **La flessibilità (Flexibility)** è la capacità di introdurre *nuovi flussi di traffico* nella rete durante il ciclo di vita dell'auto, a seguito dell'aggiunta di nuovi componenti del veicolo (ad esempio, un rimorchio, una roulotte) o di nuove applicazioni che implementano funzionalità inedite.
- **Il basso costo (Low Cost)** è sempre un vantaggio nel settore automobilistico. Ad esempio, i protocolli di rete LIN e CAN riscuotono un grande successo e sono ampiamente utilizzati poiché funzionano bene e sono economici.
- **La sicurezza / Cybersecurity (Security)** sta diventando sempre più importante con la crescita delle comunicazioni inter-dominio e con la maggiore apertura dell'auto ai flussi provenienti dal mondo esterno (ad esempio, le comunicazioni car-to-x).
- **L'efficienza energetica (Energy efficiency)** è un aspetto importante, soprattutto quando il motore non è avviato. Per risparmiare energia, le reti automobilistiche offrono modalità di sospensione (*sleep*) e risveglio (*wake-up*) e il collegamento in rete parziale (*partial networking*). Un requisito fondamentale è che, all'avvio o al riavvio dell'auto, questa debba essere completamente operativa entro due secondi.

Dalle comunicazioni punto-punto a quelle multiplate

Fino all'inizio degli anni '90: collegamenti punto-punto tra le ECU.

ECU (Electronic Control Unit): sottosistema composto da un microcontrollore e da un insieme di sensori e attuatori.

Limitazioni dei collegamenti punto-punto:

- **Grande quantità di canali di comunicazione:** Il numero di collegamenti cresce con il quadrato del numero di ECU se ogni nodo è interconnesso con tutti gli altri.
- **Mancanza di scalabilità all'aumentare del numero di ECU:** A causa del peso, del costo, della complessità e dell'affidabilità di cavi e connettori.

Nelle auto di fascia alta odierne, vengono scambiati fino a 2500 segnali da un numero di ECU che può raggiungere le 70 unità. Questi dati hanno motivato l'utilizzo di reti in cui le comunicazioni sono multiplate su un mezzo condiviso, ovvero le reti sono state progettate per trasferire le informazioni provenienti da molti sensori attraverso le stesse linee dati.

Vantaggi delle reti di bordo (in-car networks):

- **Riduzione del peso:** La sostituzione di un "cablaggio con reti LAN nelle quattro portiere di una BMW ha ridotto il peso di 15 kg." (comunicato stampa, 1998)
- **Maggiore flessibilità e scalabilità più semplice**
- **Condivisione dei dati:** Più ECU possono condividere i dati per realizzare funzioni distribuite.

LIN (Local Interconnect Protocol)

Sistema di comunicazione seriale broadcast economico e a bassa velocità, a filo singolo.

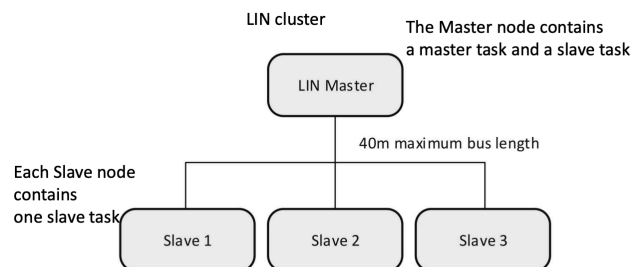
- **Ampiamente utilizzato** nelle auto di produzione.
- Presente nei sottosistemi ***body & comfort***, controllando ad esempio: portiera con tutte le sue funzionalità (es. alzacristalli, serrature, tettuccio apribile, ecc.), sedile con le sue funzionalità (es. motori per la regolazione della posizione, controllo dell'occupazione del sedile, ecc.). Avendo un dominio funzionale molto semplice (comandi per chiudere aprire finestrini tipo) ha *payload molto piccoli*.
- È *data based*: ogni nodo produce un determinato dato.
- Per ridurre i costi e il carico totale di traffico, tutti questi sottosistemi fanno parte di un'**architettura gerarchica**, in cui le sottoreti LIN sono collegate a una dorsale (*backbone*) CAN tramite un gateway LIN/CAN.

Storia

- L'adozione del LIN è avvenuta con l'aumento di sensori e attuatori a bordo dei veicoli, scenario in cui il cablaggio punto-punto era diventato troppo pesante e il protocollo CAN troppo costoso.
- Il progetto è stato avviato nel 1998 da un consorzio di case automobilistiche (BMW, Audi, Volvo, Volkswagen).
- Diverse specifiche rilasciate nel tempo:
 - Dallo standard LIN 1.1 (2000)
 - Fino allo standard LIN 2.2A (2010)
- Standard ISO di riferimento: ISO 17987 Parti 1-7.

Tecnologia

- Velocità di **20 Kbps** su un singolo filo.
- Carico utile (payload) da **1 a 8 byte** di dati + checksum a 8 bit.
- Identificatore a **6 bit**: sono disponibili 64 identificatori distinti che indicano il contenuto del messaggio o frame speciali (source addressing).
- **Topologia a bus** con architettura **Master-Slave**: un solo master, fino a un massimo di 16 slave, lunghezza totale massima di 40 metri su mezzo condiviso.



La tabella di schedulazione viene fatta *offline*. C'è un file con tutte le specifiche di trasmissione dei vari nodi, che viene dato in pasto ad un tool che produce lo scheduling.

Un file di descrizione LIN (**LDF - LIN Description File**) viene utilizzato per configurare e creare il comportamento di pianificazione (*scheduling*) del cluster LIN. Questo file definisce il baud rate, l'ordine e i ritardi temporali per la trasmissione degli header (intestazioni) da parte del task master, nonché il comportamento di risposta di ciascun task slave. La schedulazione del master consiste in tabelle con dei messaggi separati da ritardi specifici che vengono eseguiti in un loop continuo.

Il **master task**, in esecuzione sul nodo master, avvia la trasmissione degli header sul bus e tutti i task slave presenti nel cluster (compreso il task slave dello stesso nodo master) rispondono secondo quanto specificato nel LDF.

Cluster LIN: Master/Slave (un solo master, fino a 16 slave)

- Sul bus LIN vengono scambiati dei frame composti da una parte di Intestazione (**Header**) e una di Risposta (**Response**). Ogni frame viene avviato dal Master. Il task master presente nel nodo master controlla tutta la comunicazione sul bus LIN. Normalmente, il task master interroga (*poll*) ciascun task slave all'interno di un ciclo continuo trasmettendo un header.
- Il Master segue una **tabella di pianificazione (*schedule table*)**, presente nel LDF: ogni voce della tabella è chiamata slot di frame (*frame slot*). La tabella elenca gli header da inviare e i ritardi tra di essi (spazio interframe o *interframe space*).
- Il Master invia in modalità broadcast un Header contenente l'ID dei dati da spedire (indirizzamento basato sui dati o *data-based addressing*), e un solo slave risponde inviando il carico utile dei dati (*payload*), che sarebbe la response. Il nodo sorgente dei dati può essere sia il Master sia uno Slave. Il nodo che produce quell'ID invia i dati nella parte di Risposta (**Response**). Qualsiasi nodo interessato può leggere il frame dati trasmesso sul bus (modello *publisher/subscriber*). Il dato aggiornato deve viaggiare sul bus in modo tale che tutti gli slave vengano a conoscenza del nuovo dato aggiornato.
- Un task slave separato, presente all'interno del nodo master, riceve autonomamente tutti i dati pubblicati sul bus e risponde come se fosse un nodo slave indipendente.
- Per trasmettere i byte di dati, il task master deve prima aggiornare la risposta del proprio task slave interno con i valori dei dati che desidera trasmettere. Successivamente, il master pubblica l'header del frame appropriato e il task slave interno trasmette il proprio payload di dati sul bus.

Frame LIN

- **HEADER (Inviato sempre dal Master):**
 - *Break / Sync / Protected Identifier (PID)*
- **RESPONSE (Inviato dallo Slave che possiede i dati):**
 - **Data bytes:** Da 1 a 8 byte di dati utili.
 - **Checksum:** 1 byte (8 bit) calcolato sui dati appena inviati per permettere a chi riceve di verificare che il messaggio non sia stato corrotto durante la trasmissione.

A seconda della versione del protocollo impostata nel file di configurazione (LDF), il calcolo del checksum cambia leggermente:

1. **Classic Checksum (LIN 1.3 e precedenti):** Viene calcolato basandosi **solo** sui byte dei dati.
2. **Enhanced Checksum (LIN 2.0 e successivi):** Viene calcolato includendo sia i byte dei dati sia il campo **PID** (Protected Identifier) dell'header, offrendo una protezione maggiore contro gli errori.

CREAZIONE DELLA FRAME LIN

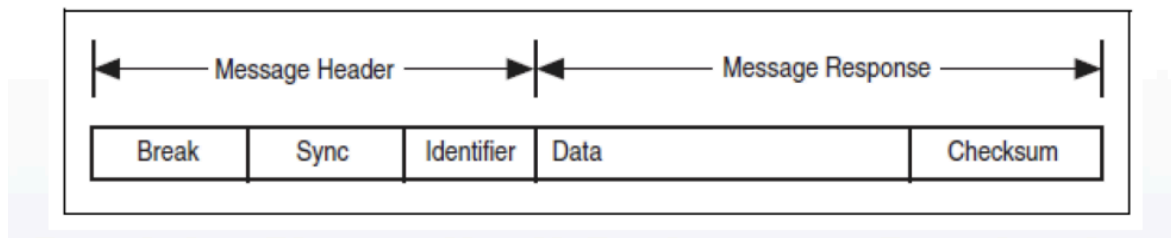
1. **Il Master avvia la comunicazione** Il *Master task* trasmette sul bus l'**Header**.

L'header è composto da tre parti:

- **Break:** Segnale per svegliare/avvisare la rete che sta iniziando un frame.
 - **Sync:** Campo di sincronizzazione per allineare il clock dei nodi.
 - **ID=0:** L'identificatore specifico del dato richiesto.
2. **Lo Slave risponde** Non appena l'header viene ricevuto, lo *Slave task* configurato per produrre i dati associati al suo id si attiva e trasmette sul bus la sua **Response**, composta da:
 - **Data Bytes:** I dati veri e propri.
 - **Checksum:** Il byte di controllo.

Prima di passare all'ID successivo vi è l'**inter-frame delay**, che per ogni ID è specificato nella **tabella di scheduling**. Significa che il Master, seguendo rigorosamente la tabella di marcia (*schedule table*), attende un numero preciso di millisecondi prima di inviare la richiesta successiva, lasciando il bus a riposo.

LIN frame



Nel dettaglio

- **BREAK:** è una sequenza di **13** bit dominanti (0), seguiti da **1** bit recessivo (1), che contrassegna l'inizio di un nuovo frame.
- **SYNC:** Il Master sincronizza gli slave con la velocità del bus inviando appositi caratteri di sincronizzazione (**Sync**, ovvero **01010101**): l'alternanza delle transizioni consente agli slave di sincronizzarsi con il master e di essere pronti alla ricezione.

- **ID**: è l'ultimo campo trasmesso dal task master nell'header. Questo campo fornisce l'identificazione per ciascun messaggio sulla rete e, in ultima analisi, determina quali nodi della rete ricevono o rispondono a ogni trasmissione. L'ID viene trasmesso sul bus sotto forma di un byte di ID protetto (**PID**), in cui:
 - i sei bit inferiori contengono l'ID grezzo (*raw ID*)
 - i due bit superiori contengono i **bit di parità**.

Tutti i task slave rimangono continuamente in ascolto dei campi ID, ne verificano la parità e determinano se sono *publisher* (produttori) o *subscriber* (utilizzatori) per quel particolare identificativo. Il bus LIN fornisce un totale di **64 ID**.

- 0 a 59 sono utilizzati per i frame di trasporto dei segnali (dati),
- 60 e il 61 sono utilizzati per trasportare i dati diagnostici
- 62 è riservato alle estensioni definite dall'utente
- 63 è riservato a futuri miglioramenti del protocollo.

Il rilevamento degli errori (*error detection*) deve essere gestito dai task slave, mentre il monitoraggio degli errori (*error monitoring*) da parte del task master non è richiesto. Al riscontro del primo errore in un frame, il task slave interrompe l'elaborazione del frame fino al rilevamento della successiva sequenza di break-sync (nell'header successivo trasmesso dal master).

Tipologie di Frame LIN

Il Master invia **diversi tipi di frame** a seconda dei differenti utilizzi.

- **Unconditional frame**: quello utilizzato più comunemente. I frame incondizionati trasportano segnali (dati) e i loro identificativi rientrano nell'intervallo da 0 a 59.
- **Event-triggered frame**: tenta di preservare la larghezza di banda del bus richiedendo una risposta di frame incondizionato a più slave all'interno dello slot temporale di un singolo frame.
 - Il frame ET funziona nel modo seguente: il master scrive un ID di attivazione da evento in un header. Gli slave possono rispondere solo se i loro dati sono stati aggiornati. Un frame ET può avere un ID nell'intervallo da 0 a 59. Ciascuno slave che potrebbe potenzialmente rispondere all'ID dell'header ET inserisce nel suo **primo byte di dati il proprio ID protetto** corrispondente (*ovvero l'ID a cui lo slave risponderebbe se il master avesse inviato un frame incondizionato*).
 - Se nessuno dei task slave risponde all'header, la parte restante dello slot di frame rimane silenziosa e l'header viene ignorato.
 - Se un solo slave pubblica una risposta, il master la riceve e, guardando il primo byte di dati, sa da quale slave è stata ricevuta (tramite l'ID protetto).
 - Se più slave pubblicano una risposta, si verifica una **collisione**, che il task slave del dispositivo master segnala come errore del bus. Il dispositivo master interroga quindi ciascuno slave singolarmente utilizzando i frame incondizionati.

Il frame **Event-triggered** è una delle funzionalità più intelligenti del protocollo LIN, introdotta per risolvere un problema strutturale: la **lentezza del bus** (max 20 Kbps). Se il Master dovesse interrogare continuamente (fare il *polling* di) tutti i 16 slave uno per uno tramite frame incondizionati solo per chiedere "*Hai novità? E tu? E tu?*", sprecherebbe quasi tutta la banda disponibile in messaggi vuoti, dato che pulsanti o sensori cambiano stato raramente.

- **Sporadic frames:** tentano di fornire un comportamento dinamico al bus LIN.

L'header di un frame sporadico deve essere inviato solo quando il task master sa che un valore all'interno del frame è stato aggiornato. I dati vengono prodotti dallo slave task del Master.

Nel frame sporadico, il **Master Task** invia l'header solo quando serve, e lo **Slave Task del nodo Master** agisce come *publisher* (produttore) inserendo i dati nella *Response*. Tutti gli altri nodi slave della rete che sono interessati a quel messaggio rimangono in ascolto come *subscriber* (utilizzatori) e si aggiornano di conseguenza.

Così si evita che il Master invii informazioni uguali periodicamente.

Il polling è bello perché evita le collisioni, ma non è efficiente se molti nodi non devono dire niente, il polling si fa solo quando hanno qualcosa da trasmettere.

- **Diagnostic frames:** hanno sempre una lunghezza di otto byte di dati e trasportano sempre dati di diagnostica o di configurazione. I loro ID sono 60 o 61.
- **User-defined frames:** che hanno un ID pari a 62, possono trasportare qualsiasi tipo di informazione. I frame **riservati (reserved frames)** hanno un ID pari a 63 e non devono essere utilizzati in un cluster LIN 2.0.

Vantaggi del LIN

1. Basso Costo (Low cost)
 - Il bus LIN funziona a **12V**, quindi non sono necessari ricetrasmittitori (*transceiver*) speciali per convertire l'alimentazione del veicolo a livelli di tensione differenti.
 - Il LIN può essere implementato su **qualsiasi controllore**, anche su quelli a 8 bit più economici.
2. Complementarità con il CAN
 - Il LIN **completa il CAN**: i cluster LIN sono solitamente interconnessi tra loro tramite una dorsale (*backbone*) CAN.
3. Efficienza Energetica (Energy efficiency)
 - Per ottimizzare il consumo energetico, il LIN offre servizi per mettere i nodi in modalità di sospensione (*sleep mode*) e per risvegliarli prima che vengano inviati nuovi dati sul bus.
 - Questo aspetto è **molto importante** nei sistemi automobilistici, specialmente quando il motore è spento.

- **Comando "go-to-sleep"**: Tutti gli slave possono essere forzati a entrare in modalità sleep dal master tramite l'invio di un frame di richiesta master diagnostico (**ID=60**) con il primo byte di dati uguale a zero. Questo frame speciale è chiamato, appunto, "comando vai in sospensione" (*go-to-sleep command*).
- **Sospensione automatica**: Gli slave entrano automaticamente in modalità sleep anche se il bus LIN rimane inattivo per più di **quattro secondi**.
- **Wakeup**: Il risveglio può essere avviato da un nodo qualsiasi sul bus (sia da uno slave che dal master).
- Specifica LIN 2.0 per il Risveglio: la richiesta di risveglio viene emessa forzando il bus in uno stato **dominante per un tempo da 250 μ s a 5 ms**.
- Ciascun nodo slave deve rilevare la richiesta di risveglio ed essere pronto a elaborare gli header **entro 100 ms**.
- Anche il master deve rilevare la richiesta di risveglio e iniziare a inviare gli header quando i nodi slave sono pronti (ovvero **entro 100 ms - 150 ms** dopo aver ricevuto la richiesta di risveglio).

Limiti del LIN

- **Bassa velocità**: massimo 20 Kbps.
- **Metodo di rilevamento degli errori di base**.
- **Non è fault-tolerant** : non viene utilizzato nelle funzioni safety critical.

In sintesi

Il bus LIN è semplice, economico e particolarmente adatto laddove non vi sia la necessità di velocità elevate. Il LIN continuerà a essere utilizzato nelle auto del futuro.

CAN

Controller Area Network

Storia (History)

- **1983** – Sviluppato da Bosch
- **1986** – Presentazione ufficiale al congresso SAE di Detroit
- **1992** – Primo utilizzo a bordo di autovetture (Mercedes)
- **1994** – Introduzione dello standard ISO
- **1995** – Estensione dello standard ISO
- **2015** – Standard ISO 11898-1:2015 (include il CAN Classico e il CAN-FD)
- **2024** – Standard ISO 11898-1:2024 (viene aggiunto il CAN XL)

Presenta una topologia con **bus multidrop**, formata da un doppino in rame, con protocollo di accesso al mezzo **CSMA/BA**.

Domini funzionali

Can viene utilizzato in diversi domini funzionali con diverse velocità.

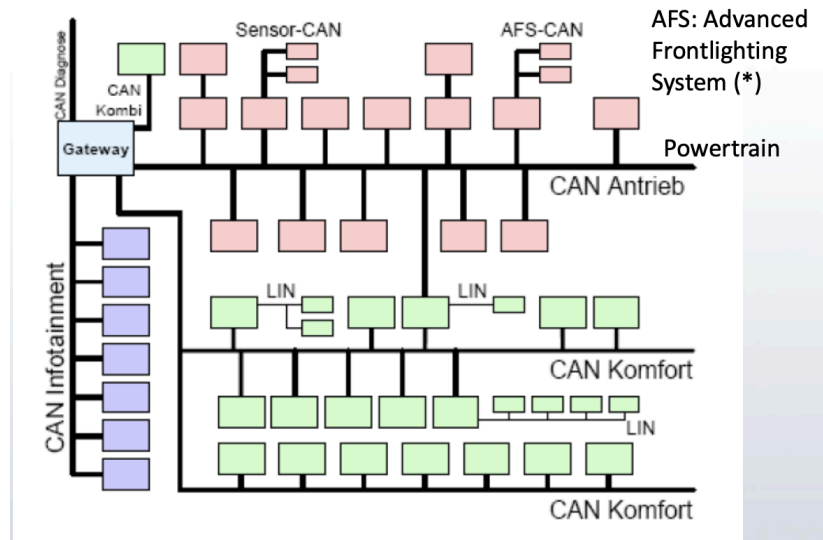
- Telaio, Sicurezza e Gruppo Motopropulsore / *Powertrain* (a 250 o 500 kbps)
- Trasporta i segnali di controllo per:
 - Sistema di avviamento/spegnimento automatico (*auto start/stop*)
 - Sistema antibloccaggio delle ruote (*ABS / antilock brake system*)
 - Moduli di controllo del motore (*ECU / engine control modules*)
 - Controllo delle emissioni
 - Assistenza al parcheggio (*parking assist*)
 - Assistenza al mantenimento della corsia (*lane assist*)
 - Sistemi di prevenzione delle collisioni (*collision avoidance*)
- Carrozzeria (*Body*), Comfort e Sicurezza (a 125 kbps)
 - Luci dei freni
 - Diagnostica
 - Infotainment...
- Guida Automatizzata (CAN ad alta velocità, CAN-FD)
 - Sensori a ultrasuoni
 - Radar
 - LIDAR

il **CAN** viene usato come spina dorsale (*backbone*) per i dati pesanti e importanti (motore, infotainment, nodi principali del comfort), mentre il **LIN** si occupa della periferia per contenere pesi e costi. Elementi principali:

- **Gateway Centrale:** È il "regista" della vettura; traduce e smista i dati tra reti diverse, isolando la presa di diagnostica (**CAN Diagnose**) e il quadro strumenti (**CAN Kombi**).
- **CAN Antrieb (Powertrain - Rosa):** Rete ad **alta velocità** (500kbps) per funzioni critiche (motore, sicurezza). Include sensori dedicati e il sistema **AFS** per i fari autoadattativi.
- **CAN Infotainment (Viola):** Rete separata per la multimedialità (radio, navigazione), isolata per evitare che il traffico pesante rallenti i sistemi vitali dell'auto.

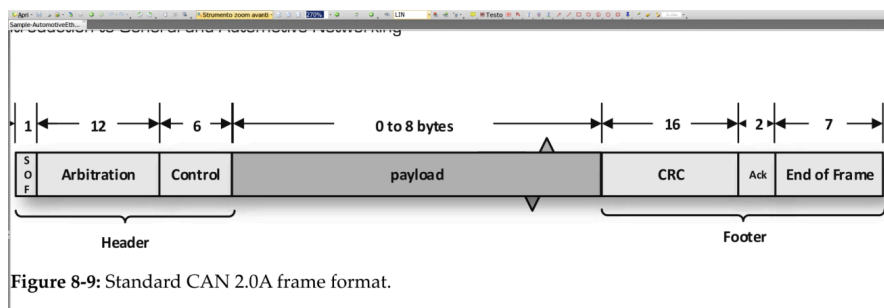
- **CAN Komfort (Comfort - Verde):** Rete a **bassa velocità** (125 kbps) per l'abitacolo. Sotto di essa si diramano le sottoreti **LIN** (quadrati piccoli) per gestire in modo economico i singoli componenti locali (alzacristalli, specchietti, luci).

In sintesi: Il **CAN** fa da spina dorsale (*backbone*) per i dati importanti e veloci, mentre il **LIN** gestisce la periferia per ridurre pesi e costi.



Tecnologia

- **Velocità fino a 1 Mbit/s** (all'interno dell'auto vengono tipicamente utilizzate velocità di 125, 250, 500 Kbps; a velocità inferiori, la presenza di 50 o più nodi è una condizione tipica).
- **Topologia a bus (multidrop):** tutti i nodi sono connessi tra loro attraverso un bus a due fili (ovvero, CAN High e CAN Low, codifica differenziale robusta).
- **Doppino intrecciato** (cablaggio in rame UTP - *Unshielded Twisted Pair*).
- **Bus seriale multimaster – CSMA/BA** (*Carrier Sense Multiple Access with Bitwise Arbitration*).
- **Da 0 a 8 byte** di carico utile dei dati (*payload*).
- **Identificatore (ID) a 11 o 29 bit.**



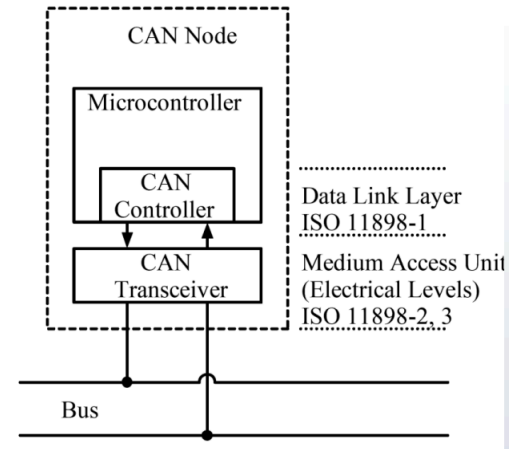
- Lunghezza massima del bus 40 metri.

- Massimo 30 nodi a 1 Mbps. Anche CAN è data based.

Nodi CAN

Ciascun nodo CAN include:

- **Unità di elaborazione centrale (Host processor):** un **microcontrollore** dotato di periferiche CAN integrate (*on-chip*) che decide il significato dei messaggi ricevuti e quali messaggi intende trasmettere.
- **Controllore CAN (CAN controller):** spesso parte integrante del microcontrollore stesso.
 - **RX (Ricezione):** il controllore CAN memorizza i bit seriali ricevuti dal bus fino a quando non è disponibile un intero messaggio, che può quindi essere prelevato dall'host processor (solitamente quando il controllore CAN attiva un interrupt).
 - **TX (Trasmissione):** l'host processor invia il messaggio da trasmettere al controllore CAN, il quale trasmette i bit in modo seriale sul bus quando quest'ultimo è libero.
- **Ricetrasmittitore (Transceiver):**
 - **RX (Ricezione):** converte i livelli di tensione del bus CAN nel flusso di dati utilizzato dal controllore CAN.
 - **TX (Trasmissione):** converte il flusso di dati proveniente dal controllore CAN nei livelli di tensione specifici del bus CAN.



Codifica differenziale

Il ricetrasmittitore chiamato **ricetrasmittitore CAN HS (High Speed - Alta Velocità)** fornisce una tensione differenziale sulle linee del bus **CAN_H** (High) e **CAN_L** (Low).

- Per un **bit recessivo** (ovvero 1), entrambe le linee sono portate a un valore nominale di **2,5 V**.
- In caso di **stato dominante** (ovvero 0), la linea **CAN_H** misura nominalmente **3,5 V** e la linea **CAN_L** **1,5 V**.

Sono previsti degli intervalli di tolleranza (ad esempio, tra 0,9 V e 2,0 V per lo stato dominante, mentre qualsiasi valore inferiore a 0,5 V è considerato recessivo). La codifica differenziale è più robusta perchè la differenza di tensione rimane costante anche se le tensioni variano (allo stesso modo).

Ampliamente utilizzato, poiché supporta trasmissioni sia periodiche sia aperiodiche.

Un frame CAN è contrassegnato da un ID, trasmesso all'interno del frame stesso, che identifica sia la priorità del frame sia il suo contenuto (*payload*).

- I file di descrizione forniscono il significato degli ID dei frame: ad esempio, un frame CAN con ID **0x321** ha un payload che contiene i giri al minuto del motore (RPM), la posizione del pedale e quella dell'acceleratore.

Arbitraggio non distruttivo basato sulla priorità.

Si comporta come una coda basata sulla priorità (*priority-based queue*).

Il **campo di arbitraggio** (12 bit) è composto dall'**ID** (11 bit) più il bit **RTR** (*Remote Transmission Request*), il quale distingue tra un frame dati (**RTR=0**) e un frame di richiesta dati (**RTR=1**).

Sono disponibili **diverse politiche di pianificazione** (*scheduling*).

Prima del 1994, nelle applicazioni automobilistiche erano tipici bassi livelli di utilizzo del bus, fino al 30% o 40%.

Applicando algoritmi di pianificazione a priorità fissa non-preemptive (in cui gli ID riflettono i vincoli temporali) e le relative tecniche di analisi in tempo reale (Tindell et al. 1994, revisione di Davis et al., 2007), si possono raggiungere **utilizzi fino al 70-80%**, garantendo il rispetto dei vincoli in tempo reale (*real-time guarantees*) per tutti i messaggi.

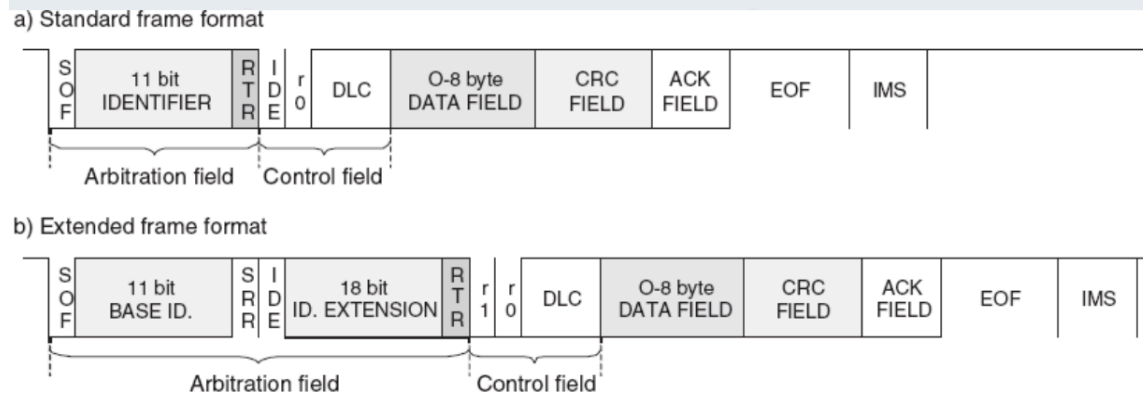
Il CAN è **multimaster**: i nodi sono paritetici. Ma l'approccio è producer-consumer: la richiesta fatta dal subscriber finisce con il recessivo, così se incontra la risposta del publisher vale meno.

Frame CAN

La specifica CAN definisce sia il formato di frame **standard** sia quello **esteso**.

Essi differiscono per la dimensione del campo ID e per altri bit all'interno del campo di arbitraggio (ad esempio, il bit **IDE**, un bit riservato che è pari a **0** nel formato base e a **1** nel formato esteso).

- **Il formato di frame standard (formato base o CAN 2.0A)**: definisce un campo identificatore a **11 bit**, il che significa che sono disponibili fino a 2048 identificatori diversi all'interno della stessa rete (di cui solo 2031 sono effettivamente utilizzabili).
- **Il formato di frame esteso (CAN 2.0B)**: assegna invece **29 bit** all'identificatore, rendendo disponibili fino a mezzo miliardo di ID differenti all'interno della stessa rete.
- Le reti interne ai veicoli (*in-car networks*) utilizzano il formato **CAN 2.0A**.



- **SOF (Start Of Frame)**: indica l'inizio del messaggio con un bit logico dominante (0).
- **Arbitration ID**: identifica il messaggio e indica la priorità del messaggio. Le frame hanno due formati : quello standard con 11 bit di arbitration ID e quello esteso con 29 bit di arbitration ID.
- **IDE (identification extension) bit**: per capire se la frame è standard o estesa.
- **RTR (Remote Transmission Request) bit**: serve a differenziare le frame dati in frame di richiesta dati. Un bit logico dominante **0** indica che la frame è **dati**, un bit logico recessivo 1 che la frame è di richiesta dati. Importante in arbitration.
- **DLC (Data Length Code)**: indica il numero di byte che contiene il data field.
- **Data Field** : contiene da 0 a 8 byte di dati.
- **CRC (Cyclic Redundancy Check)**: contiene 15 bit che consistono nella divisione tra il messaggio da inviare e il polinomio generatore. In ricezione la divisione tra il messaggio + CRC con il polinomio generatore deve fare resto zero, altrimenti ci sono stati degli errori.
- **ACK (Acknowledgment) slot + ACK delimiter**: tutti i controller CAN che ricevono correttamente un messaggio, mentre la trasmissione è ancora in atto, mandano un Ack bit alla fine del messaggio. Il nodo trasmettitore cerca la presenza di un ack bit nel bus e riprova a trasmettere se non trova nessun ack. Quindi se il nodo trasmettitore sente un bit dominante capisce che qualcuno (non si sa chi) ha recepito correttamente la sua frame. Se nessun nodo risponde (ad esempio perché il cavo è interrotto o nessuno ha capito il messaggio), il bus rimane al valore recessivo **1**. Il mittente rilegge **1**, capisce che nessuno lo ha sentito (errore di **No ACK**), genera un *Error Flag* e tenta immediatamente di ritrasmettere il frame.

BIT 0 = Positive Acknowledgment

- **CAN Signal**: pezzo di dati contenuto all'interno del data frame CAN.
- **EOF (end of frame)**: 7 bit recessivi.
- **IMS (intermission)**: 3 bit recessivi. I frame dati (*data frames*) e i frame di richiesta dati (*remote frames*) sono separati dai frame precedenti da uno **spazio interframe** (*interframe space*), il quale è composto da almeno tre bit recessivi chiamati

Intermission (intervallo). Questo consente ai nodi di avere il tempo necessario per l'elaborazione interna prima dell'inizio del frame di messaggio successivo. Dopo l'intermission, se viene rilevato un bit dominante, questo verrà considerato come il bit di "Inizio del frame" (*Start of frame*) del frame successivo; in caso contrario, la linea del bus rimane nello stato recessivo (bus inattivo o *bus idle*) fino all'inizio della trasmissione successiva.

I NODI CAN POSSONO SCRIVERE SUL BUS IN PIÙ MOMENTI: NEL MOMENTO DELL'ARBITRATION E NEL MOMENTO DELL'ACK.

CSMA/BA

La tecnica *bitwise arbitration* sfrutta le peculiarità del livello fisico del CAN, il quale fornisce concettualmente uno schema di connessione di tipo *wired-AND* tra tutti i nodi.

Il livello elettrico sul bus è:

- **dominante** se almeno un nodo sta inviando un bit dominante (0), oppure
- **recessivo** se tutti i nodi stanno trasmettendo bit recessivi (1).

La risoluzione delle collisioni è non distruttiva:

- Lo schema di arbitraggio del CAN risolve le collisioni interrompendo le trasmissioni di tutti i frame coinvolti, **eccetto quello a priorità più elevata** (ovvero, quello con l'identificatore numerico più basso).

Ciascun nodo:

- **Può iniziare la trasmissione quando il bus è inattivo (*idle*):** – Il bit *Start-Of-Frame*, di tipo dominante, può essere inviato solo dopo almeno **11 bit recessivi consecutivi** sul bus. Il SOF costituisce una sincronizzazione hardware del clock (*hard clock synchronization*) di tutti i nodi. Se più nodi vedono il bus libero e provano a inviare il SOF contemporaneamente, ciascun nodo si sincronizzerà con il primo SOF rilevato sul bus.
- **Dopo il SOF, trasmette l'identificatore del messaggio** in modo seriale sul bus, partendo dal bit più significativo (*MSB - Most Significant Bit*).
- **Durante la trasmissione, confronta il livello presente sul bus** con il valore del bit che sta inviando in quel momento.
- **Se sta trasmettendo un valore recessivo, ma il livello sul bus è dominante**, comprende di aver perso la contesa e si ritira immediatamente.

Il nodo che perde la contesa:

- **Interrompe la trasmissione.**
- **Imposta la propria porta di uscita sul livello recessivo**, in modo da non interferire con la comunicazione.
- **Passa allo stato di ricezione** per leggere il frame in arrivo (quello vincente).

Tutti i nodi che perdono la contesa riproveranno a trasmettere non appena terminerà la trasmissione del frame corrente. Praticamente non si spreca banda perché il bus può essere occupato da chi deve trasmettere e passa quello a priorità più alta, evitando collisioni.

Nel traffico real-time

Il bus CAN supporta egregiamente le **comunicazioni event-triggered**.

Poiché nel CAN tutti i nodi sono master (dal punto di vista del meccanismo MAC – *Medium Access Control*, il CAN è un **protocollo multimaster**), per essi è molto semplice segnalare eventi asincroni come, ad esempio, allarmi o condizioni di errore critiche.

Grazie allo schema di arbitraggio, nessun messaggio subirà ritardi a causa di un messaggio concorrente a priorità inferiore. Tuttavia, poiché il CAN ***non è preemptive***, un messaggio può comunque essere ritardato da uno a priorità inferiore la cui trasmissione sia però già iniziata (fenomeno del *blocking time*).

Ciononostante, dato che la dimensione dei frame nel CAN è *molto ridotta* (i frame standard sono lunghi al massimo 135 bit), il tempo di blocco subito dai messaggi estremamente urgenti è in generale piuttosto basso. Questo rende il CAN una rete **estremamente reattiva** (*very responsive*), il che spiega perché venga utilizzato in molte applicazioni di controllo in tempo reale nonostante la sua larghezza di banda sia relativamente bassa. Inoltre, CAN è molto flessibile, è facile aggiungere nuovi nodi, grazie al CSMA, a differenza del TDMA. Comunque i payload piccoli potrebbero essere un suo limite.

Distanze massime: Nella fase di **arbitration** e **ack** i nodi devono essere sincronizzati sullo stesso bit, quindi la velocità di propagazione deve poter far andare e tornare il bit. Di conseguenza l'intera rete deve essere abbastanza corta da permettere al bit di fare l'intero viaggio di andata e ritorno entro la finestra di campionamento del bit stesso.

Quando si assegnano gli identificatori ai messaggi, ai messaggi più urgenti (ovvero quelli caratterizzati dalle scadenze più stringenti - *tightest deadlines*) dovrebbero essere assegnati gli **identificatori più bassi**, in modo da garantire loro le **priorità più elevate**.

Le tecniche basate su **offset** (ritardi iniziali), desincronizzando i flussi dei frame, possono ottimizzare il carico del bus e le prestazioni.

Se il periodo degli scambi di dati periodici (e il tempo minimo di interarrivo di quelli aperiodici) è noto in anticipo, si possono utilizzare gli algoritmi **Rate Monotonic** o **Deadline Monotonic** per trovare (se esiste) un'assegnazione fattibile degli identificatori ai messaggi, in modo che la pianificazione (*schedule*) risultante sia sostenibile (ovvero, che le scadenze di tutti i messaggi siano rispettate).

Sono disponibili sul mercato molti strumenti per l'ottimizzazione e la verifica che utilizzano la simulazione, l'analisi di pianificabilità (*schedulability analysis*) e l'analisi delle tracce (*trace analysis*).

Altre caratteristiche positive del CAN:

- Meccanismi di **rilevamento e ripristino dell'errore** (*error detection and recovery*).
- Meccanismi di **isolamento dei guasti** (*fault-confinement*) per identificare i guasti permanenti.

Tuttavia, per le applicazioni critiche per la sicurezza (*safety-critical*) sono necessari **ulteriori meccanismi di tolleranza ai guasti** per far fronte a:

- Il guasto del **"babbling idiot"** (l'idiota farfugliante), che porta un nodo guasto a trasmettere continuamente un bit dominante, bloccando l'intera rete.
- Nodi che inviano frame al di fuori delle proprie specifiche (in termini di periodo o lunghezza del messaggio).

Vantaggi

1. **Standard aperto ISO 11898:** Le sue specifiche tecniche sono pubbliche, accessibili a tutti e regolamentate da un ente internazionale super partes, l'ISO. Presenta un ampio bacino di fornitori e oltre 30 anni di esperienza nelle applicazioni CAN in ambito automobilistico.
2. **Economico** : essendoci decine di produttori di semiconduttori in concorrenza tra loro, i prezzi dei componenti sono crollati, a tutto vantaggio dei costruttori di auto.
3. **Ampiamente utilizzato:** miliardi di nodi CAN installati in milioni di auto, con molte varianti (*flavors*) di CAN disponibili a diverse velocità.
4. **Il CAN è flessibile:** non richiede dispositivi di interconnessione (come gli switch), rappresentando il miglior esempio di "Plug and Play" nelle reti automobilistiche. Consente l'aggiunta o la rimozione semplificata dei nodi senza la necessità di modificare gli altri nodi, il software o altri parametri di rete.
 - *Al contrario, nelle reti Switched Ethernet con microsegmentazione, è necessaria una porta dello switch per ciascun dispositivo; di conseguenza, l'espandibilità della rete richiede di lasciare delle porte libere (non utilizzate) sugli switch.*
5. **Set di strumenti (toolset) disponibili sul mercato** per la configurazione, la verifica, la simulazione e l'analisi delle reti CAN.

Limiti del CAN

1. **Prestazioni:** La velocità massima di 1 Mbps è troppo bassa per le moderne applicazioni automobilistiche.
2. **Payload ridotto (max 8 byte):** Elevato *overhead* (a seconda del carico utile trasportato, potrebbero esserci più bit nell'intestazione e nella coda del frame rispetto a quelli presenti nel campo dati).
3. **Attivazione da evento (Event-Triggered):** Presenza di *jitter* (instabilità temporale) nella trasmissione.

Soluzioni (Solutions)

1. **CAN FD:** Sviluppato inizialmente come "CAN FD Bosch", poi standardizzato come "ISO CAN FD". Migliora le prestazioni e consente velocità più elevate e payload più lunghi, mantenendo la compatibilità con il CAN classico. Risolve i limiti L1 e L2.
2. **TT-CAN (Time-Triggered CAN):** Costruisce una struttura attivata dal tempo (*time-triggered*) al di sopra dei frame CAN classici. Risolve il limite L3.
3. **CAN XL:** Il processo è iniziato alla fine del 2018; il CAN XL è ora specificato dal CiA 610-1 (*CAN in Automation*) ed è attualmente standardizzato come parte dell'ISO 11898-1. CAN XL è un livello di collegamento dati (Data Link Layer) che fornisce payload "extra-large" fino a **2048 byte** e una velocità di trasmissione dati fino a **20 Mbps**.
 - **Casi d'uso:** Veicoli elettrici, camion e autobus, bus CAN più sicuri (security), sistemi ADAS, riprogrammazione delle ECU (*flashing*), ecc.

(*) *Nota: Prima dell'aggiornamento dello standard ISO 11898-1:2015, lo standard CAN FD presentava una debolezza legata al controllo degli errori (CRC). I controllori conformi allo standard aggiornato vengono talvolta denominati "ISO CAN FD".*

Velocità di trasmissione e lunghezza del bus

- La procedura di arbitraggio del CAN si basa sul fatto che un nodo trasmittente monitora (rilegge) il bus mentre sta trasmettendo.
- Il segnale deve essere in grado di propagarsi fino al nodo più remoto e tornare indietro prima che venga stabilito il valore del bit.
- Ciò richiede che il tempo di bit (ovvero il tempo che intercorre tra due bit consecutivi dello stesso frame) sia **lungo almeno il doppio rispetto al ritardo di propagazione**, il che limita la velocità dei dati: al massimo **1 Mbit/s su un bus lungo 40 metri**.

Lunghezza del bus (metri)	Max bit rate (bit/sec)
40	1 Mbit/s
100	500 Kbit/s
200	250 Kbit/sec
500	125 Kbit/s
6K	10 Kbit/s

Usi del CAN

Le applicazioni che utilizzano il CAN si trovano in questi domini:

- **Automotive** (Automobilistico).
- **Ferroviario:** ad esempio per il collegamento delle unità delle porte o dei controllori dei freni.
- **Aviazione:** ad esempio nei sensori di volo e nei sistemi di navigazione.

- **Aerospaziale:** ad esempio nell'analisi dei dati di volo e nei sistemi di controllo dei motori degli aeromobili.
- **Apparecchiature mediche:** i sistemi basati su CAN vengono utilizzati negli ospedali per controllare i componenti delle sale operatorie, ovvero luci, letti chirurgici, telecamere, macchine a raggi X, letti dei pazienti...
- **Ascensori e scale mobili:** utilizzano reti CAN collegate a sistemi integrati (*embedded*).
- **Manifatturiero:** linee di produzione, macchine da stampa.
- **Robotica.**

CAN LAYERS

Le specifiche CAN includono solo il **livello fisico** (*physical layer*) e il *data link layer*, non l'*application level* perché ci possono essere livelli diversi.

- Il **livello fisico** gestisce la trasmissione dei dati sul supporto di comunicazione e affronta gli aspetti meccanici, elettrici e di sincronizzazione appartenenti a questo livello. Viene implementato nei transeiver.
- Il **livello di collegamento dati** è suddiviso in due sottolivelli separati:
 - Il **Medium Access Control (MAC)** (Controllo dell'Accesso al Supporto).
 - Il **Logical Link Control (LLC)** (Controllo del Collegamento Logico).

Il livello **MAC** gestisce l'accesso al mezzo condiviso fornendo un meccanismo per coordinare l'uso del bus. Funzioni del sottolivello MAC sono:

- Codifica e decodifica dei frame (*frame encoding and decoding*).
- Arbitraggio (*arbitration*).
- Controllo e segnalazione degli errori (*error checking and signaling*).
- Isolamento dei guasti (*fault confinement*).

Il **sottolivello LLC** offre all'utente (i programmi applicativi in esecuzione nei livelli superiori) **un'interfaccia adeguata**, dotata di un insieme ben definito di servizi di comunicazione e capacità di decidere se un messaggio in arrivo sia rilevante o meno per il nodo (**filtraggio dei messaggi**), dato che i messaggi sono in broadcast, in base all'identificatore. Offre set di primitive (funzioni di interfaccia) e serve a implementare scambi di *data object* sotto l'egida del paradigma user consumer.

Sincronizzazione

Al livello fisico (PHY), il CAN si basa sulla codifica dei bit di tipo **Non-Return-to-Zero (NRZ)**.

Tutti i nodi sulla rete CAN devono operare alla stessa velocità di bit nominale (*nominal bit rate*). Poiché non viene utilizzato un segnale di clock separato, è necessario un sistema per sincronizzare i nodi tra loro.

La sincronizzazione inizia con la transizione del bit di Inizio Frame (**SOF - Start of Frame**) dopo un periodo di inattività del bus (*bus idle*).

La **risincronizzazione** avviene su ogni transizione da recessivo a dominante durante il frame.

I controllori CAN si aspettano che tale transizione avvenga in corrispondenza di un multiplo del tempo di bit nominale (nominal bit time). Quindi se non ci sono sincronizzazioni frequentemente si rischia di perdere la sincronizzazione.

Bit Stuffing

In arbitration e ack c'è bisogno di sincronizzazione, cioè che tutti i nodi sentano lo stesso bit. Poiché il CAN adotta la codifica NRZ, senza una linea di clock dedicata, l'inserimento di un bit di riempimento opposto dopo 5 bit consecutivi assicura i fronti necessari alla risincronizzazione dei nodi.

Per mantenere la sincronizzazione, nel flusso di bit trasmesso deve esserci un numero sufficiente di transizioni ("fronti" del segnale). A tale scopo, i controllori CAN applicano la cosiddetta tecnica del **bit stuffing** (inserimento di bit).

- In pratica, ogni volta che nel flusso di bit trasmesso compaiono **cinque bit consecutivi dello stesso valore** (siano essi dominanti o recessivi), il nodo trasmittente inserisce un bit aggiuntivo di riempimento (*stuff bit*) di **valore complementare**.
- **Esempio:** La sequenza

11111000011110000... viene modificata con il bit stuffing nel seguente modo
111110000011111000001...

Questi bit di riempimento (*stuff bits*) possono essere rimossi in modo semplice e sicuro dai nodi riceventi, così da riottenere il flusso di bit originale.

A causa del bit stuffing, i frame dati possono risultare più lunghi di quanto ci si aspetterebbe enumerando semplicemente i bit teorici.

Le simulazioni mostrano che, in media, vengono aggiunti solo da 2 a 4 bit di riempimento a ciascun frame, a seconda della dimensione dei campi dell'identificatore e dei dati.

Non tutti i campi di un frame CAN sono codificati secondo il meccanismo del bit stuffing: in particolare, il bit stuffing si applica **solo alle parti del frame che vanno dal bit di SOF fino alla sequenza di CRC** (inclusa). I restanti campi, invece, presentano una forma fissa e non sono soggetti a bit stuffing.

I campi successivi (CRC Delimiter, ACK Slot, ACK Delimiter ed EOF) **non subiscono mai il bit stuffing**, perché nella frame CAN ci sono dei bit che devono essere sempre dominanti o recessivi e i nodi si aspettano di trovare quei bit esattamente al loro valore fisso.

Se un nodo non legge un bit recessivo nel CRC Delimiter o nell'EOF, capisce che c'è stato un gravissimo errore di forma (Form Error) e distrugge il frame sul bus.

Valori Fissi DOMINANTI (0)

I bit dominanti fissi servono per dare comandi hardware immediati a tutta la rete (es. sincronizzare i clock o dichiarare il tipo di frame) e sono:

- **SOF (Start of Frame):** 1 bit. È il primissimo bit del frame. Rompe il silenzio del bus (*bus idle*) passando da recessivo a dominante e costringe tutti i nodi a sincronizzare istantaneamente i propri clock hardware sul suo fronte di discesa.
- **r0 (Reserved bit):** 1 bit. Nel campo di controllo (subito prima del DLC, il Data Length Code) c'è un bit di riserva per sviluppi futuri dello standard. Deve essere inviato **sempre come dominante**.

Valori Fissi RECESSIVI (1)

I bit recessivi fissi servono principalmente come "spazio di manovra" elettrico per separare i campi, permettere la risincronizzazione dei clock e dare tempo ai nodi di elaborare i dati.

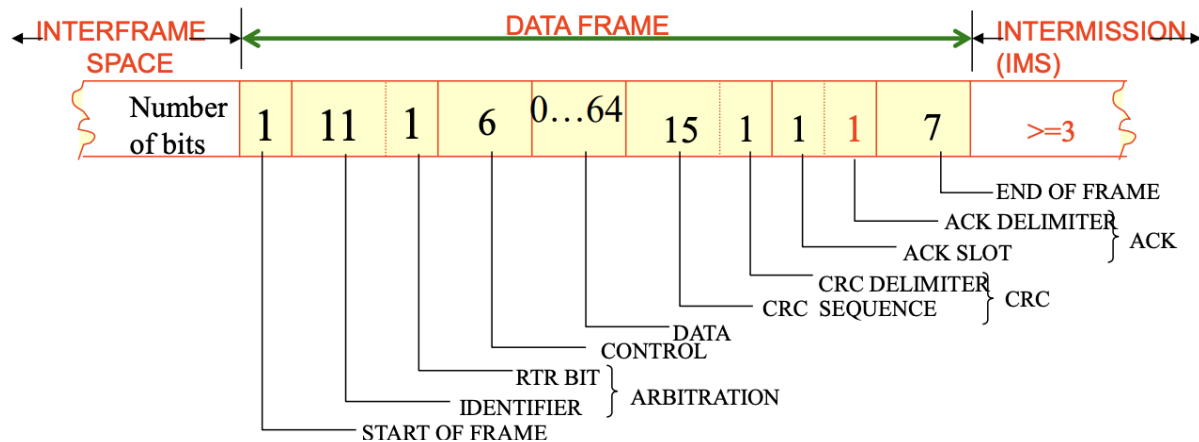
- **IDE (Identifier Extension) [nel formato standard]:** È un bit singolo che si trova subito dopo il campo dell'identificatore. Nel formato standard (CAN 2.0A) è **sempre dominante (0)**, ma diventa fisso **recessivo (1)** nel formato esteso (CAN 2.0B) per segnalare che seguono altri 18 bit di ID.
- **CRC Delimiter:** 1 bit. Si trova subito dopo i 15 bit del CRC. Serve a separare il blocco del CRC dal successivo campo di ACK.
- **ACK Delimiter:** 1 bit. Si trova subito dopo l'ACK Slot. Serve a delimitare la fine della fase di conferma prima che inizi la chiusura del frame.
- **EOF (End of Frame):** 7 bit. È una sequenza fissa di 7 bit recessivi che decreta la fine ufficiale e regolare del messaggio.
- **Intermission (all'interno dell'Interframe Space):** Almeno 3 bit. Come hai visto prima, separa un frame dal successivo e permette ai nodi l'elaborazione interna.

Frame Dati

Il campo dati (*data field*) viene utilizzato per memorizzare il carico utile effettivo (*payload*) del frame, limitato al massimo a **8 byte**.

Dopo il campo dati è presente il **CRC**, codificato su **15 bit**, seguito da un delimitatore del CRC (*CRC delimiter*) impostato al valore recessivo. Successivamente troviamo:

- Il **campo di conferma (ACK)**, composto da due bit: l'**ACK slot** e l'**ACK delimiter**, entrambi inviati a livello recessivo da parte del trasmettitore.
 - L'ACK slot viene **sovrascritto con un valore dominante** da ciascun nodo che ha ricevuto il frame correttamente (ovvero, se non è stato rilevato alcun errore fino al campo ACK).
 - Tramite il bit di ACK, il nodo trasmittente è in grado di scoprire se almeno un nodo nella rete ha ricevuto correttamente il suo frame.
- Il campo di **fine frame (EOF - End of Frame)**, composto da **7 bit recessivi**, che notifica ai nodi la fine di una trasmissione priva di errori.



Per poter ritrasmettere i nodi devono aspettare:

- 1 bit di Ack delimiter + 7 bit di end of frame + 3 bit di intermission = 11 bit totali.

Frame di richiesta dati

I **frame remoti** sono molto simili ai frame dati (*data frames*). L'unica differenza è che **non trasportano dati** (ovvero, il campo dati non è presente).

I frame remoti vengono utilizzati per **richiedere che un determinato frame venga inviato** sulla rete da parte di un nodo remoto.

Il nodo richiedente non sa chi sia il produttore (*producer*) dell'informazione associata. Spetta ai nodi riceventi scoprire chi deve rispondere.

Il campo **DLC** (*Data Length Code*) nei frame remoti è impostato sullo **stesso valore** del corrispondente frame dati.

A causa del modo in cui è codificato il bit **RTR** (**RTR=0** per i dati, **RTR=1** per la richiesta), se viene effettuata la richiesta di un oggetto nello stesso identico istante in cui la trasmissione di quell'oggetto viene avviata dal relativo produttore, la contesa viene risolta **a favore del frame dati**.

Error Frame

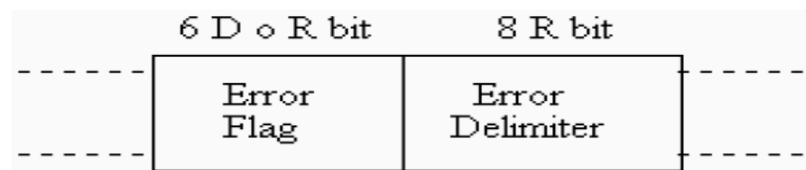
La **gestione dell'errore** (*error handling*) è integrata nel protocollo CAN per fornire robustezza attraverso la capacità di rilevare gli errori di trasmissione direttamente al livello MAC.

Ogni controllore CAN lungo il bus tenta di rilevare eventuali errori all'interno di un messaggio. Se un controllore CAN rileva un errore in un messaggio, trasmette un messaggio speciale, noto come **"Error Frame" (Frame di Errore)**, che distrugge il traffico sul bus.

L'Error Frame inizia con un **Error Flag** che viola le regole di formattazione di un messaggio CAN (ovvero, viola il *bit stuffing* o il formato fisso dei bit per i delimitatori di EOF, CRC e ACK) e costringe anche tutti gli altri nodi collegati alla rete a inviare a loro volta un frame di errore.

La violazione intenzionale dello standard CAN consiste nell'invio di **6 bit uguali consecutivi** e garantisce la distruzione del frame dati o remoto difettoso, consentendo così al trasmettitore originale di **ritrasmettere automaticamente il messaggio**.

I frame di errore sono composti da due campi: l'**Error Flag** e l'**Error Delimiter**.



Error frame format

Ciascun nodo può trovarsi in 3 stati: Error Active, Error Passive e Bus-Off.

Solo i nodi in stato *Error Active* ed *Error Passive* possono inviare Frame di Errore, e si distinguono due tipi di Error Flag, rispettivamente per le due tipologie di nodo:

- **Active Error Flag:** composto da **sei bit dominanti**.
- **Passive Error Flag:** composto da **sei bit recessivi**.

Un **Active Error Flag** impone una condizione di errore rilevata a tutti gli altri nodi, poiché i 6 bit dominanti – violando le regole del *bit stuffing* o le parti a formato fisso del frame – verranno ascoltati da tutti i nodi riceventi. Ciascun nodo che rileva la condizione di errore trasmetterà a sua volta un proprio Error Flag sul bus. Il **Passive Error Flag** è invece costituito da 6 bit recessivi ed è trasmesso da un nodo Error Passive, che non interferisce con la comunicazione degli altri nodi.

L'**Error Delimiter** è composto da **otto bit recessivi**.

Dopo la trasmissione di un error flag, ciascun nodo inizia a inviare bit recessivi e, allo stesso tempo, continua a monitorare il livello del bus finché non viene rilevato un bit recessivo. A questo punto, il nodo invia altri sette bit recessivi, completando così il delimitatore di errore.

Una volta che viene mandato l'error delimiter, il bus si calma e il nodo riprova a trasmettere.

Tipi di errore

Le specifiche CAN prevedono cinque diversi errori di trasmissione:

1. **Errore di CRC (Cyclic Redundancy Check - Controllo di Ridondanza Ciclico):** Durante la trasmissione di un frame, il nodo mittente aggiunge un codice CRC ampio 15 bit alla fine del frame stesso. I nodi riceventi ricalcolano il CRC per verificare se corrisponda o meno a quello trasmesso. In caso di mancata corrispondenza (*mismatch*), il livello MAC attende il delimitatore del CRC (*CRC Delimiter*) e successivamente avvia la trasmissione di un **Error Frame**.
2. **Errore di Forma (Form error):** I campi a formato fisso all'interno dei frame ricevuti possono essere facilmente verificati rispetto al loro valore atteso. Ad esempio, sia i delimitatori del CRC e dell'ACK, sia il campo di EOF devono trovarsi obbligatoriamente a livello recessivo. Se vengono rilevati uno o più bit non validi, viene generato un errore di forma (e un **Error Frame** viene inviato sul bit immediatamente successivo).
3. **Errore di Riconoscimento (Acknowledgement error):** Il nodo trasmittente controlla se il bit di ACK è stato sovrascritto al valore dominante da parte di un ricevitore. In caso contrario, viene emesso un errore di riconoscimento (un **Error Frame** viene inviato sul bit immediatamente successivo).
4. **Errore di Bit (Bit error):** Ciascun nodo trasmittente confronta il livello presente sul bus con il valore del bit che sta scrivendo in quel momento (ovvero, rilegge il livello del segnale trasmesso). Se il livello di bit effettivamente letto differisce da quello trasmesso, viene segnalato un errore di bit (un **Error Frame** viene inviato sul bit immediatamente successivo).
 - a. *Questo controllo non si applica al campo di Arbitraggio e allo slot di Riconoscimento (ACK slot).*
5. **Errore di Bit Stuffing (Bit stuffing error):** Ciascun nodo verifica se le regole del bit stuffing siano state violate nella porzione del frame che va dal bit di SOF fino alla sequenza di CRC. Nel caso in cui vengano letti dal bus **6 bit consecutivi dello stesso valore**, viene generato un errore (un **Error Frame** viene inviato sul bit immediatamente successivo).

Sul bus CAN non vengono trasmessi i dettagli specifici dell'errore, ma gli errori vengono registrati (*logged*) all'interno dei controllori CAN per consentire agli strumenti di diagnostica di leggerli (esistono formati standard a tale scopo). Esistono dei formati standard per i codici di errore. Dove trovo l'informazione dell'errore? La trovo nel **controller**, con opportuni tool posso leggerla e intervenire.

Il meccanismo

Non appena un qualsiasi nodo (mittente o ricevente che sia) rileva uno di questi 5 errori, fa scattare la **clausola di autodistruzione**:

1. **Urla sul bus:** Invia l'**Active Error Flag** (6 bit dominanti consecutivi).
2. **Effetto domino:** Questo viola intenzionalmente la regola del *bit stuffing*. Di conseguenza, tutti gli altri nodi rilevano a loro volta un *Bit Stuffing Error* e iniziano a urlare anche loro inviando bit dominanti.
3. **Piazza pulita:** Il bus viene totalmente saturato da questo livello dominante. Il frame corrotto viene "sovrascritto" e distrutto, impedendo a chiunque di prenderlo per buono.

4. **Reset:** Finita la sfuriata di bit dominanti (e dopo l'Error Delimiter di 8 bit recessivi), la rete torna silenziosa. Il mittente originale sa che il messaggio è fallito e lo **ritrasmette automaticamente** da capo.

Fault Confinement

Il protocollo CAN implementa un meccanismo che distingue tra disturbi temporanei e guasti permanenti dei nodi, al fine di evitare che i nodi difettosi blocchino il traffico di rete.

Ciascun nodo mantiene due contatori che tracciano il comportamento del nodo rispetto agli errori:

- Un contatore degli errori di trasmissione (**TEC** - *Transmit Error Counter*),
- Un contatore degli errori di ricezione (**REC** - *Receive Error Counter*)

In caso di errore di trasmissione o di ricezione, i contatori vengono incrementati. A seguito di una trasmissione o ricezione corretta di un messaggio, il rispettivo contatore degli errori viene decrementato (se non si trovava già a zero).

La manipolazione dei contatori degli errori è **asimmetrica**. Le regole per l'incremento e il decremento dei contatori degli errori sono le seguenti:

- Gli errori di trasmissione comportano l'addebito di **8 punti di errore**.
- Gli errori di ricezione comportano l'addebito di **1 punto di errore**.
- I messaggi trasmessi e/o ricevuti correttamente causano il decremento del relativo contatore (o dei contatori) di **un solo punto**.

Attraverso l'uso di questi contatori di errore, un nodo CAN esegue l'**isolamento dei guasti**.

Un nodo può trovarsi in uno di questi 3 stati, che rappresentano i tre livelli della politica di **Fault Confinement (Isolamento dei Guasti)** del CAN. Il protocollo CAN è stato progettato con un principio fondamentale: **evitare che un nodo difettoso o guasto possa bloccare l'intera rete**. Per farlo, ogni nodo monitora la propria affidabilità attraverso due contatori hardware interni:

- **TEC (Transmit Error Counter):** aumenta quando il nodo sbaglia a *trasmettere*.
- **REC (Receive Error Counter):** aumenta quando il nodo rileva errori mentre *ricevere*.

A seconda del valore di questi contatori, il nodo si sposta tra i tre stati, diventando progressivamente meno "aggressivo" sul bus per non disturbare gli altri.

Error Active (Stato Normale / Sano)

È lo stato iniziale in cui si trovano tutti i nodi all'accensione. Il nodo funziona perfettamente o ha avuto pochissimi errori.

- **Condizione:** $TEC / REC \leq 127$.

- **Comportamento in caso di errore:** Se il nodo rileva un errore, si comporta in modo "aggressivo" (ma corretto per lo standard): invia un **Active Error Flag** composto da **6 bit dominanti**. In questo modo sovrascrive il bus, distrugge il frame errato e avvisa tutta la rete.

Error Passive (Stato Sospetto / Ammonito)

Il nodo ha accumulato troppi errori. Il protocollo inizia a sospettare che il problema sia del nodo stesso (es. il suo ricetrasmittitore è danneggiato o il suo clock sta sballando) e non dei disturbi sulla linea.

- **Condizione:** $TEC / REC > 127$.
- **Comportamento in caso di errore:** Per evitare di distruggere continuamente il traffico degli altri nodi sani, al nodo viene tolto il potere di imporre bit dominanti in caso di errore. Se rileva un errore, può inviare solo un **Passive Error Flag** composto da **6 bit recessivi**.
- **La conseguenza:** Essendo bit recessivi, l'errore rimane "silenzioso" per il resto della rete. Il nodo non può più distruggere i frame sul bus. Però continua a monitorare il bus e **manterrà la sincronizzazione con gli altri nodi**. Inoltre, prima di poter ritrasmettere un messaggio, deve aspettare un tempo aggiuntivo (8 bit di *Suspend Transmission*), dando la precedenza ai nodi sani.

Bus Off (Stato Grave / Espulso)

Il nodo è palesemente guasto e sta fallendo continuamente ogni tentativo di trasmissione. Per preservare l'incolumità della rete, viene spento e isolato.

- **Condizione:** $TEC > 255$.
- **Comportamento:** Il nodo si **scollega** (a livello logico) elettricamente dal bus. La sua linea di trasmissione viene impostata in stato recessivo perenne. Il nodo non può più né trasmettere né ricevere nulla, diventando un fantasma per la rete. Evita il problema del **bubbling idiot**.

ESEMPIO

- Ogni volta che il Nodo A tenta di trasmettere un messaggio, fallisce (per una qualsiasi ragione).
- Ogni volta che ciò accade, incrementa il proprio contatore degli errori di trasmissione (**TEC**) di **8 punti** e trasmette un *Active Error Flag*.
- Successivamente tenterà di ritrasmettere il messaggio... e si verificherà la stessa identica cosa.
- Quando il contatore degli errori di trasmissione del Nodo A supera il valore di **127** (ovvero dopo 16 tentativi falliti consecutivi), il nodo A passa nello stato **Error Passive** (Errore Passivo).
- Da questo momento in poi, il Nodo A trasmetterà sul bus solo *Passive Error Flags*.

- Un *Passive Error Flag* comprende 6 bit recessivi e non distrugge il restante traffico sul bus; di conseguenza, gli altri nodi non "sentiranno" il Nodo A lamentarsi degli errori sul bus.
- Tuttavia, il Nodo A continua a incrementare il proprio contatore degli errori di trasmissione. Quando questo supera il valore di **255**, il nodo A entra finalmente nello stato di **Bus Off** (Disconnesso dal bus).

Cosa succede agli altri nodi?

- Per ogni *active error flag* che il Nodo A ha trasmesso (prima di diventare passivo), gli altri nodi incrementano il proprio contatore degli errori di ricezione (**REC**) di **1 punto**.
- Nel momento in cui il Nodo A entra in *Bus Off*, gli altri nodi avranno un valore nei propri contatori di errore di ricezione (**REC**) che si attesta intorno a 16, ovvero **ben al di sotto del limite di 127** previsto per lo stato *Error Passive*.
- Questo conteggio diminuirà di uno per ogni messaggio ricevuto correttamente. Il Nodo A, invece, rimarrà nello stato di **Bus Off**.

Quando viene rilevato un qualsiasi errore:

Nel caso di un errore di CRC: il livello MAC attende il delimitatore dell'ACK (*ACK Delimiter*) e solo successivamente avvia la trasmissione di un **Error Frame**.

Qualsiasi altro tipo di errore (errore di bit, errore di bit stuffing, errore di forma, errore di ACK), invece, fa sì che il livello MAC trasmetta un **Error Frame** immediatamente (sul bit successivo).

A seconda dell'errore, l'Error Frame può essere inviato sia dal **nodo trasmettitore** (sia esso in stato *Error-Active* o *Error-Passive*), qualora abbia rilevato l'errore, sia da **tutti i nodi ricevitori in stato Error-Active** che rilevano l'errore.

I nodi ricevitori in stato Error-Passive sono esclusi dalla segnalazione. Infatti, qualsiasi notifica da parte loro non avrebbe effetto, poiché il *Passive Error Flag* (composto da 6 bit recessivi) verrebbe certamente sovrascritto dal nodo trasmettitore attivo in quel momento (se *Error-Active* e consapevole dell'errore) o da un qualsiasi altro nodo ricevitore in stato *Error-Active*.

Servizi di comunicazione LLC

In conformità con le specifiche ISO, il sottolivello **LLC** del CAN fornisce esclusivamente due servizi di comunicazione:

- **L_DATA:** che viene utilizzato per trasmettere in modalità *broadcast* il valore di uno specifico oggetto su tutta la rete.
- **L_REMOTE:** che viene utilizzato per richiedere che il valore di uno specifico oggetto venga trasmesso in *broadcast* da parte del relativo produttore remoto (*remote producer*).

Queste primitive sono implementate direttamente nell'hardware da tutti i controllori CAN attualmente disponibili sul mercato.

Modello per gli scambi di informazioni

Il CAN adotta un **indirizzamento a oggetti** (*object addressing*) invece di un indirizzamento a nodi (*node addressing*).

I messaggi **non** sono contrassegnati con l'indirizzo del nodo di destinazione e/o di origine.

A ciascuna informazione scambiata sulla rete (ovvero, a ciascun oggetto) viene assegnato un **identificatore unico**, che denota in modo inequivocabile il **significato** dell'oggetto stesso all'interno dell'intero sistema.

Identificare gli oggetti scambiati sulla rete in base al loro significato, anziché al nodo a cui sono destinati, consente implicitamente il **multicasting** e rende molto semplice, per le applicazioni di controllo, gestire le interazioni tra i dispositivi secondo il paradigma **produttore/consumatore** (*producer/consumer*). Lo scambio di informazioni nel CAN avviene secondo 3 fasi:

1. **La trasmissione:** Il produttore (*producer*) di una determinata informazione codifica e trasmette il relativo frame sul bus; la tecnica di arbitraggio risolverà in modo trasparente qualsiasi contesa che dovesse verificarsi.
2. **La diffusione:** Grazie alla natura intrinsecamente *broadcast* del bus, il frame si propaga su tutta la rete e ogni nodo ne legge il contenuto all'interno di un buffer di ricezione locale.
3. **Il filtraggio:** La funzione di filtraggio per l'accettazione del frame (**FAF** - *Frame Acceptance Filtering*) presente in ciascun nodo determina se l'informazione sia rilevante o meno per il nodo stesso.
 - Nel primo caso, il frame viene passato ai livelli di comunicazione superiori.
 - In caso contrario, il frame viene semplicemente ignorato e scartato.

I **frame dati** (*data frames*) vengono utilizzati dai nodi produttori (*producer nodes*) per trasmettere in modalità *broadcast* i nuovi valori sulla rete, identificati in modo inequivocabile tramite l'identificatore.

Nel CAN, l'informazione viene inviata **non appena diventa disponibile** o dalle applicazioni di controllo o dal sistema reale (tramite i sensori); questo migliora la reattività (*responsiveness*) dell'intero sistema.

Le reti CAN funzionano altrettanto bene anche quando vengono utilizzate per interconnettere dispositivi in sistemi basati su un modello di comunicazione più convenzionale di tipo **Master-Slave**.

In questo caso, il *Master* può utilizzare i **frame remoti** per richiedere che una specifica informazione venga inviata da remoto sulla rete. Il produttore (*producer*) di tale informazione,

come conseguenza di questo frame di richiesta, risponderà con un **frame dati** che trasporta l'oggetto in questione.

La risposta che contiene il valore richiesto viene trasmessa in modalità *broadcast* su tutta la rete; di conseguenza, può essere letta da tutti i nodi interessati, oltre che da quello che ha trasmesso la richiesta remota originaria.

Significa che il CAN supporta **entrambe** le modalità:

Invio spontaneo (Push / Event-Triggered) — Il caso standard

Non appena un sensore legge un nuovo valore (es. la temperatura del motore sale) o l'applicazione di controllo calcola un nuovo dato, il nodo *Producer* prende l'iniziativa: impacchetta il dato in un **Data Frame** e prova a trasmetterlo immediatamente sul bus. Il sistema è estremamente **reattivo**. Non si perde tempo ad aspettare che qualcuno chieda il dato; se succede qualcosa di importante, l'informazione viaggia subito.

Invio su richiesta (Pull / Request-Reply) — Il caso dei Remote Frame

Esistono situazioni in cui un nodo *Consumer* ha bisogno di un'informazione che viene trasmessa raramente, oppure ha appena subito un reset e deve conoscere l'ultimo stato di un sensore. In questo caso:

1. Il nodo *Consumer* invia un **Remote Frame** (un frame di richiesta) contenente l'ID dell'oggetto desiderato.
2. Il nodo *Producer* di quell'oggetto riceve la richiesta e risponde inviando il **Data Frame** con il valore aggiornato.

Il flusso normale sul bus CAN vede i nodi inviare i propri dati **quando sono pronti o a intervalli periodici fissi** (es. la centralina invia i giri motore ogni 10 millisecondi) senza che nessuno glielo chieda. I **Remote Frame** (la richiesta esplicita) si usano come meccanismo secondario o di backup.

CAN FD

Il protocollo CAN FD nasce dall'esigenza di soddisfare diversi requisiti stringenti:

- **Accelerare il download del software** alla fine della linea di produzione (fine serie) o durante la manutenzione nelle officine per gli aggiornamenti software.
- **Supportare payload più lunghi**, superando il limite di 8 byte previsto dal CAN classico.
- **Evitare la segmentazione** dei messaggi lunghi (che altrimenti richiederebbero di essere divisi in più frame).
- **Mettere in sicurezza i normali messaggi CAN** tramite informazioni di **autenticazione**, garantendo che i dati provengano dalla corretta centralina (ECU).
- **Richieste dai costruttori di auto (carmakers)**: maggiore larghezza di banda per alcune reti di bordo basate su CAN (ad esempio, nel gruppo motopropulsore / *powertrain*).
- **Richieste dai produttori di camion/veicoli commerciali**: maggiore produttività (*throughput*) su bus CAN di lunghezze elevate.
- I controller del protocollo CAN FD devono essere in grado di **eseguire anche le comunicazioni CAN standard** (retrocompatibilità).

I Miglioramenti di CAN FD

Il CAN FD (Flexible Data-rate) migliora il CAN classico principalmente in due modi:

- **Trasmissione dati più veloce**: velocità di trasmissione superiori a 1 Mbit/s.
- **Payload più lungo (campo dati)**: capacità estesa fino a **64 byte**.

Principio di funzionamento

Durante la parte compresa tra la fase di arbitration e acknowledgement (ACK) di un frame CAN, è consentito un **unico trasmettitore** alla volta sul bus.

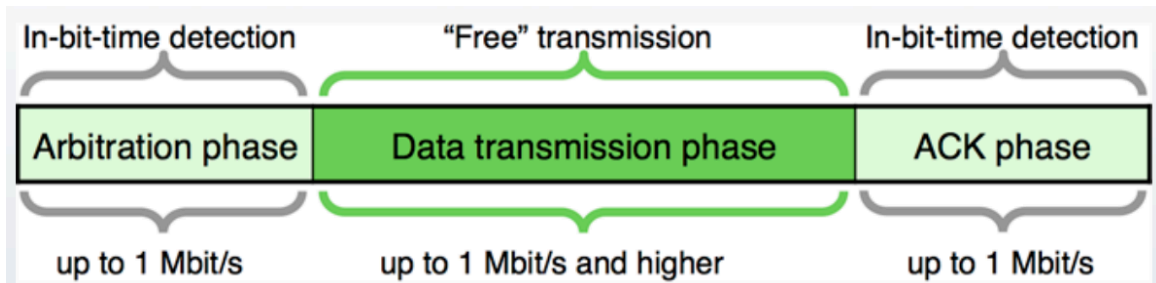
L'idea alla base del CAN FD si fonda su un principio semplice: quando un solo nodo sta trasmettendo (ovvero durante i campi DLC, Data e CRC), **la velocità di bit (bit-rate) può essere aumentata, poiché in questa fase i nodi non hanno bisogno di sincronizzarsi tra loro per gestire i conflitti di accesso**. Prima della trasmissione del bit relativo allo slot di ACK, i nodi devono essere **ri-sincronizzati**.

Di conseguenza, i frame dati CAN FD possono essere trasmessi utilizzando due differenti bit-rate:

- **Nella fase di arbitrato**: il bitrate è limitato a **1 Mbit/s** (dipende dalla topologia).
- **Nella fase di dati**: il bit-rate è limitato esclusivamente dalle caratteristiche fisiche del ricetrasmittitore (*transceiver*).

Quindi :

- Le parti della frame CAN FD vengono mandate a velocità diverse, in base ai delimitatori.
- La velocità dipende dai transceiver.
- FDF indica che la frame è FD.



Frame CAN FD

Il protocollo CAN FD introduce una frame che supporti più data bytes e bit rate flessibili.

RRS e FDF

RTR vs. RRS

- **RTR:** Il bit *Remote Transmission Request* (RTR) viene utilizzato per distinguere i frame dati dai corrispondenti frame remoti.
- **RRS:** I frame remoti **non sono supportati**. Al loro posto si trova il bit *Remote Request Substitution* (RRS), che è sempre **dominante (0)**.

r0 vs. FDF

- **r0:** è riservato e sempre **dominante (0)**.
- **FDF:** Questo bit prende il nome di *FD Format* (FDF) ed è **recessivo (1)** per indicare il formato CAN FD.

Nota: I nodi che non supportano il CAN FD generano un *error frame* (frame di errore) subito dopo il bit FDF, poiché si aspettano un bit dominante.

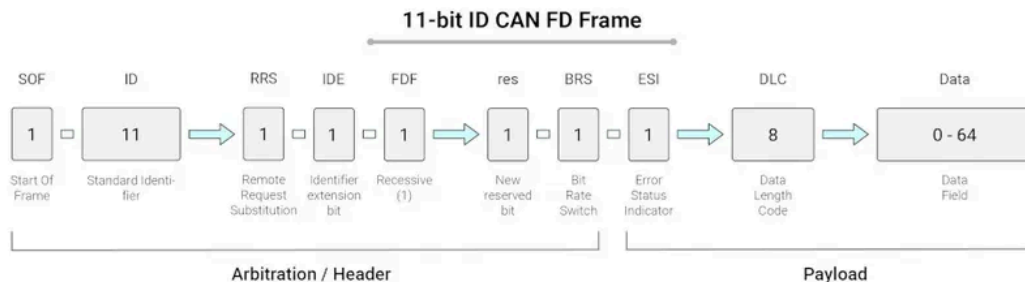
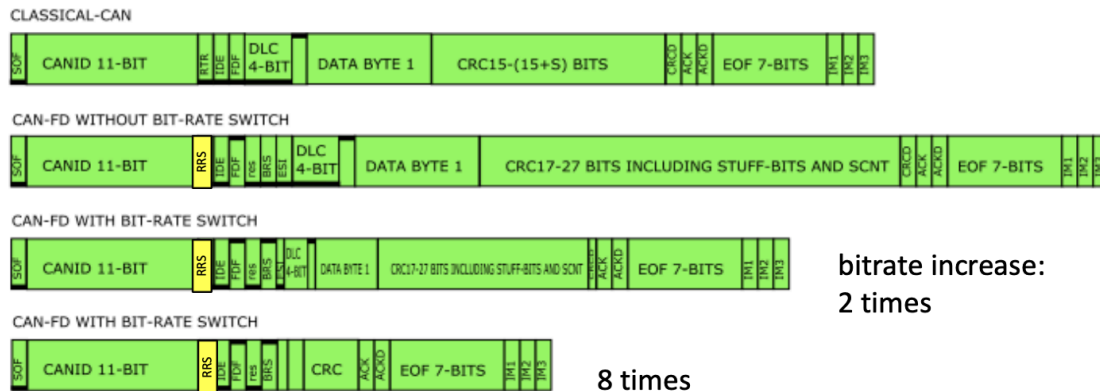
Nuovi campi

I 3 Nuovi Bit introdotti dopo il bit FDF

Dopo il bit FDF, il protocollo CAN FD aggiunge tre nuovi bit di controllo:

1. **res (Reserved Bit):** Un nuovo bit riservato per sviluppi futuri. In futuro potrebbe essere impostato su recessivo (1) per indicare l'introduzione di un nuovo protocollo.

2. **BRS (Bit Rate Switch):** Il bit di commutazione della velocità.
 - Se è **dominante (0)**, il frame dati CAN FD viene inviato interamente alla velocità di arbitrato (ovvero fino a un massimo di 1 Mbit/s).
 - Se è **recessivo (1)**, significa che la parte restante del frame (fino al campo CRC) viene inviata alla velocità superiore (fino a **5 Mbit/s**).
3. **ESI (Error Status Indicator):** L'indicatore dello stato di errore. Di default è **dominante (0)**, che indica uno stato di *error active* (errore attivo). Se il nodo trasmettitore passa in modalità *error passive* (errore passivo), questo bit diventa **recessivo (1)** per segnalarlo immediatamente alla rete.



Campi del Frame e Gestione del Bit-Rate

- **DLC (Data Length Code):** Come nel CAN Classico, il DLC del CAN FD è composto da **4 bit** e indica il numero di byte di dati presenti nel frame. Tuttavia, CAN FD mappa questi 4 bit per supportare le seguenti dimensioni del campo dati: **(0-8, 12, 16, 20, 24, 32, 48, 64 byte)**.
- **CRC (Cyclic Redundancy Check):** Il codice di controllo della ridondanza ciclica. Il CAN Classico utilizza un CRC a 15 bit. Il CAN FD utilizza invece un **CRC a 17 bit** per campi dati fino a 16 byte inclusi, oppure un **CRC a 21 bit** per campi dati da 20 byte in su. A causa della maggiore quantità di dati disponibile, sono state introdotte modifiche nel CRC del CAN FD per ridurre il rischio di errori non rilevati e mantenere una capacità di rilevamento degli errori paragonabile a quella del CAN classico.

- **ACK (Acknowledgement):** La fase dati (ovvero il payload) del frame CAN FD si interrompe al bit di ACK, il quale segna anche il confine del periodo in cui la velocità dei bit può essere aumentata.

Il cambio di bit-rate segue una transizione precisa all'interno del frame:

- **Inizio dell'alta velocità:** Il primo bit inviato alla velocità di bit più elevata (*data phase*) è il bit **ESI** (*Error Status Indicator*), seguito in sequenza dal **DLC**, dai **byte di dati** e dai **bit di CRC**.
- **Fine dell'alta velocità:** L'ultimo bit inviato alla velocità più elevata è il **delimitatore del CRC** (*CRC Delimiter*). Subito dopo, il bus ritorna alla velocità di arbitrato standard prima della trasmissione dello slot di ACK.

I frame dati CAN FD possono essere trasmessi utilizzando due differenti velocità di bit:

- **Velocità nominale (Nominal bit rate):** Utilizzata nella fase di arbitrato e nella fase di ACK; è limitata a **1 Mbit/s**, esattamente come nel CAN classico.
- **Velocità dati (Data bit rate):** In questa fase la velocità è limitata esclusivamente dalle caratteristiche del ricetrasmittitore (livello fisico - PHY).

Prestazioni e Vincoli Fisici

- **Velocità tipiche:** Si raggiungono normalmente **2-2,5 Mbps** e fino a **3,7 Mbps** con i ricetrasmittitori per il settore automotive attualmente in commercio, con picchi fino a **5 Mbps**.
- **Transceiver speciali (SIC):** Utilizzando ricetrasmittitori speciali dotati di funzionalità *Signal Improvement Capability* (SIC), è possibile raggiungere velocità comprese tra **5 e 8 Mbps**.
- **Il fattore temperatura:** Uno dei criteri limitanti più importanti è l'intervallo di temperatura desiderato. La maggior parte delle reti con topologia a bus condiviso (*multi-drop*) è limitata a **2 Mbit/s** per un intervallo di temperatura compreso tra **-40 °C e +125 °C**. Tuttavia, la riprogrammazione (*flashing*) di una centralina (ECU) non richiede il supporto alle basse temperature: ciò significa che per il flashing delle ECU è possibile spingersi **fino a 12 Mbit/s**.
- **L'impatto della topologia:** Un'ulteriore e importante limitazione della velocità è causata dalla topologia di rete scelta. Una topologia puramente a bus con derivazioni (*stubs*) molto corte consente velocità significativamente più elevate rispetto a topologie ibride con derivazioni lunghe o a stella.

Compatibilità dei Controller

Il controller del protocollo CAN FD supporta anche i frame CAN Classici; ciò significa che i controller CAN FD sono in grado di inviare e ricevere sia frame CAN classici sia frame CAN FD. Sul mercato si possono trovare tre tipi di ricetrasmittitori (*transceiver*):

- **CAN FD-enabled:** in grado di trasmettere e ricevere frame CAN FD.

- **CAN FD-tolerant:** non supportano i frame CAN FD, ma non generano errori se questi frame vengono trasmessi sul bus.
- **CAN Legacy:** i ricetrasmittitori CAN tradizionali che generano invece dei frame di errore (*error frames*) se sul bus vengono trasmessi frame CAN FD.

Entrambi i protocolli (CAN Classico e CAN FD) compaiono per la prima volta all'interno dello stesso standard con la norma **ISO 11898-1:2015**.

I nodi CAN Classico e CAN FD possono essere mescolati all'interno della stessa rete rispettando determinate condizioni. Questo consente una migrazione graduale (*passo dopo passo*) verso il CAN FD, evitando di dover sostituire ogni singola centralina (ECU) in un'unica soluzione. Si possono configurare due scenari principali:

- **Sistema 100% CAN FD:** In questo caso, i controller CAN FD possono alternare e combinare liberamente frame dati CAN Classici e CAN FD.
- **Presenza di nodi legacy (CAN Classico):** In questo scenario, i controller CAN FD possono passare alla modalità di comunicazione CAN Classica per evitare che i nodi CAN Classici generino frame di errore. Inoltre, durante attività specifiche come il *flashing* (riprogrammazione) delle ECU, i nodi CAN Classici possono essere temporaneamente disattivati (spenti) per consentire il passaggio temporaneo alla comunicazione in CAN FD ad alta velocità.

Vantaggi

1. **Payload più ampio:** Il rapporto tra dati utili (*payload*) e dati di overhead (bit di controllo e di gestione) risulta nettamente migliorato. I valori consentiti per il payload dei frame CAN FD sono: **0-8, 12, 16, 20, 24, 32, 48 e 64 byte**.
2. **Più dati, minore carico sul bus (busload):** Grazie al payload più ampio e a una trasmissione più rapida del contenuto del frame CAN FD, il tempo di trasmissione complessivo si riduce, abbassando di conseguenza il carico sul bus. Inoltre, la possibilità di trasmettere un campo dati più lungo all'interno di un singolo frame riduce drasticamente la necessità di segmentare i dati (frammentazione in più messaggi).
3. **Produttività (*throughput*) più elevata:** Utilizzando un rapporto di **1:8** tra le velocità di bit della fase di arbitrato/ACK e quella della fase dati, si ottiene un incremento della produttività di circa **sei volte**, anche considerando il fatto che i frame CAN FD utilizzano un numero maggiore di bit nell'intestazione (*header* / campo di controllo) e nel campo CRC.
4. **Supporto per funzionalità di sicurezza (*security*):** Il payload più capiente può essere sfruttato sia per trasferire chiavi crittografiche composte da più byte, sia per implementare protocolli di comunicazione sicura e autenticata tra i dispositivi della rete.
5. **Coesistenza in parallelo di controller vecchi e nuovi:** L'implementazione del protocollo richiede nuovi controller CAN FD. Questi ultimi, tuttavia, sono **retrocompatibili** e perfettamente in grado di gestire i frame CAN classici. Quando solo alcune centraline (ECU) supportano il CAN FD, queste possono comunicare tra loro a velocità elevate mentre le altre vengono messe in stato di instradamento parziale (*partial networking* / modalità *sleep*). Questa strategia è particolarmente utile per

l'aggiornamento software delle ECU in officina, dove velocità di trasmissione più elevate riducono i tempi di un processo che spesso richiede ore

6. **Continuità tecnologica:** Un ulteriore punto di forza del CAN FD è il fatto di non essere una tecnologia completamente nuova. Tutto il bagaglio di conoscenze ed esperienze acquisito dagli ingegneri in anni di lavoro sul CAN classico rimane valido e riutilizzabile. Il passaggio dal CAN al CAN FD non comporta quindi costi o sforzi eccessivi in termini di formazione del personale.

Considerazioni sulle prestazioni del CAN FD:

- **Aumento dell'efficienza con payload maggiori:** Con più di 8 byte di dati all'interno dei frame CAN, la produttività dei dati (*throughput*) aumenterà proprio grazie al miglioramento dell'efficienza nel rapporto tra dati utili e bit di controllo.
- **Il rischio con bit-rate invariato:** Tuttavia, l'invio di una maggiore quantità di dati in ogni singolo frame ridurrebbe le prestazioni complessive se non venisse combinato con l'utilizzo di una velocità di bit (*bit-rate*) più elevata.
- **Vantaggi dell'alta velocità su efficienza e latenza:** Un bit-rate più elevato non aumenta solo l'efficienza, ma incrementa anche le prestazioni generali, poiché riduce la durata temporale dei frame CAN sul bus, abbassando di conseguenza la latenza di comunicazione.
 - Se il bit-rate rimanesse invariato, le trasmissioni più lunghe occuperebbero il bus CAN per più tempo, rischiando potenzialmente di ritardare l'invio di frame dati prioritari e fondamentali per il sistema (*mission-critical*).
 - Anche in assenza di espliciti vincoli di latenza, l'uso di un bit-rate più elevato rimane comunque di grande interesse, ad esempio per incrementare la produttività dei dati e ridurre così i tempi di download del software.

Sintesi del CAN FD

- **L'aumento del bit-rate nel CAN FD offre una maggiore larghezza di banda, garantendo di conseguenza:**
 - Minore latenza e migliori prestazioni complessive.
 - Una produttività (*throughput*) più elevata.
- **Il CAN FD può contenere più dati all'interno del frame CAN, passando da 8 fino a 64 byte:**
 - Minore overhead relativo = maggiore efficienza complessiva.
 - Invio più semplice ed efficiente di blocchi di dati di grandi dimensioni.
- **Il CAN FD è dotato di un algoritmo CRC dalle prestazioni più elevate:**
 - Riduce drasticamente il rischio di errori non rilevati sulla rete.

CAN FD Light

CAN FD light è una versione ottimizzata in termini di costi del CAN FD (riduzione dei costi di circa il 50%)

- **Architettura:** 1x *Commander* (Comandante) e Nx *Responders* (Risponditori)
 - Il commander CAN FD light controlla l'intera comunicazione (tramite *polling* / interrogazione ciclica).
- **Commander:** Qualsiasi nodo CAN FD/XL esistente.
- **Responder:** Nodo CAN FD a basso costo.
 - **[Layer 1]:** Nessuna modifica, tutti i ricetrasmittitori (*transceiver*) CAN sono utilizzabili.
 - **[Layer 2] Protocollo Semplificato:** Sottinsieme del CAN FD.
 - **Nessun arbitrato, Nessun frame di errore (*error frame*), Nessun ID a 29 bit, Nessuna commutazione della velocità (bit di BRS), ecc.**
 - Viene utilizzato il **formato di frame CAN FD con ID a 11 bit**.
 - Standardizzato nella norma **ISO 11898-1:2024**.

Vantaggi

- **Basso costo:** (ad es. è utilizzabile un oscillatore/clock con tolleranza del $\pm 4\%$, ovvero 10 volte superiore rispetto al normale FD → Risparmio di 0,40 US).
- **Integrazione monolitica possibile** di: (1) Ricetrasmittitore, (2) IP CAN FD light, (3) parte analogica.
- **Velocità di bit netta (*net bit rate*) maggiore** rispetto al CAN FD.

Contesto e Applicazioni

Il CAN FD light, standardizzato nella norma **ISO 11898-1:2024**, è concepito per l'uso in reti di sensori/attuatori *deeply embedded* e *altamente sensibili ai costi*.

- **Sistemi di illuminazione per auto moderne:** Un esempio tipico sono i sistemi in cui i cluster di LED collegati (nodi *responder*) vengono controllati individualmente da un unico nodo centrale (nodo *commander*).
- **Ulteriori applicazioni potenziali:** Altri sistemi in cui un controllore ospite centrale (*central host controller*) gestisce diversi dispositivi terminali semplici e con risorse limitate, come i sistemi di riscaldamento, ventilazione e condizionamento dell'aria (**HVAC**).

Specifiche e Meccanismo di Controllo

- Lo standard CAN FD light specifica la comunicazione tra nodi commander e responder secondo uno **schema commander/responder** (master/slave).
- All'interno di questo schema, i nodi responder **non supportano l'arbitrato di rete**, ma trasmettono i frame dati CAN FD esclusivamente su richiesta del commander CAN FD light (tramite meccanismo di **polling** / interrogazione ciclica).

Ottimizzazione dei Costi dell'Hardware

- **Nessuna necessità di una sorgente di clock precisa** su ciascun nodo della rete.
- Possono essere utilizzati dispositivi a singolo chip monolitico **privi di oscillatori a cristallo**.
- Questo fattore riduce drasticamente i costi complessivi di produzione e implementazione.

Frame

- **Formato e Payload dei Frame:** I nodi responder del CAN FD light trasmettono frame dati CAN FD conformi allo standard **ISO 11898-1** e offrono un payload (campo dati) che può raggiungere i **64 byte**.
- **Identificatore (ID):** Supportano esclusivamente il formato di frame base, ovvero con **identificatore a 11 bit**.
- **Gestione del Bit-rate:** Non viene supportata la commutazione della velocità di bit (*no bit-rate switching*). Di conseguenza, viene utilizzata la **stessa velocità di bit** sia nella fase di arbitrato sia nella fase di trasmissione dei dati, e i bit all'interno del campo di controllo del frame rimangono fissi.
- **Velocità Massima Corrente:** Attualmente, la velocità di bit massima è limitata a **1 Mbit/s**, un valore ampiamente sufficiente per i requisiti delle applicazioni a cui questo protocollo è destinato.
- **Ricezione e Acknowledgement (ACK):** I nodi responder ricevono e trasmettono frame dati CAN nel formato di frame FD. Inoltre, inviano il bit di *acknowledge* (ACK) a seguito della ricezione priva di errori di un frame dati proveniente dal nodo commander.

Utilizzo

SDV (*Software Defined Vehicle*) – Esempio di Illuminazione

ECU per Illuminazione (ECU for Lighting)

- Driver LED per retroilluminazione (*Backlight*), fari abbaglianti adattivi (ADB - *Adaptive Driving Beam*), ecc.

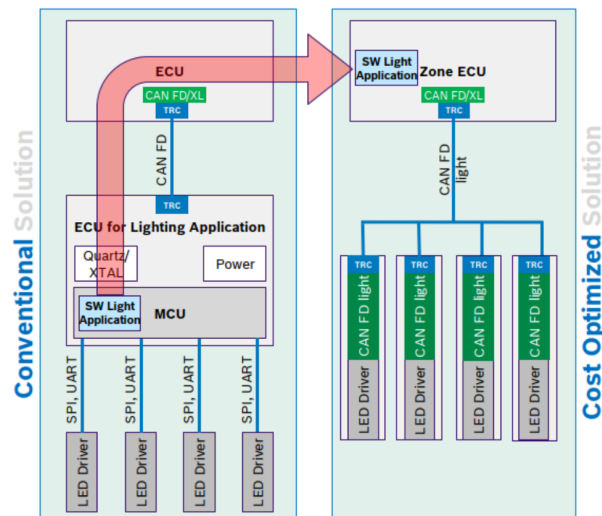
Nodi Responder CAN FD light

- **Integrazione del driver LED analogico** in un design monolitico che include il ricetrasmittitore (*transceiver*), l'IP CAN FD light e la macchina a stati.
- **Spostamento della ECU per l'applicazione di illuminazione all'interno della Zone ECU (Centralina di Zona):**
 - **Vantaggi:**
 - L'aggiornamento del software avviene **esclusivamente nella Zone ECU**.
 - **Assenza di software** nel nodo terminale (*end-node*).
 - Minore manutenzione per il costruttore (*OEM*).

- Nuove possibilità per l'architettura elettrica/elettronica (Architettura E/E).

Confronto tra i due schemi nel diagramma

- **Soluzione Convenzionale (A sinistra - *Conventional Solution*):** L'applicazione software risiede in una ECU locale dedicata all'illuminazione (dotata di microcontrollore MCU, quarzo/oscillatore XTAL e alimentazione), la quale comunica con i singoli driver LED sottostanti tramite interfacce punto a punto come SPI o UART.
- **Soluzione Ottimizzata in termini di Costi (A destra - *Cost Optimized Solution*):** L'applicazione software viene spostata centralmente nella **Zone ECU**. Il bus CAN FD light scende direttamente dalla centralina di zona e pilota i nodi terminali (i driver LED integrati con IP CAN FD light), eliminando la MCU locale, il quarzo e i cablaggi dedicati SPI/UART.



IN SINTESI:

CAN FD Light è un protocollo di comunicazione seriale lightweight basato su CAN FD, progettato specificamente per reti di sensori e attuatori ***deeply embedded***, con particolare enfasi sui sistemi di illuminazione automobilistica e HVAC (ventilazione, riscaldamento ecc). A differenza di CAN o CAN FD che utilizzano una topologia multi-commander con arbitrato, CAN FD Light adotta una architettura commander-responder, dove un nodo commander controlla centralmente la comunicazione e i noder responder rispondono solo su richiesta.

Questa architettura semplifica notevolmente l'**hardware** dei nodi finali, eliminando la necessità di **oscillatori al quarzo** esterni e costosi, poiché non è richiesto l'arbitrato di bus. Il protocollo standardizzato in ISO 11898-1:2024 supporta payload fino a 64 byte e bit-rate fissi fino a 1 Mbit/s, garantendo una comunicazione robusta e a basso costo per il controllo di dispositivi semplici come pixel LED e sensori.

CAN XL

CAN XL è un *data link layer* CAN che fornisce payload "extra large" fino a **2048 byte** e una velocità dati superiore a 10 Mbps:

- $\geq$ **12 Mbps** nella fase dati,
- $\leq$ **1 Mbps** nella fase di Arbitration e di ACK.

Dispone di un **campo di priorità a 11 bit** utilizzato esclusivamente per scopi di arbitrato (gestione dell'accesso al bus), consente un'elevata velocità di bit netta per payload ridotti.

Mentre l'**ID del Messaggio (32 bit)**, viene inviato nel campo di accettazione ("*Acceptance Field*") durante la fase dati del CAN XL.

Lunghezza del campo dati, **da 1 a 2048 byte**, consente il **tunneling** trasparente dei frame Ethernet, l'utilizzo di TCP/IP, SOME/IP e altro ancora.

Caratteristiche del CAN XL

Il CAN XL è concepito per applicazioni di rete di tipo dorsale (***backbone***) e sotto-dorsale (***sub-backbone***), ed è progettato per una facile integrazione nei sistemi di rete basati su TCP/IP.

- Il processo è iniziato alla fine del 2018, guidato da **Bosch** all'interno dell'associazione CiA (*CAN in Automation*). Il CAN XL è ora incluso nello standard **ISO 11898-1:2024**.
- **Doppia velocità**: Analogamente al CAN FD, sono specificate due diverse configurazioni per il timing dei bit (fase di arbitrato lenta e fase dati veloce).
- **Payload maggiore**: Estesa da un minimo di **1 byte fino a 2048 byte**.
- **Separazione dell'ID**: Una novità assoluta è la separazione del campo CAN-ID in due parti distinte: un campo per l'**ID di priorità a 11 bit** e un ***Acceptance Field* a 32 bit**.
- **Integrazione con i livelli superiori**: Il CAN XL offre nuove possibilità di configurazione integrate nel protocollo (*protocol-embedded*) per agevolare l'interfacciamento con i livelli architetturali superiori (Higher Layers).

Le caratteristiche di configurazione **opzionali** includono:

- La possibilità di **disattivare la segnalazione degli errori** (impedendo la propagazione degli *error frames*).
- L'abilitazione della **codifica PWM** (modulazione a larghezza di impulso) in alternativa alla classica codifica NRZ (*Non-Return-to-Zero*) del CAN. La codifica PWM consente di raggiungere velocità di bit pari a **20 Mbit/s e oltre** (a seconda del design fisico della rete).

3 Nuovi Campi

SDT: Service Data Unit Type (8 bit)

- Indica il tipo di protocollo di livello superiore che è incapsulato all'interno del Data Field.
- È l'equivalente del campo **EtherType** nei frame Ethernet.
- La specifica **CiA611-1** definisce già un valore di SDT dedicato espressamente al **Tunneling Ethernet** (per trasportare frame Ethernet completi dentro il CAN XL).

VCID: Virtual CAN Network ID (8 bit)

- Permette di mappare e separare il bus CAN fisica in più **reti virtuali indipendenti**.
- **Paragone con l'Ethernet:** È l'esatto equivalente del **VLAN ID** (Virtual LAN) nel mondo Ethernet. Consente la coesistenza di più sottoreti logiche sullo stesso bus fisico, migliorando la sicurezza e l'organizzazione del traffico.

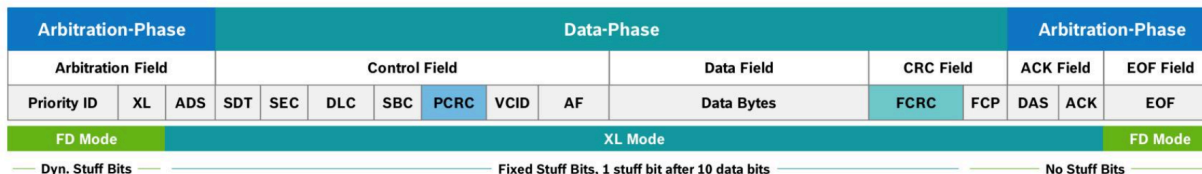
AF: Acceptance Field (32 bit)

- A seconda del valore impostato nel campo SDT, l'AF può contenere:
 - L'**ID del Messaggio** (Message ID),
 - L'**Indirizzo di Sorgente/Destinazione**,
 - Altre informazioni di indirizzamento.
- **Flessibilità:** Grazie a questa struttura, il CAN XL è in grado di supportare nativamente sia l'indirizzamento basato sul **contenuto**, tipico del CAN classico, sia l'indirizzamento basato sui **nodi**, tipico **dell'Ethernet**.

Caratteristiche Generali del CAN XL

- Il campo **SDT (Service Data Unit Type)** permette di eseguire protocolli eterogenei di livello superiore.
 - Un campo indicatore a 8 bit (SDT) specifica il tipo di SDU, ovvero il protocollo successivo di livello superiore utilizzato.
 - Di conseguenza, viene abilitato il **tunneling di frame Ethernet**, pacchetti TCP/IP o altri protocolli di livello superiore su CAN XL.
- Consente un indirizzamento basato sul contenuto (*content-based*) o basato sull'indirizzo (*address-based*) tramite l'**Acceptance Field (AF, 32 bit)**; l'interpretazione di questo campo dipende direttamente dal valore impostato nel campo SDT.
- Introduce un **nuovo formato di frame** con campi aggiuntivi dedicati.

FRAME CAN XL



Il frame si divide nettamente in tre blocchi temporali dominati da due modalità fisiche diverse.

Arbitration Phase - LENTA

La fase di Arbitrato Iniziale utilizza la velocità classica (≤ 1 Mbit/s). Usa lo **stuffing dinamico** dei bit (Dyn. Stuff Bits). *Campi:*

- **Priority ID (11 bit):** Identificatore utilizzato per stabilire la priorità di accesso al bus in caso di collisioni.
- **XL (1 bit):** Un segnale segnaposto che funge da *Frame Format Switch*, indicando il passaggio immediato dal formato FD al formato XL.
- **ADS (Arbitration Data Sequence):** Sequenza fisica di transizione che esegue lo switch del bit-rate, passando dalla fase lenta alla fase veloce dei dati.

Data Phase - VELOCE

La fase dati Centrale (Data-Phase - XL Mode) viene attivata l'alta velocità (≥ 12 Mbit/s). Per garantire la sincronizzazione a frequenze così elevate senza appesantire il sistema, si passa a uno stuffing a bit fissi (Fixed Stuff Bits): viene inserito tassativamente 1 fixed stuff bit ogni 10 bit di dati.

Campo di Controllo (Veloce)

- **SDT (SDU Type - 8 bit):** Indica il tipo di protocollo superiore incapsulato nei dati.
- **SEC (Security - 1 bit):** Segnala se sono state aggiunte funzioni di Livello 2 all'header del campo dati, come informazioni di sicurezza/autenticazione o di segmentazione.
- **DLC (Data Length Code - 11 bit):** Indica la lunghezza effettiva dei dati.
- **SBC (Stuff Bit Count):** Contatore dei bit di stuffing dinamici utilizzati nella fase di arbitrato. Serve come protezione contro specifiche tipologie di errore di sincronizzazione.
- **PCRC (Preface CRC - 13 bit):** Un primo codice CRC parziale che mette al sicuro tutti i bit precedenti dell'header, fino al campo PCRC stesso.
- **VCID (Virtual CAN Network ID - 8 bit):** ID della rete CAN virtuale, equivalente al tag VLAN ID dell'Ethernet, per segmentare logicamente i flussi sul bus.
- **AF (Acceptance Field - 32 bit):** Campo di indirizzamento a 32 bit. Può contenere un ID messaggio o un indirizzo nodo (la sua interpretazione logica dipende dal valore di SDT).

Campo Dati e Controllo Errori Finale (Veloce)

- **Data Bytes:** Il payload vero e proprio, scalabile da 1 a 2048 byte.

- **FCRC (Frame CRC - 32 bit):** Il CRC principale a 32 bit. Protegge l'intero frame, garantendo una robustezza altissima contro gli errori anche su pacchetti molto grandi.
- **FCP (Format Check Pattern):** Una sequenza di controllo utilizzata dal ricevitore per verificare di essere ancora perfettamente allineato al flusso di bit trasmesso ad alta velocità.

*il Preface CRC protegge la parte iniziale del frame necessaria a interpretarne tipo e lunghezza, consentendo il rilevamento precoce degli errori; il Frame CRC protegge invece l'intero messaggio, compreso il payload, e viene verificato al termine della ricezione per garantire l'integrità completa dei dati.

Fase di Acknowledgment - LENTA

La fase di Arbitrato Finale (Arbitration-Phase - FD Mode): Il bus ritorna alla modalità lenta e stabile per gestire l'ACK e la fine del frame. In questa coda **non vengono usati stuff bit** (*No Stuff Bits*).

- **DAS (Data Arbitration Sequence):** Sequenza fisica inversa che effettua la commutazione della velocità, riportando il bus dall'alta velocità alla velocità nominale di arbitrato.
- **ACK (Acknowledgement):** Il bit di conferma di ricezione positiva, con logica identica a quella del CAN FD.
- **EOF (End of Frame):** Indica la fine geometrica del frame di trasmissione.

CAN XL e CAN FD

CAN FD e CAN XL sono compatibili. Soluzione Tecnica:

- I nodi **CAN FD ignorano i frame CAN XL.**
- I nodi **CAN XL comprendono entrambi i formati:** sia i frame FD sia i frame XL.
- **Limitazione:** Questa coesistenza funziona con velocità **fino a 8 Mbit/s.**

Vantaggi per il Cliente

- **Reti miste FD/XL → risparmio sui cablaggi.**
- **Aggiornamento incrementale → riutilizzo delle ECU CAN FD esistenti.**

Ethernet Tunneling

Il principio si basa sull'incapsulamento puro: un intero frame Ethernet viene inserito direttamente nel campo dati (*payload*) di un frame CAN XL.

- All'interno del campo di controllo del CAN XL, il byte **SDT** indica ai nodi riceventi cosa c'è dentro il payload.

- Quando un nodo legge **SDT = 0x05**, sa istantaneamente che il payload che segue non è una serie di segnali CAN tradizionali, ma è un **Frame Ethernet** completo e intatto.

In questo modo una **Dorsale Ethernet (Ethernet Backbone)** ad altissima velocità collega i computer centrali dell'auto alla **ZONE ECU** (Centralina di Zona).

Il Bus CAN XL come derivazione: La ZONE ECU fa da ponte. Sotto di essa parte un normale ed economico bus CAN a cui sono appesi vari sensori (es. tre sensori Radar XL) e un vecchio nodo legacy CAN FD.

Tunneling sul bus: Quando la ZONE ECU deve mandare un pacchetto Ethernet (IP) a un endpoint specifico (l'ultimo sensore a destra, l'**Eth (DDS) Endpoint**), inserisce il frame Ethernet dentro un frame CAN XL (*XL ETH frame*).

Comportamento dei nodi: * I Radar standard ignorano il contenuto.

Il nodo CAN FD legacy ignora il frame XL (grazie alla compatibilità vista prima).

L'**Eth Endpoint** riceve il frame CAN XL, estrae il frame Ethernet e lo passa ai suoi strati software superiori.

Dentro l'endpoint (il sensore che riceve il tunnel):

- I bit viaggiano sul cavo CAN. Passano dal *Transceiver* fisico al modulo hardware *CAN XL Module*.
- *Software driver for CAN XL Module* riceve i frame XL, legge il campo **SDT = 0x05** ed estrae il frame Ethernet originale.
- Il frame estratto viene passato direttamente allo stack di rete superiore (es. TCP/IP) come se fosse arrivato da una vera scheda di rete Ethernet. L'applicazione software sul sensore non si accorge nemmeno che i dati hanno viaggiato su un cavo CAN.

3 tipi di protocolli su un solo cavo CAN FD, CAN XL, Ethernet: Riduzione dei costi.

Il vantaggio enorme è di tipo economico e logistico: sullo **stesso identico doppino di rame** puoi far coesistere la diagnostica o i messaggi legati al CAN FD, i flussi dati ad alta velocità del CAN XL e il traffico IP/Ethernet per i sensori intelligenti. Si eliminano cavi schermati Ethernet e switch intermedi, riducendo drasticamente peso e costi del veicolo.

Architetture zonali

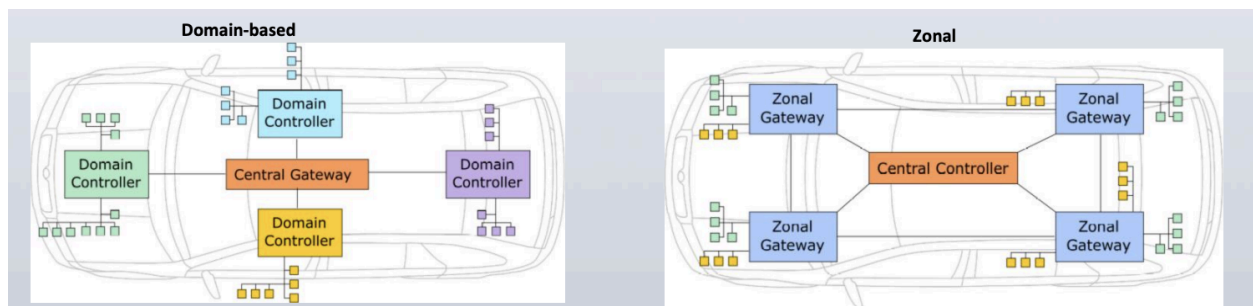
Le tradizionali architetture basate su domini prevedono numerose **ECU** e reti separate per ogni singolo sottosistema dell'autoveicolo (ad esempio: gruppo motopropulsore/*powertrain*, infotainment, carrozzeria/*body*, ecc.). Le architetture a domini non sono ottimizzate sotto il profilo del **cablaggio**, e questo comporta un impatto negativo in termini di costi e di spazio occupato.

Le innovative architetture zonali segnano un passaggio dalla distribuzione *logica* delle funzionalità (tipica dell'architettura a domini) alla loro distribuzione *fisica*: il cablaggio dei componenti elettronici non dipende più dalle loro funzioni, bensì dalla loro effettiva posizione geometrica all'interno della vettura.

L'architettura zonale semplifica il layout di sensori, attuatori e cablaggi all'interno dell'auto, portando a una significativa riduzione del peso del fascio di cavi (*wiring harness*). Inoltre, la presenza di un minor numero complessivo di ECU si traduce in un'ulteriore riduzione di costi e ingombri.

Diversi controllori di gateway zonali (*zonal gateway controllers*) gestiscono la comunicazione con i sensori e gli attuatori di una specifica area fisica, aggregando e combinando i protocolli di rete richiesti da ciascuno dei dispositivi periferici.

I gateway zonali sono collegati tra loro e con la CPU centrale (o il computer di bordo ad alte prestazioni - *High-Performance Computer*) attraverso una **dorsale Ethernet** (*Ethernet backbone*).



CAN XL non ha una velocità fissa: con i **transceiver SIC XL** è **altamente flessibile**. Se i progettisti mantengono i cablaggi puliti (lineari o daisy chain con derivazioni minime), si può spingere stabilmente il bus fino a **20 Mbit/s**, superando di 4 volte il limite tipico del CAN FD (5 Mbit/s) e offrendo una validissima alternativa economica ad Automotive Ethernet per la distribuzione locale di pacchetti pesanti.

In sintesi : proprietà di CAN XL

Bitrate: Il protocollo gestisce due anime temporali distinte, ottimizzando l'affidabilità nei momenti critici e la velocità nel trasporto dati:

- **Fase di Arbitrato (Arbitration Phase):** Limitata a $\leq 1 \text{ Mbit/s}$. È la fase lenta e conservativa, identica al CAN classico, che serve a gestire le collisioni sul bus in totale sicurezza.
- **Fase Dati (Data Phase):** Spazia da **1 a 20 Mbit/s**. La velocità esatta viene configurata dall'utente in base al *trade-off* (compromesso) con la topologia di rete scelta (come visto con i transceiver SIC XL).

Lunghezza del Campo Dati (Data Field length)

- **Estensione (Range):** Da **1 a 2048 byte**, gestibile con granularità del singolo byte. Rispetto agli 8 byte del CAN classico e ai 64 byte del CAN FD, si tratta di un salto generazionale enorme.
- **Cosa abilita :** Il supporto alle applicazioni CAN legacy.
 - L'**Ethernet frame tunneling**, che permette di incapsulare interi pacchetti Ethernet portando l'architettura IP direttamente sul nodo terminale ("*IP to the edge*").

Nuove Funzionalità: i tre pilastri che avvicinano il CAN XL alle logiche delle reti Ethernet:

- **Reti CAN Virtuali:** Abilitate tramite il campo **VCID** (*Virtual CAN Network ID*), per segmentare logicamente il traffico sullo stesso bus fisico (VLAN).
- **Indicazione del contenuto del payload:** Gestita tramite il campo **SDT** (*Service Data Unit Type*). La specifica **CiA611-1** standardizza i valori numerici dell'SDT (es. il valore *0x05* visto per indicare che il payload contiene un frame Ethernet).
- **Acceptance Field**, indirizzamento sia source che fisico, e la presenza di **due potenti algoritmi CRC** (il Preface CRC per l'header e il Frame CRC a 32 bit per l'intero pacchetto) che garantiscono un'altissima protezione contro gli errori hardware sul bus.

Il Sistema di Gestione della Batteria (BMS) e il Ruolo del Bus CAN

Il **Battery Management System (BMS)** è un sistema elettronico che interviene per ottimizzare la batteria delle auto elettriche, tenendo conto dei requisiti di sicurezza e di fattori quali lo stato di carica e le variazioni di temperatura della batteria, con l'obiettivo di rallentare la riduzione di efficienza dei componenti e rilevare eventuali segnali negativi nel funzionamento del veicolo. In un BMS ad alte prestazioni, la comunicazione è importante tanto quanto la chimica delle celle, la regolazione termica o l'elettronica di potenza. Nei moderni veicoli elettrici e ibridi, il **bus CAN** è diventato una dorsale di comunicazione fondamentale, in grado di abilitare uno scambio dati affidabile e ad alta velocità tra i componenti della batteria e i controllori del gruppo motopropulsore (*powertrain*). Il CAN viene utilizzato per supportare le comunicazioni che consentono al BMS di coordinare i calcoli dello **stato di carica (SoC - State of Charge)**, il rilevamento di anomalie di sicurezza o la distribuzione della potenza alla trasmissione elettrica.

In particolare, il CAN viene impiegato come livello di comunicazione per connettere:

- Le unità di monitoraggio delle celle (**CMU - Cell Monitoring Units**),
- I controllori della batteria (*Battery Controllers*),
- I controllori del motore (*Motor Controllers*),
- Le unità di controllo del veicolo (**VCU - Vehicle Control Units**),
- I sistemi di ricarica.

La sua scalabilità, resilienza e capacità diagnostica lo rendono ideale per gli ambienti della mobilità. La mobilità elettrificata richiede infrastrutture di comunicazione che garantiscano

prestazioni, isolamento dei guasti (*fault isolation*) e sicurezza. Il bus CAN fornisce esattamente questo; per tale motivo, le architetture basate su CAN supportano:

- Motocicli elettrici, Veicoli elettrici e ibridi,
- Trattori elettrici, Autobus e veicoli commerciali elettrici,
- Imbarcazioni elettriche.

Nel Dettaglio, il CAN Supporta il BMS Fornendo:

1. Real-Time Safety Monitoring

Il CAN consente la trasmissione rapida dei parametri di tensione, corrente e temperatura, garantendo in questo modo che condizioni di guasto come la sovratemperatura o la sovratensione vengano rilevate istantaneamente, abilitando immediate risposte di protezione del sistema.

2. Coordinamento con i Componenti del Gruppo Motopropulsore (*Powertrain*)

La comunicazione CAN assicura che il BMS, il controllore del motore, la VCU (Vehicle Control Unit) e il caricabatterie funzionino come un unico sistema coordinato. Questa sincronizzazione è essenziale nei trattori elettrici, negli autobus elettrici e nelle imbarcazioni elettriche, contesti in cui sono richiesti un controllo preciso del carico e un'elevata stabilità termica.

3. Diagnostica Avanzata e Manutenzione (*Enhanced Diagnostics and Maintenance*)

Il CAN permette una registrazione strutturata degli errori (*error logging*) e della diagnostica, migliorando l'efficienza degli interventi di manutenzione e la tracciabilità complessiva del sistema.

4. Compatibilità con l'Infrastruttura di Ricarica

La ricarica rapida (*fast-charging*) e i sistemi di ricarica a bordo (*onboard charging*) si affidano al CAN per scambiare parametri chiave quali la corrente massima di carica, i limiti di temperatura e l'identificazione della batteria. Ciò consente cicli di ricarica sicuri ed efficienti, specialmente nei veicoli elettrici e ibridi ad alta capacità.

TTCAN (Time-Triggered CAN)

Il TTCAN è stato introdotto da Bosch nel 1999 per rendere il bus CAN più idoneo alle nuove esigenze delle applicazioni automotive, ovvero:

- **Requisiti temporali stringenti** (*tight timing requirements*).
- **Comportamento rigorosamente deterministico** (*strictly deterministic behavior*).

Il TTCAN permette di stabilire con esattezza l'istante temporale in cui i messaggi critici per la sicurezza (*safety-critical*) verranno inviati, indipendentemente dal carico complessivo della rete.

Il TTCAN mira a:

- **Ridurre il jitter di latenza** (la variabilità del ritardo di trasmissione).
- **Garantire un modello di comunicazione deterministico** sul bus.
- **Mantenere la retrocompatibilità** con i dispositivi CAN esistenti.
- **Supportare il traffico misto**, combinando messaggi gestiti a eventi (*event-triggered*, ad esempio una variazione di temperatura nel meccanismo di raffreddamento) e messaggi gestiti dal tempo (*time-triggered*, ad esempio lo stato dei freni).

Posizionamento

Il protocollo TTCAN (standardizzato nella norma **ISO 11898-4**) è un protocollo di livello superiore posizionato al di sopra del protocollo CAN tradizionale, il quale rimane totalmente invariato. Questo implica che il TTCAN condivide con il CAN classico:

- **Lo stesso formato di frame** (nessun overhead o bit aggiuntivo introdotto nell'intestazione).
- **La stessa velocità dati** (ovvero una velocità di trasmissione che non supera **1 Mbit/s**).

Il TTCAN nasce per consentire una transizione graduale dal CAN verso il protocollo **FlexRay**. Ad oggi, ha avuto un impatto limitato sulle vetture di produzione di serie.

Il CAN ha un problema: non può azzerare il jitter perché si basa sempre su CSMA non preemptive, una frame può essere bloccata da una a priorità più bassa.

Il TTCAN è un approccio ibrido che impone una struttura **TDMA** (Time Division Multiple Access) sui servizi forniti dal livello di collegamento dati (*Data Link layer*) del CAN, il quale è nativamente basato su un meccanismo **CSMA** (Carrier Sense Multiple Access).

La comunicazione *time-triggered* è guidata dal **Messaggio di Riferimento** (*Reference Message*) inviato dal **Time Master**.

Il TTCAN è un sistema *distribuito*: il ruolo del Time Master è limitato esclusivamente alla sincronizzazione temporale, quindi non è master/slave. Il Messaggio di Riferimento possiede un ID unico e noto a priori a tutti i nodi della rete.

La corretta ricezione di un Messaggio di Riferimento sincronizza l'unità di tempo locale di ciascun nodo, ovvero un contatore interno (*local counter*).

Il nodo inizializza un temporizzatore (*timer*) che registra il progredire del tempo. Dopo aver ricevuto il Messaggio di Riferimento, i singoli nodi partecipanti alla rete TTCAN sanno esplicitamente, per via della loro configurazione, in quale esatto momento devono inviare i propri frame. In altri termini, il Messaggio di Riferimento generato dal Time Master della rete dà il via a una **sequenza predeterminata di trasmissioni**.

Il meccanismo di arbitration nativo del CAN viene comunque mantenuto e preservato all'interno di speciali finestre temporali dedicate, chiamate *Arbitration Windows*.

Basic Cycle e Matrix Cycle

La trasmissione dei dati è strutturata come una sequenza di *Basic Cycles*. I Basic Cycle rappresentano l'intervallo di tempo che intercorre tra due Messaggi di Riferimento (*Reference Messages*) consecutivi.

Ogni Ciclo Base inizia con il Messaggio di Riferimento inviato da un *Time Master*, seguito da un numero fisso di finestre temporali (*time windows*) configurate in **modalità offline** (in fase di progettazione).

Tutti i **Cicli Base** hanno la stessa identica durata temporale, ma possono differire tra loro per il contenuto e/o per la lunghezza dei frame inviati al loro interno.

Il **Matrix Cycle** comprende molteplici cicli base (fino a un massimo di **64**) e viene ripetuto indefinitamente fino allo spegnimento del sistema. Rappresenta la periodicità complessiva di trasmissione di una rete TTCAN. Viene chiamata matrice perché ogni blocco di una colonna ha una grandezza prestabilita.

Finestre temporali in TTCAN

All'interno di un Ciclo Base del TTCAN, il tempo viene suddiviso in tre diversi tipi di finestre:

Exclusive windows

Sono riservate in **modo esclusivo e statico** a un messaggio predefinito. Non vi è alcuna contesa o concorrenza per l'accesso alla rete CAN, il che consente l'invio di dati critici per la sicurezza (*safety-critical*) in modo totalmente deterministico e privo di jitter.

La ritrasmissione automatica dei messaggi è disabilitata (in caso di errore), garantendo così che i messaggi successivi pianificati nelle finestre esclusive non subiscano alcun tipo di ritardo o slittamento temporale. Il protocollo preferisce sacrificare un singolo dato corrotto pur di preservare la **regolarità della rete** evitando di congestionare la rete con ritrasmissioni inutili. Può ritentare la trasmissione in arbitration.

Arbitrating windows

Sono assegnate a **più di un messaggio** e la contesa sul bus viene gestita e risolta secondo il classico meccanismo di arbitrato bit a bit del CAN standard, CSMA/BA. I messaggi pianificati che hanno subito una corruzione all'interno delle loro finestre esclusive hanno la possibilità di essere ritrasmessi all'interno di una finestra di arbitration.

Un messaggio può iniziare la sua fase di arbitrato **solo se la finestra ha ancora tempo sufficiente per contenere l'intera trasmissione**, prima del termine della finestra di arbitrato stessa. Se una finestra di arbitrato sta per scadere e mancano solo pochi microsecondi, il controller hardware vieta categoricamente l'inizio di una nuova trasmissione, lasciando la fine della finestra "vuota" piuttosto che rischiare di sfiorare nel blocco di tempo successivo.

Le ritrasmissioni automatiche a catena non sono consentite all'interno delle finestre di arbitrato. Il nodo dovrà semplicemente attendere il **Ciclo Base successivo** (o la successiva finestra di arbitrato programmata nel Ciclo Matrice) per fare un nuovo tentativo.

In sintesi, donano flessibilità alla rete per gestire task event-triggered senza sprecare banda.

Finestre Libere (Free windows)

Sono finestre vuote, **riservate per future estensioni** della rete TTCAN. In fase di evoluzione del sistema, possono essere convertite in finestre di arbitrato o in finestre esclusive per ospitare e integrare **nuovi nodi**, oppure per fornire maggiore larghezza di banda ai nodi già esistenti sulla rete.

Cycle Time e Time Marks

La conoscenza richiesta al singolo nodo è ridotta al minimo. Un nodo non ha bisogno di conoscere l'intera pianificazione del traffico (traffic schedule) di tutta la rete, ma deve conoscere esclusivamente le proprie finestre di trasmissione e ricezione. Ciò permette di semplificare hardware.

I **Time Marks** sono definiti all'interno di ciascun controllore di rete come attivatori (*triggers*) di trasmissione o di ricezione, utilizzati rispettivamente per l'invio dei messaggi e per la convalida della loro ricezione.

Tx_Triggers (Transmission triggers)

- Controllano il rilascio dei messaggi durante il corso del Ciclo Matrice (*Matrix Cycle*).
- Vincolano messaggi specifici alle rispettive posizioni di trasmissione stabilite nella pianificazione del Ciclo Matrice.
- Rimangono attivi per un intervallo di tempo massimo denominato ***Tx_Enable Window***.
- È solo ed esclusivamente all'interno di questa finestra temporale che la trasmissione di un messaggio può essere avviata, rispettando le normali regole del CAN standard.

La ***Tx_Enable Window*** è un sotto-intervallo temporale cruciale: se il bus è temporaneamente occupato (ad esempio perché un altro nodo sta completando un arbitrato) e il nodo non riesce ad avviare il suo frame prima che la ***Tx_Enable Window*** si chiuda, la trasmissione per quel ciclo viene **annullata**. Questo impedisce rigorosamente al messaggio di "sforare" e invadere la finestra temporale del nodo successivo.

Rx_Triggers

- Indicano un preciso istante temporale entro il quale un determinato messaggio avrebbe dovuto essere ricevuto dal nodo.
- Vengono impostati (attivati) all'inizio del rispettivo Ciclo Base (*Basic Cycle*) e successivamente azzerati (disattivati) se il messaggio specificato viene ricevuto puntualmente entro i tempi previsti.

Livelli di sincronizzazione

Nel protocollo TTCAN sono possibili due diversi livelli di implementazione per la sincronizzazione del tempo: **Level 1** e **Level 2**.

Livello 1

- Supporta le sole funzionalità di sincronizzazione strettamente necessarie a realizzare la **pianificazione dei messaggi *time-triggered scheduling***.
- Garantisce che tutti i nodi della rete siano allineati rispetto all'inizio del **Ciclo Base**.
- Ogni volta che viene ricevuto un Reference Message i clock interni dei nodi vengono semplicemente azzerati.

Livello 2

- All'interno del Reference Message viene incorporato un ***timestamp***, che rappresenta la visione del tempo da parte del *Time Master* nell'esatto istante di trasmissione del **1° bit** del Messaggio di Riferimento stesso.
- Al momento della ricezione di due Messaggi di Riferimento consecutivi, diventa possibile una sincronizzazione del tempo globale con una **precisione nell'ordine del singolo microsecondo (1 µs)**.
- Oltre a riportare a zero i contatori, ogni nodo corregge attivamente la ***velocità di scorrimento del tempo*** di ogni singolo nodo per farla combaciare al microsecondo con quella del Time Master.
- Il Livello 2 rappresenta un'estensione del Livello 1 e fornisce in aggiunta una **base temporale globale ad alta precisione**, direttamente correlata al **secondo fisico**.

Per riassumere la differenza: mentre il **Livello 1** si preoccupa solo che i nodi inizino a contare il tempo "insieme" per non far collidere i frame nelle finestre, il **Livello 2** introduce una **calibrazione** fine basata sulla frequenza **dell'oscillatore** del Time Master (calcolando il *drift*).

In questo modo, l'orologio interno di ogni nodo non solo è sincronizzato sull'inizio del ciclo, ma scorre alla stessa identica velocità del secondo fisico.

Gestione del master failure

Nel TTCAN, il fallimento del Time Master è uno scenario critico che viene gestito in modo nativo dal protocollo attraverso un meccanismo di **ridondanza a tolleranza d'errore**. Se il Time Master principale smette di funzionare (ad esempio per un guasto hardware o un problema di alimentazione), la rete non si blocca, ma si attiva una procedura di **subentro (Master Handover)**.

In fase di progettazione della rete (*offline configuration*), non viene definito un solo Time Master, ma viene creata una gerarchia. Abbiamo:

- **L'Active Time Master:** che sta attualmente orchestrando la rete inviando i *Reference Message*.
- **I Potential Time Masters:** Altri nodi della rete (fino a un massimo di 7 nel protocollo) che possiedono la capacità hardware e la configurazione per fare da master, ma che al momento rimangono in ascolto come nodi normali.

Tutti i nodi della rete, compresi i Master Potenziali, hanno un temporizzatore interno che si resetta ogni volta che sul bus passa il *Reference Message*. Se il Time Master attivo fallisce e non invia il messaggio all'inizio del Ciclo Base:

1. Il tempo scorre oltre il limite massimo previsto per l'inizio del ciclo.
2. Nei Master Potenziali scatta un **time-out di sincronizzazione**.
3. La rete capisce istantaneamente che il Master Attivo è "morto".

A questo punto, i Master Potenziali cercano di prendere il controllo del bus per diventarne il nuovo leader. Il nuovo Master viene scelto con l'**arbitrato basato sull'ID**.

Ciascun Master Potenziale prova a inviare sul bus un *Reference Message* speciale (chiamato messaggio di subentro). Ognuno di questi messaggi ha un ID CAN differente, associato alla priorità del master definita a tavolino (es. Master Potenziale 1 ha priorità più alta di Master Potenziale 2). Grazie all'arbitrato bit a bit del CAN, il Master Potenziale con l'ID più prioritario vince la contesa senza distruggere i frame sul bus.

Il Master Potenziale che ha vinto l'arbitrato diventa ufficialmente il **nuovo Active Time Master**. Tutti gli altri nodi della rete ricevono questo nuovo *Reference Message*, azzerano i loro contatori locali e si sincronizzano sulla timeline dettata dal nuovo leader. La comunicazione *time-triggered* riprende a scorrere regolarmente, garantendo la continuità dei sistemi di sicurezza del veicolo con un'interruzione minima (pari a un solo ciclo base mancato o ritardato). Questo meccanismo assicura che il protocollo non abbia un *Single Point of Failure* (un singolo punto di vulnerabilità) a livello di sincronizzazione.

Infine TTCAN non ha avuto grande diffusione sia perché offriva 1 Mbps, poi è arrivato FlexRay e a causa della complessità di configurazione offline.

Flex Ray

Nel 1998, BMW e Daimler-Chrysler riscontrarono che nessuna delle reti automobilistiche allora disponibili sul mercato era in grado di soddisfare i requisiti dei sistemi automotive di nuova generazione per applicazioni di controllo critiche per la sicurezza (*safety-critical*, all'epoca orientate in particolar modo verso le tecnologie **X-by-Wire**, come lo sterzo o la frenata elettronici), che erano:

- **Accesso al mezzo deterministico.**
- **Scalabilità.**
- **Alta velocità.**
- **Canali di comunicazione ridondanti** (Che can non aveva)
- **Elevata robustezza** in caso di guasti transitori.
- **Accordo distribuito e tollerante ai guasti** sullo stato del protocollo.
- **Rilevamento esteso degli errori** e relativa segnalazione all'applicazione.
- **Capacità di sincronizzazione rigida in tempo reale.**

Tech

- **10 Mbit/s** (scalabile fino a **20 Mbit/s** se si utilizzano entrambi i canali).
- **0 a 256 byte** di dati utili per singolo frame.
- Combina il **TDMA** statico con il **TDMA Flessibile** (basato sul meccanismo del *mini slotting* per il traffico dinamico).
- **UTP - Unshielded Twisted Pair**, viene utilizzato 1 doppino se si sfrutta un solo canale, oppure 2 doppiini se vengono impiegati entrambi i canali.

Domini Funzionali Target

- **Progettato specificamente per sistemi X-by-Wire** (es. sistemi di guida, sterzata o frenata interamente elettronici, privi di collegamento meccanico o idraulico diretto).
- Ideale per le funzioni del telaio (*chassis*) e del gruppo motopropulsore (*powertrain*) **critiche per la sicurezza**, che richiedono determinismo temporale, velocità più elevate rispetto al CAN e ridondanza hardware.
- **Esempi applicativi:** Sistemi *Powertrain* ad alte prestazioni, sistemi di sicurezza attiva (*Drive-by-wire*, sospensioni attive, controllo adattivo della velocità di crociera / *Adaptive Cruise Control*).

Vantaggi

- **Prestazioni e Velocità:** Comunicazione rigorosamente in tempo reale ad alta velocità se confrontata con i protocolli CAN e CAN FD.
- **Supporto alla Sicurezza Critica:** Progettato per comunicazioni **safety-critical** grazie alla ridondanza dei canali e alla presenza del **Bus Guardian** (un dispositivo hardware che impedisce a un nodo guasto di trasmettere fuori dal suo slot temporale).
- **Power Management:** Supporto integrato per modalità a basso consumo (*sleep modes*) e tempi di risveglio estremamente rapidi (*fast wake-up*).
- **Topologia Estremamente Flessibile:** Supporta configurazioni di rete a bus, a stella e a stella multipla (*multistar*).

Bus Multi-drop: Il FlexRay viene comunemente utilizzato in una semplice topologia a **bus multi-drop**, caratterizzata da un singolo cavo di rete principale che collega insieme molteplici ECU. Si tratta della stessa topologia impiegata da CAN e LIN ed è ben nota ai costruttori, il che l'ha resa una scelta molto diffusa nei veicoli FlexRay di prima generazione.



Ciascuna ECU può collegarsi in derivazione (*branch*) fino a una breve distanza dalla dorsale principale (*trunk*) del bus. Le estremità della rete sono dotate di resistenze di terminazione installate per eliminare i problemi legati alle riflessioni del segnale.

Poiché il FlexRay opera ad alte frequenze (fino a 10 Mbit/s rispetto a 1 Mbit/s del CAN), i progettisti di reti FlexRay devono prestare molta attenzione a terminare e disporre correttamente le reti per evitare problemi di integrità del segnale. Il formato multi-drop si adatta inoltre molto bene ai cablaggi dei veicoli, che condividono comunemente un layout di tipo simile, semplificando l'installazione e riducendo il cablaggio in tutto il veicolo.

Rete a Stella: costituite da singoli collegamenti che convergono verso un nodo attivo centrale. Questo nodo è funzionalmente simile a un hub presente nelle reti Ethernet per PC. La configurazione a stella attiva consente di far funzionare le reti FlexRay su distanze più lunghe o di segmentare la rete in modo tale da renderla più **affidabile** in caso di guasto a una porzione della stessa. Se uno dei rami della stella viene tagliato o va in cortocircuito, gli altri rami continuano a funzionare.

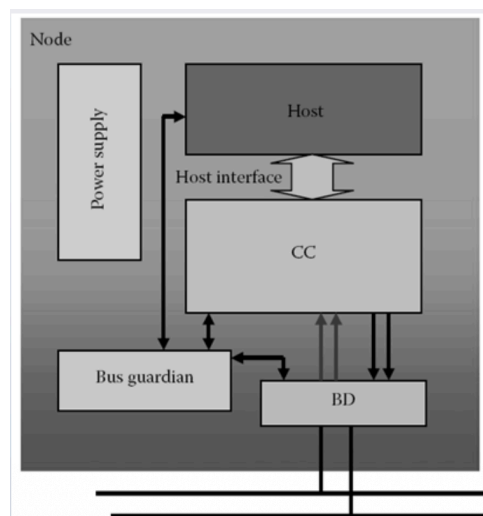


Poiché i lunghi tratti di cavo tendono a condurre un maggior rumore ambientale (come le emissioni elettromagnetiche generate dai grandi motori elettrici), l'utilizzo di più rami riduce la quantità di cavo esposto per un singolo segmento e può contribuire ad aumentare l'immunità al rumore.

Rete Ibrida : Le topologie a bus e a stella possono essere combinate tra loro per formare una **topologia ibrida**. Le future reti FlexRay saranno probabilmente costituite da reti ibride per sfruttare contemporaneamente la facilità d'uso e i vantaggi in termini di costo della topologia a bus, applicando al contempo le prestazioni e l'affidabilità delle reti a stella laddove strettamente necessario all'interno del veicolo.



Nodo FlexRay



- Una rete FlexRay è costituita da un insieme di ECU dotate di CC (**Communication Controllers**) integrati.
- Ciascun CC connette l'ECU a uno o più canali di comunicazione tramite una porta di comunicazione.
- Questa, a sua volta, si collega al *Bus Driver* che si interfaccia direttamente con il livello fisico del canale (un singolo cavo bus, oppure una stella attiva o passiva).

Bus Guardian

Un dispositivo **hardware** di protezione chiamato *Bus Guardian* monitora l'accesso TDMA del controllore per finalità di contenimento degli errori (*error containment*).

Il *Bus Guardian* è un meccanismo operante a **livello fisico**; si tratta di un componente dotato di un **oscillatore (orologio) indipendente**, strutturato in modo tale che un eventuale guasto nel controllore di comunicazione non possa influenzare il guardiano, e viceversa.

Il BG implementa il contenimento degli errori proteggendo il canale dalle interferenze causate da trasmissioni non allineate con la pianificazione temporale della rete (*network communication schedule*).

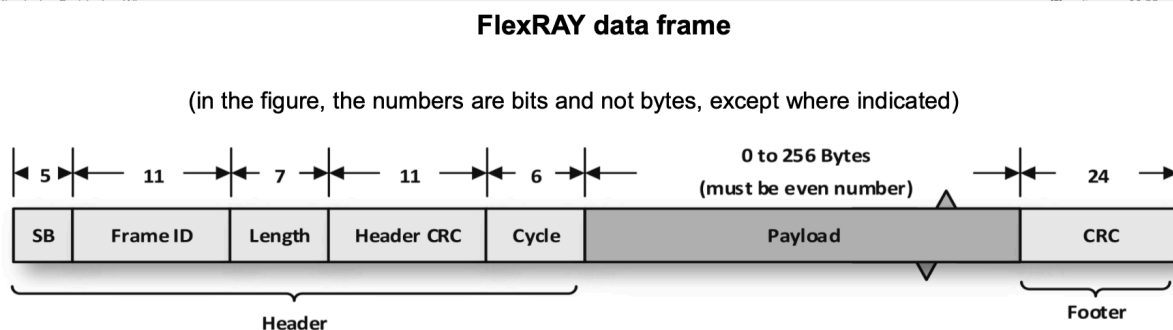
In pratica, il Bus Guardian agisce come un interruttore di sicurezza. Conoscendo la pianificazione statica dei tempi (TDMA), esso apre la linea di trasmissione per la propria ECU **solo ed esclusivamente** durante lo slot temporale assegnatole. Se il controllore di comunicazione (CC) dovesse "impazzire" a causa di un guasto e tentasse di trasmettere continuamente (il celebre problema del nodo monopolizzatore o "*Babbling Idiot*"), il Bus Guardian bloccherebbe fisicamente il segnale (disabilita il bus driver), salvando la comunicazione del resto del veicolo.

Ridondanze

Il FlexRay supporta il funzionamento di un Controllore di Comunicazione (CC) sia con un singolo canale, sia con **canali di comunicazione ridondanti** (doppio canale). I controllori possono essere configurati secondo due logiche principali di protezione:

- **Ridondanza Spaziale (Trasmissione in simultanea):**
 - Per trasmettere i dati in modo ridondante su **due canali contemporaneamente**.
 - Questa configurazione consente di mascherare un guasto temporaneo o l'interruzione fisica di uno dei due canali.
- **Ridondanza Temporale (Trasmissione differita):**
 - Per trasmettere i dati sui medesimi canali applicando un **ritardo temporale prefissato** tra le trasmissioni ridondanti.
 - Questo approccio consente di tollerare i guasti transitori (come i disturbi elettromagnetici temporanei) che potrebbero colpire entrambi i canali nello stesso istante.

Ottimizzazione della Banda : Se la specifica applicazione del veicolo non richiede l'uso di due canali indipendenti per scopi di tolleranza ai guasti (ovvero non serve la ridondanza), i due canali possono essere sfruttati in modo indipendente per trasferire dati diversi. In questo modo si ottiene il **raddoppio della larghezza di banda complessiva della rete** (portando la velocità effettiva da 10 Mbit/s a **20 Mbit/s**).

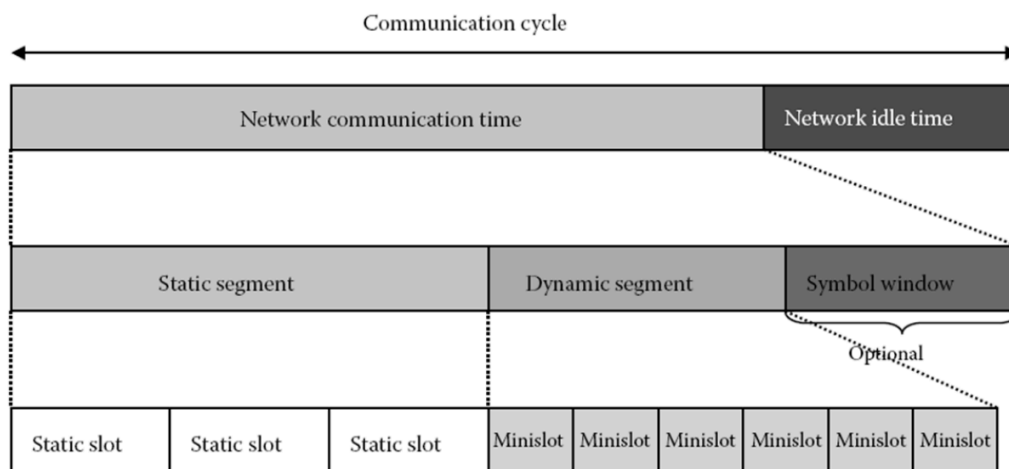


Communication Cycle

Il controllo di accesso al mezzo (MAC) del FlexRay adotta un approccio ibrido che combina il meccanismo **TDMA** e il **TDMA Flessibile**. L'elemento base dello schema TDMA è il *communication cycle* che quattro elementi specifici:

- Un **segmento statico** (*static segment*),
- Un **segmento dinamico** (*dynamic segment*),
- Una **finestra dei simboli** (*symbol window*),
- Un tempo di inattività della rete (**NIT** - *Network Idle Time*).

I cicli di comunicazione vengono eseguiti periodicamente dall'avvio (*start-up*) fino allo spegnimento (*shutdown*) del sistema. I valori tipici della durata di un ciclo variano **da 1 a 5 ms**.



Il **segmento statico** è strutturato in slot temporali a durata fissa (*fixed duration*).

La durata e il numero complessivo degli slot sono determinati dai parametri di configurazione dei controllori FlexRay. Questi parametri di configurazione devono essere rigorosamente **identici in tutti i controllori** appartenenti alla medesima rete.

Ogni singolo slot è di proprietà esclusiva di un unico controllore di comunicazione (CC) FlexRay per la trasmissione di un frame. Tale proprietà esclusiva si riferisce a **un solo canale**. Sul secondo canale, all'interno del medesimo slot temporale, la trasmissione può essere effettuata dallo stesso controllore oppure da un controllore differente.

Un singolo nodo può possedere e avere assegnati molteplici slot all'interno dello stesso segmento statico. L'associazione e l'identificazione di quale controllore sia autorizzato a trasmettere all'interno di uno specifico slot sono anch'esse stabilite tramite i parametri di configurazione dei controllori FlexRay.

Se consideriamo lo **Slot 1**:

- **Sul Canale A:** Può essere assegnato alla ECU dei freni per inviare dati critici.

- **Sul Canale B:** Nello stesso identico istante (Slot 1), la ECU dei freni può inviare lo *stesso identico frame* (ottenendo la ridondanza spaziale per mascherare i guasti).
- **Alternativa sul Canale B:** Oppure, lo Slot 1 sul Canale B può essere assegnato alla centralina dello sterzo per inviare dati completamente diversi. In questo modo si ottimizza l'efficienza temporale, permettendo a due nodi differenti di parlare nello stesso slot, sfruttando le due "autostrade" fisiche separate.

I controllori in ricezione non possiedono alcuna conoscenza diretta su quale sia il nodo mittente. Essi sono configurati semplicemente per ricevere in uno specifico slot di uno specifico ciclo. Il contenuto e il significato di un frame sono determinati univocamente dalla sua **posizione (slot)** all'interno del ciclo di comunicazione (time addressing).

Il **Frame ID** corrisponde esattamente allo *Slot ID* all'interno del quale il frame deve essere trasmesso (il primo frame ha ID = 0, il secondo ha ID = 1, e così via).

La combinazione di **Frame ID** e **Numero di Ciclo** identifica in modo univoco il frame all'interno della rete.

Il formato del frame è identico sia per il segmento statico che per quello dinamico. Tuttavia, nel **segmento statico la dimensione del frame è unica e fissa** (tutti i frame hanno la stessa lunghezza), mentre nel segmento dinamico sono ammesse lunghezze differenti dei frame.

Il segmento statico garantisce una comunicazione **deterministica** al traffico gestito dal tempo (*time-triggered*), ma richiede una prenotazione fissa e rigida della larghezza di banda.

Determinismo vs. Utilizzo della banda. Se un nodo non ha dati da trasmettere in quel determinato momento, lo slot ad esso assegnato **rimane vuoto e inutilizzato** (*idle*).

Siamo di fronte a un cambio di paradigma radicale rispetto al CAN. Nel CAN l'ID diceva "*sono il sensore X*". Nel segmento statico del FlexRay, l'ID dice semplicemente "*sono lo slot numero 5*". Chi riceve sa cosa contiene quel messaggio solo perché sa cosa è stato pianificato nello slot 5 a tavolino. Se la centralina titolare dello slot 5 è spenta o non ha dati, quel tempo scorre vuoto sul bus (silenzio elettrico) e nessun altro può usarlo. È il prezzo da pagare per avere la certezza matematica che lo slot 6 parta esattamente in orario.

Il **segmento dinamico** ha una durata complessiva fissa ed è suddiviso in un numero fisso di **minislot** (chiamati anche *macrotick*) di uguale lunghezza (il cui valore tipico è di **1 microsecondo**).

Un minislot ha una lunghezza fissa che è significativamente più breve rispetto a quella di uno slot statico. La lunghezza nativa di un minislot non è sufficiente a contenere un frame; tuttavia, se un nodo ha un frame da trasmettere, **il minislot si espande** fino a diventare uno slot reale per poter contenere il frame. Di conseguenza, nel segmento dinamico i frame possono avere lunghezze differenti. Un minislot funge da segnaposto (***placeholder***) che definisce

esclusivamente un potenziale istante di inizio di una trasmissione, ovvero l'opportunità concessa a un determinato nodo di avviare la trasmissione nel segmento dinamico.

Un nodo possiede un determinato numero di minislot (non necessariamente consecutivi) e può iniziare a trasmettere un frame all'interno di ciascuno dei **minislot di sua proprietà**. All'interno di un singolo ciclo di comunicazione, ogni minislot è di proprietà esclusiva di un unico controllore FlexRay. È possibile effettuare la **multiplazione** dei minislot in cicli differenti (ovvero, nodi diversi possono inviare frame nello stesso identico minislot, ma in cicli diversi).

Tutti i controllori presenti sulla rete mantengono una visione coerente e allineata sul minislot corrente attraverso un ***minislot counter***.

Se un controllore desidera trasmettere, nel momento in cui arriva il minislot di sua proprietà, accede al mezzo di comunicazione e avvia la trasmissione. Da quel momento si verificano le seguenti azioni:

- **Sospensione del conteggio:** Gli altri controllori sulla rete rilevano l'inizio della trasmissione sul bus e **interrompono il conteggio** dei minislot.
- **Espansione temporale:** Il minislot viene "**espanso**" fino a diventare uno slot reale, della lunghezza esattamente necessaria e sufficiente a ospitare la trasmissione dell'intero frame.
- **Ripresa del conteggio:** Il conteggio dei minislot riprende regolarmente a scorrere subito dopo il termine della trasmissione del frame (non appena il bus torna silente).

Se il proprietario di un minislot non ha dati da trasmettere, rimane semplicemente in silenzio. In questo caso, il minislot non viene espanso e il conteggio degli slot prosegue immediatamente.

Non viene consumata alcuna larghezza di banda aggiuntiva oltre alla durata microscopica del minislot stesso (quell'unico microsecondo di "pausa").

Questo protocollo (il **Flexible TDMA**) risolve il grande difetto del segmento statico (e del TTCAN), dove lo slot vuoto pesava sull'economia della rete per tutta la sua intera durata. Qui, se il nodo è silente, la rete "perde" solo 1 microsecondo prima di dare la parola al nodo successivo.

Tuttavia, l'espansione di un minislot (dovuta alla trasmissione di un frame) consuma tempo utile, **riducendo di conseguenza il numero di minislot successivi effettivamente disponibili** all'interno dello stesso segmento dinamico. Il segmento dinamico implementa un meccanismo di accesso al mezzo che è:

Priority-driven: Minore è il numero associato al minislot, maggiore sarà la sua priorità di trasmissione. Di conseguenza, i nodi posizionati all'inizio del segmento hanno più garanzie di riuscire a trasmettere rispetto a quelli in fondo.

Guidato dalle richieste ed efficiente nella gestione della banda (*Demand-driven and bandwidth-efficient*): Un minislot viene espanso (e la relativa larghezza di banda viene

consumata) **se e solo se** il controllore di riferimento ha effettivamente dati pronti da trasmettere sul bus.

La durata del singolo minislot (quando non espanso) e la lunghezza complessiva dell'intero segmento dinamico sono **parametri di configurazione globali**, il che significa che devono essere impostati con lo stesso identico valore in tutti i nodi della rete.

Configurazione del Ciclo FlexRay

All'interno del ciclo di comunicazione del FlexRay:

- **Obbligatorio** : un **segmento statico minimo** e un **network idle time**.
- **Opzionale**: **segmento dinamico** e **symbol window**.

Sono possibili tre diverse configurazioni di rete:

- **Puramente Statica**: Contiene esclusivamente slot statici per la trasmissione dei dati. Il segmento statico deve essere composto da **almeno due slot** di proprietà di due controllori differenti (questo è necessario per permettere l'avvio e la sincronizzazione iniziale della rete tra almeno due nodi).
- **Mista**: Configurazione ibrida che prevede la presenza sia di un segmento statico (per i dati critici) sia di un segmento dinamico (per il traffico impulsivo o diagnostico). Nei tipici domini applicativi del settore automotive, le **configurazioni miste rappresentano la scelta dominante**.
- **Puramente Dinamica**: Tutta la larghezza di banda utile viene assegnata al segmento dinamico. Anche questa configurazione richiede obbligatoriamente un cosiddetto **segmento statico degradato** (*degraded static segment*), il quale deve contenere almeno **due slot statici** (sempre per garantire i meccanismi nativi di startup e sincronizzazione del protocollo).

Il FlexRay è in grado di raggiungere un utilizzo della larghezza di banda pari a circa il **70% nel caso migliore** (*best-case*). L'utilizzo medio (*average utilization*) della banda si attesta stabilmente intorno al **60%**.

Il segmento statico **non può azzerato completamente**, perché il FlexRay ha bisogno di almeno due slot statici perché i nodi deputati alla sincronizzazione (i *Sync Nodes*) devono avere una finestra temporale matematica e intoccabile per scambiarsi i frame di sincronizzazione del clock, senza i quali la rete non potrebbe nemmeno avviarsi.

Sincronizzazione:

- La durata minima di un **minislot** dipende dai parametri fisici della rete (ad esempio, il ritardo di propagazione) e dalla massima deviazione della frequenza degli oscillatori (*clock frequency deviation*) nei controllori.
- L'unità temporale più piccola all'interno della rete è il **macrotick**. I controllori FlexRay si sincronizzano tra loro e regolano i propri orologi locali in modo che il macrotick scocchi nello **stesso identico istante** su ogni nodo della rete. Poiché il macrotick è sincronizzato, anche tutti i dati che dipendono da esso risultano sincronizzati.
- **Valore tipico**: 1 macrotick = 1 ms.

La **symbol window** è uno slot temporale a durata fissa all'interno del quale possono essere trasmessi simboli speciali. Questi simboli non contengono normali frame dati, ma vengono utilizzati esclusivamente per scopi di **gestione e manutenzione della rete** (*network management*), come ad esempio le procedure di sveglia o di test del livello fisico).

Il **Network Idle Time** è una finestra temporale specifica del protocollo nella quale **non è pianificato alcun traffico dati** sui canali di comunicazione. I controllori di comunicazione (CC) utilizzano questa finestra di silenzio per eseguire l'**algoritmo di sincronizzazione degli orologi** e applicare la correzione dell'offset (*offset correction*). Durante il NIT non può essere garantito il funzionamento coerente del controllo di accesso al mezzo (MAC), motivo per cui è categoricamente **richiesto il silenzio elettrico** sul bus.

Poiché la durata del NIT deve essere sottratta alla larghezza di banda netta disponibile per i dati, questo intervallo viene mantenuto **il più breve possibile**. La lunghezza minima del NIT dipende in larga misura dalla deviazione massima accumulata tra gli orologi locali (*clock drift*) al termine di un intero ciclo di comunicazione. La durata del NIT è un parametro di configurazione globale che deve essere rigorosamente **identico in tutti i controllori** della rete.

Storia

A partire dal 2006, il FlexRay è stato impiegato su vetture di produzione da costruttori come BMW, Audi, Daimler e Volvo (ad esempio, la BMW Serie 5 arriva a implementare fino a 17 nodi FlexRay). Nonostante ciò, il FlexRay non ha ancora ottenuto un'ampia accettazione globale, poiché molti costruttori automobilistici si affidano tuttora a comunicazioni basate sul protocollo CAN (e le sue evoluzioni come il CAN FD).

Il protocollo offre una **flessibilità limitata** ed è estremamente difficile e complesso da configurare (a causa della pianificazione *offline* rigida dei tempi).

Lo sviluppo delle applicazioni **X-by-Wire** (critiche per la sicurezza) è evoluto molto più lentamente del previsto, risentendo fortemente della grande recessione che ha colpito le case automobilistiche nel 2008.

L'utilizzo del FlexRay oggi è limitato quasi esclusivamente ad applicazioni strettamente legate alla sicurezza dinamica del veicolo (es. powertrain ad alte prestazioni e sistemi di controllo del telaio/sospensioni).

Nel lungo periodo è difficile prevedere un futuro solido per il FlexRay. L'introduzione di **Automotive Ethernet** come dorsale di rete (*backbone*) interna al veicolo fornirà una larghezza di banda di gran lunga superiore e sarà in grado di estendersi su molteplici domini funzionali dell'auto (al contrario del FlexRay che, ad esempio, non è strutturalmente adatto a gestire i sistemi di *infotainment*).

Da un lato, il CAN FD ha dato nuova linfa al vecchio CAN, coprendo le esigenze intermedie senza stravolgere i tool di progettazione. Dall'altro, l'esplosione delle telecamere, dei radar per

la guida autonoma (ADAS) e dei display digitali ha richiesto fin da subito centinaia di Megabit (o Gigabit) al secondo. L'industria ha quindi preferito investire massicciamente sull'**Automotive Ethernet**, che offre una banda enormemente superiore, scalabilità e la possibilità di unificare l'intera architettura dell'auto sotto un unico standard derivato dal mondo IT.

MOST (Media Oriented Systems Transport)

Il MOST è una rete **multimediale** ad alta velocità progettata specificamente per soddisfare le esigenze di **elevata larghezza di banda** e **bassa latenza** tipiche delle applicazioni di *infotainment* e dei sistemi di telecamere per la visione circostante o di retromarcia (**ADAS**), requisiti che non potevano essere soddisfatti dalle altre reti automobilistiche dell'epoca.

Il MOST funge da dorsale di rete per la trasmissione in tempo reale di flussi audio/video e segnali di controllo, facilitando così l'integrazione tra l'infotainment, i sistemi di assistenza alla guida e le altre centraline del veicolo. Il MOST è in grado di connettere in rete fino a **64 dispositivi di bordo (ECU)** sfruttando una **topologia a anello** (*ring topology*).

Servizi Supportati: Navigazione GPS, integrazione smartphone, assistenti vocali, streaming audio/video, servizi di informazioni sul traffico e sul meteo, ecc.

Quando venne sviluppato il sistema MOST, i costruttori di veicoli avevano due richieste fondamentali riguardo alle funzionalità di un sistema di comunicazione dedicato al infotainment:

1. Un'architettura con **approccio orientato ai servizi**.
2. La capacità di trasmettere simultaneamente **dati in streaming, dati a pacchetto e informazioni di controllo**.

La prima richiesta ha portato allo sviluppo del framework MOST, all'interno del quale i **blocchi funzionali** (*Function Blocks*) rappresentano la parte essenziale:

- I blocchi funzionali incorporano tutte le proprietà e i metodi di un dispositivo MOST (ad esempio, un lettore CD) che sono necessari al controllo del dispositivo stesso.
- La comunicazione con i blocchi funzionali avviene tramite il protocollo applicativo e **non richiede l'indirizzo fisico** del dispositivo MOST. Ciò consente una progettazione semplice e rapida del dominio di infotainment, garantendo al contempo un elevato livello di astrazione.

La seconda richiesta è stata soddisfatta grazie alla specifica struttura del frame MOST, la quale:

- Trasporta i dati multimediali in modo rigorosamente **sincrono** (*streaming* audio/video).
- Può trasmettere una grande quantità di dati in modo **asincrono** (*packet data*, come i dati di navigazione o internet) senza influenzare o disturbare la trasmissione sincrona.
- Offre un **Canale di Controllo** (*Control Channel*) dedicato esclusivamente ai comandi di controllo e ai messaggi di stato.

Il vero punto di forza del MOST risiede nella sua struttura di trama (frame) rigidamente suddivisa. A differenza del CAN o dell'Ethernet tradizionale, dove i dati di controllo e i flussi multimediali competono per lo stesso canale, il MOST divide ogni singolo frame in aree dedicate e separate a livello logico e fisico. In questo modo, lo streaming di un film o della musica (canale sincrono) viaggia a temporizzazione costante e senza interruzioni, mentre il download di una mappa (canale asincrono) o la pressione del tasto del volume (canale di controllo) avvengono in parallelo senza causare scatti o lag nel flusso audio/video.

Storia

Verso la fine degli anni '90, l'esigenza di supportare complesse **applicazioni audio** all'interno delle vetture divenne urgente. Non solo i **CD** audio stavano sostituendo in modo irreversibile i supporti musicali analogici (come le musicassette), ma i clienti mostravano anche un interesse sempre maggiore verso i sistemi di navigazione e l'utilizzo dei telefoni cellulari a bordo.

All'interno dell'abitacolo, i differenti flussi audio generati da queste applicazioni dovevano essere coordinati tra loro e con i segnali acustici di avvertimento aggiuntivi provenienti dalle funzioni di assistenza alla guida, garantendo al contempo un'esperienza sonora ottimale.

Questa evoluzione richiedeva, prima di tutto, un incremento significativo della velocità di trasmissione dei dati (larghezza di banda). A quell'epoca, i sistemi ottici erano gli unici in grado di promettere la velocità di dati richiesta attraverso una soluzione conforme ai requisiti di compatibilità elettromagnetica (**EMC**) e con un livello di costi ragionevole.

L'attenzione dei progettisti si focalizzò sulla **Fibra Ottica Polimerica** (POF - *Polymeric Optical Fiber*), una soluzione meno costosa e più robusta rispetto alla Fibra Ottica di Vetro (GOF - *Glass Optical Fiber*) come mezzo di trasmissione.

La fibra ottica, trasmettendo impulsi di luce anziché corrente elettrica, risolve il problema alla radice: era **totalmente immune ai disturbi elettromagnetici (EMC)** e, grazie alla plastica (POF) anziché al vetro, era abbastanza flessibile e robusta da resistere alle vibrazioni e alle curve strette dei cablaggi flessibili dell'auto.

Nel 1998, BMW, (Harman) Becker, Daimler e OASIS fondarono la **MOST Cooperation (MOST Co)** con l'obiettivo di sviluppare il MOST come tecnologia di rete per applicazioni interne al veicolo e stabilirlo come standard industriale.

L'intento non era solo quello di avere a disposizione una nuova tecnologia per il livello fisico (PHY) con una velocità di dati più elevata, ma di disporre di un sistema che coprisse **tutti gli aspetti di rete, dal livello fisico fino al livello applicativo**, così da abilitare le complesse sequenze di controllo richieste dai casi d'uso desiderati.

L'industria automobilistica decise di cooperare con un fornitore forte che, nel ruolo di una sorta di "appaltatore generale", coordinasse la progettazione e lo sviluppo del nuovo sistema di rete, mentre i requisiti venivano forniti direttamente dai costruttori di auto.

Il partner scelto fu OASIS (successivamente integrata in SMSC, oggi parte di **Microchip**), in virtù della sua significativa esperienza nei sistemi di infotainment automobilistici e nei relativi requisiti. OASIS aveva inoltre già all'attivo una proposta tecnologica a supporto della trasmissione ottica.

Gli accordi prevedevano licenze esenti da royalty (*royalty-free*) tra i membri della MOST Cooperation per tutti gli sviluppi, **ad eccezione del Data Link Layer (DLL) e del livello fisico**

(PHY), la cui proprietà intellettuale apparteneva a OASIS (e quindi oggi a Microchip). Per licenziare la tecnologia DLL/PHY era necessario un accordo basato sul pagamento di royalty. La MOST Cooperation non forniva una specifica pubblica e indipendente su cui poter eseguire test di interoperabilità e conformità del Data Link Layer.

I potenziali altri concorrenti dell'epoca si trovarono di fronte al rischio di incompatibilità con la tecnologia del **fornitore dominante**, Microchip. Nessun'altra azienda colse l'opportunità di entrare in questo mercato, motivo per cui la tecnologia MOST viene spesso **associata a un monopolio**, essendoci un solo fornitore per l'hardware di riferimento (Microchip).

Il MOST ha debuttato ufficialmente a bordo della **BMW Serie 7 nel 2001**, all'interno del celebre sistema multimediale *iDrive*. Nel 2016 è stato avviato il progetto ISO 21806 per trasferire le specifiche del MOST, inclusa la specifica del Data Link Layer, all'interno degli standard internazionali ISO. La prima release ufficiale dello standard MOST è stata pubblicata come norma **ISO 21806-1:2020**.

A differenza del CAN (sviluppato da Bosch ma licenziato liberamente a decine di produttori di chip come NXP, STMicroelectronics o Texas Instruments), il cuore del MOST è rimasto blindato nelle mani di un solo costruttore hardware (Microchip). Per le case automobilistiche, dipendere da un unico fornitore globale per i chip di rete (*Single Source*) rappresenta un enorme rischio strategico ed economico. Questa chiusura commerciale, unita alla nascita di tecnologie aperte e concorrenti, ha spinto nel tempo i costruttori a guardare altrove, accelerando la transizione verso l'**Automotive Ethernet** (standard aperto e prodotto da moltissimi vendor indipendenti).

Il protocollo MOST si è evoluto nel tempo attraverso diverse generazioni, incrementando progressivamente la velocità di trasmissione (*bitrate*) e diversificando i supporti fisici:

- **MOST25: 25Mbit/s (POF);**
- **MOST50: 50 Mbit/s (UTP, POF);**
- **MOST150 150 Mbit/s (POF, Coassiale);**

Il protocollo ha implementato tre diverse tipologie di **cablaggio**, ciascuna con precisi compromessi ingegneristici:

Fibra Ottica Polimerica (POF - Polymeric Optical Fiber)

Vantaggi: Molto meno costosa e decisamente più robusta rispetto alla fibra ottica di vetro (GOF). Soddisfa nativamente i severi requisiti di compatibilità elettromagnetica (**EMC**), poiché non genera radiazioni di interferenza verso l'esterno ed è completamente immune ai disturbi elettromagnetici esterni.

Svantaggi: Più **fragile** rispetto a un comune cavo in rame UTP (se viene piegata oltre un certo raggio di curvatura critico, la luce si disperde e il canale smette di funzionare correttamente). Risulta complessivamente più **costosa** e più complessa da riparare o gestire in fase di assistenza (*service*).

Doppino Intrecciato Non Schermato (UTP - Unshielded Twisted Pair)

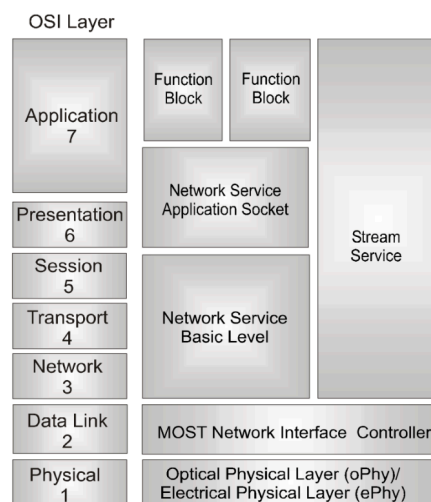
- **Vantaggi:** Più *economico*, decisamente più *flessibile* nei passaggi stretti del telaio e molto più facile da riparare/giuntare nelle officine rispetto alla fibra ottica.

Cavo Coassiale (Coaxial Cabling)

- **Vantaggi:** Presenta la caratteristica fondamentale di poter trasportare l'alimentazione elettrica (**power**) e il segnale dati ad altissima frequenza sul medesimo cavo, riducendo il numero di fili verso la centralina periferica.
- **Svantaggi:** Risulta più costoso e più difficile da mantenere o riparare rispetto al classico doppino in rame UTP.

Livelli MOST

A differenza di protocolli come CAN, LIN e FlexRay, che si limitano a definire esclusivamente i livelli inferiori (livello fisico e di collegamento), il MOST risponde a tutte le esigenze di una rete di infotainment di bordo all'interno di un unico sistema che **copre tutti i 7 livelli del modello ISO-OSI.**



L'architettura interna si suddivide in quattro macro-componenti logici e hardware:

- **Blocchi Funzionali**: Rappresentano l'interfaccia diretta con l'applicazione **software**. Essendo lo strato più alto con cui interagiscono i servizi del veicolo, appartengono interamente all' *Application Layer* (7).
- **Network Services**: Fungono da middleware di comunicazione posizionato tra il controllore di interfaccia di rete (NIC) e i blocchi funzionali (FBs). Questo strato software copre le funzionalità che vanno dal Livello 3 (Rete) fino a una parte del Livello 7 (Applicazione) del modello ISO-OSI.
- **Network Interface Controller (NIC / INIC)**: Questo componente hardware gestisce ed esegue tutte le funzioni native del **Data Link Layer (DLL)** (Livello 2 ISO-OSI).
 - **NIC** (*Network Interface Controller*) per lo standard MOST25.
 - **INIC** (*Intelligent Network Interface Controller*) per MOST50 e MOST150.

- **Optical Physical Layer** (oPhy), **Electrical Physical Layer** (ePhy).
- **Stream Services:** Sono servizi aggiuntivi verticali *integrati* nell'architettura, progettati specificamente per gestire la trasmissione sincrona dei flussi audio/video (A/V), garantendo *l'assenza di ritardi* o sfasamenti temporali durante la riproduzione multimediale.

Service Based

Il MOST è la prima tecnologia di rete interna al veicolo che supporta metodologie **service-based**, consentendo la richiesta di funzioni e servizi su richiesta (*on demand*) durante il funzionamento.

L'interfaccia verso l'utente del sistema MOST è l'**Interfaccia Uomo-Macchina (HMI)**, che presenta le funzioni del sistema a un elevato livello di astrazione (tipicamente sul cruscotto dell'auto).

L'HMI coordina i vari **Controllori** che gestiscono i dispositivi MOST (**Slave**).

Le interfacce verso le funzioni disponibili dei dispositivi e dei Controllori sono descritte nel dettaglio nei Blocchi Funzionali MOST (*FBlocks*).

I **Blocchi Funzionali sono oggetti deputati al controllo di funzioni** dedicate all'interno della rete MOST. Essi incapsulano le **proprietà** e i **metodi** di un dispositivo MOST (ad esempio, un lettore CD) che sono necessari per controllare il dispositivo stesso o per gestire la rete. Esistono FBlock preposti al controllo delle *applicazioni* e altri dedicati alla gestione della rete.

La comunicazione con i blocchi funzionali avviene tramite il protocollo applicativo a un **elevato livello di astrazione. Nei livelli superiori, un messaggio MOST fa riferimento a:**

- L'identificativo del dispositivo (**device ID**),
- Il **blocco funzionale** all'interno del dispositivo,
- **All'operazione implementata** con i relativi parametri, e così via.

I blocchi funzionali semplificano notevolmente lo sviluppo di applicazioni audio distribuite.

Addressing

Al livello 2 (Data Link Layer) dell'architettura di riferimento ISO, la rete MOST supporta molteplici modalità di indirizzamento:

- **Indirizzamento in base alla posizione del nodo**

Ciascun nodo all'interno della rete viene indirizzato in base alla sua posizione fisica reale, espressa in un formato a 16 bit.

Il **Timing Master** assume sempre la posizione **0x0000**, il nodo immediatamente successivo la posizione **0x0001**, e ai nodi seguenti vengono assegnate posizioni incrementali sequenziali in base al loro ordine di collegamento fisico **lungo l'anello**.

- Tramite la posizione del nodo è possibile determinare molto facilmente il ritardo di propagazione (*delay*) tra la sorgente del segnale e la destinazione (*sink*): la sorgente comunica alla destinazione la propria posizione e quest'ultima calcola il numero di nodi di rete attraverso cui il segnale è dovuto passare.

- **Indirizzi logici (*Logical addresses*)**

Vengono assegnati in modo statico durante la fase di **inizializzazione** del nodo subito dopo l'avvio della rete. L'indirizzo logico può fare riferimento alla specifica **funzione** del nodo (ad esempio, primo monitor video **0x0200**, secondo monitor video **0x0201**) oppure alla sua **posizione fisica** all'interno del veicolo (ad esempio, **0x0100** abitacolo lato sinistro, **0x0200** abitacolo lato destro, **0x0500** bagagliaio).

- **Indirizzamento di gruppo (*Group addressing*)**

I dispositivi appartenenti alla stessa classe (come gli amplificatori o le unità di infotainment) possono condividere lo stesso indirizzo di gruppo, permettendo di controllarli simultaneamente inviando un singolo frame sul bus.

- **Indirizzamento Broadcast**

Consente la trasmissione simultanea ed efficiente dei dati a tutti i nodi della rete.

- **Indirizzamento MAC (*MAC addressing*)**

Lo standard **MOST150** supporta nativamente anche l'indirizzamento basato su indirizzi MAC, utilizzato nello specifico per l'incapsulamento del traffico Ethernet (meccanismo *Ethernet over MOST*). *Most150 può quindi mandare anche pezzi frame ethernet.*

RING SCAN: L'indirizzamento basato sulla posizione del nodo riflette in modo chiaro le proprietà geometriche della topologia ad anello del MOST. Poiché ogni nodo riceve i dati dal nodo precedente e li **ritrasmette** a quello successivo, la rete mappa matematicamente se stessa durante la fase di startup.

Sapere esattamente quanti "salti" (*hops*) separano una sorgente (ad esempio, il lettore DVD nel bagagliaio) dalla destinazione (lo schermo sui sedili posteriori) è un'informazione fondamentale per i servizi di streaming: permette ai nodi di stimare con precisione microscopica il ritardo di **propagazione** introdotto dalla ripetizione del segnale lungo l'anello e di calibrare di conseguenza i **buffer** di sincronizzazione audio/video.

Un anello logico, tipicamente mappato su un anello fisico (configurazione a stella possibile anche tramite hub), in grado di connettere in rete fino a **64** dispositivi.

Il MOST è una rete sincrona, ovvero organizza il traffico di rete in una sequenza continua di frame che vengono generati da un **Timing Master** (tipicamente la *Head Unit* del sistema di infotainment) con una frequenza fissa. Vengono utilizzate due frequenze:

- **48 kHz** (audio professionale, su MOST50 e 150)
- **44,1 kHz** (CD audio, su MOST25).

Quando il *Timing Master* riceve indietro il frame che ha viaggiato lungo tutto l'anello, genera il frame successivo. Alla frequenza di 44,1 kHz, il frame MOST viene trasmesso 44100 volte al secondo. Nel MOST25, un frame da 512 bit, questo si traduce in una velocità dati circa 22,58 Mbit/s.

Gli altri dispositivi, i *Timing Slaves*, sincronizzano il proprio funzionamento al preambolo (*Preamble*) di questi frame. Tutti i dispositivi condividono un impulso di clock di sistema comune derivato direttamente dal flusso dati. I dispositivi sono in fase tra loro e trasmettono in modo sincrono.

L'architettura di base di una rete MOST è un **anello logico**, che trasmette i dati da un dispositivo all'altro.

- La comunicazione lungo l'anello è rigorosamente **unidirezionale**.
- I frame vengono trasmessi continuamente dal *Timing Master* al nodo successivo dell'anello; quest'ultimo, a sua volta, inserisce i propri dati sorgente all'interno del frame e lo trasmette al nodo successivo della rete.

Topologia

La struttura ad anello logico viene solitamente realizzata sotto forma di un **anello fisico**. La corrispondenza tra anello logico e fisico non è obbligatoria. È possibile realizzare, ad esempio, una topologia combinata ad anello e stella utilizzando un **hub centrale**. In una configurazione a **stella**, i nodi possono essere aggiunti o rimossi semplicemente senza che il resto della rete ne venga influenzato, il che aumenta l'affidabilità complessiva del sistema.

Il principio cardine di trasmettere il flusso dati da un nodo a quello successivo viene sempre mantenuto, sia che la rete sia disposta in una topologia a stella, sia in qualsiasi forma mista di anello e stella.

Per ottenere un'elevata disponibilità (**high availability**) in caso di guasti, è possibile realizzare una configurazione a **doppio anello**. In questo scenario, ciascun nodo deve essere equipaggiato con due ricevitori ottici e due trasmettitori ottici. Se si verifica un'interruzione fisica o un problema di comunicazione tra due nodi, l'anello può essere "richiuso" riconfigurando il flusso dati attraverso i segmenti ridondanti (meccanismo di **loopback**), salvando l'integrità della rete.

Masters

Il protocollo MOST supporta funzionalità fondamentali per l'**efficienza del veicolo**, quali la gestione dell'energia (**power management**), le modalità a basso consumo (**sleep modes**) e tempi di risveglio estremamente rapidi (**fast wake-up**).

Queste funzioni non devono essere necessariamente gestite dalla medesima unità hardware, sebbene nella pratica comune lo siano. Per questa ragione, all'interno della rete, oltre alla figura del *Timing Master*, sono definite ulteriori funzioni di coordinamento per la gestione delle connessioni, della rete e dell'alimentazione:

- **Connection Master:** Amministra e coordina molteplici connessioni simultanee all'interno dei canali sincroni della rete.
- **Network Master:** Controlla, monitora e gestisce lo stato operativo complessivo del sistema (*system status*).
- **Power Master:** Supervisiona e coordina le delicate procedure di avvio (*start-up*) e di spegnimento (*shut-down*) della rete MOST.

Questa suddivisione logica dei ruoli (i cosiddetti *Masters*) serve a strutturare la rete in modo modulare.

MOST Frame

Un frame MOST contiene diverse porzioni, denominate "canali", ciascuna delle quali risponde a uno scopo specifico:

- **Control Channel**: Utilizzato per trasferire pacchetti dedicati a comandi, messaggi di stato e di diagnostica verso specifici dispositivi, nonché per avviare ed stabilire la connessione di streaming dati tra il mittente e il ricevitore.
- **Streaming Data Channel (Sincrono/Isocrono)**: Dedicato ai flussi continui di dati che richiedono un'elevata larghezza di banda e una trasmissione sincronizzata nel tempo.
 - o I flussi in tempo reale possono essere **sincroni** (aventi la stessa frequenza di campionamento della rete MOST) oppure **isocroni** (con una frequenza svincolata dal clock di sistema della rete, come ad esempio i flussi MPEG, supportati esclusivamente dallo standard MOST150).
 - o La **larghezza di banda** viene **riservata**, in modo tale da risultare sempre disponibile e dedicata a tali flussi. Di conseguenza, nel trasporto dei flussi dati **non si verificano interruzioni, collisioni o ritardi**.
 - o Le connessioni vengono gestite attraverso appositi messaggi di controllo scambiati lungo il Canale di Controllo.
- **Packet Data Channel (Asincrono)**: Destinato a trasmissioni che richiedono un'elevata larghezza di banda in modalità *impulsiva/spontanea* (*burst-like*, come i dati legati a eventi o informazioni strutturate in pacchetti).

Lo standard **MOST150** integra inoltre il cosiddetto **MOST Ethernet Packet Channel (MEP)** a larghezza di banda regolabile, progettato specificamente per trasportare dati basati sul protocollo IP e offrire supporto ai protocolli IP impiegati per le funzionalità di sleep e wake-up (consentendo il transito di dati a pacchetto fino a **120 Mbit/s**).

Il confine dinamico tra il canale di streaming (sincrono) e il canale dati a pacchetto (asincrono) viene definito utilizzando un parametro chiamato **Boundary Descriptor**.

Ogni frame (trama) supporta contemporaneamente diverse tipologie di traffico:

- **Controllo**: Trasmissione dei dati di controllo, come ad esempio la gestione dell'attivazione e della disattivazione delle connessioni, e lo scambio dei messaggi di controllo per i blocchi funzionali (*Function Blocks*). I dati relativi ai comandi trasmessi attraverso il Canale di Controllo vengono distribuiti lungo diversi frame successivi.
- **Streaming**: Connessioni di streaming sincrone (o isocrone nello standard MOST150) tra una sorgente (*source*) e uno o molteplici destinatari (*sinks*).
- **Packet Data**: Trasmissione asincrona di dati a pacchetto (ad esempio, i dati di configurazione del sistema di navigazione) senza influenzare in alcun modo la trasmissione sincrona dei dati. Nello standard MOST150, il Canale Dati a Pacchetto trasporta anche dati basati sul protocollo IP.

Il formato specifico del frame varia tra le generazioni **MOST25**, **MOST50** e **MOST150**.

MOST25

Il livello superiore genera messaggi MOST della lunghezza di 32 byte.

Tuttavia, **i messaggi MOST non coincidono con i frame MOST**. Un singolo messaggio MOST potrebbe dover essere distribuito (frammentato) su più frame MOST successivi.

Il frame MOST 25 ha una lunghezza complessiva di **64 byte** ed è partizionato in **64 slot** (dove 1 slot equivale esattamente a 1 byte).

- **2 slot** per uso amministrativo.
- **2 slot** riservati alle informazioni di controllo.
- **24 fino a 60 slot** per **dati sincroni**. Il flusso continuo viene frammentato in segmenti (*sliced*); ogni frame trasporta una frazione con il corretto numero di byte, permettendo ai ricevitori di ricostruire il flusso originario in tempo reale.
- **0 fino a 36 slot** sono destinati ai **dati asincroni** (come le informazioni cartografiche dei sistemi di navigazione o il traffico TCP/IP).

Ciò significa che un messaggio di controllo da **32 byte** generato da un dispositivo viene **suddiviso e distribuito su 16 frame successivi**; di conseguenza, il dispositivo successivo deve attendere la fine di questa sequenza prima di poter utilizzare a sua volta i 2 byte del canale di controllo.

Nello standard MOST25, alla frequenza di campionamento di 44,1 kHz, la larghezza di banda netta dedicata al **Canale di Controllo** è pari a **705,6 kbit/s**.

I messaggi di controllo richiedono obbligatoriamente una conferma di ricezione (*acknowledgment*). In caso di mancata ricezione dell'ACK, si verificano automaticamente dei tentativi di ritrasmissione.

I restanti **21,2 Mbit/s** della larghezza di banda totale sono suddivisi e condivisi tra la porzione Sincrona e quella Asincrona.

Accesso al mezzo

Streaming Channel (Porzione Sincrona)

L'accesso al mezzo è basato su uno schema **TDMA**. All'interno di ogni singolo frame MOST, un determinato dispositivo ha il diritto esclusivo di trasmettere all'interno di slot specifici preassegnati. Per via della natura in tempo reale dei flussi, **non sono previste ritrasmissioni** dei dati (se un byte viene corrotto, viene scartato per non introdurre ritardi).

Packet Channel (Porzione Asincrona)

L'accesso è guidato da un meccanismo **token-based**. Il token è incorporato direttamente nell'*header* del frame. Quando un dispositivo entra in possesso del token, acquisisce il diritto di trasmettere un singolo messaggio MOST, il quale può contenere un massimo di **1014 byte** di dati utente, prima di dover cedere il token al nodo successivo. Il singolo messaggio può essere trasmesso sfruttando la totalità dei byte (slot) correntemente assegnati alla porzione di dati asincroni.

Tale messaggio viene necessariamente distribuito e trasmesso lungo diversi frame MOST successivi. **Esempio matematico:** Se un messaggio ha una lunghezza massima di 1014 byte e il dispositivo ha a disposizione la larghezza di banda massima teorica (ovvero 36 slot asincroni per frame), il calcolo sarà:

$$\frac{1014}{36} \approx 28.17$$

Saranno quindi necessari **29 frame MOST** consecutivi per completare l'invio dell'intero messaggio. A differenza del canale di controllo, in questa porzione asincrona **non sono previsti né ACK né ritrasmissioni** a livello di protocollo MAC.

Ci sono quindi tre modi diversi all'interno dello stesso mezzo fisico:

1. **Master-Slave classico con ACK** nel canale di controllo.
2. **TDMA** nel canale sincrono (perché lo streaming audio/video non può tollerare ritardi dovuti a contese, e un pixel o un campione audio corrotto è preferibile rispetto a un blocco del flusso).
3. **Token Ring** nel canale asincrono. Questa scelta impedisce che due nodi tentino di scaricare dati (come le mappe del navigatore) contemporaneamente, evitando collisioni sull'anello e ottimizzando l'uso dei pochi slot asincroni rimasti liberi dopo la configurazione del *Boundary Descriptor*.

MOST150



Figure 8-15: A frame in a MOST150 network is divided up into channels, and travels around the ring delivering consecutive pieces of data in each channel. The Control, Synchronous / Isochronous, Asynchronous / Legacy channel and MOST Ethernet Packets (MEP) channels all have their own respective headers and footers.

L'intestazione (*header*) del frame MOST150 è analoga a quella del MOST50, ma presenta un byte aggiuntivo e le informazioni relative al **preambolo**. La parte restante del frame viene suddivisa tra i dati di streaming (*stream data*) e i dati a pacchetto (*packet data*) in base al valore impostato nel *Boundary Descriptor* (Descrittore di Confine).

Nello standard MOST150, il confine tra la porzione sincrona/isocrona (**Syn/ISOC**) e quella asincrona (**ASYNC**) può essere modificato dinamicamente **senza la necessità di dover interrompere e ristabilire le connessioni sincrone attive**. Questa prerogativa di modificare il valore del *Boundary Descriptor* spetta esclusivamente alla funzione del *Timing Master*.

Con una velocità massima di trasferimento dati pari a **150 Mbit/s**, la terza generazione MOST150 consente lo streaming fluido e senza interruzioni di contenuti audio e video ad alta definizione (HD).

In aggiunta alla maggiore larghezza di banda, il MOST150 introduce un meccanismo di trasporto **isocrono** progettato specificamente per supportare estese e pesanti applicazioni video (come i flussi video compressi **MPEG** provenienti dalle telecamere ADAS).

Il MOST150 offre un canale **Ethernet** dedicato per il trasporto efficiente di dati a pacchetto basati sul protocollo IP. Questo canale è in grado di veicolare i pacchetti Ethernet tradizionali (conformi allo standard **IEEE 802.3**) in modo nativo, ovvero senza apportare loro alcuna modifica o incapsulamento pesante.

MOST25 vs MOST150

MOST150 risponde ai due grandi limiti delle versioni precedenti:

1. **La flessibilità del Boundary Descriptor:** nel MOST25 cambiare il confine tra sincrono e asincrono significava far cadere tutte le telefonate o i flussi audio attivi (un grande limite nei sistemi dinamici). Nel MOST150 il firmware **dell'INIC** è abbastanza intelligente da ridistribuire i byte "al volo" a ciclo continuo, garantendo la continuità del servizio.
2. **L'isocronismo:** L'aggiunta del canale isocrono permette di gestire flussi video con clock indipendenti rispetto a quello di rete (fondamentale per i flussi video digitali compresi).
3. **Ethernet:** La vera novità è l'integrazione del canale Ethernet (IEEE 802.3) nativo: permette all'auto di dialogare con il mondo esterno (diagnostica DOIP, aggiornamenti OTA, hotspot Wi-Fi di bordo) usando gli stessi identici pacchetti del mondo IT, eliminando i software di traduzione e i relativi ritardi di elaborazione.

Punti di Forza

- **Estremamente efficiente per lo streaming A/V:** Assenza totale di collisioni sul bus grazie al metodo di accesso TDMA e disponibilità di un'elevata larghezza di banda dedicata.
- **Ampia diffusione attuale**
- **Funzionalità Plug-and-Play:** I dispositivi e le centraline possono essere agganciati o rimossi dalla rete con estrema facilità.

Debolezze

- **Fornitore unico:** Una sola azienda (**Microchip**) sviluppa e fornisce il silicio (i chip hardware) per il MOST, limitando fortemente le opzioni di scelta sul mercato.
- **Interoperabilità limitata:** Il fornitore dominante ha rilasciato la specifica dettagliata del Data Link Layer (DLL) esclusivamente dietro il pagamento di royalty, limitando di fatto la libertà delle altre aziende di semiconduttori di eseguire test indipendenti di interoperabilità e conformità.
- **Distanza limitata tra i nodi:** A causa dei vincoli fisici intrinseci della topologia ad anello (attenuazione ottica ed elettrica), la distanza massima tra due nodi adiacenti è ristretta.

Questa limitazione può rappresentare una sfida nei veicoli di grandi dimensioni o quando si integrano componenti situati molto distanti tra loro.

- **Complessità elevata:** Il protocollo richiede competenze e conoscenze altamente specializzate sia per l'implementazione software/hardware sia per le procedure di diagnostica e risoluzione dei problemi (*troubleshooting*). I costruttori e i fornitori dell'automotive devono investire massicciamente in formazione e risorse dedicate.
- **Collo di bottiglia della banda:** Il limite massimo di **150 Mbit/s** sta diventando insufficiente per i moderni e densi sistemi di infotainment. Questo rende le architetture **Automotive Ethernet** a Gigabit e multi-Gigabit (supportate dagli standard di sincronizzazione **IEEE AVB/TSN**) decisamente più competitive e futuribili.

Ethernet non sostituisce LIN perché LIN fa bene il suo lavoro, ma FlexRay o Most possono essere sostituiti.

Ethernet non è proprietario, è gestito dall'IEEE (no costi di licenza).

Ethernet in Automotive

- Crescita dei requisiti di traffico
- Oltre 70 ECU nelle auto di fascia alta odierne, fino a 150 nei moderni veicoli di lusso.
- Implementazione di centinaia di funzioni distribuite per fornire, ad esempio, comfort, sicurezza e infotainment.
- Oggigiorno, ben il 50% del costo di un veicolo è legato al contenuto elettronico e si prevede che questo numero continuerà ad aumentare.
- Per incrementare la facilità di guida e la sicurezza, molte applicazioni richiedono una risoluzione migliorata delle immagini e dei sensori.
- Sistemi di avviso di superamento della corsia (*Lane departure warning*), riconoscimento dei segnali stradali/semafori, sistemi di prevenzione delle collisioni (*Collision avoidance*).
- **La guida automatizzata pone nuove sfide, poiché richiede una corretta percezione dell'ambiente circostante in qualsiasi condizione di guida.**
- **Per ottenere questo risultato, non solo sono necessari molti più sensori rispetto agli attuali sistemi ADAS, ma tali sensori sono anche di diversi tipi, come radar, sensori a ultrasuoni, LiDAR e telecamere.**
- Dato l'elevato numero di sensori e i requisiti di larghezza di banda di alcuni di essi (ad esempio, telecamere e LiDAR), sono necessarie reti di livello automobilistico (*automotive-grade*) ad alta velocità.
- La richiesta di comunicazione inter-ECU è aumentata notevolmente e continuerà ad aumentare in futuro.
- È necessario supportare le comunicazioni tra funzioni appartenenti a domini differenti.

Point-to-Point Technologies

- Oggi, le telecamere o i display ad alta definizione (HD), così come i sofisticati sensori LiDAR (*Light Detection and Ranging*), richiedono velocità di trasmissione dati **> 1 Gbit/s**.
- **Serializzatore/Deserializzatore** (Serializer/Deserializer), chiamato anche LVDS (Low-Voltage Differential Signaling), pixelink o collegamento video ad alta velocità (high-speed video link), **è il nome generico delle tecnologie punto-a-punto che supportano la trasmissione ad alta velocità di dati binari per trasferire informazioni accurate al singolo pixel sui livelli più bassi del modello ISO/OSI.**
- L'utilizzo primario di un'architettura SerDes è fornire la trasmissione dei dati su una singola linea o su una coppia differenziale, riducendo così al minimo il numero di pin di I/O (Input/Output) e di interconnessioni fisiche.
- L'LVDS è il principio fisico alla base delle interfacce SerDes. Di fatto, l'LVDS specifica le caratteristiche elettriche di uno standard di segnalazione seriale e differenziale che opera a bassa potenza e può raggiungere velocità molto elevate utilizzando economici cavi in rame a doppino intrecciato.
- Sebbene sia spesso utilizzato come sinonimo di SerDes, l'LVDS è esclusivamente una specifica del livello fisico (*Physical Layer*); molti standard di comunicazione dati e applicazioni lo utilizzano come base, aggiungendo al di sopra di esso un livello di collegamento dati (*Data Link Layer*) come definito nel modello OSI.

Comunicazione Interdominio

La comunicazione è richiesta anche tra domini funzionali differenti all'interno del veicolo. Ad esempio, le **sospensioni attive** presenti su diversi veicoli di fascia alta richiedono un'interazione in tempo reale tra le telecamere del sistema ADAS (che scansionano il fondo stradale), i sensori del *powertrain* (coppia, velocità) e gli attuatori del telaio (*chassis*).

A causa della coesistenza di molteplici tecnologie di rete non direttamente compatibili tra loro (come CAN, FlexRay, MOST), la comunicazione interdominio deve essere necessariamente supportata e veicolata da **gateway**. Si tratta di gateway complessi che richiedono una conoscenza approfondita a livello applicativo (*application knowledge*) per poter tradurre e instradare correttamente i messaggi tra i diversi protocolli.

Esempio pratico: Un dispositivo connesso alla rete MOST deve inviare una richiesta per configurare un parametro a un dispositivo connesso alla rete FlexRay. Questo flusso di comunicazione prevede nello specifico tre passaggi consecutivi:

- **Fase 1:** Il dispositivo MOST effettua una chiamata a funzione (*function call*) verso un blocco funzionale MOST ospitato all'interno del gateway che funge da mediatore.
- **Fase 2:** Il gateway deve decodificare e ricodificare i parametri del MOST, tradurre il messaggio applicativo MOST in uno o più segnali FlexRay e provvedere al loro invio sulla relativa rete.
- **Fase 3:** Il gateway deve eseguire un processo analogo e inverso per recapitare al dispositivo MOST la risposta inviata dal dispositivo FlexRay.

Per eseguire correttamente questi passaggi, il gateway deve possedere una conoscenza approfondita dell'applicazione (*application knowledge*). Questo rappresenta un problema critico in fase di ingegnerizzazione, poiché il gateway deve essere riadattato e riconfigurato ogni qualvolta venga apportata una modifica a livello applicativo nei nodi.

Inoltre, si verifica una penalizzazione delle prestazioni (*performance penalty*) a causa della conversione di protocollo operata dai gateway.

Ad esempio, le interfacce di infotainment devono garantire al conducente un'interazione pronta e tempestiva. Tuttavia, a volte tali interfacce rispondono lentamente agli input dell'utente, poiché i sistemi di infotainment devono attendere la ricezione delle informazioni da altri componenti appartenenti a differenti sottosistemi in altri domini funzionali.

Un altro esempio è la mancanza di sincronizzazione tra i quadri strumenti (*instrument clusters*) e gli schermi del cruscotto su alcune vetture, che può causare un'occasionale perdita di coerenza visiva per le indicazioni e gli avvisi, degradando così la qualità dell'esperienza utente (UX).

Una tecnologia di rete comune per le unità di controllo elettronico (ECU) all'interno del veicolo, in grado di offrire un'elevata larghezza di banda, latenza limitata (*bounded latency*), affidabilità e operazioni sincronizzate delle ECU attraverso un tempo di rete comune, risolverebbe il problema e semplificherebbe l'architettura elettronica dell'auto.

Diagnostica e Aggiornamenti Software Richiedono Banda

La quantità di software integrato nelle automobili odierne sta crescendo rapidamente a causa dei costanti progressi delle funzionalità fornite dai sistemi elettronici del veicolo. Il tempo speso per la riprogrammazione (chiamata aggiornamento flash o *flashing*) delle ECU automobilistiche è già diventato un fattore di costo critico.

La riprogrammazione si rende necessaria per caricare nuove applicazioni all'interno dell'unità di controllo elettronico sia durante la fase di produzione dell'auto, sia successivamente presso le officine di riparazione. Per garantire efficienza e riduzione dei costi sono richiesti una diagnostica rapida e tempi di aggiornamento decisamente più brevi. Il salto generazionale nell'architettura di rete mostra un abbattimento drastico dei tempi di attesa:

- **BMW Serie 7 di 4ª generazione tramite bus CAN:** Richiedeva circa **10 ore** per trasferire e scrivere appena **81 MB** di dati.
- **BMW Serie 7 di 5ª generazione tramite Automotive Ethernet:** Consente di trasferire e scrivere ben **1 GB** di dati in soli **20 minuti**.

Comunicazioni verso sistemi esterni al veicolo

La comunicazione tra i veicoli e il mondo esterno continua ad aumentare.

- Monitoraggio remoto
- Gestione delle flotte (Fleet management)
- Applicazioni automobilistiche basate su Internet
- Comunicazioni Veicolo-a-X (Car-to-X)

Esempio: il Traffic Light Phase Assistant, che suggerisce la velocità più idonea per sfruttare l'onda verde su una determinata strada.

Le reti di telematica e di infotainment automobilistico devono diventare più aperte verso i dispositivi non automobilistici e devono supportare applicazioni basate su IP e sul Web.

In passato, l'elettronica di consumo e l'elettronica dell'autoveicolo rappresentavano mondi separati, con requisiti e focus differenti. Oggi stanno convergendo, grazie al ruolo crescente delle reti interne al veicolo da un lato e alla diffusione dei dispositivi mobili dall'altro.

Le ragioni di Ethernet in Automotive

- **Maggiore larghezza** di banda fornita da Ethernet (100 Mbit/s, 1 Gbit/s, 2.5, 5 e 10 Gbit/s, e oltre...) rispetto alle attuali reti interne al veicolo consente di ottenere prestazioni ottimali. Risulta idonea per i sistemi ADAS basati su telecamere, per la multimedialità/infotainment e per i sistemi di guida automatizzata, i quali fanno crescere continuamente il volume di dati scambiati nelle comunicazioni automobilistiche.
- Grazie al funzionamento simultaneo in modalità **full-duplex** e all'architettura basata su switch (**switched operation**), il throughput aggregato complessivo può risultare nettamente superiore rispetto alla larghezza di banda nominale del singolo collegamento.
- Ogni frame Ethernet trasporta all'interno della propria intestazione l'indirizzo del destinatario (**l'indirizzo MAC di destinazione**). Il principio di funzionamento degli switch consente loro di recapitare i frame all'esatta destinazione prevista, indipendentemente dalle variazioni o dalle riconfigurazioni della topologia di rete. Grazie a questa modalità

operativa, risulta estremamente semplice aggiungere ulteriori switch a una rete preesistente nel momento in cui si renda necessario connettere nuovi dispositivi. Ciò permette di espandere l'architettura e creare reti di dimensioni arbitrarie.

- Molti costruttori automobilistici implementano Ethernet come **dorsale principale** del veicolo, sebbene continuino a utilizzare altre tecnologie di rete (come CAN FD o LIN) all'interno di specifici sottosistemi periferici.
- Ampio utilizzo, **standardizzazione IEEE**, apertura. Una vasta comunità di sviluppatori e utenti fa progredire la tecnologia e offre vantaggi in termini di: elevato numero di chip di buona qualità disponibili sul mercato, costi ridotti sia nello sviluppo che nella produzione dei prodotti.
 - o Al contrario, il MOST e il FlexRay non vengono utilizzati in ambiti di comunicazione diversi da quello automobilistico e non offrono le proprietà sopra menzionate: Ridotta penetrazione nel mercato, costi elevati.
- Il supporto offerto all'Internet Protocol (IP) e alle suite di comunicazione come TCP/IP, AVB e TSN che funzionano su qualsiasi tipo di Ethernet. Grazie al suo approccio stratificato in cui il livello fisico è separato da tutti gli altri livelli.
- Al livello Fisico si possono apportare modifiche sostanziali, ad esempio nuovi tipi di cablaggio e velocità operative più elevate, lasciando invariati tutti i software e i protocolli dei livelli superiori.
- Con Ethernet non è necessario implementare un'intera nuova pila di protocolli (*protocol stack*) se la rete a basso livello subisce modifiche.
- Sono standardizzati e disponibili livelli fisici Ethernet di tipo automobilistico (*automotive-grade*) in grado di funzionare correttamente in condizioni gravose, sotto elevate radiazioni elettromagnetiche e stress termo-meccanici.
- **Le reti automobilistiche operano a temperature elevate e con un'elevata EMC. Le prestazioni termiche Ethernet si sono dimostrate adeguate: funziona bene ad alte temperature, grazie al basso consumo energetico.**
- Ethernet offre una comprovata capacità di adattarsi alle esigenze future (**future-proof**), di evolversi, cambiare e supportare requisiti di larghezza di banda sempre crescenti, mantenendo al contempo la retrocompatibilità con i sistemi preesistenti (*legacy*).

Di conseguenza, Automotive Ethernet rappresenta la principale soluzione di rete di bordo per le architetture automobilistiche attuali e future.

Automotive Ethernet PHYs

Il Primo Livello Fisico: BroadR-Reach

Per ragioni legate al contenimento dei costi e del peso complessivo del veicolo, viene utilizzata un'unica coppia di fili intrecciati (doppino).

Il **primo livello fisico (PHY)** basato su tecnologia Ethernet che ha dimostrato di funzionare in modo ottimale per le reti automobilistiche è stato lo standard **BroadR-Reach (BR)** sviluppato da Broadcom.

Obiettivi del BroadR-Reach:

- Costi ridotti.

- Peso contenuto.
- Velocità di trasmissione più elevata (**100 Mbit/s**).
- Piena compatibilità con il livello MAC dello Standard Ethernet e con i livelli superiori.
- Supporto a topologie di rete flessibili ed espandibili (topologie a stella e a stella gerarchica, al posto del classico bus condiviso).
- Elevata affidabilità in ambienti operativi ostili.

L'utilizzo di un cavo a **doppino intrecciato non schermato (UTP)** consente di ridurre i costi di connettività fino all'**80%** e il peso del cablaggio del **30%**, risultando decisamente più leggero ed economico rispetto al cablaggio tradizionale di tipo LVDS (Fonte: Broadcom, 2012).

L'adozione del doppino UTP permette inoltre di ridurre in modo significativo la lunghezza complessiva e la complessità dell'intero impianto elettrico di bordo (*wiring harness*).

– LVDS è point-to-point: l'inserimento di un nuovo dispositivo significa un nuovo cablaggio.

Esso fornisce:

- **Trasmissione bidirezionale** su una coppia di fili UTP.
- Notevole **risparmio sui costi**, sullo spessore e sul peso del cablaggio rispetto alle tecnologie concorrenti (MOST, LVDS).
- Una soluzione robusta ed economica a **100 Mbps** che soddisfa i requisiti EMC/EMI delle reti automobilistiche. Lunghezza massima del cavo pari a **15 metri**.

(*) "Bilanciato" (*Balanced*) è il termine utilizzato dall'IEEE per indicare un segnale di tipo "Differenziale".

Automotive Ethernet: Gigabit e MultiGig PHY

- Lo standard **IEEE 802.3bp-2016** (emendamento a IEEE Std 802.3-2015) aggiunge le specifiche del Livello Fisico punto-a-punto a 1 Gbps per il funzionamento su un singolo cavo in rame a doppino intrecciato (UTP, STP) nelle applicazioni automobilistiche (**1000BASE-T1**).
 - Focus sui requisiti automobilistici, ad esempio su temperatura, compatibilità elettromagnetica (EMC), capacità diagnostiche, latenza del PHY, ciclo di vita, tempi di risveglio (*wake-up time*), ecc.
- Lo standard **IEEE Std 802.3ch-2020** (emendamento a IEEE Std 802.3-2018) aggiunge le specifiche del livello fisico e i parametri di gestione per il funzionamento a 2,5 Gbps, 5 Gbps e 10 Gbps su una singola coppia bilanciata di conduttori (STP) idonea per applicazioni automobilistiche (**MultiGig Automotive Ethernet, 2.5G/5G/10GBASE-T1**).
- Lo standard **IEEE 802.3cy-2023** (emendamento a IEEE Std 802.3-2022) aggiunge le specifiche del livello fisico e i parametri di gestione per il funzionamento a 25 Gbps su una singola coppia bilanciata di conduttori (STP) idonea per applicazioni automobilistiche (Automotive Ethernet elettrica a 25 Gbps).
- Lo standard **IEEE 802.3cz-2023** (emendamento a IEEE Std 802.3-2022) aggiunge le specifiche del livello fisico e i parametri di gestione per l'Automotive Ethernet multi-Gigabit su fibra ottica di vetro (GOF) a 2,5 Gbps, 5 Gbps, 10 Gbps, 25 Gbps e 50 Gbps.

10BASE-T1S

10BASE T1S nasce per fornire una soluzione economica basata su Ethernet con una topologia a bus condiviso (**multidrop bus topology**), analogamente alle reti automobilistiche tradizionali, operante a 10 Mbps.

Nelle architetture punto-punto, ogni sensore richiede una porta dedicata su uno switch. Negli strati più semplici dell'auto (es. sensori di parcheggio, pulsanti dei finestrini, attuatori dei sedili), posizionare switch ovunque è proibitivo dal punto di vista economico e dello spazio. Con il multidrop, i sensori condividono la stessa linea senza bisogno di componenti di commutazione intermedi.

La **modalità Multidrop (Bus condiviso)** consente di collegare più centraline (almeno 8 nodi) sullo stesso identico segmento di cavo lungo fino a 25 metri, eliminando la necessità di utilizzare uno **switch di rete** centrale per quel segmento. Il 10BASE-T1S elimina la necessità di utilizzare ECU con funzioni di gateway complessi per trasferire i dati tra tipologie di reti differenti. Una rete omogenea consente inoltre l'instradamento IP (*IP routing*) verso l'intero veicolo, realizzando il concetto di **"IP ovunque"** (*IP everywhere*).

Alcuni dei requisiti chiave includevano:

- Una comunicazione più **veloce** rispetto alle tecnologie esistenti, ad esempio CAN(FD).
- La sostituzione delle tecnologie di rete di bordo tradizionali come il **FlexRay**.
- Un'alternativa al 100BASE-T1 per le ECU in cui il 100BASE-T1 non risulta efficiente dal punto di vista dei costi e dell'energia.

Lo standard **IEEE Std 802.3cg-2019** si riferisce a una nuova variante dello standard Ethernet che offre 10 Mbit/s su un livello fisico a singolo doppino.

Questa specifica colma il divario delle moderne interfacce fisiche Ethernet nella fascia più bassa dello spettro di larghezza di banda. Una delle varianti specificate all'interno dello standard IEEE è il 10BASE-T1S, dove:

- **10**: Indica la velocità di trasmissione nominale della rete, ovvero **10 Mbit/s**.
- **BASE**: Sta for *Baseband* (banda base), e indica che il mezzo trasmissivo trasporta esclusivamente quel segnale analogico, senza modulazione di frequenza su più canali contemporanei.
- **S** sta per "short reach" (breve distanza) e si rivolge principalmente all'utilizzo in ambito automobilistico. Viene definita anche una variante "long reach" (lunga distanza), chiamata 10BASE-T1L, progettata per distanze fino a 1 km e destinata all'uso industriale.
- **T1**: Indica il tipo di mezzo fisico utilizzato, nello specifico un **singolo doppino intrecciato** di rame (*Single Twisted Pair*).

Modalità Operativa 1: Bus Multidrop

- Lo standard 10BASE-T1S specifica una topologia a bus condiviso (*multidrop*) in modalità **half-duplex**, che è ottimo per il **risparmio energetico**: quando nessun nodo sta trasmettendo dati, la rete è elettricamente silente (ovvero non vi è alcuna transizione di tensione sul bus).
- Ma in full duplex anche se non vengono trasmessi dati utili, i transceiver inviano costantemente un flusso continuo di **simboli di controllo/idle** (es. per mantenere il sincronismo del clock descritto dal *Timing Master* e l'adattamento dell'equalizzatore di linea). Questo significa che il circuito analogico del PHY consuma costantemente energia.
- Lo standard stabilisce che in modalità *multidrop* si possano connettere:
 - **Almeno fino a 8 nodi**,
 - **Lunghezza 25 metri**, con derivazioni di **10 cm** per il collegamento dei nodi.
 - Tutti i nodi collegati condividono la larghezza di banda totale di **10 Mbit/s**.

Modalità Operative 2 e 3: *Point-to-Point*

- Lo standard 10BASE-T1S specifica inoltre una topologia **punto-a-punto obbligatoria** con funzionamento in modalità **half-duplex**.
- Definisce infine una topologia **punto-a-punto opzionale** con funzionamento in modalità **full-duplex** (più complessa e costosa).
- Entrambe le varianti punto-a-punto supportano una portata fino a **15 metri** con un massimo di 4 connettori in linea, consentendo la realizzazione di topologie ad albero, a stella e di altro tipo.

PLCA

Per la modalità multidrop lo standard 10BASE-T1S specifica un metodo di accesso al canale denominato **PLCA** *Physical Layer Collision Avoidance*, progettato specificamente per evitare le collisioni sulla rete condivisa.

10BASE T1S riesce a implementare una comunicazione deterministica a livello fisico, non MAC. Prima tutti i meccanismi di accesso al mezzo venivano implementati a livello due. Il mac layer è CSMA/CD perché si occupa di tutto il livello fisico.

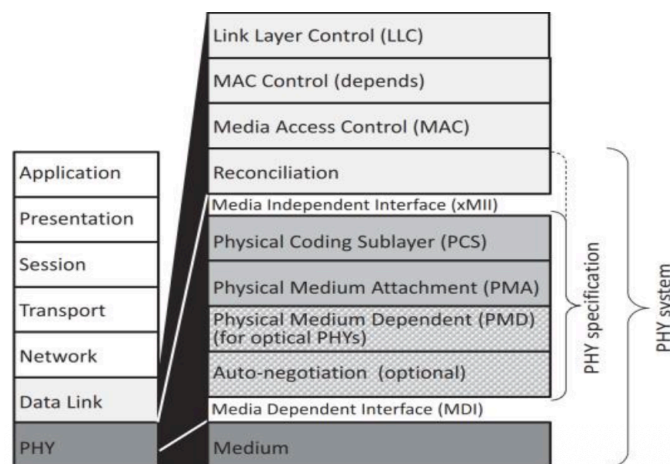
Il meccanismo PLCA garantisce, sulla topologia multidrop half-duplex, una **latenza massima limitata** (*bounded maximum latency*), il cui valore dipende strettamente dal numero di nodi presenti sulla rete e dalla quantità di dati che devono essere trasmessi.

In aggiunta al livello fisico 10BASE-T1S, lo standard IEEE 802.3cg definisce il sottolivello opzionale **PLCA Reconciliation Sublayer** per *garantire il determinismo* (tra MII e MAC)

Il PLCA opera nel *Reconciliation Layer* del sistema PHY IEEE 802.3.

Sottolivelli principali:

- **Reconciliation Layer:** traduce i comandi astratti del MAC nei segnali elettrici standard dell'interfaccia e mappa i segnali dell'interfaccia MII verso le primitive di servizio del sottolivello *Physical Signaling* (PLS) utilizzate nel MAC.
- **xMII (Media Independent Interface):** Connette elettricamente e logicamente il livello PHY con il livello MAC (*Media Access Control*).
- **Livello fisico:**
 - **PCS (Physical Coding Sublayer):** Riceve i dati digitali dalla *Media Independent Interface* (MII) e li codifica in simboli per mandarli al PMA. Il PCS si occupa inoltre di decodificare il segnale ricevuto in un flusso di bit pronto per essere inoltrato ai livelli superiori tramite la MII.
 - **PMA (Physical Medium Attachment):** Elabora il segnale analogico per la trasmissione. Viene utilizzata la codifica **Differential Manchester** (Manchester Differenziale), la quale combina i segnali di clock e di dato in un flusso auto-sincronizzante (*self-synchronizing*).
 - **PMD - Physical Medium Dependent (Opzionale):** È necessario quando si utilizzano mezzi trasmissivi differenti che richiedono una gestione aggiuntiva e specifica del segnale (ad esempio, per uno standard di trasmissione ottica, il PMD definisce la conversione tra i segnali analogici elettrici e quelli ottici).
 - **Auto-negotiation layer (Opzionale):** Stabilisce e seleziona le migliori *capacità di comunicazione* supportate da entrambi i nodi, come la velocità dei dati (*data rate*), la modalità duplex, ecc.
 - **MDI - Media Dependent Interface:** Rappresenta l'interfaccia fisica e meccanica (connettore/pin) tra il canale di comunicazione e la tecnologia del livello fisico.
 - **Il Mezzo:** È il canale di trasmissione fisico effettivo (il singolo doppino di rame).



Il principio fondamentale del bus condiviso (*multidrop*) 10BASE-T1S è che le unità (i nodi) hanno la possibilità di trasmettere in sequenza, in base ai loro ID di nodo, secondo una logica di tipo **Round Robin** (a rotazione ciclica).

Questo meccanismo viene realizzato attraverso il *Reconciliation Sublayer* del sistema PHY IEEE 802.3, mentre il livello MAC è totalmente inconsapevole della presenza del PLCA e continua a eseguire un "classico" protocollo CSMA/CD.

Come funziona il PLCA

A ciascun nodo viene assegnato un **ID**, partendo dall'**ID 0 per il Coordinator** e incrementando i valori fino al numero di nodi - 1.

Il *Coordinatore* ha la responsabilità di inviare il **Beacon** (*segnale di sincronizzazione*), il quale consiste in speciali simboli di dati a livello fisico che permettono ai PHY dei vari nodi di aggiornare i propri timer e contatori PLCA, così da coordinare le rispettive attività di trasmissione (Tx) e ricezione (Rx). Il beacon è un segnale del livello fisico che viene gestito direttamente dal sottolivello di **Riconciliazione**, **non è un pacchetto (o frame) gestito dal livello MAC**.

Tutti i partecipanti alla rete, chiamati **Drop-nodes**, riconoscono il beacon inviato dal Coordinatore. In linea di principio, qualsiasi unità può fungere da nodo Coordinatore; tuttavia, nella pratica, è molto probabile che questo ruolo venga assunto dal nodo che collega il segmento multidrop al resto della rete Ethernet principale (agendo da ponte/switch).

Ogni Beacon avvia un ciclo di rete, in cui ciascun nodo avrà un'opportunità di trasmettere (**Transmit Opportunity, TO**) partendo dal nodo con ID 0 fino al nodo con l'ID più alto, secondo una modalità **round-robin**. Ogni nodo ha due possibilità:

1. Se un nodo ha dati da trasmettere, invierà un comando di **Commit** durante la propria TO per far sapere agli altri nodi che trasmetterà dei dati.
Dopo il *Commit*, il nodo trasmittente invierà i dati fino alla dimensione massima del frame configurata e poi altri due simboli speciali: **end of stream delimiter, ESD**, e il **ESDOK** se non si sono verificati errori.
2. Se un nodo non ha dati da trasmettere in quel momento, non farà nulla.
Dopo un numero configurabile di tempi di bit (*bit times*, il valore predefinito è 32, ma sono possibili altri valori, ad esempio 20), la TO scadrà e si passa al nodo successivo. Un tempo di 32 bit (ovvero 3,2 microsecondi) è significativamente più breve del pacchetto Ethernet di dimensione minima, pertanto la larghezza di banda viene utilizzata in modo ottimale.

Dopo che ciascun nodo ha avuto la propria TO, il ciclo ricomincia con un nuovo Beacon.

PLCA vs TDMA

- Il PLCA evita le **collisioni** fisiche sul mezzo, offre **determinismo**.
- Il PLCA assicura l'equità (*fairness*) dei pacchetti in modalità *round-robin*.
- Il PLCA non è un TDMA: Il TDMA viene tipicamente eseguito dal livello MAC, mentre il PLCA viene eseguito al livello PHY ed è interamente asincrono rispetto ai processi del MAC.

- La durata del ciclo PLCA è variabile: dipende dalla *lunghezza totale dei dati inviati* dai nodi e dal *numero di nodi* che trasmettono.
- Il PLCA supera il TDMA in termini di prestazioni, con ritardi più brevi (lunghezza del ciclo regolabile rispetto a una lunghezza del ciclo fissa) e un migliore utilizzo della larghezza di banda e maggiore flessibilità.
- Il ciclo PLCA **si contrae o si espande nel tempo** in base al numero di nodi che trasmettono in ciascun ciclo e alla quantità di dati effettivamente trasmessi da ogni singolo nodo.

Ottimizza la banda eliminando i tempi morti garantendo al contempo una **latenza massima limitata** (*bounded latency*), poiché ogni nodo ha la certezza matematica di poter trasmettere almeno un pacchetto a ogni ciclo di Beacon.

In Ethernet la dimensione massima di un frame è rigidamente limitata, generalmente 1500 byte per i frame standard. Sapendo che la velocità della rete è fissa a 10 Mbit/s, possiamo calcolare al millisecondo il tempo massimo che un nodo impiega a trasmettere il frame più grande possibile.

Burst Mode

Il 10BASE-T1S consente una modalità **Burst** opzionale, la quale permette alle singole unità di trasmettere un numero configurato di pacchetti aggiuntivi (fino a **255**) all'interno di una TO. *È utile per ridurre l'overhead di protocollo.*

È utile per consentire ai nodi che hanno la necessità di trasmettere una quantità di dati nettamente superiore rispetto agli altri di vedersi assegnare una maggiore larghezza di banda di rete, fino a un massimo di 255 pacchetti di dati aggiuntivi per singolo ciclo.

È ideale per i nodi che generano dati isocroni con un periodo più breve rispetto alla durata complessiva del ciclo PLCA. Poiché questi nodi richiedono una frequenza di trasmissione più elevata di quella normalmente consentita dal tempo di ciclo PLCA, la modalità **Burst** evita sia l'utilizzo di **buffer** di grandi dimensioni, sia l'introduzione di ulteriore complessità sul lato di trasmissione (TX) e di ricezione (RX).

Meccanismo di Segnalazione del Burst

- Il nodo invia il Commit, seguito dal successivo pacchetto di dati.
- Se il nodo ha la necessità di trasmettere un pacchetto di dati aggiuntivo, terminerà il pacchetto con **ESDBRS (ESD Burst Restart) / ESDOK** per indicare che ci sono ulteriori dati in arrivo.

Questo prosegue finché il nodo trasmittente non finalizza il suo ultimo pacchetto di dati, fino a quando non trasmette un numero pari al massimo consentito.

Ecco gli schemi sequenziali lineari con le frecce per ogni singolo caso, pronti da visualizzare :

1. NO BURST (Trasmissione Standard di un singolo pacchetto)

[COMMIT] —> [Pacchetto Dati] —> [ESD][ESDOK]

2. BURST: Transizione tra due pacchetti

[Pacchetto 1] —> [ESDBRS][ESDOK] —> [COMMIT] —> [Pacchetto 2]

3. BURST: Caso 1 (Raggiungimento Limite Massimo)

[COMMIT] —> [Ultimo Pacchetto Consentito] —> [ESD][ESDOK]

4. BURST: Caso 2 (Chiusura anticipata perché il buffer si è svuotato)

[Pacchetto X] —> [ESDBRS][ESDOK] —> [COMMIT] —> [ESD][ESDOK]

Reconciliation Sublayer e MAC

Il PLCA garantisce che ogni ECU trasmetta solo durante la propria **Opportunità di Trasmissione (TO)**. Per farlo senza modificare il livello MAC standard (che ragiona ancora in logica CSMA/CD), sfrutta il **Reconciliation Sublayer (RS)** come intermediario intelligente attraverso i seguenti passaggi:

- **Buffering e Controllo del Flusso:** Quando il MAC invia i dati credendo che il canale sia libero, l'RS li intercetta e li trattiene in un buffer locale, in attesa che arrivi la TO specifica di quel nodo.
- **Lo Stratagemma della "Collisione Logica":** Per evitare che il MAC continui a inviare dati saturando il buffer, l'RS simula una **falsa collisione** interna (locale). Il MAC, credendo che ci sia stato un impatto reale, si ferma ed entra in uno stato di *random back-off* (attesa casuale).
- **Prontitudine del MAC:** Poiché il tempo di back-off è brevissimo (inferiore alla durata di un pacchetto Ethernet), il MAC si rimette subito in stato di "pronto a trasmettere" prima ancora che la linea fisica si sia liberata.
- **Occupazione Artificiale del Canale:** Anche quando il canale torna fisicamente libero, l'RS continua a segnalarlo come "occupato" al MAC. Questo impedisce al nodo di trasmettere prima del tempo, lasciando la precedenza alle unità con TO antecedenti.
- **Arrivo della TO e Segnale di COMMIT:** Quando finalmente scatta la TO del nodo, l'RS segnala al MAC che il canale è libero. Il MAC attende il tempo standard (Inter-Packet Gap) e parte. Durante questa minima attesa, il PHY invia sul bus un comando di **COMMIT** per bloccare preventivamente tutti gli altri nodi e blindare la trasmissione.

Automotive Ethernet

Automotive Ethernet è considerata la soluzione verso una rete di bordo omogenea che sostituirà le molteplici piccole reti connesse tramite gateway utilizzate nelle auto odierne. È disponibile un'ampia gamma di velocità di trasmissione dati (*data rates*) per l'uso automobilistico. In ordine cronologico:

- 100 Mbps (standardizzato come IEEE 802.3bw-2015)
- 1 Gbps (standardizzato come IEEE 802.3bp-2016)
- 10 Mbps (standardizzato come IEEE 802.3cg-2019, noto anche come 10BASE-T1S)
- 2,5, 5 e 10 Gbps (standardizzato come IEEE 802.3ch-2020, noto anche come Multi-Gig Automotive Ethernet PHY).

Le reti di bordo dei veicoli hanno iniziato la transizione dalle architetture elettroniche tradizionali (basate su domini) alle architetture zonali, che utilizzano collegamenti Ethernet per supportare la guida completamente autonoma. Ciò ha generato la necessità di velocità di trasmissione superiori a 10 Gb/s nell'ambiente automobilistico.

- 25 Gbps su una singola coppia bilanciata di conduttori in rame (standardizzato come IEEE 802.3cy-2023).
- Funzionamento a 2,5 Gbps, 5 Gbps, 10 Gbps, 25 Gbps e 50 Gbps su fibra ottica di vetro in ambiente automobilistico (standardizzato come IEEE 802.3cz-2023).

Asymmetrical Mode

Automotive Ethernet è in grado di gestire velocità di trasmissione dati simmetriche, il che significa che trasporta i dati alla stessa velocità in entrambe le direzioni su un cavo automobilistico a singolo doppino. Questa capacità la rende la tecnologia preferita per la dorsale di rete (*network backbone*).

Ethernet può anche operare in modalità asimmetrica quando necessario. Ad esempio, la modalità asimmetrica è utile per i collegamenti di telecamere e sensori. Su questi collegamenti, i dati (video) vengono inviati ad alta velocità (nell'ordine dei Multi-Gbps) dalla telecamera verso il SoC/GPU. Nella direzione opposta (dal SoC alla telecamera) vi sono solo segnali di controllo che devono essere inviati a velocità molto inferiori (nell'ordine dei Mbps) e che possono beneficiare di meccanismi di risparmio energetico. La modalità asimmetrica e le modalità operative ad alta efficienza energetica convergono nei più recenti standard per l'Automotive Ethernet.

Energy-Efficient Ethernet (EEE)

La necessità di specificare l'Energy-Efficient Ethernet (EEE) deriva dall'introduzione dei collegamenti **full-duplex** in Ethernet. Su un collegamento *half-duplex*, le unità trasmettono esclusivamente quando hanno dati da inviare. Un collegamento Ethernet *full-duplex*, nella sua modalità di funzionamento normale, consuma energia in entrambe le direzioni anche quando il collegamento è inattivo (*idle*) e non viene trasmesso alcun dato. Ciò accade perché, in modalità *full-duplex*, le unità che non hanno dati da inviare devono comunque trasmettere continuamente simboli di *idle* (inattività), al fine di monitorare costantemente le condizioni di trasmissione e

mantenere i PHY sincronizzati, causando sprechi di energia. Lo standard **IEEE 802.3az** – *CSMA/CD Access Method and Physical Layer Specifications Amendment 5: Media Access Control Parameters, Physical Layers, and Management Parameters for Energy-Efficient Ethernet* – include la definizione di una modalità denominata **Low Power Idle (LPI)** e dei relativi meccanismi per comunicare e gestire l'ingresso, l'uscita e il funzionamento all'interno di questa modalità a basso consumo. Il principio alla base dell'Energy-Efficient Ethernet (EEE):

- Ridurre al minimo assoluto l'attività sul collegamento (*link*) quando non vi è traffico dati in transito.
- Ripristinare la massima velocità operativa del collegamento nel modo più rapido possibile non appena si presenta del nuovo traffico.

Il Protocollo di Segnalazione LPI

Lo standard IEEE **802.3az** specifica un protocollo di segnalazione progettato per ottenere un risparmio energetico durante i periodi di inattività (*idle time*).

Questo avviene attraverso lo scambio di indicazioni LPI, utilizzate per segnalare la transizione verso la modalità a basso consumo quando non ci sono dati da inviare.

L'LPI indica esattamente il momento in cui un collegamento può entrare in stato di inattività e quando deve riprendere la normale operazione (*resume*) dopo un ritardo predefinito.

Modalità Low Power Idle (LPI): Quando due unità collegate a un link rilevano che il traffico è composto esclusivamente da pacchetti di inattività (*idle packets*), esse possono concordare di entrare nella modalità LPI.

Durante lo stato LPI:

Un periodo definito di assenza totale di trasmissione (denominato *zero transmission*, ad esempio della durata di 20 ms) è seguito da un periodo definito di aggiornamento attivo (*active refresh*, ad esempio di 200 μ s), il quale garantisce che i PHY rimangano perfettamente sincronizzati.

Grazie a questo aggiornamento regolare (*refresh*), il collegamento può essere riattivato molto rapidamente non appena vi sono nuovamente dati da inviare. Lo standard specifica i seguenti tempi massimi di avvio (*start-up times*) per il ripristino della piena operatività:

- **16,5 μ s** per lo standard Ethernet a 100 Mbps.
- **30 μ s** per lo standard Ethernet a 1 Gbps.

In base al rapporto tra gli intervalli di assenza di trasmissione (*zero transmission*) e gli intervalli di aggiornamento (*refresh*), è possibile ottenere un risparmio energetico sul collegamento che può raggiungere **fino al 90%**.

Il PHY Automotive Ethernet 100BASE-T1 non specificava una modalità a basso consumo.

Il 1000BASE-T1 include l'EEE come funzionalità opzionale.

Il 10BASE-T1S in modalità half-duplex non richiede l'EEE, poiché la trasmissione avviene esclusivamente quando ci sono dati da inviare. Per la modalità opzionale full-duplex del 10BASE-T1S, l'EEE è opzionale.

Gli standard Ethernet a 2,5 Gbps, 5 Gbps e 10 Gbps (noti collettivamente come MultiGBASE-T1) includono l'EEE come funzionalità opzionale. Questi standard consentono inoltre un risparmio energetico in ricezione permettendo una modalità di risveglio lento ("*slow wake*"), particolarmente utile sui collegamenti asimmetrici.

Asymmetrical mode and Energy-Efficiency

Il MultiGBASE-T1 abilita inoltre un risveglio lento ("*slow wake*"). Oltre all'energia risparmiata in trasmissione, lo *slow wake* consente di ottenere un risparmio energetico in ricezione e, all'occorrenza, di essere pronti per un riavvio rapido.

Normalmente in modalità LPI, mentre il trasmettitore può rimanere spento tra l'invio dei vari segnali di *refresh*, il ricevitore deve restare sempre sveglio e pronto a riattivarsi rapidamente in qualsiasi momento, al fine di non perdere l'istante in cui la comunicazione riprende.

Quando si trova in modalità "*slow wake*", il risveglio è possibile solo subito dopo un segnale di *refresh*. Ciò significa che il ricevitore deve rimanere sveglio e pronto esclusivamente a intervalli definiti, potendo risparmiare energia nel tempo intermedio.

Una caratteristica dell'EEE che ha guadagnato maggiore attenzione con il MultiGBASE-T1 è la possibilità di implementare l'Energy-Efficient Ethernet in modo asimmetrico.

Caratteristiche e Vantaggi:

- Maggiore è la velocità di trasmissione dati (*data rate*), maggiore è l'energia necessaria per la sola comunicazione, ed è più probabile che la trasmissione dei dati risulti altamente asimmetrica.
- Si prevede che per molti utilizzi del MultiGBASE-T1 la trasmissione dei dati avverrà prevalentemente in un'unica direzione.
- Dal punto di vista del consumo energetico, è vantaggioso che l'ingresso nella modalità LPI (*Low Power Idle*) possa avvenire separatamente per ciascuna direzione.
- Il meccanismo di LPI asimmetrico specificato per il MultiGBASE-T1 consente l'ingresso in modalità LPI per singola direzione di trasmissione, indipendentemente dalla comunicazione nella direzione opposta.
- Se il supporto a un maggiore risparmio energetico tramite l'EEE asimmetrico è stato considerato essenziale per le velocità del MultiGBASE-T1, lo è ancora di più per i livelli di velocità superiori a 10 Gbps.

Domain-based vs zonal architectures

I principali megatrend nel settore automobilistico sono la **personalizzazione**, l'**elettrificazione**, la **guida autonoma** e la **connettività totale**. Per supportare queste nuove funzionalità, i produttori di automobili (OEM) devono modificare radicalmente la loro architettura elettrica ed elettronica (E/E), allontanandosi dalle soluzioni domain based, le quali sono diventate ingombranti e difficili da gestire a causa delle continue aggiunte (*add-ons*) stratificatesi nel corso delle generazioni di piattaforme. **L'architettura basata su domini offre una capacità limitata di far evolvere le funzionalità dell'auto e di integrarne di nuove.**

L'industria automobilistica si sta muovendo verso una nuova architettura comune, denominata **architettura zonale**. A differenza delle architetture basate su domini, le architetture zonali determinano la connettività dei dispositivi in base alla loro **posizione fisica** all'interno del veicolo piuttosto che alla loro funzione logica.

Vantaggi:

- Nelle architetture zonali, la maggior parte delle funzioni del veicolo è consolidata in **poche ECU di calcolo ad alte prestazioni** (*High-Performance Computing ECUs / Central Compute Nodes*). Queste centraline eseguono software che viene sviluppato prevalentemente in-house dagli stessi OEM per differenziare l'esperienza di guida.
- Le architetture zonali segnano un passaggio dalla distribuzione logica delle funzionalità, tipica dell'architettura a domini, alla loro distribuzione fisica: il cablaggio dei componenti elettronici non dipende dalle loro funzioni, ma dalla loro posizione fisica all'interno della vettura. Le architetture domain-based, invece, è frammentata e non è ottimizzata per il cablaggio. Questo ha un forte impatto in termini di costi e spazio occupato.
- Diversi controllori **gateway zonali** (*zonal gateway controllers*) gestiscono la comunicazione con i sensori e gli attuatori di una specifica area fisica, aggregando e combinando i protocolli di rete richiesti da ciascuno dei dispositivi.
- I gateway zonali sono collegati tra loro e con la CPU centrale (o computer ad alte prestazioni) attraverso una dorsale Ethernet (Ethernet backbone). **L'architettura zonale semplifica il layout dei sensori**, degli attuatori e dei cablaggi all'interno dell'auto, con una significativa riduzione del peso del fascio di cavi. Poiché è presente un numero inferiore di ECU, si ottiene inoltre una riduzione dei costi e dello spazio richiesto.

Ciascun gateway zonale (*zonal gateway*) deve gestire molteplici protocolli di rete tradizionali (*legacy*) utilizzati per interconnettere sensori, attuatori e ECU. Per fare questo, in ogni zona verranno impiegati gateway Ethernet operanti a velocità di 10 Mbps, 100 Mbps, 1 Gbps, fino a 10 Gbps, i quali saranno collegati alle reti automobilistiche tradizionali (come **LIN, CAN, CAN FD e FlexRay**) per eseguire funzioni di conversione di protocollo e funzioni di controllo rapido nella periferia dell'auto. **Lo standard Ethernet a 10 Mbps si adatta perfettamente all'architettura zonale per fungere da bus condiviso (*multidrop bus*) all'interno delle singole zone**, con un potenziale elevato per sostituire nel tempo le altre tecnologie di rete tradizionali (CAN, FlexRay).

Le comunicazioni tra le diverse zone passeranno invece attraverso una dorsale Ethernet ad altissima velocità (*ultra-high speed Ethernet backbone*), operante a velocità comprese **tra 10 Gbps e 25 Gbps**.

Il Ruolo dell'Ethernet Oltre i 10 Gbps nelle Architetture Zonali

Le architetture zonali consolidano molte delle funzioni delle ECU multi-dominio (*cross-domain*) in un esiguo numero di ECU di rete a livello di supercomputer. Per sfruttare appieno il potenziale delle reti di bordo (*In-Vehicle Networks - IVN*) basate su Ethernet, i nodi all'interno dell'auto dovranno "parlare" nativamente in Ethernet, inclusi sensori e telecamere; pertanto, saranno necessarie anche velocità di trasmissione dati superiori a 10 Gbps.

Software Defined Vehicle

Nelle architetture zonali, uno o più server di bordo ospiteranno il software per tutte le funzioni del veicolo. Un server è un computer multi-ruolo che sfuma i confini tra i vari domini, frammentandoli ulteriormente in singoli moduli software.

Il server si collega a un esiguo numero di ECU che fungono da gateway zonali all'interno del veicolo. I gateway possono essere collegati in un'architettura ad anello o a stella, a seconda dei requisiti di ridondanza. Questo ridurrà in modo significativo i costi di sviluppo, dell'hardware e del cablaggio.

Le funzioni diventano pacchetti software (SW); di conseguenza, l'aggiunta di funzionalità basate sul software a un determinato modello di veicolo per creare una differenziazione commerciale diventa una semplice modifica software all'interno di una progettazione E/E comune. L'aggiunta di nuove funzionalità sarà realizzata tramite aggiornamenti del firmware, anche dopo la fase di produzione (*post-production*).

Un **Software-Defined Vehicle (SDV)** è un qualsiasi veicolo che gestisce le proprie operazioni, aggiunge funzionalità e abilita nuove caratteristiche principalmente o interamente attraverso il software. Il software dovrebbe essere sviluppato principalmente all'interno dagli stessi OEM (produttori di automobili) e in modo separato dall'hardware, al fine di rendere possibili aggiornamenti frequenti.

Disaccoppiando il software dall'hardware, i produttori di veicoli possono adottare pratiche di sviluppo più agili e modulari, nonché sviluppare applicazioni ed ecosistemi che operano indipendentemente dall'hardware stesso.

- Il **livello hardware** (*hardware layer*) di un SDV consiste generalmente in un computer di infotainment di bordo, un computer ADAS, i controlli per gli esterni e gli interni, un meccanismo di sterzo centrale e un modulo di connettività.
- Al di sopra di questo livello hardware, un livello software esteso (*expanded software layer*) controlla tutti i processi, incluse le funzioni generali e le API.
- Mantenere la separazione tra le funzioni e l'hardware sottostante attraverso la **virtualizzazione** migliorerà inoltre le prestazioni, poiché i produttori di veicoli potranno focalizzarsi sul miglior hardware possibile senza doversi preoccupare della compatibilità.

L'idea alla base del Veicolo Definito dal Software è quella di disporre di un computer centrale dotato di una potenza di calcolo nettamente superiore, più robusto e più flessibile di quanto il veicolo richieda al momento della sua "nascita" (produzione).

Ciò consentirà di:

- elaborare i volumi estremamente elevati di dati raccolti dai vari sensori del veicolo, in particolare dai sistemi di assistenza alla guida (ADAS), dai servizi multimediali e di connettività, dal telaio (*chassis*)...
- ospitare ed integrare nuove funzioni nel corso del tempo senza rallentare il sistema.

Per aggiungere nuove caratteristiche al veicolo, il *Software-Defined Vehicle* si affida al sistema operativo del veicolo (*vehicle operating system*), in modo simile a quanto avviene in un computer o in uno smartphone, ma adattato alla plancia di comando.

Le nuove applicazioni che verranno sviluppate saranno disponibili all'interno di un negozio di applicazioni (*application store*) sul cloud dell'OEM e saranno perfettamente compatibili con il sistema operativo del veicolo.

Vantaggi: I SDV superano i loro predecessori basati sull'hardware (*hardware-defined*) sotto molteplici aspetti, ovvero:

- **Sicurezza migliorata:** attraverso funzionalità quali i sistemi anticollisione e l'assistenza alla guida. Grazie all'enorme assorbimento di dati raccolti dall'auto, alla potenza di calcolo e all'intelligenza artificiale offerte dal *Software-Defined Vehicle*, la sicurezza a bordo risulterà potenziata.
- **Maggiore comfort:** ad esempio tramite sistemi di infotainment di bordo che integrano lo streaming di musica e video.
- **Comprensione più approfondita delle prestazioni del veicolo:** attraverso la telematica e la diagnostica, consentendo una manutenzione preventiva più efficace. Ad esempio, l'usura di determinati componenti o i guasti possono essere identificati in tempo reale, il che aiuterà ad anticipare le riparazioni o a correggere direttamente le anomalie.
- **Connettività continua (cloud):** per fornire e ricevere servizi di informazione da e verso la vettura.

Gli SDV, attraverso funzionalità quali la personalizzazione del veicolo, la vendita di servizi e gli aggiornamenti Over-The-Air (OTA), abilitano inoltre l'ottimizzazione e nuovi ricavi post-vendita per gli OEM (produttori di automobili).

- **Personalizzazione:** la capacità di aggiungere nuove funzionalità con aggiornamenti *Over-The-Air*, consentendo ai conducenti stessi di aggiungere in modo fluido e flessibile caratteristiche e servizi (ad esempio, nuove funzionalità dell'abitacolo, gestione della carica della batteria, infotainment) forniti come servizio, sia tramite abbonamento (*subscription*) sia per acquisto singolo.

Grazie ad aggiornamenti regolari lungo l'intero ciclo di vita, il **valore residuo** del veicolo può essere migliorato; in altri termini, il veicolo perderà meno valore nel tempo se è stato arricchito con nuove funzioni.

- **Ottimizzazione:** i produttori possono incrementare le prestazioni del veicolo dopo che l'auto ha lasciato la fabbrica attraverso uno sviluppo continuo, migliorando così perpetuamente l'esperienza di guida via cloud, mediante aggiornamenti delle caratteristiche del veicolo, upgrade software e miglioramenti.

Questo rappresenta il **più significativo cambio di paradigma** per l'industria automobilistica, poiché le auto tradizionali "definite dall'hardware" (*hardware-defined*) tendono, al contrario, a rimanere invariate per tutto il loro ciclo di vita.

Ethernet offre molti PHY ad alta larghezza di banda, ma la velocità da sola non è sufficiente a garantire un ritardo limitato (*bounded latency*).

Una preoccupazione immediata quando si valuta Ethernet come tecnologia di rete all'interno di un veicolo è che, nella sua forma base, lo **Switched Ethernet** (Ethernet commutato) non fornisce garanzie sui ritardi di comunicazione.

Tuttavia, lo sviluppo da parte di IEEE di famiglie di standard quali **Audio Video Bridging (AVB)** e **Time-Sensitive Networking (TSN)** ha risolto il problema. Questi standard combinano diverse tecniche di prenotazione della banda (*bandwidth reservation*), controllo del traffico (*traffic policing*), regolazione del flusso (*shaping*) e pianificazione (*scheduling*) al fine di fornire garanzie sulla latenza dei frame Ethernet attraverso la rete.

In particolare,

- L'**Ethernet Audio Video Bridging (AVB)** offre una Qualità del Servizio (*Quality of Service - QoS*) potenziata per i flussi Audio/Video ed è particolarmente adatto per i sistemi multimediali e di infotainment.
- Per i domini critici per la sicurezza (*safety-critical*), che richiedono non solo un'elevata larghezza di banda, ma anche determinismo e alta disponibilità (*high availability*), le estensioni **IEEE Time-Sensitive Networking (TSN)** rappresentano la soluzione.

802.1Q

L'**IEEE** è la più grande associazione professionale al mondo dedicata al progresso educativo e tecnico dell'ingegneria elettrica ed elettronica, delle telecomunicazioni, dell'ingegneria informatica e delle discipline affini. L'IEEE è sponsor tecnico di conferenze, programmi educativi, riviste tecniche e pubblica standard che vengono prodotti dai suoi comitati di standardizzazione. Il **Comitato per gli Standard IEEE 802 LAN/MAN** sviluppa e mantiene standard di rete e pratiche raccomandate per reti locali (LAN), metropolitane (MAN) e altre tipologie di reti d'area. Alcuni tra gli standard più noti sono:

- **802.1:** *Bridging* e Architettura (generalmente la parte superiore del livello di collegamento / *link layer*).
- **802.3:** Ethernet.
- **802.11:** Wireless LAN (WLAN / Wi-Fi).
- **802.15:** Wireless Personal Area Network (WPAN / i.e. Bluetooth).

I Gruppi di Lavoro (*Working Groups* - WG) all'interno dell'IEEE sono responsabili dello sviluppo degli standard in una determinata area. Il **Gruppo di Lavoro IEEE 802.1** sviluppa standard e pratiche raccomandate nelle seguenti aree:

- Architettura delle reti 802 LAN/MAN
- Interconnessione (*internetworking*) tra reti 802 LAN
- Reti MAN e altre reti geografiche (WAN)
- Sicurezza nelle reti 802
- Gestione complessiva della rete 802
- Livelli di protocollo superiori ai livelli MAC e LLC.

In questo ambito siamo interessati allo standard **IEEE 802.1Q**, lo standard di rete che supporta le reti locali virtuali (**VLAN**) su una rete Ethernet IEEE 802.3.

Titolo completo: 802.1Q-2022 (lo standard più recente pubblicato finora) - *IEEE Standard for Local and Metropolitan Area Networks—Bridges and Bridged Networks*.

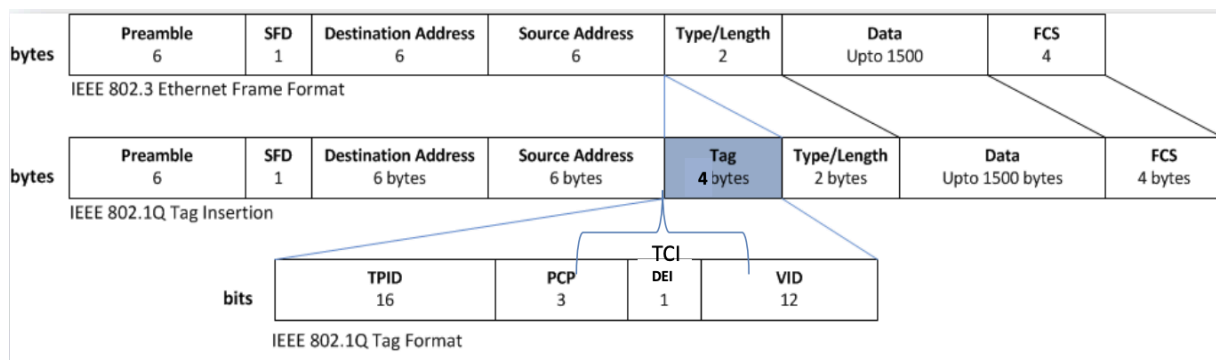
Nel corso del tempo sono state definite diverse versioni dello standard IEEE 802.1Q. Nelle diapositive successive faremo riferimento allo standard del 2014, che definisce:

- un sistema di etichettatura delle VLAN (*VLAN tagging*) per i frame Ethernet.
- le procedure che devono essere utilizzate dagli switch nella gestione di tali frame.
- uno schema di prioritizzazione per la Qualità del Servizio (*Quality of Service* - QoS).
Tuttavia, senza reali garanzie in tempo reale (*real-time guarantees*).

Nota: Successivamente vedremo che i progetti IEEE 802.1 sono identificati utilizzando nomi di progetto come, ad esempio, 802.1Q, 802.1AS, 802.1Qat, 802.1Qav.

- Subito dopo la dicitura "802.1" seguono una, due, tre o persino quattro lettere.

- Le **lettere maiuscole** identificano gli standard (indipendenti / *standalone*), mentre le **lettere minuscole** identificano gli emendamenti (*amendments*, precedentemente chiamati supplementi) a standard esistenti.
- Le forme a tre e quattro lettere sono state introdotte per identificare meglio gli emendamenti.
- In questo schema, le prime una o due lettere (sempre maiuscole) identificano lo standard che viene modificato, e le ultime due lettere (sempre minuscole) identificano il progetto specifico che apporta tale emendamento a quello standard.
- La notazione **802.1Q-REV** viene utilizzata per identificare una revisione di uno standard esistente: si tratta di modifiche al testo originale decisamente più estese rispetto a quelle che possono essere intraprese attraverso un semplice emendamento.



Frame

Lo standard **IEEE 802.1Q** introduce nel frame Ethernet un campo **Tag lungo 4 byte**. Questo campo viene sfruttato per fornire la prioritizzazione del traffico e la gestione delle VLAN.

Il campo Tag contiene:

- **TPID (Tag Protocol Identifier)**: Indica il tipo di frame; assume il valore specifico **0x8100** per identificare i frame dotati di tag 802.1Q.
- **TCI (Tag Control Information)**, il quale a sua volta è suddiviso in **PCP**, **DEI** e **VID**.
 - **PCP (Priority Code Point)**: È un campo a **3 bit** (con valori compresi nell'intervallo da 0 a 7) che si riferisce alla classe di servizio (*Class of Service - CoS*) e si mappa sul livello di priorità del frame. Valori di PCP differenti vengono utilizzati per assegnare priorità diverse a differenti classi di traffico. Un valore più alto indica una priorità maggiore.
 - **DEI (Drop Eligible Indicator)**: È un campo a **1 bit** che consente di contrassegnare i frame che possono essere scartati (*eligible to be dropped*) in caso di congestione della rete.
 - **VID (VLAN Identifier)**: È un campo a **12 bit** che indica la specifica VLAN a cui appartiene il frame.

Audio Video Bridging (AVB)

L'AVB estende lo standard **IEEE 802.1Q** per fornire il supporto ai flussi audio/video (A/V) su reti *Switched Ethernet*, fornendo le specifiche per servizi di streaming sincronizzati nel tempo e a bassa latenza.

IEEE 802.1 Audio Video Bridging (AVB) Task Group (TG)

- **Inizio:** nato nel 2005
- **Obiettivo:** rivolgersi al mercato dell'audio e del video professionale
- **Settori d'applicazione:** elettronica di consumo e *infotainment* automobilistico

Lo standard **IEEE 802.1 AVB** introduce una serie di concetti per garantire la **Qualità del Servizio**:

- **Livelli di priorità:** per distinguere i flussi *time-sensitive* dal traffico ordinario, consentendo di gestirli in modo differente. Ai flussi TS viene assicurata una **latenza limitata** (*bounded latency*), mentre il traffico ordinario viene gestito secondo una logica *best-effort* (senza garanzie).
- **Bandwidth Reservation:** per riservare una determinata quantità di banda garantita lungo una porzione di rete, destinata esclusivamente alla gestione del traffico ad alta priorità.
- **Protocolli per la gestione del tempo di rete:** per supportare operazioni sincronizzate (ad esempio, la riproduzione A/V o la navigazione passo-passo).

Latenza limitata (Bounded latency)

Per un percorso fino a **sette hop** (salti) all'interno della rete, lo standard IEEE AVB garantisce un limite massimo fisso per la latenza. Vengono definite due classi ad alta priorità, chiamate **classi SR** (Stream Reservation). *La somma dei contributi di latenza nel caso peggiore (worst-case) per singolo hop deve tradursi in una latenza end-to-end complessiva:*

- **Classe A (SR A) : 2 ms**
- **Classe B (SR B) : 50 ms**

Una tipica configurazione AVB di **7 hop** dal *talker* (sorgente) al *listener* (destinatario) soddisfa appieno questi requisiti di latenza.

Con l'AVB, l'IEEE ha portato Ethernet nel dominio delle applicazioni in tempo reale!

Le principali specifiche che compongono lo standard sono le seguenti:

- IEEE 802.1**AS**: **Timing and Synchronization for Time-Sensitive Applications**
- IEEE 802.1**Qat**: **Stream Reservation Protocol (SRP)**
- IEEE 802.1**Qav**: **Forwarding and Queuing Enhancements for Time-Sensitive Streams (FQTSS)**

Ognuna di queste tecnologie può essere utilizzata singolarmente, ma è la loro combinazione a realizzare un sistema AVB completo. L'interazione tra i diversi standard AVB è definita nel documento:

IEEE 802.1BA-2011: Audio/Video Bridging (AVB) Systems

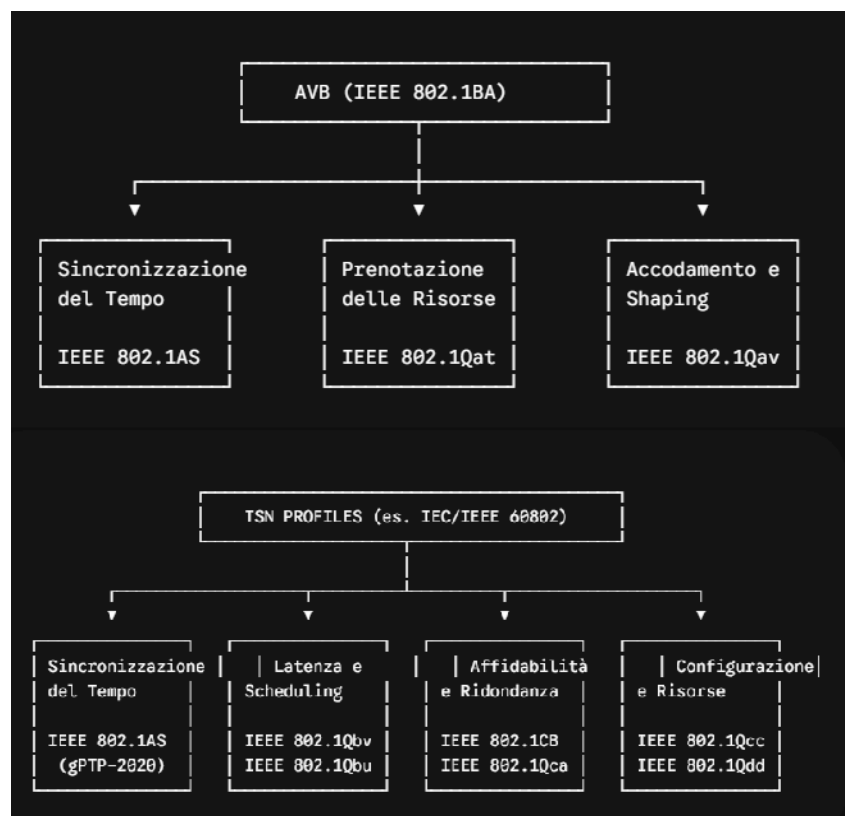
Definisce come gli standard IEEE 802.1 AVB sopra menzionati possano cooperare per realizzare un sistema AVB funzionante.

Stabilisce come definire i **profili**, selezionare le funzionalità dei protocolli, le opzioni, le configurazioni, i valori predefiniti (*defaults*), i protocolli e le procedure per bridge, stazioni e reti LAN, al fine di costruire reti in grado di fornire servizi A/V.

IEEE 802.1BA - Audio Video Bridging (AVB) Systems

Questo standard specifica le funzionalità da supportare e i parametri di configurazione per gli standard IEEE 802.1 al fine di implementare i sistemi AVB; tali specifiche prendono il nome di **Protocol Implementation Conformance Statement (PICS)** (Dichiarazione di Conformità dell'Implementazione del Protocollo).

Il fornitore di un'implementazione di protocollo per la quale si dichiara la conformità allo standard AVB deve fornire il documento PICS, attestando quali funzionalità e opzioni del protocollo siano state effettivamente implementate, insieme ai risultati dei test di conformità del protocollo (*protocol conformance test results*).



802.1Qat : Stream Reservation Protocol

L'**SRP** (Stream Reservation Protocol) è un meccanismo di controllo delle ammissioni *che garantisce la presenza di una larghezza di banda sufficiente lungo l'intero percorso tra la sorgente e la destinazione di qualsiasi flusso A/V prima che il trasferimento abbia inizio.*

L'SRP ha il compito di configurare le prenotazioni attraverso una rete commutata di livello 2 (Layer-2 switched network). Le risorse di rete (*) vengono allocate e configurate sia nei nodi terminali (sorgente e destinazioni del flusso dati), sia negli switch lungo il percorso che collega la sorgente alle destinazioni.

Viene inoltre fornito un meccanismo di segnalazione end-to-end per rilevare il successo o il fallimento del tentativo di prenotazione.

(*) larghezza di banda, spazio nelle tabelle interne dello switch...

Concetti base e terminologia AVB

Ogni flusso A/V (flusso di classe SR), prima di essere inviato in rete dalla sorgente del flusso (**Talker**), deve essere registrato sugli switch (bridge) che lo inoltreranno lungo la rete fino alla destinazione del flusso (**Listener**).

La **registrazione** è possibile se e solo se ogni singolo switch lungo il percorso dal Talker al Listener dispone di larghezza di banda e risorse sufficienti. Se la registrazione viene completata con successo, la quantità di banda richiesta dal nuovo flusso viene sottratta, su ciascuna porta attraversata dal flusso stesso, dalla quantità totale di banda disponibile.

Una volta determinata la percentuale di larghezza di banda da riservare alle classi SR, la parte rimanente resta a disposizione del traffico ordinario (*best-effort*). Viene stabilito un limite massimo alle prenotazioni complessive per tutte le classi SR (tipicamente il **75%**) al fine di prevenire il blocco totale del traffico non-SR (ovvero il traffico *best-effort*).

I **Talker** annunciano alla rete la disponibilità dei propri flussi inviando messaggi di **Talker Advertise** a un singolo Listener (unicast) o a un indirizzo multicast. Questi messaggi costituiscono delle vere e proprie dichiarazioni delle caratteristiche dei flussi e si propagano attraverso gli switch.

I **Listener** si iscrivono ai flussi desiderati inviando messaggi di **Listener Ready**, con i quali richiedono formalmente la prenotazione della larghezza di banda necessaria per ricevere tali flussi.

I messaggi di **Talker Advertise** contengono, tra gli altri, i seguenti campi:

- **StreamID**: composto *dall'indirizzo MAC della sorgente e dall'ID del flusso*, utilizzato per distinguere più flussi provenienti dallo stesso indirizzo.
- **Stream Priority**: ovvero l'ID della Stream Reservation..

- **Rank (emergency/non-emergency):** indica il livello di importanza del flusso dati. Il sistema di classificazione (*Ranking*) consente ai Bridge (switch) di sostituire i flussi meno importanti con quelli più importanti senza richiedere l'intervento delle stazioni terminali.
- **Traffic Specifications (Tspec):** specifiche del traffico, costituite da due parametri:
 - La dimensione massima del frame (**Max Frame Size**).
 - Il numero massimo di frame per intervallo di misurazione della classe.
 - L'intervallo di misurazione della classe è il periodo di trasmissione definito per la specifica classe e il suo valore dipende dalla Classe SR.

Per esempio, la Classe A è destinata a flussi con requisiti di latenza più stringenti e ha un intervallo di misurazione più breve; ciò significa che i suoi pacchetti di streaming sono più piccoli e vengono trasmessi più frequentemente. I valori numerici effettivi sono definiti dall'emendamento IEEE 802.1Qav-2009 (FQTSS):

- **125 µs** per la Classe SRA (*8000 frame al secondo*)
- **250 µs** per la Classe SR B (*4000 frame al secondo*).
- **Latenza accumulata:** misurata in nanosecondi (ns), viene aggiornata a ogni hop (salto) per calcolare la quantità totale di latenza che il flusso può subire nel tragitto dal Talker al Listener.

Meccanismo

Una volta che il Talker ha emanato il Talker Advertise, gli switch propagano le dichiarazioni, inoltrando o modificando il messaggio di *Talker Advertise* in base al loro stato specifico.

I Listener dichiarano la propria intenzione individualmente, ma i bridge (switch) intermedi tra il Listener e il Talker **aggregano le registrazioni dei Listener** per un determinato flusso man mano che queste si propagano a ritroso verso il Talker.

Gli switch lungo il percorso configurano le prenotazioni di banda per tali flussi e inseriscono l'indirizzo di destinazione del flusso nella **forwarding table** del bridge per ciascuna porta di uscita (*egress port*) che conduce ai Listener. La **forwarding table** consentirà il passaggio dei dati di streaming esclusivamente attraverso le porte dello switch che conducono ai Listener designati.

- Se le risorse sono disponibili, gli switch inoltrano il messaggio di **Talker Advertise**.
 - *Se un potenziale Listener riceve un Talker Advertise, significa che tutte le risorse lungo il percorso dal Talker al Listener sono disponibili.*
- Se non vi è sufficiente larghezza di banda per quel determinato flusso, modificano il *Talker Advertise* in un messaggio di **Talker Failed**.
 - Se un Listener riceve Talker Failed, quest'ultimo manda a ritroso verso il Talker un messaggio di **Listener Asking Failed**.
 - Il bridge riceve il *Listener Asking Failed* e invia quindi un messaggio di **Ready Failed** per segnalare al Talker che almeno un percorso a valle verso un Listener

ha una *prenotazione attiva* ma almeno un percorso verso un altro Listener a valle dispone di risorse *insufficienti per stabilire una prenotazione*.

In sintesi:

- Se si riceve un **Asking Failed**, significa che lo stream è bloccato per **tutti** i destinatari posizionati lungo quella tratta.
- Se si riceve un **Ready Failed**, significa che lo stream ha abbastanza banda per viaggiare lungo la rete e raggiungere **almeno uno** dei destinatari, ma non tutti.

Esempio: Ipotizziamo che vi siano un Talker T e due Listener, L1 e L2.

- Il percorso verso L1 è: T - B1 - B2 - B3 - B4 - L1.
- Il percorso verso L2 è: T - B1 - B2 - B3 - B5 - B6 - L2.

Scenario 1

Tutti i bridge B1, B2 e B4 che collegano a L1 dispongono di larghezza di banda e risorse sufficienti per il flusso, pertanto L1 riceve il messaggio di *Talker Advertise* e risponde con un *Listener Ready*. Al contrario, il bridge B5 sul percorso verso L2 ha una larghezza di banda insufficiente, quindi la registrazione presso quel determinato bridge non è possibile.

Il bridge B5 trasforma il *Talker Advertise* in un messaggio di **Talker Failed** che viene inviato a L2.

- L2 invia a ritroso verso il Talker un messaggio di **Listener Asking Failed**.
- Il bridge B3, che è collegato sia a B4 sia a B5, riceve il messaggio di *Listener Ready* da L1 (tramite B4) e il messaggio di *Listener Asking Failed* da L2 (tramite B5).
- Il Bridge B3 invia quindi un messaggio di **Ready Failed** per segnalare al Talker che almeno un percorso verso un Listener a valle (ovvero L1) ha una prenotazione attiva, ma almeno un percorso verso un altro Listener a valle (L2) dispone di risorse insufficienti per stabilire una prenotazione.

Scenario 2

Le risorse sono insufficienti sia su B4 sia su B5. In questo caso, il Talker riceverà da B3 un messaggio di **Asking Failed**, il quale indica che non è stata stabilita alcuna prenotazione.

802.1Qcc - Ottimizzazioni e Miglioramenti delle Prestazioni dello SRP

Lo standard **IEEE 802.1Qcc** (*Stream Reservation Protocol (SRP) Enhancements and Performance Improvements*) è un **emendamento allo standard 802.1Qat** che estende le funzionalità dell'SRP per scopi di gestione delle risorse e configurazione della rete.

In particolare, lo standard IEEE 802.1Qcc descrive i protocolli per fornire supporto alle classi di prenotazione dei flussi (*Stream Reservation classes*) e ai flussi (*streams*) configurabili.

Lo standard IEEE 802.1Qcc definisce tre modelli di configurazione della rete, ovvero:

- il modello completamente distribuito (*fully distributed*),
- il modello a rete centralizzata/utente distribuito (*centralized network/distributed user*)
- il modello completamente centralizzato (*fully centralized*).

Modello Completamente centralizzato

A differenza dei modelli decentralizzati (dove i nodi negoziano tra loro), qui i nodi non sanno come instradare o pianificare il traffico: c'è un "cervello" che calcola tutto dall'alto per garantire latenza zero e assenza di congestione.

Le due entità centrali sono il **CNC** e il **CUC**.

- L'entità chiamata **Centralized User Configuration (CUC)** ha il compito di scoprire i nodi terminali (*end nodes*) e i *requisiti delle applicazioni* per calcolare i *requisiti dei flussi* (periodo, dimensione del frame, vincoli di latenza e ridondanza come l'802.1CB), i quali vengono a loro volta trasmessi a un Centralized Network Configurator.
- Il **Centralized Network Configurator (CNC)** riceve le informazioni e calcola e propaga la *configurazione di rete* ai bridge (switch). Il CNC agisce come il cervello della rete, mantiene una visione globale della topologia e si interfaccia con gli switch tramite protocolli di gestione (ad esempio, NETCONF o RESTCONF). Esso calcola matematicamente i percorsi ottimali e i programmi di schedulazione temporale (le *Gate Control List* dell'802.1Qbv) per ogni singolo switch.

Una funzione fondamentale del CNC è il **controllo di ammissione (*admission control*)**: esso valuta se il flusso richiesto possa essere ospitato senza violare i vincoli esistenti. Se la rete è in grado di supportare il nuovo flusso, il CNC calcola la pianificazione (*schedule*) necessaria e invia (*push*) la configurazione agli switch interessati.

Il modello completamente centralizzato è il più adatto per le **applicazioni automotive**.

Svantaggi di SRP

L'utilizzo del protocollo SRP (*Stream Reservation Protocol*) nel settore automotive comporta tre sfide principali (*):

Inaccettabilità del rifiuto di una prenotazione

Il rifiuto di una richiesta di prenotazione IEEE 802.1Qat all'interno di un'autovettura non è ammissibile. Sarebbe disastroso se, ad esempio, una funzione di assistenza alla guida (ADAS) che utilizza i dati della telecamera fallisse l'attivazione solo perché un passeggero sul sedile posteriore sta guardando un video in alta definizione (HD).

È pertanto essenziale che la rete AVB e i tassi di trasmissione previsti (inclusi quelli dei dispositivi consumer) siano accuratamente pianificati in anticipo, e che queste considerazioni si riflettano direttamente nella progettazione dell'architettura di rete (*network design*).

Overhead (Carico di servizio)

I frame della Classe A sono di piccole dimensioni e vengono inviati a intervalli temporali brevi. Avere un overhead significativamente superiore rispetto al carico utile (*payload*) costituisce uno spreco di larghezza di banda e si traduce in un carico di elaborazione inutilmente elevato in tutti i dispositivi coinvolti. È più vantaggioso utilizzare frame più lunghi.

Per migliorare il rapporto tra payload e overhead, è quindi vantaggioso introdurre nuove classi di traffico, come quella identificata con la lettera "**C**" nel profilo automotive di Avnu (la quale non fa parte dello standard IEEE Qat).

La prenotazione dinamica dei flussi all'avvio del sistema richiede troppo tempo

All'interno dei veicoli, le applicazioni che utilizzano la rete Ethernet e i loro requisiti in termini di frequenza di trasmissione dati (*data rate*) sono noti a priori; questo include anche le applicazioni multimediali che i passeggeri potrebbero portare a bordo.

Ogni auto dispone solo di un numero limitato di posti a sedere per i passeggeri, ciascuno dei quali può utilizzare solo un numero limitato di dispositivi.

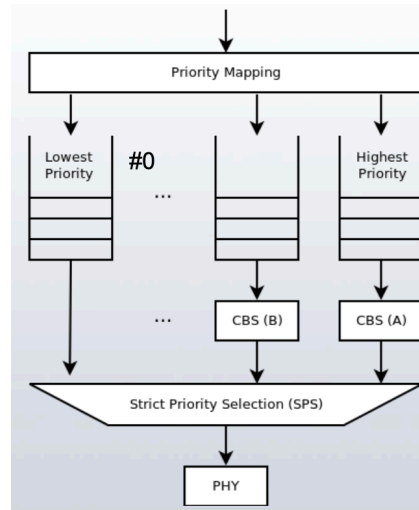
È quindi teoricamente possibile prevedere una **prenotazione statica dei flussi**. Tuttavia, le informazioni sui flussi riservati devono essere fornite a tutti i nodi coinvolti.

Una possibile soluzione è la **prenotazione preconfigurata**, che consiste nel pre-memorizzare i dati in modo che siano accessibili a ogni avvio del veicolo. Per generare questi dati pre-memorizzati è possibile, ad esempio, **eseguire il protocollo SRP una sola volta** come parte della configurazione della rete durante il processo di produzione (in fabbrica) e memorizzarne il risultato finale.

802.1Qav (FQTSS: Forwarding and Queuing Enhancements for Time Sensitive Streams)

L'**FQTSS** (*Forwarding and Queuing Enhancements for Time-Sensitive Streams*) pianifica lo scheduling dei dati di streaming A/V sensibili al tempo, garantendo la tempestività della trasmissione attraverso la rete. Il traffico ordinario non di streaming viene gestito separatamente rispetto al traffico AVB riservato.

Mappa i frame sulle classi di traffico in base alla priorità, da un minimo di 2 a un massimo di 8 code.



Ogni coda ha una priorità differente, da 0 (la più bassa) a N (la più alta). Di queste code, le classi SR con priorità maggiore vengono gestite con **Credit-Based Shaping (CBS)** e sono due, la classe A e B.

Le classi che supportano il CBS hanno una **priorità più alta** rispetto alle classi di traffico che supportano la *Strict Priority* (SP, priorità stretta) o qualsiasi altro algoritmo di selezione della trasmissione.

Vincoli :

- Almeno una classe di traffico deve supportare il CBS.
- Almeno una classe di traffico deve supportare la SP (Strict Priority).

Credit-Based Shaper

Il CBS è un algoritmo di selezione della trasmissione che si applica al traffico di prenotazione del flusso (**SR**, *Stream Reservation*) sulle porte di uscita dei nodi terminali e degli switch, *al fine di prevenire picchi improvvisi di traffico (traffic bursts)*.

Ogni classe di traffico SR ha un parametro di credito associato, che può variare tra due limiti fisici: **loCredit** (credito minimo) e **hiCredit** (credito massimo).

- **Accumulo di Credito (Pendenza Positiva):** Se ci sono pacchetti in coda in attesa di essere trasmessi, ma la porta è attualmente occupata a trasmettere un altro pacchetto (ad esempio, un frame Best Effort o di un'altra classe), il credito aumenta linearmente con una pendenza pari a **idleSlope**.
- **Consumo di Credito (Pendenza Negativa):** Quando la coda trasmette i propri pacchetti sul link, il credito diminuisce linearmente con una pendenza pari a **sendSlope**.
- **Azzeramento del Credito:** Se la coda diventa vuota, ci sono due casi:
 - Se il credito è positivo viene **azzerato istantaneamente**. Non si può accumulare credito "gratis" se non si ha effettivamente traffico da inviare.
 - Se il credito è negativo, il credito risale verso lo zero con pendenza pari a **idleSlope** e si ferma esattamente a zero.

Il valore iniziale del credito per tutte le classi SR è pari a zero all'avvio del sistema o quando una porta di uscita viene abilitata.

I valori di **idleSlope** e **sendSlope** dipendono dalla larghezza di banda prenotata per quella specifica classe.

Condizione per trasmettere

Per poter avviare la trasmissione di un pacchetto da una coda gestita da CBS, devono essere soddisfatte contemporaneamente due condizioni:

1. Ci deve essere almeno un pacchetto completo in coda.
2. Il credito attuale della coda deve essere **maggiore o uguale a zero**.

Se il credito scende sotto lo zero *durante* la trasmissione, essa **non viene interrotta**. Il frame viene inviato interamente, portando il credito in territorio negativo (fino al limite loCredit). Una volta terminato il pacchetto, la coda non potrà più trasmettere finché il credito non tornerà a essere positivo.

LoCredit è un **limite calcolato** che rappresenta *lo scenario peggiore possibile* di discesa del credito. In pratica, il caso peggiore si verifica quando il credito della coda AVB è esattamente a **zero** e inizia a trasmettere il suo frame più grande possibile (maxFrameSize). Durante tutta la trasmissione di questo frame capiente, il credito scende con pendenza negativa (**sendSlope**). Al termine del pacchetto, il credito avrà raggiunto il punto più basso possibile in assoluto: quel punto è, per definizione, **loCredit**.

Meccanismo

Se una coda trasmette il suo credito desce, con una pendenza negativa chiamata **sendSlope**. Più il frame è lungo, più tempo impiega a essere trasmesso, più il credito si spinge verso il basso (fino al limite minimo loCredit). Finché il credito non torna a zero, la linea è

virtualmente "bloccata" per quella classe (non può abusare della banda), lasciando spazio al traffico ordinario.

Invece se una coda ha pacchetti ma deve aspettare perchè in questo momento la linea è occupata il suo credito aumenta. Il credito sale con una pendenza positiva chiamata *idleSlope*.

Se la coda è vuota e il credito è sceso sotto zero, il credito continua a salire al ritmo di *idleSlope* fino a raggiungere esattamente lo zero.

C'è solo un caso in cui il credito non segue una pendenza: se la coda è vuota ma ha un credito positivo (magari accumulato stando a lungo in attesa), quel credito viene azzerato istantaneamente. Questo evita il fenomeno del **credit banking**: impedisce cioè a una sorgente rimasta silente per ore di accumulare un credito "infinito", che le permetterebbe poi di trasmettere una raffica gigantesca di dati violando la latenza massima della rete.

Vantaggi

1. **Per l'Alta Priorità (AVB)**: La latenza diventa totalmente prevedibile. Sappiamo esattamente quanti millisecondi al massimo un frame dovrà aspettare per via dei "debiti" accumulati dagli altri.
2. **Per il traffico normale (Best-Effort)**: Viene eliminato il rischio di *starvation* (morte per fame). I dispositivi consumer o i dati meno importanti troveranno sempre uno spazio per passare, perché lo shaper "frena" ciclicamente i flussi critici.

L'AVB offre:

- **Trasporto efficiente del traffico in tempo reale (A/V)** con una latenza massima garantita.
- **Basso jitter**, a condizione che vengano effettuate un'accurata pianificazione dell'esecuzione periodica e una mappatura adeguata verso le code ad alta priorità all'interno dei bridge.
- **Grazie al protocollo di prenotazione delle risorse**, che è in grado di gestire dinamicamente la Qualità del Servizio (QoS), nuovi dispositivi possono connettersi alla rete in qualsiasi momento.

802.1AS : Synchronization

IEEE 802.1AS: Una suite di protocolli di sincronizzazione del clock che permette alle stazioni terminali (end stations) e ai bridge di sincronizzare i propri orologi locali tra loro (creando una nozione comune di tempo).

Le implementazioni di IEEE 802.1AS-2011 hanno dimostrato di poter raggiungere un tempo di riferimento con *un'accuratezza superiore a 1 μ s*.

Lo standard 802.1AS specifica un **paradigma leader-follower** con elezione del leader. Il **Grand Master (GM)** funge da tempo di riferimento per tutti gli altri dispositivi della rete.

Il **Best Master algorithm** (algoritmo per la scelta del miglior master) seleziona automaticamente il dispositivo grandmaster a partire da un insieme di dispositivi abilitati a essere grandmaster (ovvero il Talker o uno dei bridge AVB). L'algoritmo configura dinamicamente la gerarchia di sincronizzazione nella rete, chiamata *synchronization spanning tree*.

Viene quindi determinata una **gerarchia di sincronizzazione** nella rete, dove il Grand Master definisce la temporizzazione e può avvenire la **misurazione dei ritardi di collegamento** (*link delays*) tra i dispositivi.

Lo standard 802.1AS consente:

- **di coordinare la trasmissione dei frame**
- **alle applicazioni di utilizzare una base temporale sincronizzata** per un meccanismo di marcatura temporale (*timestamping*) degli eventi critici
- **alle stazioni terminali (end stations) di coordinare l'esecuzione delle attività**, come ad esempio l'acquisizione sincronizzata dei valori dei sensori in tutto il sistema distribuito.

Requisito fondamentale: STAZIONI TIME AWARE

Per sincronizzare la rete in conformità con lo standard IEEE 802.1AS, tutti i nodi, i bridge e le stazioni terminali (*end-stations*) devono essere **stazioni time-aware**.

Una stazione time-aware è un sistema che fa esplicito riferimento al tempo e implementa specifici meccanismi (la misurazione del ritardo di collegamento per ciascuna porta e il calcolo della deriva dell'orologio – *clock drift* – con il proprio vicino).

Vengono definiti due protocolli:

- **Best Master Clock Algorithm (BMCA)**: seleziona il nodo di riferimento temporale.
- **Generalized Precision Time Protocol (gPTP)**: sincronizza l'orologio dei nodi fornendo loro il valore temporale del nodo di riferimento, chiamato **Grand Master (GM)**. Opzionalmente, l'orologio del GM può essere sincronizzato con il tempo UTC.

BMCA - Best Master Clock Algorithm

Lo standard IEEE 802.1AS opera in una rete con topologia ad albero (*spanning-tree*) in cui il **Grand Master (GM)** rappresenta la radice (*root*) dell'albero.

Per selezionare il miglior GM, viene eseguito un algoritmo di selezione chiamato **BMCA**.

Il GM può cambiare ogni volta che si verifica una modifica nella rete (ad esempio, in caso di guasto del GM corrente, o per l'aggiunta di un altro GM con un orologio migliore, ecc.).

Come funziona il BMCA:

Il GM invia ciclicamente un messaggio chiamato **ANNOUNCE** che contiene alcuni indicatori della sua "Qualità dell'orologio" (*Clock quality*).

Se è presente un solo GM non c'è competizione.

Ogni nodo abilitato a fungere da Grandmaster che riceve un rispettivo messaggio ANNOUNCE confronta le informazioni del GM attualmente migliore con i valori di qualità del proprio orologio.

Se i valori di qualità del proprio orologio danno un risultato migliore rispetto a quello del GM corrente annunciato nel messaggio, l'unità che ha effettuato il confronto si annuncia come nuovo miglior Grandmaster.

Il processo viene ripetuto finché non viene trovato il GM effettivamente migliore all'interno della rete.

I parametri del GM trasmessi nei messaggi di ANNOUNCE vengono confrontati (in ordine di importanza decrescente) in base a:

- **Priorità** (impostabile via software/gestione),
- **Qualità dell'orologio** (es. accuratezza del clock – *clockAccuracy*)
- **Indirizzo MAC**.

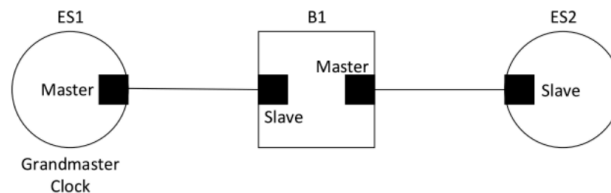
Per l'uso in ambito automobilistico (*automotive*), è sensato preselezionare il Grandmaster IEEE 802.1AS (e l'albero di copertura del clock – *clock spanning tree*) e scegliere come Grandmaster una centralina (ECU) di cui ogni vettura è dotata di serie.

Potrebbe talvolta accadere che una centralina opzionale offra in realtà un orologio con una qualità migliore; tuttavia, una selezione dinamica del Grandmaster graverebbe sui tempi di avvio (*start-up time*) e richiederebbe un maggiore sforzo nei test di rete.

Un maggior numero di varianti (ad esempio su chi sia il Grandmaster) crea più possibilità di malfunzionamenti, e la robustezza è un requisito fondamentale nell'industria automobilistica.

La gerarchia di sincronizzazione

Una volta selezionato il GM, l'albero di sincronizzazione diventa un albero di copertura minimo (*minimum spanning tree*) all'interno della rete, realizzato assegnando a ciascuna porta della rete uno di questi tre stati: **master**, **slave** o **passive**.



La stazione terminale 1 (**ES1**) può essere eletta come orologio grandmaster.

- La porta che collega ES1 allo switch 1 (**B1**) viene impostata sullo stato **master**.
- La porta che va da B1 a ES1 viene impostata sullo stato **slave**.
- La porta che va da B1 alla stazione terminale 2 (**ES2**) viene impostata sullo stato **master**.
- La porta che collega ES2 a B1 viene impostata sullo stato **slave**.

Qualsiasi altra porta nei dispositivi (nel caso in cui, ad esempio, B1 disponga di ulteriori porte) verrebbe impostata sullo stato **passive**.

Generalized Precision Time Protocol

Il protocollo viene eseguito periodicamente e inizia con il grandmaster che trasmette un messaggio di sincronizzazione, **fsync**, su tutte le porte il cui stato di sincronizzazione è impostato su **master**. Il master acquisisce una marcatura temporale (*timestamp*) al momento della trasmissione di fsync dalla porta. Il valore tipico è di 8 messaggi fSync al secondo (ovvero uno ogni 125 ms).

La marcatura temporale (**timestamping**) in qualsiasi dispositivo è implementata a livello hardware per raggiungere una precisione nell'ordine dei nanosecondi (ovvero superiore a 1 μ s).

Quando un dispositivo riceve fsync (su una porta con stato di sincronizzazione impostato su **slave**), acquisisce un timestamp dell'istante esatto di ricezione e avvia la trasmissione di nuovi frame fsync su tutte le porte impostate in stato **master**.

Anche in questo caso, il dispositivo acquisisce un timestamp al momento della trasmissione di fsync da una porta. Tramite i timestamp, ogni dispositivo intermedio conosce il cosiddetto tempo di residenza (*residence time*) di fsync all'interno del dispositivo stesso, semplicemente sottraendo il timestamp di ricezione da quello di trasmissione.

Man mano che fsync si propaga lungo l'albero di copertura della sincronizzazione (*synchronization spanning tree*), ogni dispositivo dell'albero esegue questo processo, inserendo

i timestamp e inoltrando fsync fino a quando i nodi foglia (*leaf nodes*) dell'albero non ricevono il messaggio.

A questo punto, tutti i dispositivi della rete hanno percepito un evento di risincronizzazione reciproco (la ricezione di fsync) che sincronizza gli orologi locali dei dispositivi tra loro.

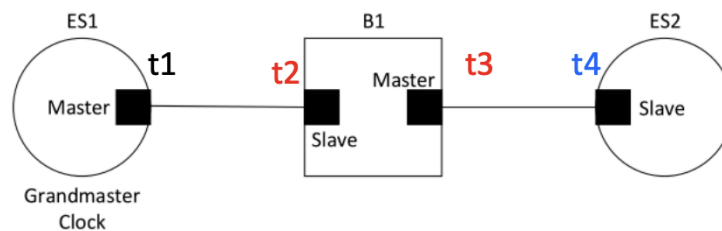
Tuttavia, i dispositivi più vicini al GrandMaster ricevono fSYNC prima rispetto ai dispositivi posizionati più lontani, di conseguenza i dispositivi più vicini azzerano/regolano i propri orologi prima degli altri.

Per tenere conto delle latenze di trasmissione di fSYNC, un secondo messaggio, il frame di follow-up **fFLUP**, viene inviato dal grandmaster e distribuito lungo l'albero di copertura della sincronizzazione in tutta la rete.

fFLUP contiene

- il ***preciseOriginTimestamp*** (t_1 nella figura), ovvero l'istante esatto in cui il GrandMaster ha avviato la trasmissione di fSYNC
- Il ***correctionField***, campo di correzione che accumula anche i **tempi di residenza** del frame (espresso in ns).

In questo modo, man mano che fFLUP attraversa l'albero di copertura della sincronizzazione, il ***correctionField*** viene aggiornato di dispositivo in dispositivo con il tempo specifico in cui fSYNC ha risieduto nel rispettivo dispositivo.



Lo standard 802.1AS include un meccanismo per misurare continuamente la latenza del collegamento (***link latency***) e aggiungerla al ***correctionField***, nonché per misurare la deriva dell'orologio (***clock drift***) al fine di ottenere i valori correttivi per compensare le derive degli orologi dei dispositivi rispetto all'orologio del grandmaster.

Time-Sensitive Networking

Un insieme di standard, in corso di sviluppo da parte del gruppo di lavoro TSN dell'IEEE 802.1 WG, che definiscono i meccanismi per fornire servizi deterministici attraverso le reti IEEE 802. In altre parole, garantiscono il trasporto dei pacchetti con latenza delimitata (vincolata), una bassa variazione del ritardo dei pacchetti (jitter), un'elevata affidabilità e una bassa perdita di pacchetti.

Il TSN consente l'implementazione di "reti convergenti", in grado di supportare simultaneamente il traffico della tecnologia operativa (OT - Operations Technology) e altre tipologie di traffico.

Time-Sensitive Networking (TSN) è una "cassetta degli attrezzi" (toolbox) flessibile.

Bounded Low Latency in IEEE standards

Nella suite AVB (Audio Video Bridging), lo standard **IEEE 802.1Qav FQTSS** (*Forwarding and Queuing Enhancements for Time-Sensitive Streams*) definisce miglioramenti delle prestazioni, protocolli e meccanismi che garantiscono trasmissioni in tempo reale.

Il TSN migliora ulteriormente le prestazioni in tempo reale attraverso gli standard **IEEE 802.1Qbv** e **IEEE 802.1Qbu**.

- **IEEE 802.1Qbv** definisce le comunicazioni guidate da una pianificazione temporale (*schedule-driven*).
- **IEEE 802.1Qbu** standardizza un meccanismo di prelazione (*preemption*) che consente ai messaggi critici dal punto di vista temporale di interrompere le trasmissioni non critiche in corso.

Le applicazioni di controllo industriale e automotive sono caratterizzate da traffico critico dal punto di vista temporale (*time-critical*):

- I dati trasmessi alimentano i parametri di cicli di controllo (*control loops*) critici.
- I dati di controllo vengono trasmessi secondo una pianificazione temporale ripetitiva (periodica).
- La consegna tardiva di tali frame può causare instabilità, imprecisione o il fallimento del funzionamento dei relativi cicli di controllo.

Queste applicazioni richiedono una consegna dei frame altamente prevedibile, in termini di:

- Tempo di trasmissione del frame.
- Latenza complessiva e jitter (variazione del ritardo) subiti fino alla destinazione del frame.

In altri termini, esse richiedono il supporto per il **Traffico Pianificato** (*Scheduled Traffic*).

Il traffico pianificato (*Scheduled Traffic*) è una classe di traffico che:

- Richiede di pianificare la trasmissione dei frame sulla base della temporizzazione derivata dallo standard IEEE 802.1AS.
- È tipicamente sensibile al tempo (*time-sensitive*) e richiede latenza e jitter vincolati, pertanto l'interferenza da parte di altre classi di traffico deve essere evitata.

Il lavoro svolto all'interno del gruppo di lavoro TSN (TSN WG) porta Ethernet nel dominio delle applicazioni **hard real-time!**

Per supportare questo traffico, è necessario aggiungere nuove funzionalità all'AVB, poiché sono richiesti miglioramenti sia a livello di bridge (switch) che di stazioni terminali (*end stations*).

Il problema dei flussi di traffico interferenti deve essere risolto.

La latenza deve essere ridotta al minimo possibile.

Per quantificare la latenza massima per il traffico sensibile al tempo (*time-sensitive*), è necessario distinguere due casi:

Caso 1: Senza traffico interferente (nessun altro traffico oltre a quello sensibile al tempo):

- È la lunghezza massima del frame sensibile al tempo a influenzare la latenza massima, specialmente con switch di tipo *store-and-forward*.
- La lunghezza del frame sensibile al tempo è il parametro che deve essere ottimizzato durante la progettazione della rete per soddisfare il limite di ritardo (*delay bound*) richiesto dall'applicazione.
- Risultato positivo, poiché i flussi sensibili al tempo sono tipicamente caratterizzati da frame di piccole dimensioni!

Caso 2: In presenza di traffico interferente (altre classi di traffico):

- Il contributo principale alla latenza massima per il traffico sensibile al tempo deriva dalla dimensione massima dei frame interferenti (tipicamente, il traffico *best-effort*).
- Questo è l'effetto dello scheduling non preventivo (*non-preemptive*) applicato alle porte di uscita (*egress ports*) dei dispositivi di rete (switch e stazioni terminali).
- Nel caso peggiore, un frame sensibile al tempo può essere ritardato in ogni switch dalla trasmissione in corso di un frame di dimensioni massime non appartenente alla classe sensibile al tempo.

Alcuni dati numerici:

- **Senza traffico interferente best-effort (Caso 1):** ipotizzando un frame sensibile al tempo (AVB/TSN) lungo 300 byte, il valore della latenza dello switch è di circa **4 μs**.

- **In presenza di traffico interferente best-effort (Caso 2):** ipotizzando una rete a 1 Gigabit Ethernet e un frame interferente con dimensione massima di 1522 byte (traffico best-effort), il valore della latenza dello switch sale a circa **13 μ s**.

Lezioni apprese:

- Le trasmissioni dei frame sensibili al tempo devono essere protette dall'interferenza dei frame non sensibili al tempo.
- Questo obiettivo è raggiungibile attraverso l'**isolamento temporale** per la classe di traffico sensibile al tempo.

In che modo l'AVB gestisce il traffico interferente?

In AVB, la latenza massima che un frame della classe SR (*Stream Reservation*) può subire dalla sorgente alla destinazione è garantita... ma il jitter e l'istante di trasmissione non sono deterministici!

I frame SR possono risentire di:

- Il ritardo dovuto alla trasmissione in corso di un frame a **priorità inferiore** oppure
- L'interferenza di frame appartenenti alla **stessa classe** di traffico, ovvero posizionati nella stessa coda, all'interno della stazione terminale o della porta dello switch (*bridge port*). Flussi multipli possono essere mappati sulla stessa classe SR, e di conseguenza i loro frame possono interferire tra loro quando sono accodati e in attesa di essere trasmessi nella medesima coda.

Soluzioni

Opzione 1: Reti dedicate e altamente ingegnerizzate, utilizzate esclusivamente per la trasmissione del traffico di controllo critico dal punto di vista temporale (time-critical).

- **Problema:** Inefficienza
- Il costo per la realizzazione di una rete di controllo dedicata può essere elevato.
- Mancanza di spazio per il dispiegamento (installazione).
- La banda richiesta da tale traffico è spesso ridotta.

Opzione 2: Un'unica rete "convergente", in cui il traffico critico è miscelato con altre classi di traffico, ma è comunque in grado di soddisfare i propri requisiti temporali.

- **Sfida:** Gestire il traffico interferente.
- **Soluzione:**
 - Imporre l'**isolamento temporale** per il traffico critico, ovvero garantire che, in momenti specifici, solo quella determinata classe di traffico acceda alla rete.
 - Creare una **finestra temporale protetta** che sia utilizzata esclusivamente da quella classe di traffico.

802.1Qbv – Transmission Gates

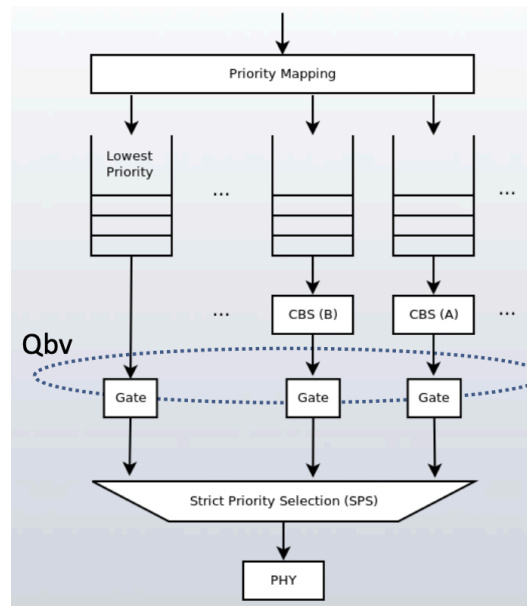
Lo standard **IEEE 802.1Qbv** (*Enhancements for Scheduled Traffic*) consente a un bridge (switch) o a una stazione terminale (*end station*) di pianificare le trasmissioni da ciascuna coda sulla base di una scala temporale nota.

Lo standard Qbv pianifica la trasmissione attraverso *l'attivazione/disattivazione delle code di trasmissione* all'interno degli switch e delle stazioni terminali, attraverso un *transmission gate* associato a ciascuna coda su una determinata porta. Il gate di trasmissione può trovarsi in uno di questi due stati:

- **Aperto (Open):** i frame in coda vengono selezionati per la trasmissione, in base all'algoritmo di selezione della trasmissione associato a quella specifica coda.
- **Chiuso (Closed):** i frame in coda non vengono selezionati per la trasmissione.

Come funziona il meccanismo dei gate: Il comportamento definito per i cancelli stabilisce che un frame di una determinata classe di traffico può essere trasmesso se e solo se:

- il cancello associato a quella classe di traffico è **aperto**;
- vi è **tempo sufficiente** per trasmettere l'intero frame prima della prevista chiusura del cancello stesso.



Il comportamento di questi gate segue una pianificazione temporale rigida e ciclica definita offline da una tabella hardware chiamata **Gate Control List (GCL)**.

La Gate Control List è associata a ciascuna porta ed è una lista ordinata di operazioni sui cancelli, chiamate Gate Operation, che vengono eseguite ciclicamente.

La **Gate Operation** definisce:

- **Gate States**: una maschera di massimo 8 bit che indica lo stato (aperto/chiuso) di ciascuna coda.
- **Time Interval**: la durata per cui quella configurazione deve essere mantenuta, fino all'operazione successiva.

La sequenza di *gate operation* termina alla fine della lista e **si ripete periodicamente** a partire dall'inizio della sequenza (parametro configurabile).

Protected Window e Banda di Guardia

Le operazioni sui cancelli realizzano l'isolamento temporale creando una *Protected Window* e determinando una banda di guardia (guard band).

1. **Finestra Temporale Protetta (Protected Window)**: Il gate della coda a massima priorità (es. Coda 7, traffico di controllo) viene impostato su **Open**, mentre i gate di tutte le altre code (traffico multimediale o Best Effort) vengono impostati su **Closed**. In questo modo, i pacchetti critici trovano la linea fisica completamente sgombra, eliminando sul nascere qualsiasi ritardo di accodamento o interferenza (*jitter*). Il traffico prioritario viene praticamente isolato, garantendo i vincoli real time.
2. **Finestra del traffico standard**: Successivamente, la GCL chiude il gate prioritario e apre i gate dedicati al traffico Best Effort o AVB asincrono (regolato ad esempio dal Credit-Based Shaper di 802.1Qav).

Se un frame Best Effort molto lungo iniziasse la trasmissione un istante prima dell'apertura del gate prioritario, ne ritarderebbe comunque l'uscita, violando il determinismo (fenomeno noto come *interferenza residua*).

Per risolvere questo problema, la Gate Operation calcola e introduce una **Guard Band** (banda di guardia) prima dell'inizio della finestra protetta. Durante la Guard Band, i gate non prioritari vengono chiusi *in anticipo* per impedire l'avvio di qualsiasi trasmissione che non possa concludersi prima dello scattare della finestra critica. Per determinare la *lunghezza della banda di guardia* viene utilizzata una politica *statica* più semplice che utilizza la durata del frame di dimensioni massime. Un'implementazione *dinamica* potrebbe decidere al volo (*on the fly*) se i messaggi in coda siano idonei o meno alla trasmissione (in base alla loro dimensione effettiva).

La Banda di Guardia consiste nel momento oltre il quale il frame Ethernet non avrebbe più il tempo di essere trasmesso per intero, per questo garantisce il determinismo. Lo svantaggio è che comporta spreco di banda. Per questo in alternativa, nei sistemi TSN più moderni, si applica lo standard **IEEE 802.1Qbu (Frame Preemption)**, che permette alla Gate Operation di interrompere temporaneamente (spezzare) a metà un frame non prioritario per far passare il traffico della coda a tempo e poi riprenderlo subito dopo.

Lo **scopo** di 802.1Qbv è portare Ethernet nel mondo dell'**hard real-time**, garantendo che i dati critici (es. i comandi automotive o industriali) arrivino a destinazione con latenza e jitter quasi

azzerati. Per farlo, introduce il concetto di **Time-Aware Shaper (TAS)**, introduce un concetto di **sincronizzazione temporale globale**. Il TAS prevede:

- **Sincronizzazione di base:** Sfruttando l'orologio comune fornito dallo standard IEEE 802.1AS, tutti gli switch e i nodi della rete sanno esattamente che ore sono al microsecondo.
- **I "Cancelli" temporali (Gates):** Davanti a ciascuna delle code di trasmissione di una porta dello switch (fino a 8) viene posizionato un "cancello" elettronico che può essere aperto (o) o chiuso (C).
- **La Gate Control List (GCL):** Lo switch esegue ciclicamente una tabella oraria preconfigurata. In determinati slot temporali, la GCL chiude i cancelli del traffico standard (internet, video, best-effort) e tiene aperto **solo** il cancello del traffico critico.

In questo modo, quando i pacchetti dati più urgenti devono essere spediti sul cavo, trovano l'intera infrastruttura **completamente libera da interferenze**, eliminando i ritardi di accodamento causati dal traffico concorrente. Per evitare che un frame standard lungo sfori nello slot protetto, lo standard applica una **Guardband** (banda di guardia) che blocca preventivamente i pacchetti non critici in ritardo.

802.1Qbu - Frame Preemption

Frame preemption overview

Con **preemption** si intende la sospensione della trasmissione di un frame non critico dal punto di vista temporale per consentire la trasmissione di uno o più frame critici, prima che la trasmissione del frame non critico venga ripresa.

Ha lo scopo di fornire una trasmissione a latenza ridotta per i frame critici dal punto di vista temporale all'interno di una rete bridged standard che sia in grado di supportare comunicazioni in tempo reale insieme al traffico tradizionale. Le procedure per la prelazione sono specificate in due standard:

- **IEEE 802.1Qbu - Frame Preemption** : definisce le procedure per *sospendere la trasmissione di un frame*, nonché le procedure per *sbloccare o riprendere la trasmissione del frame* meno critico una volta che i frame urgenti sono stati trasmessi.
- **IEEE 802.3br - Specification and Management Parameters for Interspersing Express Traffic**: consente ai frame dati urgenti e critici dal punto di vista temporale di suddividere in frammenti più piccoli i frame non critici in transito su un singolo collegamento fisico.

Lo standard **IEEE 802.1Qbu** introduce le modifiche necessarie in IEEE 802.1Q per supportare lo standard IEEE 802.3br.

Funzionamento

A ciascuna coda delle classi di traffico supportate dalla porta viene assegnato un valore di **stato di prelazione del frame** (*frame preemption status*), che può essere:

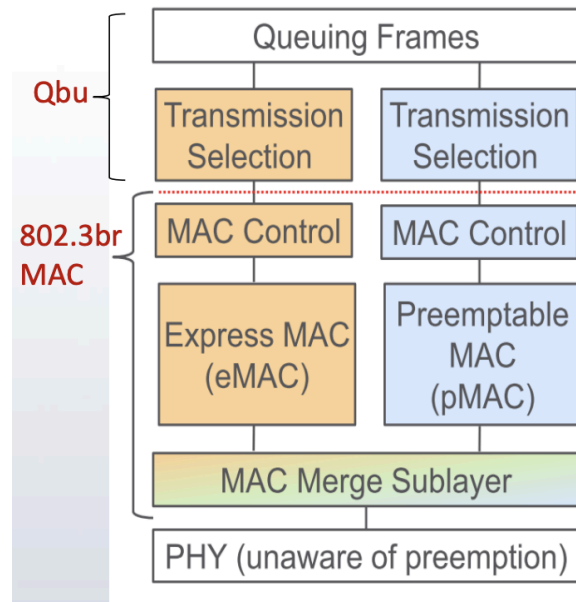
- **express** a priorità più alta,
- **preemptable** (interrompibile).

Il traffico **Express** ha una priorità più alta rispetto al traffico **preemptable**, indipendentemente dal fatto che la prelazione sia abilitata o meno.

Lo standard IEEE 802.3br fornisce due interfacce di servizio MAC, ovvero:

- **Preemptable MAC (pMAC)**, per i frame interrompibili.
- **Express MAC (eMAC)**, per i frame express (critici).

I frame interrompibili devono essere trasmessi utilizzando il servizio **pMAC**, mentre i frame express attraverso il servizio **eMAC**. Anche con la prelazione disabilitata, se sia l'eMAC che il pMAC hanno un frame pronto per la trasmissione e nessun frame è attualmente in fase di trasmissione, viene scelto il frame dell'eMAC.



Lo standard **802.3br** introduce il **MAC Merge sublayer**, che collega un eMAC e un pMAC a un singolo livello fisico (PHY) attraverso un sotto-livello di riconciliazione (*reconciliation sublayer*).

Il PHY è del tutto inconsapevole della presenza della prelazione.

Il sotto-livello MAC Merge e i relativi MAC forniscono il supporto per la prelazione dei frame (*frame preemption*) così come definito dallo standard **IEEE 802.1Qbu**.

Meccanismo

Esistono **due modi per sospendere la trasmissione** del traffico interrompibile (*preemptable*) in presenza di traffico critico (*express*):

- **Prelazione:** interrompendo il traffico interrompibile attualmente in fase di trasmissione.
- **Prevenzione:** impedendo l'avvio della trasmissione del traffico interrompibile.

L'interfaccia di servizio MAC Merge (**MMSI - MAC Merge Service Interface**) fornisce una primitiva che determina il trattenimento o il rilascio della trasmissione del traffico interrompibile.

In conformità allo standard **IEEE 802.1Qbu**, un frame *preemptable* **non è idoneo alla trasmissione** si verifica una delle seguenti condizioni:

- **Per Preemption:** la trasmissione di un precedente frame interrompibile **non è ancora stata completata** a causa della prelazione subita da parte di un frame di tipo *express* (critico).
- **Per Prevenzione:** è stata inviata al MAC una richiesta di *hold back* del frame interrompibile;

Come funziona la preemption

Il traffico **Express** ha una priorità più alta rispetto al traffico **preemptable** (interrompibile).

Tutti i livelli di priorità che vengono mappati sulla stessa coda hanno lo stesso stato, che sia esso preemptable o express.

È presente una *tabella dei parametri di prelazione* dei frame per ciascuna porta di uno switch (*bridge*) o di una stazione terminale (*end station*). I valori di prelazione possono essere modificati tramite la gestione di rete (*management*).

La trasmissione in corso di un frame *preemptable* può essere interrotta solo se:

1. sono già stati trasmessi almeno 60 bytes del frame
2. rimangono almeno 64 bytes da trasmettere (incluso il CRC del frame).

Un frame interrompibile non può essere spezzato immediatamente. Devono essere trasmessi almeno **60 byte** di dati utili. Aggiungendo i 4 byte del nuovo CRC, il frammento minimo iniziale diventa di **64 byte** (la dimensione minima assoluta di un frame Ethernet valido, per evitare che i vecchi apparati lo scartino come errore *runt* o collisione).

Lo standard IEEE 802.3br definisce una *dimensione minima* del frammento finale pari a 64 ottetti e consente al ricevitore di richiedere che i frammenti non finali siano più grandi di questo valore di 0, 64, 128 o 192 ottetti. Tipicamente, si assumono 64 ottetti come dimensione minima per i frammenti non finali.

Ritardo di trasmissione nel caso peggiore:

60 byte minimi + 63 byte rimasti da trasmettere = 123 bytes

Quando un frame *express* è pronto per la trasmissione e un frame *preemptable* è già in fase di trasmissione, il ritardo prima dell'inizio della trasmissione del frame *express* è **al peggio pari al tempo di trasmissione di 123 byte**.

La funzionalità di prelazione è **più utile a velocità operative inferiori** (ovvero a 100 Mbps).

Ad esempio, la durata (tempo di trasmissione) di un pacchetto da 2000 ottetti a 100 Mbps è di **160 µs**, mentre a 1 Gbps è di **16 µs**. **La preemption perde efficacia su reti multi-gigabit.**

Quando la funzionalità di prelazione non viene utilizzata, il tempo necessario per trasmettere un singolo pacchetto di lunghezza massima costituisce il limite superiore (*upper bound*) del ritardo aggiuntivo prima di poter trasmettere un frame *express*. A velocità operative più elevate, questo ritardo aggiuntivo diventa più piccolo in proporzione alla velocità, riducendo il vantaggio offerto dal meccanismo di prelazione.

A queste velocità, il ritardo di accodamento diventa così piccolo che l'introduzione della preemption introduce quasi più overhead hardware (gestione dei frammenti, riassemblaggio,

calcolo dell'mCRC) rispetto al semplice attendere la fine del pacchetto. È per questo che nei moderni backbone TSN automotive a 2.5G o 10G si tende a preferire lo *Scheduled Traffic* (802.1Qbv) o semplici politiche di shaping rispetto alla frammentazione fisica.

Ai fini della correzione dell'errore:

Ogni frammento non finale trasporta un particolare valore di CRC (valore **mCRC** - *mini CRC* o *modified CRC*) che viene utilizzato per determinare la corretta ricezione di tutti gli ottetti (byte) del frame fino all'ultimo ottetto trasmesso in quel frammento.

Il campo CRC del frammento finale contiene il campo **FCS** (*Frame Check Sequence* - Sequenza di Controllo del Frame) dell'intero frame MAC originario che è stato interrotto (*preempted*).

Vengono forniti meccanismi di **conteggio dei frame** (*frame count*) e di **continuazione del frammento** (*fragment continuation*) per rilevare la perdita di pacchetti ed evitare il riassemblaggio di frammenti non validi.

Qbv e Qbu insieme: vantaggi

- **Consumo efficiente della larghezza di banda:** grazie alla combinazione con la prelazione, i cancelli di trasmissione per ciascuna classe possono chiudersi in un momento successivo. Infatti, con la prelazione abilitata, il parametro da tenere in considerazione è il tempo di trasmissione del frammento di frame massimo (*max frame fragment*) per la classe interrompibile (coda), e non quello dell'intero frame massimo.

La banda di guardia statica equivale al tempo di trasmissione di un pacchetto da oltre 1500 byte.

Grazie a 802.1Qbu il frame può essere tagliato ed è possibile rimpicciolire la banda di guardia al caso peggiore che arriva a 123 byte, la dimensione del frammento non interrompibile.

Di conseguenza, la finestra protetta per il traffico pianificato comporta un minore spreco di banda passante.

- **Minore latenza per il traffico interrompibile:** il quale, frammento dopo frammento, può essere trasmesso in anticipo rispetto a quanto avverrebbe se dovesse essere inviato come un blocco unico, riducendo la latenza complessiva.

802.1CB - Frame Replication and Elimination for Reliability

Lo standard **IEEE 802.1CB - Frame Replication and Elimination for Reliability (FRER)**

(Replica ed Eliminazione dei Frame per l'Affidabilità) fornisce le seguenti funzionalità:

- Identificazione e replica dei frame per trasmissioni ridondanti.
- Identificazione dei frame duplicati.
- Eliminazione dei frame duplicati.

Lo standard IEEE 802.1CB può utilizzare molteplici funzioni per realizzare *l'identificazione* dei frame, ad esempio:

- indirizzo sorgente (*Source Address*) e VLAN ID,
- indirizzo di destinazione (*Destination Address*) e VLAN ID, ecc.

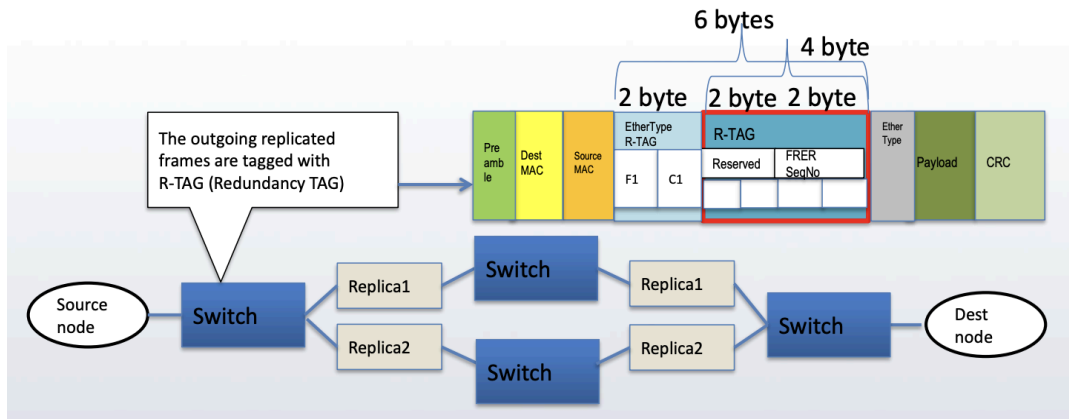
Grazie alla capacità di identificare i frame, il FRER provvede alla duplicazione dei frame e alla loro trasmissione su **percorsi disgiunti** (*disjoint paths*), nonché all'eliminazione dei frame duplicati a livello del livello MAC della stazione terminale di destinazione.

- Lo standard IEEE 802.1CB offre inoltre una modalità operativa opzionale di tipo **proxy** (*proxy operation mode*) che delega allo switch (*bridge*) il compito di eseguire la duplicazione e l'eliminazione in modo completamente trasparente rispetto alle stazioni terminali.
- In modalità proxy, la duplicazione viene eseguita dal primo switch lungo il percorso dei dati, mentre l'eliminazione dei frame ridondanti viene eseguita dall'ultimo switch sul percorso, alleggerendo il lavoro dei dispositivi finali.
- Il microprocessore dell'host deve semplicemente contrassegnare (*mark*) i flussi da inviare sui percorsi ridondanti.

Modalità Proxy

Lo switch conforme allo standard IEEE 802.1CB connesso al nodo sorgente inserisce un tag di ridondanza (*Redundancy tag* - **R-TAG**) da 4 byte in *ciascuna replica del frame*, posizionandolo tra EtherType e Payload.

Ogni R-TAG contiene un numero di sequenza (**SequenceNumber / FRER SeqNo**) che indica l'ordine di trasmissione della replica del frame rispetto agli altri frame appartenenti allo stesso flusso (*stream*). Il numero di sequenza viene utilizzato dallo switch finale connesso al nodo di destinazione per rilevare le repliche "già viste" dello stesso frame.



Implementazione FRER con percorsi ridondanti

Il **Talker** (sorgente) esegue:

- **SEQUENCE GENERATION**
 - Prende i pacchetti appartenenti a un determinato flusso di dati (*stream*) e genera un numero di sequenza univoco e progressivo (es. 1, 2, 3, 4...) per ciascun frame in uscita.
- **STREAM SPLITTING**
 - Prende i pacchetti a cui è associato il numero di sequenza e crea delle copie generando di flussi membri (*member streams*) da inviare attraverso percorsi distinti. Se un collegamento si interrompe lungo il cammino, l'altra copia del frame continuerà a viaggiare sull'altro percorso senza alcuna perdita di dati.
- **SEQUENCE ENCODING**
 - Ad ogni copia viene assegnato un numero di sequenza tramite il tag di ridondanza (**R-TAG**).

Un **Listener** (destinatario) o un **bridge intermedio** esegue:

- **STREAM IDENTIFICATION**: applicazione di una funzione di identificazione del flusso per distinguere uno specifico stream, ad esempio utilizzando gli indirizzi MAC sorgente o destinazione e il VLAN ID di un pacchetto.
- **SEQUENCE DECODING**: estrazione dell'identificativo di sequenza di un pacchetto da confrontare con le informazioni di sequenza del flusso identificato.
- **STREAM RECOVERY**: decisione sul fatto che un pacchetto sia un duplicato e debba quindi essere scartato (*dropped*) o inoltrato (*forwarded*). Questa operazione va infine a ricombinare (*merge*) i flussi membri nel flusso originale.

802.1Qci - Per-Stream Filtering and Policing

Lo standard **IEEE 802.1Qci - Per-Stream Filtering and Policing (PSFP)** (Filtraggio e Controllo per Singolo Flusso) specifica le procedure che consentono a un bridge (switch) di eseguire il *controllo* e il filtraggio in *ingresso* per bloccare una sorgente di traffico qualora venga rilevato del traffico imprevisto o non conforme. Il meccanismo è composto da tre componenti:

1. I **filtri di flusso (*stream filters*)** identificano i frame in ingresso e mappano ciascun frame identificato verso un cancello di flusso e un misuratore di flusso.

Le funzioni di filtraggio opzionali includono lo scarto (*dropping*) dei frame che superano la dimensione massima configurata e il blocco permanente di un flusso qualora un frame superi anche solo una volta la dimensione massima configurata.

2. I **cancelli di flusso (*stream gates*)** possono essere aperti o chiusi seguendo la pianificazione ciclica della *Stream Gate Control List (SGCL)*. I cancelli di flusso inoltrano o scartano il frame a seconda del tempo di arrivo del frame stesso. I frame che arrivano durante uno slot temporale in cui il cancello si trova in stato aperto vengono inoltrati, mentre i frame che arrivano in stato chiuso vengono scartati.

Opzionalmente, il cancello di flusso può utilizzare un valore di priorità di intervallo (**IPV - *Interval Priority Value***) per selezionare la coda di trasmissione per il frame in base allo slot temporale.

3. I **misuratori di flusso (*flow meters*)** verificano se la trasmissione del frame è conforme alla frequenza ammessa per quel flusso utilizzando un meccanismo a *token bucket*, decidendo poi se inoltrare o scartare il frame.

In sintesi, lo standard PSFP:

- Consente il rilevamento e la mitigazione degli errori, bloccando un flusso o una porta per imporre il contenimento dell'errore (*error containment*), evitando così che un guasto si propaghi all'interno della rete.
- Impedisce a dispositivi malfunzionanti, difettosi o malevoli (potenziali minacce alla sicurezza) di sovraccaricare la rete, garantendo al contempo che i picchi di traffico in un flusso non causino aumenti di latenza o overflow dei buffer per gli altri flussi.

802.1Qch - Cyclic Queuing and Forwarding - CQF

Lo standard **IEEE 802.1Qch**, denominato **Cyclic Queuing and Forwarding (CQF)**, introduce un meccanismo per ricevere e trasmettere i frame in modo alternato per un intervallo di tempo fisso, chiamato tempo di ciclo (*cycle time*). I real time frame che vengono raccolti durante un ciclo i vengono poi trasmessi nel ciclo successivo $i+1$, al fine di ridurre al minimo il jitter di trasmissione (migliorando così il determinismo) e garantire ritardi end-to-end vincolati.

Ciclo 1:

- I frame in ingresso passano attraverso i *stream gate* e vengono inseriti nella coda 2 (il cui cancello di trasmissione è chiuso), mentre il cancello di trasmissione 1 è aperto e abilita la trasmissione dei frame ricevuti durante il ciclo precedente.

Ciclo 2:

- Al termine del ciclo i , lo stato del cancello di flusso cambia, pertanto al ciclo $i+1$ (numero dispari) i frame in ingresso passano attraverso il cancello di flusso e vengono inseriti nella coda 1 (il cui cancello di trasmissione è chiuso), mentre il cancello di trasmissione 2 è aperto e abilita le trasmissioni dei frame che erano stati accodati nella coda 2 durante il ciclo precedente. **Se la coda di trasmissione ha tempo a sufficienza per trasmettere tutti i frame accodati entro il proprio tempo di ciclo, la latenza dipende esclusivamente dalla durata del tempo di ciclo e dal numero di passaggi (*hops*) tra il talker (sorgente) e il listener (destinatario).**

SPIEGAZIONE COMPRENSIBILE

Per eliminare le code, l'802.1Qch impone a ogni switch di usare **due code di priorità (Buffer A e Buffer B)** dedicate al traffico critico su ogni porta di uscita. Queste due code funzionano ad alternanza ciclica basandosi sul tempo, che è sincronizzato in tutta la rete grazie all'802.1AS.

Immagina il tempo diviso in intervalli fissi (chiamati **cicli**, ad esempio di 100 microsecondi l'uno):

- **Nel Ciclo 1:** Lo switch **riceve** i pacchetti e li accumula nel **Buffer A**. Contemporaneamente, svuota e **trasmette** i pacchetti che erano già pronti nel **Buffer B**.
- **Nel Ciclo 2 (Si invertono i ruoli):** Lo switch **riceve** i nuovi pacchetti e li mette nel **Buffer B**. Contemporaneamente, svuota e **trasmette** i pacchetti accumulati nel ciclo precedente nel **Buffer A**.

Questo meccanismo a "ping-pong" si ripete all'infinito in modo sincronizzato su tutti gli switch della rete.

Se la rete è progettata correttamente (ovvero la banda è sufficiente a svuotare la coda prima che il ciclo finisca), un pacchetto impiegherà **esattamente un ciclo di tempo per ogni switch (hop) che attraversa**.

802.1 Qcr - Asynchronous Traffic Shaping - ATS

Lo standard **IEEE 802.1Qcr-2020**, denominato **Asynchronous Traffic Shaping (ATS)**, definisce un meccanismo per modellare il traffico di rete al fine di garantire una bassa latenza e trasmissioni prive di congestione senza richiedere un tempo di rete comune, ovvero **senza una sincronizzazione temporale globale**.

La funzionalità ATS è suddivisa in due fasi del processo di inoltro dei frame: il **pianificatore ATS** (*ATS scheduler*) e l'**algoritmo di selezione della trasmissione ATS** (*ATS transmission selection algorithm*).

Prima che il frame venga inserito in coda, un **pianificatore ATS** (*ATS scheduler*) determina il tempo di idoneità del frame (*frame eligibility time*), vale a dire l'istante in cui il frame può essere trasmesso in conformità con la frequenza media dei dati ammessa per il suo flusso (Committed Information Rate, **CIR**) e con la dimensione massima del picco (Committed Burst Size, **CBS**).

Il pianificatore ATS opera a monte delle code di trasmissione della porta e riceve i frame dagli stessi filtri di flusso (*stream filters*) e cancelli di flusso (*stream gates*) utilizzati per il PSFP (802.1Qci). Ciò consente di scartare i frame che superano la dimensione configurata e anche di separare il traffico in base alla priorità.

Dopo che il frame è stato inserito in coda, l'**algoritmo di selezione ATS** sceglie i frame i cui tempi di idoneità sono inferiori al tempo corrente. Di fatto, un frame può essere trasmesso quando il tempo corrente è maggiore o uguale al tempo di idoneità del frame stesso. Se più frame risultano idonei contemporaneamente, viene scelto quello con il tempo di idoneità più remoto (ossia il più antecedente).

Ogni flusso di dati (*stream*) prioritario ha il proprio shaper dedicato.

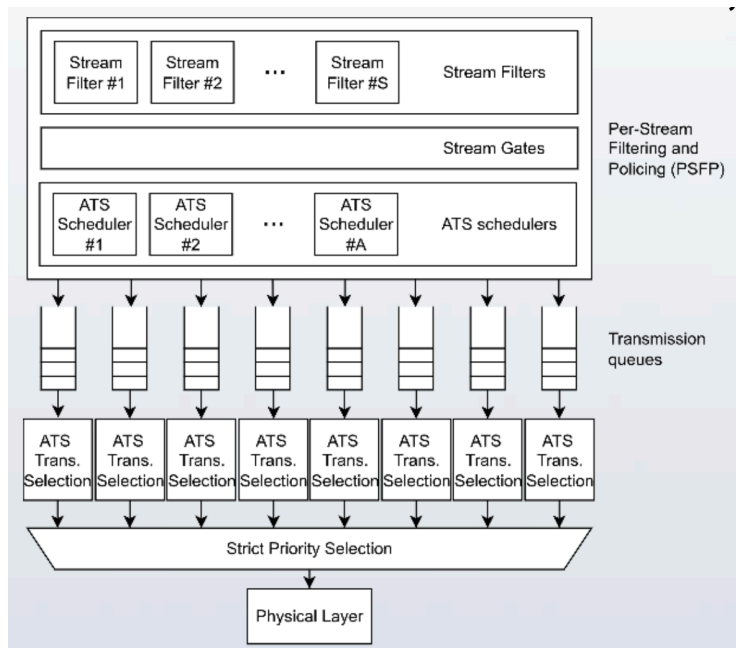
- Questo shaper assegna a ogni pacchetto in arrivo un'ora teorica di trasmissione ammissibile (chiamata **Eligibility Time**).
- Se i pacchetti arrivano troppo vicini tra loro (effetto *bunching*), lo shaper calcola che i pacchetti successivi non sono ancora "idonei" e li distanzia forzatamente, ripristinando lo spazio temporale originario tra un frame e l'altro.

I pacchetti vengono trattenuti in queste code finché non scatta la loro specifica *Eligibility Time*. Solo in quel momento vengono spostati nella coda di trasmissione vera e propria della porta per essere inviati sul cavo.

Vantaggi:

- **Indipendenza dal Clock:** Funziona anche se la sincronizzazione dell'orologio (802.1AS) fallisce o non è presente. Ogni switch calcola i tempi di idoneità basandosi esclusivamente sul proprio clock locale relativo.

- **Latenza massima limitata (Bounded Latency):** Fornisce garanzie matematiche sulla latenza massima peggiore (*worst-case latency*) per i flussi critici, azzerando i ritardi causati dall'accumulo improvviso di pacchetti.
- **Efficienza della banda:** Non "spreca" finestre temporali vuote come fa il Time-Aware Shaper (802.1Qbv) quando non c'è traffico critico. Se la linea è libera e i pacchetti sono idonei, passano subito.
- **Scalabilità enorme:** Il computer centrale (CNC) non deve calcolare una complessa pianificazione oraria globale per tutta la rete. Deve solo configurare i parametri di banda (es. quanti byte al secondo) per ogni shaper nei singoli switch.



Considerazioni sul TSN

In sintesi, il TSN offre:

- **Latenza ridotta e vincolata** (*Bounded low latency*)
- **Jitter estremamente basso** (*Very Low jitter*)
- **Sincronizzazione temporale affidabile** (*Reliable time synchronization*)
- **Ultra-affidabilità per il traffico dati** (*Ultra-reliability for data traffic*)
- **Gestione delle risorse** (*Resource management*)
- **Convergenza** (*Convergence*)
- **Interoperabilità** (*Interoperability*)
- **Sicurezza** (*Security*)

Queste caratteristiche rendono il TSN un'opzione ideale per casi d'uso quali:

- **Reti di automazione industriale per l'Industria 4.0**

- **Reti automotive (reti di bordo dei veicoli)**

IEC e IEEE stanno collaborando allo standard congiunto IEC/IEEE 60802, il quale definisce i profili TSN (*TSN profiles*) per l'automazione industriale. Tali profili guidano nella selezione dei protocolli, delle funzionalità e delle opzioni TSN necessari per implementare reti convergenti in grado di supportare simultaneamente il traffico della tecnologia operativa (OT - *Operations Technology*) e altro tipo di traffico (IT - *Information Technology*).

Profili Automotive per il TSN (What's going on: Automotive profiles for TSN)

Il progetto **P802.1DG "Time-Sensitive Networking Profile for Automotive In-Vehicle Ethernet Communications"** (Profilo di Rete Sensibile al Tempo per le Comunicazioni Ethernet a Bordo dei Veicoli) definisce i profili TSN per le comunicazioni automotive, al fine di garantire l'interoperabilità e la connettività per le applicazioni automobilistiche su reti convergenti.

Il progetto P802.1DG risponde alla necessità di standardizzare la selezione e l'utilizzo degli standard e delle funzionalità IEEE 802 per implementare reti convergenti automotive che siano sicure e ad altissima affidabilità.

La guida automatizzata (*automated driving*) richiede una **corretta percezione dell'ambiente circostante** in qualsiasi condizione di guida.

Implicazioni:

- Un numero di sensori decisamente maggiore rispetto agli attuali sistemi ADAS.
- Traffico di rete critico per la sicurezza (*safety-critical traffic*).

Diversi tipi di sensori:

- Radar
- Sensori a ultrasuoni
- LiDAR
- Telecamere (*video cameras*)

L'elevato numero di sensori e i requisiti di larghezza di banda di alcuni di essi (telecamere, LiDAR) richiedono **reti di livello automotive ad alta banda passante** (*high-bandwidth automotive-grade networks*).

“Senza una potente rete automotive, non iniziate nemmeno a pensare alla guida autonoma”.

Oltre a un'elevata larghezza di banda (*high bandwidth*), le applicazioni per la guida automatizzata devono soddisfare:

- **Vincoli temporali** (*Temporal constraints*)
- **Sincronizzazione degli orologi** (*Clock Synchronization*)

- **Ridondanza** (*Redundancy*)
- **Rilevamento e mitigazione degli errori** (*Error Detection and Mitigation*)

Supporto del TSN per i Requisiti della Guida Automatizzata

Vincoli Temporal: Temporizzazione Deterministica e Bassa Latenza

I sistemi di guida automatizzata includono molti sensori che generano traffico con frequenze dati (*data rates*) e modelli di arrivo differenti:

- Traffico periodico regolare (*Regular periodic traffic*)
- Traffico a picchi (*Burst traffic*)

Supporto TSN:

- Lo standard **IEEE 802.1Qbv** supporta il traffico pianificato (*scheduled traffic*), come il controllo a ciclo chiuso (*closed-loop control* per sterzo e frenata) e la comunicazione inter-processore, garantendo determinismo e bassa latenza. Si adatta molto bene anche al traffico periodico altamente regolare.
- Il **Credit-Based Shaper (CBS)** ereditato dall'AVB gestisce efficacemente il traffico a picchi (*burst traffic*).
- Lo standard **IEEE 802.1Qbu**, in combinazione con **IEEE 802.3br**, offre la funzionalità di prelazione dei frame (*frame preemption*) per ridurre il tempo di ritardo di accesso per il traffico urgente, come i frame relativi all'attuazione dello sterzo e della frenata.
- Lo standard **IEEE 802.1Qcr (ATS)** è in grado di gestire i picchi di traffico (*traffic bursts*) generati da radar e LiDAR applicando la modellazione (*shaping*) su base singolo flusso (*per-flow basis*).

I parametri relativi alla pianificazione (*scheduling*) sono gestiti staticamente o da un'applicazione al di sopra del Livello 2.

Sincronizzazione dell'Orologio (Clock synchronization)

- **Nozione comune del tempo per la sensor fusion:** L'algoritmo di fusione dei sensori (*sensor fusion*) necessita di dati dai sensori provvisti di timestamp (marcatura temporale), per poter correlare dati che si riferiscono alla stessa finestra temporale. I sensori devono sincronizzare il proprio orologio rispetto a una base dei tempi comune.
- Inoltre, i sistemi di **Livello 4 e 5** richiedono anche la **ridondanza** per consentire al sistema di rimanere operativo (*fail-operational*) e raggiungere una condizione di rischio minimo (*minimal risk condition*) anche in caso di guasto di un collegamento (*link*) o del *grandmaster*.

Supporto TSN:

- Lo standard **IEEE 802.1AS-2011** fornisce una sincronizzazione con precisione al microsecondo. La perdita del grandmaster attivo viene gestita eleggendone uno nuovo, ma la procedura di cambio (*handover*) prevista in IEEE 802.1AS-2011 non è molto rapida e potrebbe causare salti temporali (*time jumps*) in alcuni nodi.

- Lo standard **IEEE 802.1AS-2020** supporta molteplici grandmaster sincronizzati e percorsi di sincronizzazione multipli per consentire un passaggio senza interruzioni (*seamless handover*) a bassa latenza e un rapido ripristino della sincronizzazione temporale in modalità di guasto (*failure modes*).

Ridondanza (Redundancy)

- I sistemi di guida autonoma includono applicazioni in tempo reale sensibili al ritardo (ad esempio la *machine vision* / visione artificiale) che non possono tollerare il ritardo dovuto alle ritrasmissioni dei frame perduti.

Supporto TSN:

- Lo standard **IEEE 802.1CB** (*Frame Replication and Elimination for Reliability* - FRER) provvede alla duplicazione dei frame, alla trasmissione su percorsi disgiunti e all'eliminazione dei duplicati a livello MAC. I pro e i contro di questo approccio devono essere attentamente valutati.

Rilevamento e Mitigazione degli Errori (Error Detection and Mitigation)

- I sistemi di guida automatizzata richiedono il tempestivo rilevamento degli errori dovuti a guasti hardware casuali (ad esempio, sovraccarico di traffico), guasti software sistematici o attacchi alla sicurezza informatica (*cyber-security attacks*). Sono necessarie anche misure per prevenire la propagazione degli errori sulla rete.

Supporto TSN:

- Lo standard TSN **IEEE 802.1Qci** supporta il monitoraggio e la misurazione per singolo flusso (*per-stream metering and monitoring*), il rilevamento e la mitigazione degli errori, il controllo in ingresso (*ingress policing*) e il filtraggio.

In sintesi, l'insieme degli standard TSN:

- **Offre numerose funzionalità** che soddisfano i molteplici requisiti delle diverse tipologie di applicazioni per la guida automatizzata (*Automated Driving*).
- **Consente la realizzazione di una "rete convergente"** (*converged network*) all'interno della quale i differenti tipi di traffico delle applicazioni di guida automatizzata possono coesistere, rispettando i propri vincoli.
- **Offre metodi comuni e standardizzati** per l'implementazione di sistemi di guida automatizzata critici per la sicurezza (*safety-critical*).

Industrial Ethernet

Fieldbus: reti industriali per il controllo distribuito in tempo reale.

Un fieldbus è tipicamente una rete multidrop (bus) che collega dispositivi di campo, come sensori, attuatori, controllori di campo (PLC, regolatori, ecc.) e azionamenti. Non è stato possibile definire un unico standard fieldbus.

Tutti i protocolli industriali chiamati fieldbus sono definiti nella norma IEC 61158. Questo documento è strutturato in sette parti in base al modello di riferimento OSI (Open System Interface), come mostrato nella Tabella 21.1.

TABLE 21.1	Structure of IEC 61158
IEC 61158-1	Introduction
IEC 61158-2-x	PhL: Physical Layer
IEC 61158-3-x	DLL: Data Link Layer Service
IEC 61158-4-x	DLL: Data Link Layer Protocols
IEC 61158-5-x	AL: Application Layer Services
IEC 61158-6-x	AL: Application Layers Protocol
IEC 61158-7	Network Management
Note: x indicates the related CPF.	

Nelle parti da 2 a 6, tutte le reti sono identificate da "tipi". Esistono quindi 16 diversi tipi di reti suddivisi in sei parti differenti.

La strada verso l'Ethernet in Tempo Reale (RTE)

Motivazioni:

- **L'aumento della larghezza** di banda consente l'utilizzo di una dorsale (backbone) Ethernet industriale.
- L'adozione di Ethernet per la comunicazione tra controllori, e persino per la comunicazione con i dispositivi di campo, supporta una capacità di **connessione diretta a Internet** nell'area di campo, ad esempio per interfacce utente remote tramite browser Web.

Tuttavia, l'Ethernet convenzionale (*plain Ethernet*) non era in grado di supportare applicazioni di controllo che richiedessero un comportamento in tempo reale di tipo *hard* (vincoli temporali rigidi).

Per far fronte a queste limitazioni, molti progetti di ricerca hanno proposto soluzioni per l'introduzione della qualità del servizio (QoS), modifiche all'elaborazione dei pacchetti all'interno degli switch o la sincronizzazione tra i dispositivi.

Infine, il comitato IEC/SC65C ha completato un progetto di standardizzazione ed è stata prodotta la norma **IEC 61784**. All'interno di essa esistono diverse soluzioni concorrenti.

Di seguito:

- Una panoramica dello standard IEC 61784.
- Le caratteristiche tecniche chiave di alcune soluzioni presenti in questo standard in grado di gestire i requisiti del tempo reale.

Ethernet in Tempo Reale - Standardizzazione

Nel documento IEC 61784 sono raccolti diversi set di profili (stack di protocolli).

Gli stack di protocolli sono molto più utili per il progettista di reti, in quanto forniscono una soluzione completa e pronta all'uso.

Nella norma **IEC 61784-1** vengono definiti i set di profili per la produzione continua e discreta (manifatturiera) rilevanti per l'uso dei bus di campo nei sistemi di controllo industriali.

All'interno di questo profilo sono definite anche alcune versioni basate sulla tecnologia Ethernet.

Nel secondo volume, la norma **IEC 61784-2**, vengono definiti ulteriori profili per le reti di comunicazione basate su ISO/IEC 8802.3 (Ethernet) nelle applicazioni in tempo reale.

Per identificare tutti questi profili, viene introdotta una classificazione tramite Famiglie di Profili di Comunicazione (CPF - *Communication Profile Families*) in base alla Tabell..

Ogni CPF è libera di definire un insieme di profili di comunicazione (CP). Con il passare del tempo, nelle nuove versioni dello standard sono stati aggiunti ulteriori CP, portando il numero delle CPF fino a 22.

PERFORMANCE INDICATOR

- Gli utilizzatori di una rete RTE hanno requisiti diversi per applicazioni diverse.
- È necessario un modo per capire quali CP (Profili di Comunicazione) soddisfino al meglio i requisiti delle diverse applicazioni.
- A questo scopo, nella norma IEC 61784-2 questi requisiti vengono definiti come indicatori di prestazione.
- I requisiti per una determinata classe di applicazioni definiscono un elenco di indicatori di prestazione. La domanda è: l'offerta soddisfa la domanda?
- Ogni CP deve definire quali indicatori di prestazione soddisfa e a quali condizioni. Ogni indicatore di prestazione ha i propri limiti o intervalli di valori. Esiste un'interdipendenza tra questi indicatori di prestazione.

IN SINTESI: i **Performance Indicator (Indicatori di Prestazione)** sono parametri standardizzati utilizzati per quantificare, confrontare e certificare le capacità di una rete industriale nel gestire il traffico di controllo real-time.

Questi indicatori sono stati formalizzati a livello internazionale nello standard **IEC 61784-2** per consentire agli esperti di valutare oggettivamente quale tecnologia sia adatta a una specifica applicazione (es. motion control rispetto a un impianto di processo lento).

Performance Indicators of the IEC 61784 Standard

- **Data Capacity:** Delivery time , Throughput RTE , Non-RTE bandwidth
- **Timing:** Time synchronization accuracy, Non-time-based synchronization accuracy: Redundancy recovery time
- **Topology:** Number of end-stations, Basic network topology, Number of switches between end-stations.

Delivery time

- Il tempo necessario per trasmettere una SDU (*Service Data Unit*, il payload del messaggio) da un nodo (sorgente) a un altro nodo (destinazione).
- Viene misurato *all'interfaccia del livello applicativo* (senza considerare il tempo di elaborazione software interno al nodo).
- Il massimo delivery time deve essere dichiarato per due casi specifici:
 - **assenza** di errori di trasmissione (nel caso migliore, se va tutto bene);
 - **perdita di un frame** con successivo recupero. (caso peggiore con ritrasmissione necessaria).

Throughput RTE

- Quantità totale di dati applicativi ricevuti su un singolo collegamento, espressa in byte al secondo.
- Si riferisce al "*payload netto*" a livello di Applicazione: l'overhead introdotto dal protocollo è escluso.

Banda non-RTE

- La percentuale di larghezza di banda che può essere utilizzata su un singolo collegamento per le comunicazioni non in tempo reale (non-RTE).

Accuratezza della sincronizzazione temporale

Indica la *deviazione massima tra i clock* di *due nodi qualsiasi* (ovvero lo *skew* della sincronizzazione del clock).

Accuratezza della sincronizzazione non basata sul tempo

Indica il jitter massimo del comportamento ciclico di due nodi qualsiasi, innescato da eventi periodici sulla rete per stabilire il comportamento ciclico stesso (indicazione del jitter massimo).

In altre parole: quantifica la precisione con cui i nodi della rete riescono a coordinare le proprie azioni **senza utilizzare un orologio condiviso**, ma basandosi esclusivamente sulla **ricezione ciclica di un determinato messaggio di rete**.

Tempo di ripristino della ridondanza

Indica il tempo massimo che intercorre tra un singolo guasto permanente e il momento in cui la rete torna a essere completamente operativa.

Numero di stazioni terminali RTE

Numero massimo di stazioni terminali RTE supportate da un determinato CP (Profilo di Comunicazione).

Topologia di rete di base supportata da un CP

Comprende la topologia a stella gerarchica, ad anello, *daisy-chain* o qualsiasi combinazione di queste topologie.

Il numero di switch tra le stazioni terminali RTE supportato da un CP definisce il possibile layout (configurazione) della rete ed è a sua volta un indicatore importante.

Dependency Matrix

1. Cosa influenza il **Delivery time**?
 - Il **numero di switch** tra le stazioni terminali : ogni switch introduce un ritardo di memorizzazione e inoltre aumentando il tempo totale di consegna del pacchetto.
2. Cosa influenza il **numero di stazioni terminali**?
 - La **Topologia di base**.
 - Il **Numero di switch** intermedi.
 - L'**Accuratezza della sincronizzazione temporale** (mantenere sincronizzati i clock diventa più difficile all'aumentare del numero di nodi).
3. Cosa influenza il **numero degli switch**?
 - É strettamente connesso a: **Delivery time** massimo accettabile, dal numero totale di stazioni, dalla topologia e dalla precisione di sincronizzazione richiesta, tranne che dal throughput e dalla banda non rte.
4. **Il legame tra Throughput RTE e Banda Non-RTE :**
 - La larghezza di banda totale del canale è fissa, di conseguenza, se aumenti la quantità di dati in tempo reale (Throughput RTE), riduci inevitabilmente lo spazio disponibile per il traffico non prioritario (Banda non-RTE), e viceversa.
5. **L'accuratezza di sincronizzazione:** dipende dalla topologia della rete e dal numero di switch.

Dependent PI	Influencing PI						
	Delivery time	Number of endstations	Basic network topology	Number of switches between endstations	Throughput RTE	NON-RTE bandwidth	Time synchronization accuracy
Delivery time		NO	NO	YES	NO	NO	NO
Number of endstations	NO		YES	YES	NO	NO	YES
Basic network topology	NO	NO		NO	NO	NO	NO
Number of switches between endstations	YES	YES	YES		NO	NO	YES
Throughput RTE	NO	NO	NO	NO		YES	NO
NON-RTE bandwidth	NO	NO	NO	NO	YES		NO
Time synchronization accuracy	NO	NO	YES	YES	NO	NO	

Figure 19.1: Example of a Performance Indicator (PI) Dependency Matrix according to IEC 61784 Part 2

Questa matrice è uno *strumento decisionale per i progettisti di reti industriali*. Dimostra graficamente che nelle reti Real-Time Ethernet **non si può ottimizzare un parametro isolatamente**: ogni scelta progettuale (come aggiungere uno switch o aumentare i nodi) comporta un compromesso (*trade-off*) su altri indicatori di prestazione critici.

IMPORTANTE: Inoltre, nella norma IEC 61784-2, tutti i CP **devono** elencare anche almeno un insieme tipico di indicatori di prestazione, così come definiti dallo standard stesso. Ciò consente agli utenti finali una selezione più agevole della rete più appropriata per la propria applicazione. Senza questa regola della norma **IEC 61784-2**, ogni consorzio industriale pubblicherebbe i propri dati usando metriche, condizioni di test e terminologie diverse, rendendo un confronto oggettivo praticamente impossibile

Requisiti dell'Applicazione Utente

Gli utilizzatori di una rete RTE (Ethernet in Tempo Reale) hanno requisiti differenti a seconda delle diverse applicazioni. Una possibile classificazione si basa sul **delivery time**:

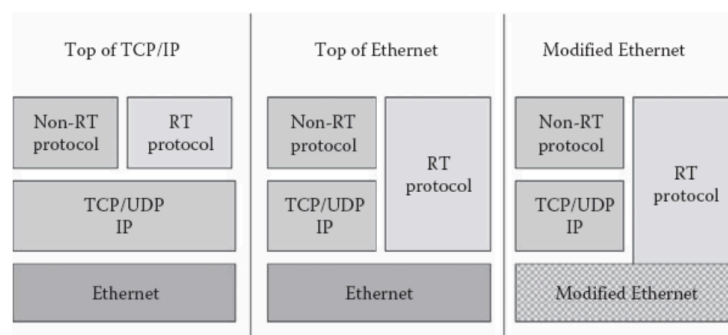
1. Prima classe: **bassa velocità, per il controllo umano**, con **delivery time ≈ 100 ms**.
 - L'essere umano è coinvolto nell'osservazione del sistema (10 immagini al secondo possono già essere viste come un filmato a bassa qualità), per il monitoraggio dei processi.
 - In questa classe rientrano la maggior parte delle applicazioni nell'automazione di processo e nel controllo degli edifici.
 - Questo requisito può essere soddisfatto senza troppi problemi utilizzando un sistema standard con canale di comunicazione TCP/IP (UDP/IP).
2. Seconda classe: **per il controllo di processo**, **delivery time ≤ 10 ms**.
 - Questo è un requisito fondamentale per la maggior parte dei sistemi di controllo delle macchine utensili, come i PLC o il controllo basato su PC.

- Per raggiungere questo comportamento temporale, sono necessari sforzi specifici nelle apparecchiature RTE.
 - Ci sono due alternative per ottenere queste performance: o risorse informatiche potenti e costose per gestire il protocollo TCP/IP in tempo reale, o lo stack protocollare deve essere semplificato e ridotto per ottenere questi tempi di reazione su risorse semplici ed economiche.
3. Terza classe: **controllo del movimento (*motion control*)**.
- Per sincronizzare più assi cinematici su una rete, è necessario un tempo di ciclo inferiore a **1 ms** con un jitter non superiore a **1 μs**. Questo traguardo può essere raggiunto solo con una rete Ethernet avente un bit rate minimo di **100 Mbps**, **a condizione che (*if and only if*)** vengano modificati sia l'accesso al mezzo del protocollo (MAC) sia la struttura hardware.

Approcci RTE

Per realizzare una soluzione RTE, esistono in linea di principio tre diversi approcci:

- **On top of TCP/IP:** consiste nel mantenere invariati i protocolli TCP/UDP/IP e concentrare tutte le modifiche per il tempo reale nel livello superiore (il protocollo UDP/IP è preferito quando è richiesta una riduzione del ritardo).
- **On top of Ethernet:** i protocolli TCP/UDP/IP vengono bypassati e si accede direttamente alle funzionalità Ethernet (approccio *dual stack*).
- **Modified Ethernet:** l'infrastruttura Ethernet stessa vengono modificati per garantire le prestazioni in tempo reale.



On Top of TCP/IP

Diverse soluzioni RTE utilizzano lo stack protocollare TCP/UDP/IP senza alcuna modifica (es. Modbus TCP, EtherNet/IP, ...).

Con questo stack protocollare è possibile comunicare oltre i confini della singola rete in modo trasparente, anche attraverso i router, esattamente nello stesso modo in cui avviene con la tecnologia Internet.

Tuttavia, la gestione efficiente di questo stack di comunicazione richiede risorse significative in termini di potenza di elaborazione e memoria.

Questo stack di comunicazione introduce ritardi non deterministici nella comunicazione.

Modbus/TCP (profili 15/1 e 15/2)

Modbus/TCP (profilo 15/1), definito da Schneider Electric, è oggi una delle soluzioni Ethernet più ampiamente utilizzate nelle applicazioni industriali e soddisfa i requisiti della classe di *controllo umano*.

MODBUS è un protocollo **client/server** semplice e offre servizi per la **lettura o la scrittura** di oggetti dati.

Questo protocollo è molto semplice e deve essere esteso con definizioni di servizio per soddisfare le esigenze di diverse applicazioni.

In aggiunta allo storico protocollo MODBUS, sono state definite nuove **estensioni real time sotto il profilo 15/2**. Queste estensioni utilizzano il protocollo RTPS (*Real-Time Publish-Subscribe*), il quale fornisce due modelli di comunicazione principali al di sopra di UDP/IP:rate

- Il protocollo **publish-subscribe** (pubblicazione-sottoscrizione), che trasferisce i dati dai *publisher* (editori) ai *subscriber* (sottoscrittori).
- Il protocollo **CST (Composite State Transfer)**, che trasferisce le informazioni di stato da un *writer* (scrittore) a un *reader* (lettore).

Il protocollo RTPS consente al sottoscrittore (*subscriber*) di definire la frequenza massima di ricezione (*max reception*), il tipo di servizio (ovvero *best-effort* o rigido/*strict*), la persistenza dei dati (validità temporale) e la priorità dei dati (*data strength*).

On Top of Ethernet

EtherType

Queste realizzazioni RTE non alterano in alcun modo l'hardware di comunicazione Ethernet, ma vengono implementate specificando un tipo speciale (**EtherType**) all'interno del frame Ethernet.

L'EtherType è un campo di due byte in un frame Ethernet e viene utilizzato per indicare quale protocollo di livello superiore è incapsulato nel payload del frame stesso.

Il tipo di protocollo standard per il protocollo IP è **EtherType = 0x0800** (2048 in decimale, IPv4)

Questi protocolli RTE utilizzano, accanto allo stack protocollare IP standard, un proprio stack identificato da un rispettivo ed uno specifico tipo di protocollo (EtherType dedicato).

Nelle reti **RTE (Real-Time Ethernet)** industriali (come PROFINET, EtherCAT o lo stesso TSN), la coesistenza di queste due modalità (lo stack standard e quello real-time) viene gestita proprio grazie all'**EtherType**.

L'EtherType viene impiegato dal data link layer per determinare a quale protocollo consegnare il payload sul lato ricevente.

Quando una scheda di rete riceve un pacchetto dal cavo, guarda il campo EtherType per capire a chi deve consegnarlo. Questo dà vita a due percorsi completamente diversi all'interno dello stesso dispositivo.

1. Lo Stack Standard (Traffico non critico) . L'EtherType: Ha il valore 0x0800 (per IPv4).
La scheda di rete passa il frame al livello di rete (IP), che poi lo passa al livello di trasporto (TCP o UDP), e infine arriva all'applicazione. Viene usato per tutto ciò che non è tempo-critico.
2. Lo Stack RTE Dedicato (Traffico Real-Time) : L'EtherType: Ha un valore specifico registrato ufficialmente dal consorzio industriale (es. 0x8892 per PROFINET, 0x88A4 per EtherCAT, 0x88F7 per il protocollo di sincronizzazione gPTP). Non appena la scheda di rete legge questo EtherType dedicato, scavalca completamente i livelli IP e TCP/UDP. Il pacchetto viene iniettato direttamente dal Livello 2 (Data Link) al Livello 7 (Applicativo) o a un motore software real-time dedicato.

L'EtherType è stato definito inizialmente dallo standard di formattazione **Ethernet II** e successivamente adattato per lo standard **IEEE 802.3**.

Inizialmente i due byte dell'EtherType rappresentavano la dimensione del payload.

Ma, al fine di consentire l'utilizzo simultaneo dei formati *Ethernet II* e *IEEE 802.3* sullo stesso segmento Ethernet, è stato introdotto uno standard unificante, l'**IEEE 802.3x-1997**, il quale ha imposto che i valori di EtherType debbano essere superiori o uguali a **1536** (0x0600).

Questo specifico valore è stato scelto perché la lunghezza massima (MTU) del campo dati di un frame Ethernet 802.3 è di 1500 byte. Di conseguenza:

- Valori ≤ 1500 rappresenta la **dimensione del payload** del frame,
- Valori ≥ 1536 rappresenta un **EtherType**.

La numerazione degli EtherType inizia generalmente dal valore **0x0800** (2048 in decimale), utilizzato per il protocollo IPv4.

Ethernet POWERLINK (EPL) è stato definito da **Bernecker e Rainer (B&R)** ed è oggi supportato dall'Ethernet Powerlink Standardisation Group (EPSG).

Il codice di Powerlink è *open source* e liberamente disponibile sotto licenza BSD. Lo stack software è reperibile su [SourceForge.net](https://sourceforge.net).

Essendo interamente basato su software, può essere utilizzato sia con hardware a 100 Mbps sia con hardware Gigabit.

EPL è un protocollo ciclico. Il tempo di ciclo, definito come il tempo necessario per scambiare i dati di input/output tra tutti i nodi coinvolti nell'anello di controllo (*control loop*), viene impostato in *fase di configurazione*.

EPL utilizza un meccanismo di *polling* (schedulazione master-slave) e una suddivisione in slot temporali (*timeslicing*) su un segmento Ethernet condiviso. Tipicamente, in tutti i nodi sono integrati degli **hub**.

Il master, chiamato **Managing Node (MN)**, pianifica tutte le trasmissioni ed è l'unica stazione "attiva". Tutti gli altri nodi sono chiamati **Controlled Nodes (CN)** e sono autorizzati a trasmettere solo su esplicita richiesta del MN.

Il MN garantisce un accesso in tempo reale **privo di collisioni** per i dati critici dal punto di vista temporale (*time-critical*) attraverso trasmissioni *pianificate*, consentendo il passaggio di frame non in tempo reale (TCP/IP o UDP/IP) solo durante un intervallo di tempo riservato a questo scopo.

Un ciclo POWERLINK si compone di tre fasi:

1. **FASE INIZIALE**: Nella prima fase, il MN (*Managing Node*) invia un frame di "**Inizio Ciclo**" (**SoC - Start of Cycle**) a tutti i CN (*Controlled Nodes*) per sincronizzare i dispositivi. Questo indica che ciascun nodo deve prepararsi a trasmettere il proprio frame.
2. **FASE ISOCRONA**: I dati critici dal punto di vista temporale (*time-critical*) vengono poi scambiati nella seconda fase, denominata **fase isocrona**.
3. **FASE ASINCRONA**: La terza fase di un ciclo è la **fase asincrona**, all'interno della quale vengono trasferiti i dati non critici dal punto di vista temporale, come i dati TCP/IP o i dati di configurazione dei parametri.
4. **IDLE TIME**.

Basic Cycle TIME

- La durata di un ciclo è un parametro, generalmente indicato come "**Cycle Time**", solitamente impostato dall'utente all'inizio dell'operazione di rete.
- La lunghezza di tali fasi può variare, a condizione che la lunghezza totale di tutte le fasi rimanga entro i limiti del tempo di ciclo di base (**basic cycle time**).

- Il tempo di ciclo di base dipende dalla **quantità di dati isocroni**, di **dati asincroni** e dal **numero di nodi** da sottoporre a polling durante ciascun ciclo.
- Tempi di ciclo tipici hanno valori : 500 μ s, 200 μ s.
- Il tempo minimo richiesto per eseguire un ciclo rappresenta il tempo minimo che deve trascorrere tra due esecuzioni consecutive di un'azione ciclica (cioè lettura / scrittura di dati da / verso un nodo di rete) e possono quindi essere definiti come tempo minimo di campionamento raggiunto dal sistema di rete.

Struttura di un basic cycle

Fase Iniziale

- Il MN invia un frame *multicast* di **Start-of-Cycle (SoC)** per segnalare l'inizio di un ciclo.
- La temporizzazione di questo frame costituisce la base dei tempi (*time base*) per la sincronizzazione della rete.

Fase Isocrona

- Viene inviato un frame di **Poll-Request (PReq)** in modalità *unicast* a ogni nodo configurato e attivo.
- Il nodo indirizzato risponde con un frame di **Poll-Response (PRes)** in modalità *broadcast*.
- Poiché tutti gli altri nodi rimangono in ascolto di tutti i dati durante questa fase, il sistema di comunicazione fornisce una relazione di tipo produttore-consumatore (*producer-consumer*).
- L'intervallo temporale che include il frame $PRqe_{Cni}$ e il frame $PRqes_{Cni}$ prende il nome di **time slot** per il nodo indirizzato.
- Non tutti i CN (*Controlled Nodes*) vengono necessariamente sottoposti a polling ad ogni ciclo. Lo standard supporta due classi di comunicazione: **continua** e **multiplexata**:
 - Nella classe continua, un CN viene sottoposto a polling a **ogni ciclo**;
 - Nella classe multiplexata, un CN viene sottoposto a **polling ogni n cicli**.
- In un singolo ciclo possono quindi coesistere sia CN sottoposti a polling a ogni ciclo, sia altri nodi sottoposti a polling con un periodo che è multiplo del ciclo di base.
- Poiché i **requisiti sugli intervalli di campionamento dei dati variano da applicazione** ad applicazione, non tutti i CN (*Controlled Nodes*) devono essere sottoposti a polling in ogni ciclo. Le applicazioni di **motion control** hanno bisogno di fornire costantemente informazioni aggiornate agli azionamenti e di campionare i loro dati di feedback a ogni ciclo, mentre per la **misurazione della temperatura** i dati dei sensori devono essere campionati solo una volta ogni 3 cicli.
- **Con il multiplexing, i CN con dati a bassa priorità condividono uno slot temporale (time slot) all'interno della fase isocrona. Lo schema di multiplexing viene configurato all'avvio del sistema.**
- I CN si alternano in modo tale che un dispositivo utilizzi il primo ciclo, un secondo dispositivo il secondo ciclo e un terzo dispositivo il terzo ciclo (ovvero una volta ogni 3 cicli) per trasferire i propri dati. Il MN (*Managing Node*) assegna e gestisce gli slot temporali condivisi per questi gruppi di nodi.

- Il multiplexing ottimizza l'uso della larghezza di banda (efficienza) nella fase isocrona.
- Il multiplexing consente di inviare sul bus i dati ad alta frequenza a ogni ciclo, mentre i dati a bassa frequenza vengono inviati sul bus solo ogni n-esimo ciclo.
- Il multiplexing è un'ottimizzazione perché mantiene il cycle time il più basso possibile.

Fase Asincrona

- Il frame di **Inizio Fase Asincrona (SoA - Start-of-Asynchronous)** è il primo frame della fase asincrona e funge da segnale per tutti i CN che la fase isocrona è terminata.
- In questo modo, la trasmissione dei dati isocroni e di quelli asincroni non interferirà mai, garantendo una temporizzazione precisa della comunicazione (*precise communication timing*).
- Durante la fase asincrona, l'accesso al segmento di rete EPL può essere concesso a un singolo CN o al MN per il trasferimento di **un unico messaggio asincrono alla volta Asynchronous Send**.
- Un nodo Slave può inviare un messaggio asincrono (ASnd) solo se ha preventivamente prenotato il proprio turno all'interno della sua Poll Response (PRes) durante la fase isocrona.
- Il protocollo preferito per i messaggi asincroni è UDP/IP.

Fase Idle

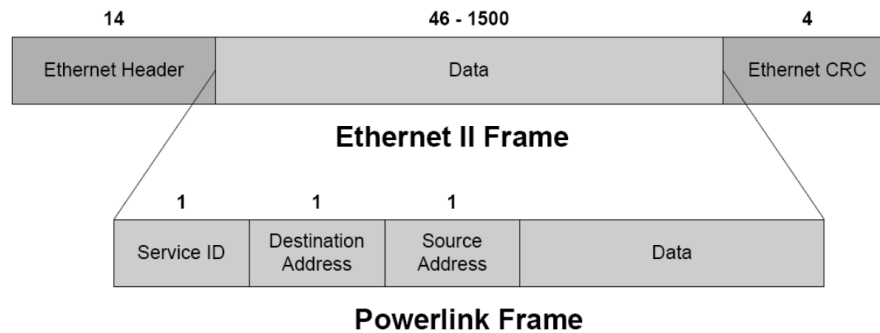
Alla fine del tempo di ciclo (subito dopo la conclusione della fase asincrona) c'è un periodo di pausa chiamato Idle Time (o tempo di inattività). Questo spazio vuoto è un elemento strutturale fondamentale per garantire il determinismo assoluto della rete.

L'Idle Time funge da "ammortizzatore": assorbe le variazioni temporali introdotte dalla fase asincrona (dove i messaggi possono avere lunghezze variabili) facendo in modo che, indipendentemente da quanto sia durata la fase asincrona, il ciclo totale si chiuda **esattamente** allo scadere del tempo impostato.

Per evitare che la fase asincrona sia troppo lunga e che il ciclo vada in **overrun**, nel tempo di ciclo totale vi è sempre un margine di Idle Time teorico sufficiente a gestire il frame asincrono più grande possibile (1518 byte di un frame Ethernet standard).

Incapsulamento

I dispositivi EPL utilizzano l'hardware Ethernet standard. Il frame EPL viene incapsulato all'interno del campo dati (*data field*) del frame Ethernet II.



Implementazione di rete

Modalità protetta

Per funzionare correttamente nella comunicazione in tempo reale (RT), i dispositivi EPL devono operare in **Modalità Protetta (*Protected Mode*)**, vale a dire che il segmento RT deve essere separato dal traffico normale tramite un bridge, un router o un gateway.

Una rete EPL è una "rete Ethernet protetta" definita con un controllore che funge da MN (*Managing Node*) e diversi dispositivi di campo implementati come *Controlled Nodes* (CN).

Al fine di salvaguardare il meccanismo di accesso al canale del MN, i nodi non-EPL non sono ammessi all'interno della rete Ethernet protetta. I dispositivi EPL che richiedono la comunicazione RT non possono coesistere sullo stesso segmento con dispositivi Ethernet non-RT, poiché questi ultimi corromperebbero il meccanismo di accesso di EPL. Tuttavia, i dispositivi EPL possono operare come normali dispositivi Ethernet.

Modalità Aperta

In **Modalità Aperta (*Open Mode*)**, il traffico RT condivide il segmento con il traffico normale, ma la comunicazione in tempo reale risulta compromessa, perché Ethernet nativo rimane CSMA/CD.

Implementazione di rete

- **NO SWITCH:** EPL non necessita di switch per evitare le collisioni o per fornire la sincronizzazione della rete: il MN (*Managing Node*) controlla autonomamente questi aspetti. Le reti EPL possono essere realizzate utilizzando **hub standard**.
- Per facilitare l'implementazione del bus, viene suggerito che ogni dispositivo integri al suo interno un hub.
- È possibile collegare in cascata **fino a 10 hub**.
- Gli switch, sebbene non siano proibiti, non sono raccomandati per EPL poiché introducono jitter e di conseguenza riducono il determinismo.
- **TOPOLOGIE:** Sulla rete è possibile realizzare qualsiasi combinazione di topologie senza necessità di configurazione, come strutture a stella, ad albero, a *daisy chain* o ad anello.

- EPL supporta **l'hot plugging** (collegamento a caldo), vale a dire che consente agli operatori di sostituire sensori o aggiungere unità meccatroniche senza dover prima spegnere o arrestare la rete.
- Nuovi dispositivi o componenti sostitutivi si identificano inviando i propri dati di dispositivo al *managing node* durante la *fase asincrona*.
- EPL presenta un'eccellente robustezza contro la compatibilità elettromagnetica (EMC).

Ridondanza

Per garantire un'elevata disponibilità, un requisito fondamentale per le reti di automazione, POWERLINK supporta due tipi di ridondanza:

- **ridondanza di mezzo**
- **ridondanza di master**

Ridondanza ad anello (*Ring redundancy*): Un'opzione semplice ed economica. I nodi sono collegati sotto forma di anello, con entrambe le estremità esterne della linea dati che attraversano la rete collegate al controllore. Ogni CN dispone di **due porte Ethernet collegate internamente da un hub**. Per formare un anello, i CN sono collegati in cascata (*daisy-chain*) e l'ultimo CN viene ricollegato al MN.

- In questa configurazione, la rete continuerà a funzionare anche se un singolo cavo dovesse rompersi o scollegarsi accidentalmente.
- Per questo layout è sufficiente un **cavo dati aggiuntivo per convertire una cascata di applicazioni in un anello**, a condizione che il controllore disponga di due interfacce che supportano il funzionamento ridondante.
- Quando viene rilevato un guasto sulla linea, il sistema passa dalla linea dati guasta alla linea dati ridondante **entro un singolo ciclo**.
- Viene tipicamente utilizzata per soluzioni che comportano una notevole **sollecitazione meccanica**.
- Se un nodo o un cavo si guastano, i pacchetti inviati da una delle connessioni del gestore dell'anello (*ring manager*) non vengono ricevuti sulla sua altra connessione.
- *Da quel momento in poi, il gestore dell'anello invia i pacchetti in entrambe le direzioni.*

Ridondanza completa del mezzo: nei sistemi critici che richiedono funzionalità di alta disponibilità, la ridondanza completa del mezzo si ottiene attraverso l'uso di **due reti fisiche distinte**. Le stesse informazioni ridondanti vengono quindi trasferite contemporaneamente su entrambe le reti.

In caso di guasto della rete, la commutazione (*switchover*) è istantanea e non richiede tempi di fermo o riconfigurazioni. Ogni nodo è collegato a entrambe le reti fisiche tramite un selettore di collegamento (*link selector*).

Dettaglio sul funzionamento: sono presenti 2 cavi fisici e 2 reti collegate a ciascun nodo tramite un selettore di collegamento. Il selettore rileverà quando la rete su un cavo è giù (interrotta) e commuterà automaticamente sulla rete dell'altro cavo.

Ridondanza del Master: utilizzata nei sistemi ad alta disponibilità (ad esempio nell'industria di processo), si basa su due o più Managing Node ridondanti.

Solo uno di questi nodi funge da MN attivo (**AMN - Active MN**); gli altri rimangono in uno stato di attesa attiva (**hot standby**) e agiscono come normali CN dal punto di vista del MN attivo.

L'unica differenza tra un MN di standby (**SMN - Standby MN**) e un CN è che lo Standby MN monitora continuamente tutte le funzioni della rete e dei CN, in modo da poter assumere la funzione di MN Attivo in qualsiasi momento senza necessità di riavvio (*reboot*). In caso di emergenza, l'ID del nodo riservato al MN viene trasferito al volo al MN ridondante più vicino.

Dettaglio sul funzionamento: se il MN si guasta, l'intera rete si arresta. Per evitare ciò, EPL consente di avere **fino a 9 master ridondanti aggiuntivi**, in cui il controllo viene passato automaticamente a un MN disponibile qualora il MN principale dovesse andare giù. Questa commutazione avviene **entro un singolo ciclo di bus** e causa un'interruzione minima alla rete.

Modified Ethernet

La maggior parte delle soluzioni che forniscono servizi in tempo reale di tipo *hard* (vincoli temporali rigidi) si basa su modifiche apportate all'hardware del dispositivo o dello switch. Tali modifiche sono obbligatorie per tutti i dispositivi presenti all'interno del segmento in tempo reale. Esempio , EtherCAT, PROFINET IO (IRT)

EtherCAT

- EtherCAT è una tecnologia Ethernet industriale in tempo reale adatta a soddisfare requisiti di *hard* e *soft* real-time nella tecnologia di automazione, nel settore dei test e delle misurazioni e in molte altre applicazioni.
- Il focus principale durante lo sviluppo di EtherCAT è stato posto su **tempi di ciclo brevi(*)**, **basso jitter** per una **sincronizzazione accurata** e **bassi costi hardware**.
- Sviluppata originariamente da Beckhoff Automation, oggi la tecnologia EtherCAT fa capo all'EtherCAT Technology Group (ETG), che conta più di 5000 aziende associate in tutto il mondo, tra cui produttori di dispositivi, utenti finali e fornitori di tecnologia.

Applicazioni tipiche di EtherCAT: applicazioni di **automazione industriale** che coinvolgono **dispositivi di I/O**, sensori e controllori a logica programmabile (PLC), nonché il controllo del movimento in tempo reale (*real-time motion control*).

EtherCAT offre:

- Efficienza della larghezza di banda molto elevata (superiore al 90%)
- Tempi di ciclo brevi (≤ 100 microsecondi)
- Basso jitter per una sincronizzazione accurata ($\leq$ microsec)

(*) **Tempo di ciclo (Cycle time):** tempo di aggiornamento dei dati, vale a dire il tempo necessario per scambiare i dati di I/O tra il controllore e tutti i dispositivi collegati in rete esattamente una volta.

Elaborazione On The Fly

- Le tipiche reti di automazione sono caratterizzate da lunghezze dei dati ridotte, molto inferiori al payload minimo di un frame Ethernet.
- L'utilizzo di un singolo frame per ciascun nodo in ogni ciclo può portare a un basso utilizzo della larghezza di banda.
- EtherCAT migliora l'Ethernet tradizionale implementando **l'elaborazione on-the-fly**, in cui i nodi della rete EtherCAT leggono i dati e i comandi da un frame mentre quest'ultimo li attraversa.
- Ogni Slave legge i dati a lui indirizzati "al volo" e inserisce i propri dati nel frame mentre il frame stesso si muove verso valle (*downstream*), non si aspetta quindi di ricevere tutto il frame completo.
- Il frame subisce solo il **ritardo di propagazione dell'hardware**: poiché i dati vengono elaborati direttamente in hardware (tramite l'ESC - *EtherCAT Slave Controller*) anziché

attendere l'elaborazione dello stack software, il **ritardo è estremamente basso**, rendendo questo approccio ideale per il *motion control* e per I/O veloci.

Più in dettaglio:

- Il **Master EtherCAT** trasmette un frame Ethernet contenente i dati e i comandi di lettura, scrittura o lettura/scrittura (R, W, R/W) per tutti i nodi presenti sulla rete.
- Ciascun nodo legge il dato o il comando a lui indirizzato "**al volo**" (***on the fly***), scrive i propri dati di risposta all'interno del frame stesso e lo trasmette al dispositivo successivo a valle (***downstream***).
- Un frame non viene ricevuto completamente prima di essere elaborato. Al contrario, l'elaborazione inizia non appena possibile. Anche la trasmissione verso il nodo successivo viene eseguita con un ritardo minimo di pochi tempi di bit (***bit times***).
- Il frame attraversa tutti i nodi in sequenza.
- Quando arriva all'ultimo nodo di una linea (***trunk***), il frame viene invertito e rimandato indietro.
- I nodi elaborano le informazioni contenute nel frame esclusivamente mentre questo transita in una sola direzione, ovvero dal Master verso gli Slave (***downstream***).
- L'ultimo Slave della catena ritrasmette il frame al Master sfruttando la funzionalità ***full-duplex*** di Ethernet.
- In ***upstream*** verso il Master, il frame viene semplicemente inoltrato.
- Ogni Slave viene attraversato dai frame due volte, in direzioni opposte, che sono note rispettivamente come **elaborazione (*processing*)** e **inoltro (*forwarding*)**.
- Ogni Slave è dotato di **due porte di rete e due interfacce**, vale a dire un'interfaccia a monte (***upstream***, nella direzione del Master) e un'interfaccia a valle (***downstream***, nella direzione opposta).
- L'elaborazione del frame avviene esclusivamente sul percorso a valle (***downstream path***), mentre il percorso a monte (***upstream***) viene utilizzato unicamente per propagare i frame all'indietro verso il Master.
- Le stazioni EtherCAT (sia il Master che gli Slave) devono essere dispositivi ***full-duplex***, cioè capaci di ricevere e trasmettere dati contemporaneamente.
- Il master riceve il messaggio e ricomincia dopo un **inter frame gap**.

Vantaggi:

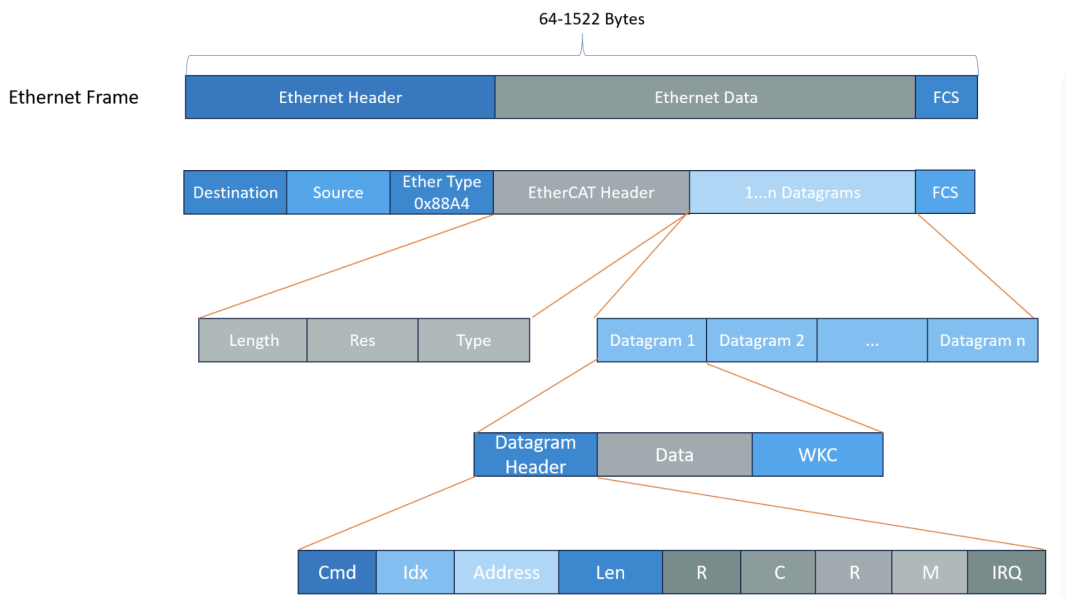
- **Migliore utilizzo della larghezza di banda**, dato che un singolo frame per ciclo è spesso sufficiente per l'intera comunicazione),
- **Elimina la necessità di switch o hub** all'interno di un singolo segmento grazie all'ESC, un chip hardware dedicato, no store and forward.
- EtherCAT supporta diverse **topologie**, ovvero: a ***daisy-chain***, a linea, ad albero e a stella.

EtherCat Slave Controller

I dispositivi Slave EtherCAT utilizzano un cosiddetto EtherCAT Slave Controller (ESC) per elaborare i frame al volo e interamente in hardware, al fine di ottenere ritardi bassi e prevedibili. Per realizzare uno Slave EtherCAT, è necessario un circuito integrato specifico per l'applicazione (**ASIC**, Application specific integrated circuit) dedicato all'accesso al mezzo, il quale integra uno switch a due porte direttamente all'interno del dispositivo stesso.

Il Master utilizza un controllore di accesso al mezzo standard (**Ethernet MAC standard**) senza alcun processore di comunicazione aggiuntivo. Ciò consente di implementare un Master su qualsiasi piattaforma hardware dotata di una porta Ethernet disponibile, indipendentemente dal sistema operativo in tempo reale (RTOS) o dal software applicativo utilizzato.

Frame EtherCAT



Il Master trasmette frame **Ethernet standard**.

I frame ethernet standard incapsulano al loro interno frame **EtherCAT**.

Le frame EtherCat contengono i **telegrammi** (talvolta chiamati **datagram**).

Ciascun telegramma EtherCAT è un comando costituito da:

- **Header** (indirizzo, comando) specifica l'operazione da eseguire (Lettura - R, Scrittura - W, oppure Lettura/Scrittura - R/W) e lo Slave che deve eseguirla.
- **Data field** è l'area di memoria utilizzata per l'operazione di lettura/scrittura (R/W).
- **Working Counter (WKC)** viene incrementato dallo Slave a ogni operazione R/W andata a buon fine, per segnalare al Master che lo Slave ha effettivamente elaborato il comando. Viene inoltre utilizzato per il rilevamento degli errori.

L'**indirizzamento diretto**, basato sulla **posizione fisica** dello Slave nella rete al momento della *configurazione*, consente l'accesso a *uno specifico Slave*.

Più comunemente, per i dati ciclici si utilizza l'**indirizzamento implicito**, ovvero un indirizzamento logico che permette di accedere a *più dispositivi contemporaneamente* e consente anche **Hot Connect** 🤖 in caso di aggiunta o rimozione di dispositivi.

Nell'Indirizzamento implicito, il Master vede l'intera rete non come un insieme di dispositivi separati, ma come **un unico enorme area virtuale grande 4 Gigabyte**. Questa memoria virtuale prende il nome di **Immagine di Processo Globale (Process Image)**.

Ogni datagramma indirizza una specifica **sotto-area** dell'immagine di processo (*process image*) nel segmento EtherCAT. Durante l'avvio della rete, a ciascun dispositivo Slave vengono assegnati uno o più indirizzi all'interno di questo spazio globale.

Quindi in una rete EtherCat tutti i dati sono contenuti all'interno di un unico pacchetto e ogni slave legge quello che gli interessa, cioè il datagramma che appartiene alla sua sottoarea di processo.

Riepilogo del funzionamento dello Slave EtherCAT

Quando uno Slave EtherCAT riconosce un comando di suo interesse, esegue le relative azioni "al volo" (*on-the-fly*), leggendo e/o scrivendo porzioni del frame Ethernet mentre questo viene ritrasmesso.

Ciascuno Slave elabora e propaga il frame in avanti in un tempo estremamente ridotto (solitamente $<1 \mu s$). L'ultimo Slave della catena trasmette il frame all'indietro, il quale si propaga di Slave in Slave fino a raggiungere il Master. Successivamente, dopo un intervallo di tempo inter-frame (**IFG - Interframe Gap**), il ciclo ricomincia.

Ciascun Slave controlla il **CRC** (controllo di ridondanza ciclico) del frame.

- Se il frame risulta corrotto, lo **Slave** non lo utilizza, ovvero non passa i dati contenuti nel frame all'applicazione dello Slave stesso. Inoltre lo Slave incrementa un contatore di errore e contrassegna il frame come corrotto per rendere consapevoli gli altri Slave dell'accaduto.
- Il **Master** vedrà che il frame è corrotto e lo scarcerà. Analizzando i valori del contatore di errore di ciascun nodo, il Master è in grado di rilevare dove si è verificato originariamente il guasto sulla linea.

EtherCAT è *Modified Ethernet*

NODO MASTER: nodo ethernet qualsiasi che implementa EtherCat.

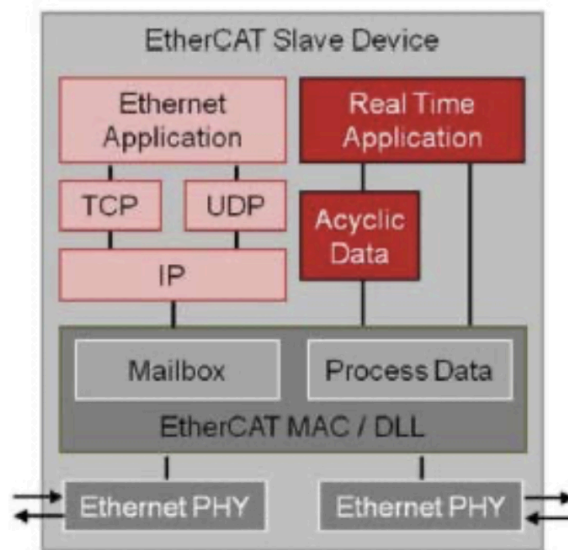
NODO SLAVE: HARDWARE SPECIALIZZATO. Deve avere due porte Ethernet e deve essere in grado di lasciare passare le frame attraverso esso per leggere o scrivere sulla frame passante.

LIVELLO FISICO: 100BASE-TX rame, fibra ottica 100BASE-FX.

LIVELLO MAC: viene implementato o in un ASIC specializzato o in un FPGA, comunque circuiti integrati dedicati.

Al di sopra del livello MAC sono presenti:

- **Stack specifico** dell'applicazione industriale in tempo reale, che gestisce il comportamento specifico dell'applicazione.
- **Stack standard TCP/IP e UDP/IP** per supportare i profili di dispositivo basati su Ethernet (le comunicazioni TCP/IP possono usare il meccanismo asincrono della mailbox).



EtherCAT P: consente la trasmissione simultanea di dati e alimentazione sullo stesso cavo.

Tipologie di traffico EtherCAT

Le trasmissioni dati in EtherCAT possono essere **cicliche** e **asincrone (acicliche)**.

- I **dati ciclici** sono i dati di processo real time (*process data*) che vengono trasferiti a intervalli periodici (o tempi di ciclo). EtherCAT è ottimizzato per la gestione di dati di processo ciclici di breve lunghezza (tramite lettura/scrittura con buffering).

Per quanto riguarda i **dati asincroni (aciclici)**:

- **ASINCRONI CRITICI:** **asincroni** che sono **critici** e presentano requisiti real time (come, ad esempio, i dati di diagnostica) vengono schedulati nello stesso modo del **traffico ciclico**. Il Master invia ciclicamente uno o più frame EtherCAT che contengono tali telegrammi.
- **ASINCRONI NON CRITICI:** Il traffico asincrono non in tempo reale, che può essere di grandi dimensioni e viene solitamente scambiato in risposta a un comando del controllore, viene gestito mediante speciali frame chiamati **mailbox**.

Lo Slave ha un grosso blocco di dati pronto nella sua casella postale interna e lo segnala al Master. Appena il traffico real-time lo consente, il Master genera un **frame EtherCAT specifico ed esclusivo per la Mailbox** di quel determinato Slave. Questo frame viaggia in modo indipendente ed esegue una transazione sicura: svuota la casella postale dello Slave (o la riempie, in caso di download dal Master) usando un protocollo di controllo che garantisce che nessun byte vada perso, anche se il trasferimento richiede più cicli di rete per essere completato.

- Le connessioni TCP/IP possono essere "incapsulate" (*tunneled*) attraverso il canale asincrono della mailbox senza impattare in alcun modo sul trasferimento dei dati in tempo reale. (GUARDARE STACK SOPRA)

Sincronizzazione

Un algoritmo di sincronizzazione dei clock distribuito e **completamente basato su hardware** (Distributed Clocks) consente di sincronizzare i nodi, mantenendo il jitter significativamente inferiore a 1 μ s. Il tempo del primo Slave viene distribuito ciclicamente a tutti gli altri dispositivi del sistema, con una compensazione automatica dei ritardi di propagazione del segnale.

I numeri e le caratteristiche di EtherCAT

- **DISPOSITIVI:** Un Master EtherCAT può indirizzare uno spazio di indirizzamento fino a 4 GB, supportando in questo modo fino a **65.535 dispositivi**.
- **DISTANZE MASSIME:** Lo strato fisico *Fast Ethernet* (PHY) consente a due dispositivi di trovarsi a una **distanza massima di 100 m** l'uno dall'altro. Distanze maggiori sono possibili grazie all'utilizzo della fibra ottica.
- **BANDA:** Utilizzo della larghezza di banda: **superiore al 90%**.
- Non vi è alcuna restrizione sull'ordine con cui gli Slave vengono indirizzati rispetto alla loro effettiva posizione fisica all'interno della rete.
- EtherCAT è adatto sia per architetture di sistema **centralizzate** che **decentralizzate**: può supportare la comunicazione *master-to-slave* e *slave-to-slave* (in base alla posizione dei nodi, quest'ultima potrebbe dover passare attraverso il Master e richiedere due cicli).
- EtherCAT consente la **ridondanza di cavo** collegando un *cavo dall'ultimo nodo a una porta Ethernet aggiuntiva presente nel Master*, estendendo così una topologia a linea in una **topologia ad anello**. In caso di rottura di un cavo o di malfunzionamento di un nodo, l'evento viene rilevato da un componente software aggiuntivo (*software add-on*)

presente nello stack del Master. Gli Slave non necessitano di alcuna modifica e non sanno nemmeno di operare all'interno di una rete ridondante.

- È inoltre possibile realizzare la **ridondanza del Master** tramite la modalità **Hot Standby**.

PRAGRAFO IN PIÙ : HOT STAND BY

- C'è un Master Attivo che è il controllore che in quel momento ha il comando della rete e Stand By Master, il master di backup, ascolta passivamente tutto il traffico che transita sul bus.
- Il Master di riserva riceve le stesse identiche risposte dagli slave che riceve il Master attivo e aggiorna la sua copia **dell'immagine di processo** (process image) in tempo reale: la sua memoria e il suo stato interno sono identici a quelli del master attivo. Se il Master Attivo smette improvvisamente di funzionare l'SMN (di riserva) se ne accorge immediatamente perché non vede più transitare il frame di inizio ciclo entro il timeout previsto. Essendo già sincronizzato e informato sullo stato di tutta la fabbrica), lo Standby Master assume il ruolo di Master Attivo al volo. Il subentro (switchover) avviene solitamente entro un singolo ciclo di bus.

Hot Standby: Il sostituto è acceso, collegato alla rete e ha i dati già aggiornati. Il tempo di recupero del guasto (recovery time) è pari a zero o limitato a un ciclo di rete. L'impianto non si ferma e non perde campioni dati.

Warm Standby (Attesa Tiepida): Il sostituto è acceso, ma non è sincronizzato in tempo reale. In caso di guasto, deve prima caricare l'ultima configurazione o lo stato salvato. Richiede qualche secondo di fermo macchina.

Cold Standby (Attesa Fredda): Il sostituto è spento o va rimpiazzato fisicamente. In caso di guasto, l'impianto si arresta, si cambia il pezzo, si accende e si ricarica il software da zero (richiede minuti o ore).

Nelle applicazioni industriali critiche l'Hot Standby è l'unica opzione accettabile perché un arresto improvviso della rete, causerebbe danni gravi.

Fieldbus

Fieldbus: Una rete multipunto (*multidrop network* o bus) utilizzata per collegare i dispositivi come sensori, attuatori e controllori di campo (PLC, regolatori, ecc.).

Rete multipunto (*Multidrop network*): ciascun nodo immette le proprie informazioni all'interno di un mezzo trasmissivo comune (condiviso).

La Preistoria del Fieldbus

- **Prima dell'era del fieldbus**, dominavano numerosi standard, come ad esempio lo **standard 4–20 mA** per i sensori analogici. Il segnale analogico varia all'interno di un intervallo di 4-20 mA in proporzione alla variabile di processo rappresentata (0-100% dell'intervallo).
- Questi standard portavano a una **quantità enorme di cavi** all'interno delle fabbriche.

DCS: Distributed Control System (Sistema di Controllo Distribuito)

- Progettazione e installazione del cablaggio **onerose/costose**.
- Manutenzione o evoluzione **difficili**.
- Gli utenti finali richiesero una soluzione per **semplificare** queste operazioni.

I Vantaggi del Fieldbus

- **Riduzione dei costi di installazione**, minor numero di cavi e riduzione del peso.
- **Facilità di aggiunta/rimozione** dei dispositivi di campo.
- **Comunicazione bidirezionale** con i dispositivi di campo.
- **Miglioramento dell'accuratezza** delle informazioni inviate alla sala di controllo, con una migliore qualità e quantità del flusso informativo.
- **Potenziamento della manutenibilità** dei dispositivi di campo.
- **Riduzione degli errori** di installazione.
- **Accesso remoto ai dati di misura** tramite interfacce palmari/portatili.
- Possibilità di implementare **strategie di controllo più avanzate** grazie a dati di campo qualitativamente migliori.

Esigenze di Traffico del Fieldbus e Domini Applicativi

- **Stazioni:** Sensori, attuatori, PLC, ecc.
- **Traffico:** Essenzialmente **periodico**.
- **Esigenze di:** *Broadcasting* (trasmissione circolare), distribuzione degli algoritmi di controllo e sincronizzazione tra i processi applicativi.
- **Traffico asincrono:** Ordini di produzione, report di attività, download di programmi.

Analizziamo le caratteristiche dei seguenti tipi di domini applicativi in tempo reale:

- **Produzione discreta (*Discrete manufacturing*):** Caratterizzata da una **dinamica più veloce** (es. catene di montaggio, robotica).
- **Industria del controllo di processo (*Process control industry*):** Caratterizzata da una **dinamica più lenta** (es. impianti chimici, raffinazione, gestione dei fluidi).

Domini applicativi

Discrete Manufacturing Applications

- La produzione discreta coinvolge parti e sistemi che possono essere facilmente contati, toccati o visti.
- Tra due operazioni, il prodotto si trova in uno stato **stabile**: ciò significa che non viene danneggiato se viene stoccato (messo a magazzino) tra queste fasi.
- Esempi di prodotti realizzati tramite produzione discreta includono automobili, mobili, aerei, giocattoli, smartphone e sistemi di difesa.
- In teoria, un prodotto discreto può essere smontato alla fine del suo ciclo di vita in modo che i suoi componenti di base possano essere riciclati.
- **Requisiti per il Fieldbus nelle Applicazioni di Produzione Discreta**
 - **In base al dominio di fabbrica**, è richiesto l'utilizzo di supporti speciali (mezzi di trasmissione) adatti alle esigenze di **compatibilità elettromagnetica (EMC)**.
 - I vincoli di **affidabilità** si applicano al fieldbus interno a una macchina sia per la qualità dei prodotti, sia per la **sicurezza delle persone (operatori)**.
 - Altri criteri, come la **disponibilità (*availability*)**, di norma non sono critici, ma restano comunque importanti per la produttività.

Process Control Industries

- Nei processi continui, il prodotto viene creato utilizzando una formula/ricetta per raffinare gli ingredienti grezzi, e viene lavorato continuamente attraverso una sequenza di operazioni (eseguite da macchine diverse) senza alcuno stato stabile tra due operazioni successive.
- In questa categoria rientrano i processi dell'industria siderurgica (ferro e acciaio), molti processi chimici e biologici, l'industria alimentare e delle bevande (*food and beverage*), l'industria cartaria e la produzione di energia.
- Il prodotto finale non può essere scomposto o ridotto nei suoi componenti di base.
- **Requisiti per il Fieldbus nelle Industrie di Controllo di Processo:**
 - In termini ambientali, i processi continui – specialmente nell'industria chimica – sono il dominio della **sicurezza intrinseca** e della protezione contro le perturbazioni elettromagnetiche.
 - La **ridondanza** è spesso desiderata, e talvolta necessaria, per garantire la **disponibilità** e la **sicurezza** (delle persone, dell'ambiente e degli strumenti di produzione).

Controllo di Processo e Sicurezza Intrinseca

Una rete è a **sicurezza intrinseca** quando i quattro componenti principali del bus (dispositivi di campo, cavi, terminatori di bus, ecc.) rientrano entro limiti predefiniti per quanto riguarda **tensione, corrente, potenza, induttanza e capacità**.

Le apparecchiature e i cablaggi a sicurezza intrinseca non sono in grado di rilasciare energia elettrica o termica sufficiente, sia in condizioni normali che di guasto (anomale), da causare l'accensione di una specifica miscela atmosferica infiammabile.

Compatibilità Elettromagnetica (EMC)

La EMC (*Electromagnetic Compatibility*) si riferisce alla capacità di varie apparecchiature elettroniche di **funzionare correttamente** nel rispettivo ambiente elettromagnetico di destinazione. Ciò significa soddisfare specifiche relative alle **emissioni** e alla **suscettibilità** alle frequenze dello spettro radio.

- **Le problematiche legate all'Emissione riguardano:**
 - La generazione indesiderata di energia elettromagnetica da parte di una sorgente.
 - Le contromisure da adottare al fine di ridurre tale generazione.
- **Le problematiche legate alla Suscettibilità (o immunità), al contrario, riguardano:**
 - Il corretto funzionamento dell'apparecchiatura elettrica, definita "vittima", in presenza di disturbi elettromagnetici imprevisti.

La Standardizzazione del Fieldbus secondo la norma internazionale **IEC 61784** organizza i vari protocolli di comunicazione industriale in famiglie di profili denominate **CPF** (*Communication Profile Families*). Abbiamo due macro-categorie principali basate sulle diverse parti della norma:

1. IEC 61784-1 **Fieldbus**

Questa sezione raggruppa i protocolli Fieldbus tradizionali (basati principalmente su comunicazione seriale), utilizzati storicamente per connettere i dispositivi di campo.

Esempi: CPF1 FOUNDATION Fieldbus, CPF2 ControlNet™, CPF3 PROFIBUS, CPF6 INTERBUS, CPF9 HART.

2. IEC 61784-2 RT **Ethernet**

Questa sezione elenca i profili basati su **Industrial Ethernet in tempo reale (RT - Real Time)**. Rappresentano l'evoluzione dei fieldbus tradizionali verso le tecnologie Ethernet, necessarie per gestire volumi di dati maggiori e velocità superiori.

ESEMPI: **CPF12 EtherCAT**, **CPF13 EPL (Ethernet POWERLINK)**, **CPF15 Modbus**.

Esistono molti standard Fieldbus. Perché?

- **La necessità di una tecnologia fieldbus** è stata identificata da numerose aziende (utenti finali) operanti in **molti settori diversi**:
 - Controllo di processo, produzione discreta, automazione degli edifici (*building automation*), sistemi embedded (es. settore automotive), reti di servizi pubblici (*utilities*), sistemi di trasporto, ecc.
- **La varietà di possibili dispositivi ("host") da collegare** (ovvero la grande diversità di sensori, attuatori e controllori).

Inizialmente non esisteva alcuno standard:

- Ciascun fornitore di tecnologia comprese l'importanza strategica del fieldbus nei sistemi di automazione industriale e sviluppò le proprie **soluzioni proprietarie** in un determinato settore.
- Una volta sviluppati i propri prodotti, i fornitori spinsero per farli standardizzare; di conseguenza, si è creata una vasta gamma di standard, sia **de facto** che **de iure** (legali/ufficiali).

All'inizio degli anni '80, in Europa presero il via diversi progetti.

In Francia, nel 1982 nacque il progetto **FIP** sotto l'egida del Ministero francese della Ricerca e dell'Industria. *FIP* è l'acronimo di *Factory Instrumentation Protocol* (Protocollo di Strumentazione di Fabbrica) ed è oggi noto come **WorldFIP**.

Un processo simile portò nel 1984 al progetto del bus di campo **PROFIBUS** (*Process Field Bus*) in Germania e nel 1983 al progetto **P-Net** in Danimarca.

Nello stesso periodo, nel 1983, l'azienda **Bosch** sviluppò le specifiche del **CAN** (*Controller Area Network*) per le automobili prodotte in Germania.

Il termine **"fieldbus"** (bus di campo) comparve per la prima volta nel 1985 durante una riunione dell'IEC (*International Electrotechnical Commission*).

Sviluppo del Fieldbus

- **Il processo di standardizzazione iniziò** in quel periodo sia all'interno di questi diversi paesi, sia a livello internazionale:
 - Con il gruppo **IEC TC 65/SC65C/WG6**, in contemporanea con la **ISA** (*Instrumentation Society of America*) negli Stati Uniti (all'interno del gruppo ISA SP50 - *ISA-Standard Practice*).
- Vennero prese in considerazione **due tipologie di fieldbus**:
 - **Il fieldbus H1**: a bassa velocità di trasmissione dati (es. 1.2 kbps), destinato principalmente al collegamento di sensori nel controllo di processo.
 - **Il fieldbus H2**: ad alta velocità di trasmissione dati (es. da 9.6 kbps fino a 12 Mbps), destinato alla produzione (*manufacturing*) o all'interconnessione di diverse reti H1.

Profibus

Profibus è uno standard di automazione basato su **Fieldbus**.

PROFIBUS è lo standard di automazione basato su fieldbus di *PROFIBUS & PROFINET International (PI)*, la più grande comunità di automazione al mondo.

PROFIBUS è un'implementazione specifica e standardizzata della tecnologia fieldbus.

PROFIBUS collega i controllori o i sistemi di controllo a diversi dispositivi di campo decentralizzati (sensori e attuatori) tramite un **unico cavo**.

Il Fieldbus parla digitale: Il bus di campo, o fieldbus, è un sistema di comunicazione digitale di tipo seriale tra apparati.

Sistema non-fieldbus: Comunicazione unidirezionale

- I compiti dei dispositivi di campo (*field devices*) e del sistema di controllo (*control system*) sono nettamente separati.
- Vengono trasferiti **solo valori analogici** (dati misurati).
- Collegamento **unidirezionale** (a senso unico).

Sistema fieldbus: Comunicazione digitale e bidirezionale

- I dispositivi di campo (*field devices*) sono parte integrante del sistema di controllo.
- **Collegamento di comunicazione bidirezionale**.
- **Dialogo digitale** tra il controllore e i dispositivi di campo.
- I dispositivi di campo assumono un **nuovo ruolo** nel sistema di automazione; questo rappresenta un importante cambio di paradigma.

PROFIBUS DP e PROFIBUS PA

- **PROFIBUS DP** (*Decentralized Periphery*) viene utilizzato principalmente per dispositivi di **input/output ad alta velocità** e per collegare dispositivi intelligenti come gli azionamenti (*drives*). Può utilizzare diversi livelli fisici (*physical layers*) come l'**RS-485**, il **wireless** o la **fibra ottica**. L'RS-485 è il più comune.
- **PROFIBUS PA** (*Process Automation*) si riferisce alle seguenti caratteristiche aggiuntive:
 - Bus autoalimentato tramite il livello fisico **MBP** (*Manchester encoded Bus Powered*), in conformità alla norma IEC 61158-2.
 - Progettazione a **sicurezza intrinseca**.
 - Configurazione attraverso il bus.

Il livello fisico RS-485: L'**RS-485** è il livello fisico di un'interfaccia di comunicazione seriale standard. Definisce le caratteristiche elettriche per una comunicazione a **2 fili**, in **modalità half-duplex** e **multi-point** utilizzando cavi a doppino intrecciato (**twisted pair**).

L'RS485 invia segnali **differenziali**, in cui le tensioni alte e basse rappresentano lo 0 e l'1 binari, consentendo una comunicazione efficiente e a lunga distanza anche in ambienti rumorosi (soggetti a disturbi).

Alimentazione da Bus con Codifica Manchester

Manchester Bus Powered (MBP) è una tecnologia per la comunicazione bus di campo (in particolare **PROFIBUS PA**) che *combina dati e alimentazione* su un singolo cavo a 2 fili, utilizzando la codifica Manchester per l'integrità del segnale e una tensione in corrente continua (DC) per alimentare i dispositivi di campo.

Viene comunemente utilizzata per l'automazione di processo, consentendo a un massimo di 32 nodi per segmento di operare in sicurezza in aree **pericolose** (a rischio di esplosione).

- I segnali di **dati** e di **clock** sono combinati per formare un flusso di dati auto-sincronizzante.
- Ogni bit viene trasmesso entro un intervallo di tempo predefinito (*bit-time*).
- Ogni bit codificato contiene una **transizione al centro del periodo del bit**.
- La **direzione della transizione** determina se il bit è uno "0" o un "1".
- La prima metà rappresenta il valore reale del bit, mentre la seconda metà rappresenta il complemento del valore reale del bit.
- La codifica di ciascun bit di dati è da basso ad alto (*low-to-high*) o da alto a basso (*high-to-low*) per durate uguali.
- Si tratta di un segnale **auto-sincronizzante** (*self-clocking*).
- La **transizione al centro del bit** trasporta il **segnale di clock**.
- Le **transizioni iniziali**, che possono apparire ai confini dei periodi dei bit a seconda dei dati trasmessi, servono solo a regolare il livello di tensione iniziale e non contengono contenuto informativo.
- Poiché ogni bit è sempre composto da proporzioni uguali di segnale positivo e negativo, **il valore medio di questo segnale è sempre costante ($I_s = 10\text{mA}$)**.
- Questa media viene utilizzata per alimentare i dispositivi. Da qui deriva il nome MBP: **Manchester coded Bus Powered** (Alimentato da Bus con codifica Manchester).

Profibus usa due physical layers:

- IN PROFIBUS DP: RS-485 (meno flessibile: solo daisy chain).
- IN PROFIBUS PA: MBP (più flessibile: consente di avere diverse topologie).

Confronto

MBP: Permette di far viaggiare dati e alimentazione, riducendo i costi di cablaggio.

RS-485: Trasmette esclusivamente i dati.

MBP: Sicurezza intrinseca.

RS-485: Può essere reso sicuro, ma richiede l'aggiunta di componenti esterni che limitino l'energia verso l'area a rischio.

MBP: Ha una velocità fissa e relativamente bassa (31.25 kbps). Questo perché nel controllo di processo (es. monitorare la temperatura di un enorme silos di liquido) le variabili cambiano lentamente ed è più importante la robustezza e l'alimentazione a distanza rispetto alla velocità.

RS-485: Può raggiungere velocità **elevatissime** (fino a 12 Mbps). È fondamentale nell'automazione discreta (es. robotica, linee di confezionamento) dove i millisecondi fanno la differenza per sincronizzare i motori.

MBP: tollerante grazie alla natura del suo segnale auto-sincronizzante.

RS-485: doppino intrecciato schermato per proteggersi dai forti disturbi elettromagnetici delle fabbriche

PROFIBUS DP

PROFIBUS DP distingue tre gruppi di tipi di dispositivi:

- **Master di Classe 1 (DPM1):** È un controllore centrale che scambia informazioni in modo ciclico con le stazioni distribuite (slave) secondo un ciclo di messaggi specificato. Dispositivi DPM1 tipici sono i controllori a logica programmabile (**PLC**) o i PC. A un DPM1 è consentito l'accesso attivo al bus, grazie al quale può leggere i dati di misura (ingressi) dei dispositivi di campo e scrivere i valori di setpoint (uscite) degli attuatori a intervalli di tempo fissi. Questo ciclo che si ripete continuamente costituisce la base della funzione di automazione.
- **Master di Classe 2 (DPM2):** Sono dispositivi di ingegneria, configurazione o funzionamento. I DPM2 non devono necessariamente essere collegati in modo permanente al sistema bus. Vengono messi in funzione durante la fase di collaudo (*commissioning*), per la manutenzione e per la diagnostica, al fine di configurare i dispositivi collegati, valutare i valori misurati e i parametri, e richiedere lo stato dei dispositivi. Anche il DPM2 ha un accesso attivo al bus.
- **Slave (Schiavi):** Sono i dispositivi periferici (moduli di I/O, azionamenti/inverter, interfacce uomo-macchina ovvero HMI, valvole, trasduttori, analizzatori) che acquisiscono le informazioni di processo e/o utilizzano le informazioni di uscita per intervenire nel processo stesso. Esistono anche dispositivi che forniscono esclusivamente informazioni di ingresso o informazioni di uscita. Per quanto riguarda la comunicazione, gli slave sono **dispositivi passivi**; rispondono solo alle interrogazioni dirette (da parte dei master).

Sistemi Mono-master vs Sistemi Multi-master

Sistemi Mono-master

- **Solo un master è attivo** sul bus durante il funzionamento del sistema bus.
- Il PLC costituisce il componente di controllo centrale. I dispositivi slave sono collegati al PLC tramite il mezzo di trasmissione.
- Questa configurazione di sistema consente di ottenere **i tempi di ciclo del bus più brevi**.

Sistemi Multi-master

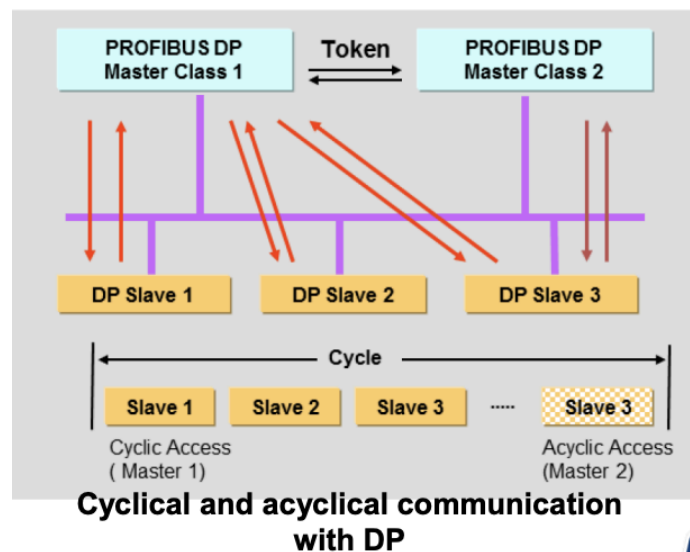
- **Diversi master condividono lo stesso bus.**
- Essi rappresentano sia sottosistemi indipendenti (composti da un master e dai relativi slave assegnati), sia dispositivi master aggiuntivi di configurazione e diagnostica.
- I master si coordinano tra loro **passandosi un "token"** (gettone) l'uno con l'altro. Solo il master che possiede il token può comunicare.

Protocollo di Comunicazione DP

PROFIBUS utilizza un unico protocollo di comunicazione aperto (PROFIBUS DP, Periferia Decentralizzata) per tutte le applicazioni

Il protocollo utilizza il modello **"Master-Slave"**: un dispositivo (master) controlla uno o più altri dispositivi (slave).

Il protocollo utilizza il modello **"Token Passing"** (Passaggio del gettone): il "token" viene trasmesso lungo la rete; la stazione che possiede il token controlla l'accesso alla rete stessa.



PROFIBUS DP utilizza due tipi di servizi di trasmissione per l'invio dei telegrammi di messaggio, definiti al **Livello 2** (Layer 2 - Livello Data Link):

SRD (Send and Request Data with acknowledgement)

- Con il servizio SRD, l'invio e la ricezione dei dati avvengono all'interno di un unico ciclo di telegramma.
- Il Master invia un frame contenente dei dati (ad esempio, i comandi di uscita per un modulo attuatore) e, **nello stesso identico ciclo di trasmissione**, richiede al dispositivo Slave di rispondere inviando i suoi dati (ad esempio, lo stato degli ingressi digitali)

SDN (Send Data with No acknowledgement)

- Questo servizio viene utilizzato quando un messaggio deve essere inviato simultaneamente a un gruppo di slave (**multi-cast**) o a tutti lo slave insieme (**broadcast**).
- Gli slave non rispondono e non inviano alcuna conferma di ricezione per i messaggi di tipo broadcast o multi-cast.

Applicazioni Chiave di PROFIBUS

1. Automazione di Fabbrica

- Produzione di veicoli
- Impianti di imbottigliamento
- Sistemi di stoccaggio / magazzini automatizzati,
- Quadri elettrici / Apparecchiature di manovra (Switchgear)
- Produzione di vetro cavo

2. Automazione di Processo

- Industria chimica
- Industria petrolchimica
- Industria cartaria e tessile
- Prodotti alimentari (Foodstuffs)
- Centrali elettriche
- Impianti di depurazione / trattamento acque (Sewage plants)

3. Tecnologia degli Azionamenti

- Macchine utensili
- Macchine per l'imballaggio / confezionamento
- Impianti di stampaggio / pressatura
- Produzione della carta

4. Applicazioni di Sicurezza

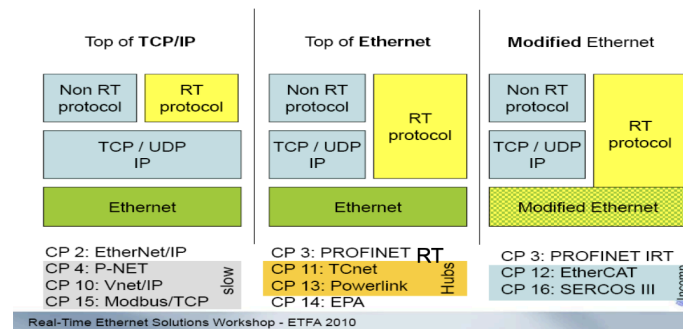
- Assemblaggio di veicoli
- Costruzione di macchine utensili

PROFINET

Vengono offerte due soluzioni PROFINET complementari

- **PROFINET RT (Real-time):** Un'implementazione definita "On top of Ethernet" (al di sopra di Ethernet standard) per comunicazioni in tempo reale di tipo **soft real-time**.
- **PROFINET IRT (Isochronous Real-Time):** Un'implementazione basata su "Modified Ethernet" (Ethernet modificato) per comunicazioni in tempo reale di tipo **hard real-time**.

Esse sono definite come diverse Classi di Conformità (*Conformance Classes*), ovvero delle estensioni/aggiunte (add-on) sviluppate al di sopra di una classe base chiamata **PROFINET IO**.



Profinet IO

PROFINET IO è un protocollo di comunicazione su Ethernet per lo scambio di dati tra controllori e dispositivi di I/O basato sul modello **Producer/Consumer** (Produttore/Consumatore). Le classi di dispositivi definite per PROFINET IO sono:

- **PROFINET IO Controller (Controllore):** È il PLC su cui gira il programma di automazione. Questo dispositivo è paragonabile al *master di classe 1* in PROFIBUS. L'IO Controller fornisce i dati di uscita (*output*) ai dispositivi di I/O configurati nel suo ruolo di produttore (*producer*), ed è il consumatore (*consumer*) dei dati di ingresso (*input*) dei dispositivi di I/O.
- **PROFINET IO Device (Dispositivo):** È un dispositivo di campo collegato a uno o più controllori. È paragonabile, come funzione, a uno *slave* in PROFIBUS. L'IO Device è il produttore (*producer*) dei dati di ingresso ed è il consumatore (*consumer*) dei dati di uscita.
- **PROFINET IO Supervisor (Supervisore):** È un dispositivo di programmazione (PD), un personal computer (PC) o un'interfaccia uomo-macchina (**HMI**) utilizzato per scopi di manutenzione o diagnostica. Corrisponde al *master di classe 2* in PROFIBUS.

Un'unità d'impianto (*plant unit*) contiene almeno un **PROFINET IO controller** (configuration, process data and alarms) e uno o più **PROFINET IO devices**. I **PROFINET IO supervisors** vengono solitamente integrati solo temporaneamente per scopi di collaudo (*commissioning*) o di risoluzione dei problemi (*troubleshooting*).

Panoramica delle Comunicazioni in PROFINET IO

Un dispositivo PROFINET può scambiare con un controllore PROFINET dati ciclici, dati aciclici e allarmi.

- **Scambio ciclico di dati di I/O con proprietà in tempo reale (Real-Time):** Si tratta di comunicazioni pianificate (*scheduled*) e ripetitive tra produttore (*producer*) e consumatore (*consumer*) sulla base di un intervallo di tempo assegnabile; non richiedono conferma di ricezione (*unacknowledged*).
- **Comunicazione aciclica dei dati:** Utilizzata per la lettura e la scrittura di parametri o di dati per la diagnostica o per l'identificazione e la manutenzione (I&M). Si tratta di una comunicazione non pianificata (*unscheduled*), eseguita su **richiesta** (*on demand*) e trasmessa tramite frame di lettura/scrittura (**Read/Write**) utilizzando il protocollo **UDP/IP**.
- **Modello di allarme flessibile:** Utilizzato per la segnalazione di errori del dispositivo o della rete. Prevede tre livelli di allarme (manutenzione richiesta, manutenzione urgente richiesta e diagnostica) e viene trasmesso come dato in tempo reale (**Real-Time**).

Profinet RT

PROFINET RT gestisce lo scambio di dati in tempo reale di tipo soft (*soft real-time*) al di sopra di Ethernet (on Top of Ethernet).

- Un frame Ethernet PROFINET RT in arrivo presenta l'**EtherType** specifico di PROFINET, ovvero **0x8892**. All'arrivo sul nodo di destinazione, il frame viene reindirizzato direttamente dal livello Ethernet (Livello 2) all'applicazione PROFINET (Livello 7).
- **Il frame salta i livelli TCP/IP**, evitando il tempo variabile necessario alla loro elaborazione; di conseguenza, la latenza di comunicazione e la prevedibilità (*predictability*) migliorano in modo significativo.

Prestazioni e Caratteristiche

- Le prestazioni di PROFINET RT dipendono dalla progettazione della rete ma, in linea generale, si possono raggiungere **tempi di ciclo da 128 ms fino a 250 µs** e un **jitter < 1 ms**.
- La soluzione RT si basa direttamente su **Ethernet standard**.
- Vengono utilizzati i meccanismi di Qualità del Servizio (**QoS**) e di prioritizzazione di Ethernet (tramite il **VLAN tag**) per dare la precedenza al traffico in tempo reale. Tuttavia, la QoS non risolve completamente i problemi di gestione delle risorse e di latenza: questo è il motivo per cui l'utilizzo è limitato alle comunicazioni *soft real-time*.

PROFINET IRT (*Isochronous Real-Time*)

PROFINET IRT rappresenta l'estensione per comunicazioni *hard real-time* su un *Ethernet modificato*.

- **Una parte della larghezza di banda Ethernet viene riservata esclusivamente** al traffico Isochronous Real-Time (IRT) attraverso un'estensione dell'hardware Ethernet standard. Ciò è reso possibile da una sincronizzazione estremamente precisa dei clock (orologi) nei nodi IRT.
- In ogni ciclo, durante un determinato intervallo di tempo (chiamato "**red interval**"), il traffico normale viene bloccato e non può essere inviato sul canale. Nella fase rossa, solo i frame Isochronous Real-Time possono accedere alla rete ed essere trasmessi.
- Inoltre, i partecipanti alla rete inviano i frame IRT **esattamente a orari precalcolati**.
- PROFINET IRT **elimina i ritardi variabili**, soddisfa tutti i requisiti di sincronizzazione e consente una comunicazione deterministica con un jitter dei tempi di ciclo ridotto fino a 1 μ s.

Il **ciclo di comunicazione IRT** (chiamato ciclo di **Sendclock**) è suddiviso in 3 fasi.

1. Red Interval: La Fase Isocrona

In questa prima fase vengono trasmessi tutti i frame IRT. Questi frame passano attraverso lo switch senza che vi sia alcuna interpretazione delle informazioni di indirizzo contenute nel frame Ethernet.

Gli switch sono impostati secondo una **tabella di marcia (*timetable*) predefinita e configurata**: a ogni istante di offset, il frame pianificato viene inviato da una porta all'altra senza interpretazione dell'indirizzo. Invio automatico, no interpretazione.

2. Green Interval

In questa seconda fase avviene la *comunicazione basata sugli indirizzi*. I dispositivi di commutazione (switch) passano alla comunicazione basata sull'indirizzamento e si comportano come switch Ethernet standard, dando la priorità ai frame RT rispetto a quelli TCP/UDP/IP. In particolare:

- **Fase Real-Time (RT) dell'intervallo verde**: I frame in tempo reale (RT) vengono trasmessi con alta priorità.
 - Si tratta di scambi ciclici non confermati (*non-acknowledged*) di dati di I/O basati sul modello produttore/consumatore (*producer/consumer*).
- **Fase Non-Real-Time (NRT) dell'intervallo verde**: Vengono trasmessi i frame Ethernet non in tempo reale (NRT), come il normale traffico IT o TCP/IP.

Isolamento Temporale e Protezione della Rete

L'isolamento temporale della fase isocrona garantisce che l'"**intervallo rosso**" sia **protetto dai ritardi** causati dalle trasmissioni dell'intervallo verde, consentendo così un livello estremamente elevato di determinismo.

Nel successivo "intervallo verde" aperto, i frame vengono trasmessi in conformità allo standard IEEE 802 e alle priorità specificate, **a condizione che la trasmissione si completi prima dell'inizio dell'intervallo rosso successivo**. In caso contrario, i frame vengono temporaneamente memorizzati (in un buffer) e verranno inviati durante l'intervallo verde successivo.

Funzionamento e Caratteristiche di Rete

- **Funzionamento:** Tipicamente reti Ethernet connesse tramite Switch a **100 Mbps in full-duplex**, ma è supportato anche il funzionamento a **1 Gbps**.
- **Lunghezza del cavo:** Fino a 100 metri su cavi in rame; distanze maggiori si ottengono utilizzando la fibra ottica.

Topologie Supportate da PROFINET

PROFINET supporta le seguenti topologie di rete:

- **Linea (*Line*):** Collega principalmente i dispositivi di campo sfruttando gli switch integrati a bordo dei dispositivi stessi.
- **Stella (*Star*):** Realizzata mediante l'utilizzo di uno switch PROFINET centralizzato.
- **Anello (*Ring*):** Utilizzata principalmente per la realizzazione della **ridondanza di supporto** (*media redundancy*).
- **Albero (*Tree*):** Intesa come combinazione delle topologie precedenti. I dispositivi di campo situati in una linea Profibus subordinata possono essere integrati nel sistema PROFINET in modo trasparente attraverso un **IO-Proxy**, che funge da rappresentante del sistema bus sottostante.

La Tecnologia Proxy (Gateway)

- I **Proxy** sono dei gateway (convertitori di protocollo) che rappresentano i dispositivi all'interno della rete PROFINET.
- I sistemi di controllo possono utilizzarli per accedere ai dispositivi di campo sia in modalità **ciclica** che **aciclica**, oltre che per finalità di diagnostica e configurazione.
- La tecnologia Proxy permette il **riutilizzo degli impianti già esistenti** e garantisce così un'eccellente protezione degli investimenti già effettuati.
- Per l'automazione di processo, questo consente di integrare i sistemi bus di campo preesistenti come **PROFIBUS DP/PROFIBUS PA** e altri ancora.

Dispositivi PROFINET

Per Profinet IRT

- **È necessario uno speciale switch PROFINET-Ethernet.**
 - Si raccomanda di integrare questo speciale switch PROFINET-Ethernet in ogni dispositivo (in genere, lo switch è integrato direttamente nel controllore Ethernet).
- Tutti i dispositivi di commutazione (switch) PROFINET IRT sono sincronizzati tramite un meccanismo **IEEE 1588 modificato**, al fine di avere i propri cicli e le tabelle di marcia IRT (*timetables*) sincronizzati con un **jitter massimo di 1 µs**.
- Il numero di switch collegati in cascata deve essere ridotto.
- Per le comunicazioni in tempo reale forte (*hard real-time*), PROFINET IRT richiede una **pianificazione temporale basata sulla topologia di rete**; si tratta di una pianificazione di sistema complessa, eseguita da un algoritmo centrale all'interno del sistema di ingegneria (il software di configurazione).

PROFINET over TSN

Attualmente è stata pubblicata la specifica PROFINET V2.4 con funzionalità TSN integrata al Livello 2 (Layer 2), denominata "PROFINET over TSN".

La tecnologia TSN viene integrata come tecnologia di Livello 2 aggiuntiva all'interno dell'architettura PROFINET.

PROFINET su TSN è integrato in chip standard. Questo apre la strada a nuove aree applicative e a nuovi progetti hardware per i controllori; inoltre, i dispositivi di campo possono essere implementati in modo più flessibile grazie a questi chip standard.

Ciò consente sia ai produttori che agli utilizzatori di sfruttare i vantaggi del TSN in modo completamente **interoperabile**.

L'integrazione del TSN permette di passare dalla configurazione del singolo dispositivo a una **configurazione a livello di intera rete** eseguita via software nel controllore o nel dispositivo PROFINET, semplificando così il lavoro dei tecnici e aumentando la flessibilità.

Funzionalità TSN integrate

Le caratteristiche TSN integrate includono, tra le altre:

- **IEEE 802.1AS:** Per la selezione del Master clock (orologio principale) e la sincronizzazione dei clock.
- **IEEE 802.1Q:** (Ad esempio, il supporto per il traffico pianificato - *Scheduled Traffic*, la prelazione dei frame - *Frame Preemption* e la configurazione di rete).
- **IEEE 802.1CB:** (Per la replicazione e l'eliminazione dei frame ai fini della ridondanza d'impianto).

802.11be (Wi-Fi 7)

Benefici delle comunicazioni wireless:

- **flessibilità**,
- costi ridotti del cablaggio, supporto alla mobilità.

Casi d'uso Industriali del Wireless

- Realtà Aumentata (AR) / Realtà Estesa (XR) / Realtà Virtuale (VR)
- Industrial Internet of Things (IIoT) (Internet delle Cose in ambito Industriale)
- Automated Guided Vehicle (AGV) (Veicoli a Guida Automatica) / Autonomous Mobile Robot (AMR) (Robot Mobili Autonomi)

Wireless technologies

- Wireless Local Area Networks (WLANs)
 - **IEEE 802.11 (Wi-Fi)**
- Wireless Personal Area Networks (WPANs)
 - IEEE 802.15.1 (Bluetooth)
 - IEEE 802.15.4 (ZigBee, Thread, WirelessHART)
- Low-Power Wide-Area Networks (LPWANs)
 - LoRaWAN
- Wireless Wide Area Networks (WWANs)
 - Cellular networks (e.g., 4G, 5G, ...)

Gruppo di Lavoro IEEE 802.11 (*IEEE 802.11 Working Group*)

- **Avviato nel 1990.**
- Definisce le specifiche del **livello PHY** (Fisico) e del **livello MAC** (*Medium Access Control* - Controllo dell'Accesso al Mezzo).

Wi-Fi Alliance (WFA)

- **Fondata nel 1999** (all'epoca denominata *Wireless Ethernet Compatibility Alliance*, WECA).
- Nel 2000 è stato coniato il termine **"Wi-Fi"**.
- La WECA è stata rinominata in *Wi-Fi Alliance* nel 2002.
- Si tratta di un **consorzio industriale**.
- Sviluppa suite di test per **certificare l'interoperabilità** dei prodotti commerciali.
- Seleziona un sottoinsieme di funzionalità definite dall'IEEE e le rende **obbligatorie** per l'ottenimento della certificazione commerciale.

Caratteristiche Chiave del 802.11be (Wi-Fi 7)

Livello PHY (Fisico)

- Larghezza di banda del canale a **320 MHz**
- Modulazione **4096-QAM**
- Multi-RU (Resource Units) e Preamble Puncturing

Livello MAC (Medium Access Control)

- **Multi-Link Operation (MLO)**: Consente ai dispositivi di inviare e ricevere dati contemporaneamente attraverso diverse bande di frequenza e canali (es. 2.4 GHz, 5 GHz e 6 GHz), riducendo drasticamente la latenza e aumentando l'affidabilità.
- **Restricted Target Wake Time (R-TWT)**: Un'evoluzione del meccanismo di risparmio energetico che permette di riservare finestre temporali esclusive per il traffico critico e sensibile alla latenza, ideale per i contesti industriali.
- **Gestione della QoS Avanzata (Enhanced QoS Management)**: Ottimizza la Qualità del Servizio per garantire la priorità e la puntualità nella trasmissione dei pacchetti di dati in tempo reale.

Multi-Link Operation (MLO) - Funzionamento Multi-Link

- **Le precedenti generazioni di dispositivi Wi-Fi** supportano già il funzionamento su diversi collegamenti (*links*), come ad esempio gli Access Point (AP) dual-band. Tuttavia, questi collegamenti operano in modo indipendente e senza alcuna coordinazione; ciascun link possiede il proprio livello MAC (*Medium Access Control*) separato.
- **Lo standard IEEE 802.11be introduce la funzionalità Multi-Link Operation (MLO)**: tutti i collegamenti disponibili (nelle bande a 2.4 GHz, 5 GHz e 6 GHz) vengono gestiti da un'unica istanza unificata del livello MAC a bordo del dispositivo Multi-Link (denominato **MLD - Multi-Link Device**), sia che si tratti dell'Access Point (AP MLD) sia che si tratti della stazione client (non-AP MLD).

Multi-Link Device (MLD) - Dispositivo Multi-Link

- I dispositivi in grado di supportare la tecnologia MLO vengono definiti **Dispositivi Multi-Link (MLD)**.
- Un MLD è un'entità **logica** capace di supportare più di una stazione (*STA - Station*) affiliata e può operare sfruttando una o più delle sue stazioni affiliate.

Tipologie di MLD

- **AP MLD (Access Point Multi-Link Device)**: Un dispositivo MLD in cui ogni singola stazione affiliata funge da Access Point.
- **Non-AP MLD (Client Multi-Link Device)**: Un dispositivo MLD in cui ogni singola stazione affiliata è una stazione client (non-AP STA).

I Vantaggi del Multi-Link Operation (MLO)

- **Aumento del rendimento (*Throughput boost*):** L'aggregazione di più collegamenti (*links*) genera un aumento cumulativo della velocità di trasmissione per i flussi di dati ripartiti sui vari canali, riducendo al contempo la latenza.
- **Riduzione della latenza (*Latency improvement*):** L'utilizzo di più collegamenti in parallelo aumenta le probabilità di accedere immediatamente a un canale radio libero, abbattendo così i tempi di attesa.
- **Maggiore affidabilità (*Increased reliability*):** Duplicando i pacchetti di dati critici contemporaneamente su più collegamenti diversi si aumenta in modo significativo l'affidabilità della trasmissione.
- **Separazione e differenziazione del traffico (*Traffic separation/differentiation*):** L'assegnazione dei flussi di dati a collegamenti differenti in base alle specifiche esigenze dell'applicazione permette di ottenere un'efficace separazione e/o differenziazione del traffico (ad esempio, riservando un canale esclusivo per i dati di controllo industriali e un altro per il traffico IT normale).

Restricted Target Wake Time (R-TWT)

Lo standard **IEEE 802.11be** introduce la funzionalità **R-TWT** per garantire prestazioni di latenza prevedibili (*predictable latency*) per specifici flussi di dati sensibili al fattore tempo. Attraverso l'R-TWT, un Access Point (AP) e una Stazione (STA) negoziano delle finestre temporali dedicate, note come **Periodi di Servizio (SP - *Service Periods*)**, per lo scambio dei dati.

La negoziazione definisce i seguenti parametri:

- **Intervallo di Risveglio (*Wake Interval*):** Indica con quale frequenza la stazione (STA) si attiva.
- **Durata del Risveglio (*Wake Duration*):** Indica la durata effettiva dei Periodi di Servizio (SP).
- **Tempo di Risveglio Obiettivo (*Target Wake Time - TWT*):** Indica l'orario di inizio del primo Periodi di Servizio.

Unità di Misura e Quantizzazione

La durata del risveglio (*Wake Duration*) è espressa in unità discrete (chiamate *wake duration unit*) che possono corrispondere a:

- Un'Unità di Tempo (**TU - *Time Unit***), pari a **1024 μ s**
- Una frazione di una TU, pari a **256 μ s**.

Il tempo di inizio (**TWT**), invece, è strettamente quantizzato in incrementi fissi di **1024 μ s**.

Per ottenere l'isolamento del traffico in tempo reale (*real-time traffic isolation*), l'Access Point (AP) pubblicizza la pianificazione temporale inserendo specifici Elementi TWT (TWT Elements) all'interno dei suoi frame di gestione periodici (noti come Beacon). Questi elementi R-TWT trasmettono in modalità broadcast a tutta la rete gli esatti confini temporali di ciascuna

pianificazione R-TWT attiva, ovvero i parametri temporali di TWT, Durata del Risveglio (Wake Duration) e Intervallo di Risveglio (Wake Interval).

Durante un Periodo di Servizio (SP) pubblicizzato per l'R-TWT, qualsiasi stazione (STA) che non fa parte di quella specifica pianificazione R-TWT deve rinviare la propria trasmissione qualora quest'ultima non possa essere completata prima dell'inizio del Periodo di Servizio stesso.

Enhanced Qos Management

Come il Wi-Fi 7 (802.11be) riesca a garantire l'**isolamento del traffico in tempo reale** e a evitare che i dispositivi "normali" disturbino le comunicazioni critiche (come quelle dei robot industriali) attraverso il meccanismo del **R-TWT**. Possiamo dividere il funzionamento descritto nella slide in due concetti chiave:

1. Come la rete viene informata della pianificazione (I Beacon)

Per far sì che tutti i dispositivi della fabbrica rispettino i tempi di trasmissione, l'Access Point (AP) deve comunicare a tutti la "tabella di marcia".

- L'AP inserisce delle informazioni speciali (chiamate **TWT Elements**) all'interno dei **Beacon**. I Beacon sono i classici pacchetti di gestione che l'Access Point invia continuamente nello spazio per dire "io esisto, questa è la rete".
- All'interno di questi elementi vengono trasmessi a tutti i dispositivi i tre parametri visti nella slide precedente: l'orario di inizio (**TWT**), quanto dura la finestra (**Wake Duration**) e ogni quanto si ripete (**Wake Interval**).

2. La regola di precedenza (Isolamento del traffico)

Una volta che tutti i dispositivi conoscono gli orari della finestra riservata (il Periodo di Servizio o SP), scatta una regola ferrea per i dispositivi che **non** fanno parte di quel flusso prioritario:

 **La regola del rinvio:** Se una stazione comune (ad esempio un PC o un sensore non prioritario) vuole inviare un dato, deve prima calcolare se il tempo necessario a trasmetterlo occuperà o meno la finestra riservata in arrivo. **Se la sua trasmissione rischia di sfiorare anche solo di un millisecondo dentro l'inizio del periodo R-TWT, il dispositivo deve obbligatoriamente rinviare l'invio e attendere che la finestra prioritaria sia terminata.**

Perché questo concetto è fondamentale?

Nelle versioni precedenti del Wi-Fi, se un dispositivo prioritario doveva parlare, doveva comunque "competere" via radio con tutti gli altri dispositivi, rischiando ritardi imprevedibili (mancanza di determinismo). Con questa regola descritta nella slide, il Wi-Fi 7 crea una vera e propria **"corsia preferenziale blindata"**: nell'istante in cui inizia la finestra R-TWT, il canale radio è matematicamente libero da interferenze interne alla rete, garantendo ai flussi industriali una latenza bassissima e totalmente prevedibile.

Seminari

RTOS

A differenza di un sistema operativo tradizionale (come quello dei nostri PC) ottimizzato per la produttività generale, un RTOS è progettato per garantire il **determinismo** : la certezza assoluta che un compito cruciale venga eseguito esattamente entro la sua scadenza (*deadline*).

L'architettura di un RTOS si basa su tre pilastri fondamentali:

1. **I Task (o Thread):** L'applicazione non è più un unico blocco di codice sequenziale, ma viene frammentata in compiti autonomi, loose coupled e funzionalmente coesi.
2. **Le Priorità:** In fase di progettazione, lo sviluppatore assegna a ogni task un livello di importanza o urgenza.
3. **Lo Scheduler:** È il cuore pulsante del kernel. Il suo compito è far sì che il task a priorità più alta si trovi sempre nello stato *Running* (in esecuzione). Per farlo, sfrutta un interrupt periodico chiamato *tick interrupt* per scansionare il sistema e decidere se applicare la prelazione (*preemption*), ovvero interrompere un task meno urgente a favore di uno prioritario.

I vantaggi concreti di un RTOS

Utilizzare un sistema come **FreeRTOS** — che è estremamente leggero e occupa appena tra i 4 e i 9 KB di memoria — offre vantaggi strategici enormi:

- **Modularità ed Estensibilità:** Astrarre le informazioni temporali dal codice permette di isolare i moduli software, rendendo il sistema infinitamente più facile da aggiornare e mantenere.
- **Gestione flessibile degli interrupt:** Possiamo mantenere le routine di interrupt estremamente brevi, delegando l'elaborazione pesante direttamente al livello dei task.
- **Primitive di sincronizzazione native:** L'RTOS mette a disposizione strumenti integrati come code, semafori e mutex per far comunicare i task tra loro e con l'hardware in totale sicurezza.

Steer By Wire

Lo **Steer-by-Wire** è un sistema di sterzo in cui i comandi del conducente vengono trasmessi **esclusivamente tramite segnali elettrici**, eliminando qualsiasi collegamento meccanico o idraulico tra volante e ruote. Il movimento del volante viene rilevato da sensori e convertito in segnali digitali che comandano gli attuatori delle ruote. Il sistema è costituito da due sottosistemi fondamentali:

- **Hand Wheel Actuator (HWA)**
 - rileva la rotazione del volante tramite sensori;
 - invia digitalmente il comando al sistema di sterzo;
 - restituisce al conducente il **feedback stradale** simulato (road feeling).

- **Road Wheel Actuator (RWA)**

- riceve i comandi dal volante;
- controlla gli attuatori elettrici che orientano le ruote;
- garantisce che la posizione delle ruote corrisponda al comando impartito dal guidatore.

Nello sterzo convenzionale:

- volante e ruote sono collegati meccanicamente;
- le forze provenienti dalla strada vengono trasmesse direttamente al volante.

Nel sistema Steer-by-Wire:

- il collegamento è solo elettronico;
- il feedback al conducente viene generato artificialmente tramite l'HWA;
- il controllo è affidato a sensori, centraline e attuatori.

Vantaggi

- **Riduzione del peso**
 - Eliminando componenti meccanici si riduce il peso del veicolo.
 - Minore peso significa minori consumi energetici e minore usura di pneumatici e freni.
- **Maggiore spazio interno**
 - I cavi occupano meno spazio rispetto ai componenti meccanici.
 - Nei veicoli autonomi il volante può addirittura scomparire nel cruscotto.
- **Maggiore comfort**
 - Le vibrazioni provenienti dal fondo stradale non vengono trasmesse direttamente al volante.
 - Il feedback viene ricreato elettronicamente.
- **Fault Tolerance (ridondanza)**
 - Il sistema può essere progettato con componenti elettronici duplicati.
 - Se un componente si guasta, un altro può continuare a garantire il controllo dello sterzo.
- **Maggiore sicurezza**
 - Il controllo elettronico rende lo sterzo più rapido e preciso.
 - Consente integrazione con sistemi ADAS e guida autonoma.
- **Maggiore manovrabilità**
 - Il rapporto di sterzo può essere modificato via software.
 - È possibile sterzare completamente con meno rotazione del volante, migliorando le manovre a bassa velocità.

AUTOSAR

AUTOSAR è una partnership internazionale fondata nel **2003** da importanti costruttori automobilistici (BMW, Toyota, Ford, Volkswagen, General Motors, ecc.) e numerosi fornitori del

settore. Nasce per affrontare la crescente complessità dei sistemi elettronici presenti nei veicoli moderni, caratterizzati da decine di centraline che devono comunicare tra loro.

Con l'aumento delle funzionalità elettroniche (ADAS, guida autonoma, infotainment, steer-by-wire, e-mobility), il software automobilistico è diventato sempre più complesso.

AUTOSAR è stato introdotto per:

- rendere il software indipendente dall'hardware della **centralina**;
- favorire il riutilizzo del software su piattaforme differenti;
- ridurre tempi e costi di sviluppo;
- aumentare affidabilità e sicurezza dei sistemi elettronici.

1. **Astrazione (Abstraction):** AUTOSAR separa il software applicativo dall'hardware. In questo modo lo stesso software può funzionare su microcontrollori diversi con modifiche minime.
2. **Configurazione:** Molte funzionalità vengono configurate anziché programmate da zero. Questo rende lo sviluppo più veloce e standardizzato.
3. **Qualità del software** Grazie alla standardizzazione del **Basic Software (BSW)**, il software risulta:
 - più affidabile;
 - più facilmente testabile;
 - più semplice da mantenere.

Architettura AUTOSAR

AUTOSAR utilizza un'architettura software a livelli (Layered Architecture), composta da quattro livelli principali.

1. Application Software (ASW) . È il livello più alto. Contiene le applicazioni che implementano le funzioni del veicolo, ad esempio:

- controllo dello sterzo;
- gestione del motore;
- sistemi ADAS;
- climatizzazione.

Le applicazioni non conoscono direttamente l'hardware.

2. Runtime Environment (RTE)

È il "ponte" tra applicazioni e software di base. Le sue funzioni principali sono:

- gestire lo scambio dei dati;
- permettere la comunicazione tra componenti software;
- isolare completamente le applicazioni dall'hardware.

3. Basic Software (BSW) È il livello che offre tutti i servizi fondamentali necessari alle applicazioni. Comprende:

- gestione memoria;
- diagnostica;
- comunicazione CAN/LIN;
- servizi crittografici;
- gestione del sistema operativo.

È il livello più **importante** dell'infrastruttura AUTOSAR.

4. Microcontroller: È il livello hardware sul quale viene eseguito tutto il software. Comprende CPU e periferiche elettroniche della ECU.

Livelli del Basic Software: Il Basic Software è suddiviso in diversi sottolivelli.

- **Services Layer:** Fornisce servizi generali come:
 - diagnostica (UDS);
 - gestione errori;
 - memoria non volatile (NVRAM);
 - comunicazioni CAN e LIN;
 - servizi crittografici.

È quasi completamente indipendente dall'hardware.

- **ECU Abstraction Layer:** Nasconde le caratteristiche specifiche dell'hardware della **ECU**. Espone API standard per accedere ai dispositivi elettronici senza che il software applicativo debba conoscerne il funzionamento.
- **Microcontroller Abstraction Layer (MCAL):** È il livello più vicino all'hardware. Contiene i driver del microcontrollore e rende il software indipendente dal tipo di processore utilizzato.

Complex Device Drivers (CDD)

Sono driver speciali utilizzati quando le funzionalità richieste non sono previste dallo standard AUTOSAR oppure richiedono tempi di risposta molto rapidi.

Essendo strettamente dipendenti dall'hardware, risultano meno portabili rispetto agli altri moduli AUTOSAR.

Communication Stack (CAN)

Il PowerPoint approfondisce anche lo stack di comunicazione AUTOSAR per il bus CAN.

I principali moduli sono:

- **COM**: converte i segnali dell'applicazione in PDU (Protocol Data Unit) e viceversa.
- **PDU Router (PduR)**: instrada i messaggi tra i diversi moduli di comunicazione.
- **CanTP**: frammenta e ricompone messaggi CAN più grandi di 8 byte.
- **CanIf**: funge da interfaccia tra i protocolli di alto livello e il driver CAN, gestendo trasmissione, ricezione e modalità operative del controller CAN.
- **CAN Driver**: comunica direttamente con l'hardware del bus CAN, trasmettendo e ricevendo i frame.

SOME/IP

Perché serve un "Middleware" nelle auto moderne?

Nelle auto del passato, i vari componenti elettronici (centraline o **ECU**) comunicavano tra loro in modo diretto (punto-punto) e "rigido". Oggi questo non è più sostenibile per tre motivi principali:

- **Il muro dell'hardware (I cablaggi):** Un'auto moderna ha più di 100 centraline e oltre 5 km di cavi. Questo "groviglio" è la terza componente più pesante e costosa del veicolo. Aggiungere anche solo un nuovo sensore richiederebbe di riprogettare l'intero sistema hardware e logico.
- **La "Trappola del Silicio" (Dipendenza dall'hardware):** Storicamente, il software delle auto veniva scritto su misura per lo specifico microchip utilizzato. Se si decideva di cambiare fornitore di chip, bisognava riscrivere il software da zero (Zero Portabilità).
- **La barriera dell'integrazione (Dipendenza del software):** Con la comunicazione vecchio stile, chi produceva un dato doveva sapere esattamente quale altra centralina ne avesse bisogno. Se si aggiungeva una nuova funzione, bisognerebbe riconfigurare tutte le centraline collegate.

La Soluzione: L'introduzione di un'un'architettura a livelli con un **interfaccia software standard (come AUTOSAR)**. Il **Middleware** agisce come un "traduttore" universale: scollega il software dall'hardware e trasforma l'auto in una piattaforma flessibile dove le funzioni diventano "Servizi" fluidi (stile App Store).

2. Cos'è e come funziona SOME/IP?

SOME/IP sta per *Scalable Service-Oriented MiddlewarE over IP*. È un insieme di regole (protocollo) che permette alle centraline dell'auto di comunicare tra loro attraverso la rete Ethernet del veicolo, usando un approccio "orientato ai servizi".

Invece di inviare dati a pioggia a una centralina specifica, si usa il modello **Server e Client**:

- Un **Server** offre un servizio (ad esempio: "Fornisco i dati della telecamera" o "Fornisco la temperatura interna").
- Un **Client** cerca il servizio che gli interessa e lo "consuma".

Caratteristiche principali di SOME/IP:

- **Messaggi veloci ed efficienti (Serializzazione):** Trasforma i dati interni della memoria della centralina in pacchetti di rete in modo estremamente rapido, ideale per le comunicazioni ad alta velocità delle auto.
- **Gestione dei messaggi grandi (SOME/IP-TP):** Normalmente i messaggi via rete (UDP) hanno un limite di dimensione. SOME/IP-TP permette di "spezzettare" in automatico i messaggi molto grandi (fino a 4 Gigabyte, come i pesanti dati dei sensori Radar o Lidar) e ricostruirli all'arrivo.
- **Eventi vs Campi (Fields):**
 - Un *Evento* è come una notifica istantanea (es. "Qualcosa è successo adesso", poi il dato sparisce).
 - Un *Campo (Field)* rappresenta uno stato permanente (es. la temperatura attuale dell'abitacolo), di cui posso chiedere il valore (*Getter*), modificarlo (*Setter*) o ricevere una notifica se cambia.

3. SOME/IP-SD (Service Discovery): Trovare in automatico

Per far funzionare questo sistema serve un vigile urbano digitale: il **SOME/IP-SD** (*Service Discovery*). Permette alle centraline di scoprirsi a vicenda dinamicamente senza configurazioni manuali.

Funziona tramite messaggi periodici divisi in fasi e meccanismi di **Offerta/Ricerca**:

1. **Offer Service**: Il Server grida sulla rete: *"Ehi, io offro questo servizio su questo indirizzo!"*.
2. **Find Service**: Se un Client si accende e ha bisogno di qualcosa, chiede: *"Chi offre il servizio X?"* e il server risponderà direttamente a lui.
3. **Iscrizione (Subscription Handling)**: Il Client dice al Server *"Mi iscrivo al tuo gruppo di eventi"*; il Server accetta l'iscrizione (*Subscribe Ack*) e da quel momento in poi invierà i dati aggiornati al Client.

4. Implementazioni e Demo pratica

Le slide mostrano che SOME/IP unisce "due mondi" diversi nell'auto:

1. **Il mondo Embedded profondo (AUTOSAR)**: Centraline piccole con risorse limitate e tempi di risposta istantanei (es. controllo motore o freni).
2. **Il mondo HPC (Linux)**: Computer di bordo potenti, dedicati all'infotainment o alla guida autonoma avanzata.

Grazie a SOME/IP, questi due mondi parlano la stessa identica lingua senza bisogno di adattatori.

La presentazione si conclude con una **Demo** in cui viene analizzato il traffico dati reale con un software chiamato *Wireshark*. Nella demo si vede concretamente lo scambio di messaggi descritto sopra (l'offerta del servizio, l'iscrizione del client e il passaggio finale del dato, ad esempio il valore di una temperatura pari a 22.5 °C) tra un server e un client simulati su schede elettroniche industriali (Aurix TC387).

LoRa

Cosa sono le reti LPWAN e a cosa servono?

Nel mondo dell'Internet delle Cose Industriale (IIoT) , i diversi protocolli wireless si dividono i compiti in base a tre fattori: raggio d'azione, velocità dei dati e consumo di energia. Il Wi-Fi gestisce grandi quantità di dati ma consuma molto , mentre il Bluetooth copre distanze molto brevi.

Le reti **LPWAN** (*Low-Power Wide-Area Networks*) si posizionano in una nicchia specifica:

- **Lungo raggio (Long Range):** coprono distanze enormi, raggiungendo anche sensori posizionati sotto terra o all'interno di edifici a più piani.
- **Bassi consumi (Low Power):** I dispositivi consumano così poco che le loro batterie possono durare fino a 10 anni.
- **Pochi dati (Low Data Rate):** Sono progettate per trasmettere pochissimi dati alla volta (es. semplici letture di temperatura o contatori).

LoRaWAN è uno dei protocolli LPWAN più famosi e utilizzati al mondo.

2. La differenza fondamentale tra LoRa e LoRaWAN

Spesso i due termini vengono confusi, ma indicano cose diverse (una legata all'hardware e una al software):

1. **LoRa (Livello Fisico / Hardware):** È la tecnologia di modulazione radio (brevettata dall'azienda Semtech). Usa una tecnica chiamata *Chirp Spread Spectrum* (CSS) per trasmettere segnali radio a lungo raggio che resistono benissimo al rumore e alle interferenze.
2. **LoRaWAN (Livello MAC / Software):** È il protocollo di comunicazione, cioè l'insieme di regole software che definisce l'architettura della rete. Serve a collegare in modo sicuro e criptato i dispositivi industriali a Internet.

3. Come è fatta un'architettura LoRaWAN?

La rete utilizza una topologia definita "**a stella di stelle**" (*star-of-stars*):

- **End-Devices (I nodi):** Sono i sensori sul campo (es. rilevatori di fumo, contatori smart) che inviano pacchetti radio.
- **Gateway (I ponti):** Ricevono i segnali radio dai sensori e li convertono in pacchetti di rete standard (IP) per spedirli via Internet. I sensori non sono accoppiati a un solo gateway; un messaggio può essere captato da più gateway contemporaneamente.
- **Network Server:** Il cervello centrale della rete che riceve i dati dai gateway, elimina i messaggi duplicati e gestisce la rete.

I limiti regionali e il Duty Cycle

In Europa si usa la banda di frequenza libera degli **868 MHz**. Per evitare che un dispositivo occupi la linea continuamente, le leggi europee (regolamenti ETSI) impongono un **Duty Cycle**. Questo significa che un sensore può trasmettere solo per una piccolissima percentuale di tempo ogni ora (spesso l'1% o lo 0,1%), rimanendo in silenzio per il resto del tempo.

4. Le 3 Classi di Dispositivi (A, B, C)

Per ottimizzare la durata della batteria in base all'uso, LoRaWAN divide i dispositivi in tre classi:

- **Classe A (Tutti i dispositivi):** È la più efficiente dal punto di vista energetico. Il sensore dorme quasi sempre; si sveglia, invia un dato (Uplink) e subito dopo apre due brevissime finestre temporali (RX1 e RX2) per ascoltare se il server ha un messaggio per lui

(Downlink). Se il server deve inviargli un comando, deve aspettare che il sensore si svegli e trasmetta qualcosa.

- **Classe B (Beacon):** Oltre alle finestre della Classe A, il dispositivo si sincronizza con un segnale a tempo (**Beacon**) emesso dal gateway. Questo gli permette di aprirsi per ricevere comandi a intervalli di tempo regolari e prevedibili (*ping slots*).
- **Classe C (Ascolto Continuo):** Il dispositivo è alimentato dalla rete elettrica (non a batteria) e rimane continuamente in ascolto. Non ha ritardi nella ricezione dei messaggi dal server, ma consuma molta energia.

ADAPTIVE DATA RATE

- LoRa richiede la configurazione di una serie di parametri di trasmissione che hanno un impatto significativo sulle prestazioni della rete. Per assegnare dinamicamente agli *end device* potenza di trasmissione (TP) e lo *spreading factor* (SF), la specifica LoRaWAN suggerisce di sfruttare un algoritmo di Adaptive Data Rate (ADR).
- Tuttavia, non viene effettivamente definito alcun algoritmo ADR. Al contrario, un documento di raccomandazione fornito da Semtech [2] delinea un algoritmo per selezionare lo SF e la TP con l'obiettivo di ridurre al minimo il consumo energetico, mantenendo l'affidabilità della comunicazione e la velocità di trasmissione dei dati (*data rate*) il più elevate possibile. Ai dispositivi finali che si trovano più vicini ai gateway vengono assegnati valori più bassi di SF e TP, mentre quelli più lontani utilizzano valori più alti di SF e TP per aumentare la robustezza.

Ulteriori considerazioni: L'ADR non è adatto per scenari dinamici con dispositivi finali (*end-device*) mobili che si spostano all'interno dell'area di rilevamento in condizioni di rete altamente variabili.

- In questo contesto, le reti neurali profonde (DNN - *Deep Neural Networks*) possono essere sfruttate per modellare con grande precisione (*fine granularity*) le non-linearità del canale wireless e calcolare i parametri di trasmissione in modo più accurato.

5. Applicazioni e Ricerca Avanzata dell'Università

Le slide mostrano scenari reali di utilizzo in ambienti complessi o ostili: dall'estrazione di petrolio e gas al monitoraggio di parchi solari, fino alla manutenzione predittiva nelle fabbriche intelligenti (Industry 4.0).

La parte finale delle slide evidenzia i **limiti nativi di LoRaWAN**: non è nato per comunicazioni in tempo reale a causa dei limiti di trasmissione ed è soggetto a rallentamenti se i dispositivi si muovono. I ricercatori dell'Ateneo hanno quindi presentato tre soluzioni innovative per superare questi ostacoli:

1. **ADR basato su Deep Learning:** Un algoritmo che usa reti neurali (DNN) per impostare automaticamente la potenza di trasmissione e la velocità ideale per i sensori quando si muovono in contesti difficili.

2. **Strategia DL+ per i Gateway:** Un modulo software che calcola un punteggio matematico per scegliere dinamicamente il miglior gateway a cui affidare un comando critico da mandare al sensore.
3. **Protocolli RT-LoRa e MRT-LoRa:** Nuove strategie di accesso alla rete (usando regole a divisione di tempo o modelli gerarchici a più salti - *multi-hop*) per permettere a LoRa di supportare finalmente flussi di dati in tempo reale in ambito industriale.

Domande:

Timeline Scheduling (cycling scheduling)

Definizione di Taskset Schedulabile e Test di schedulabilità

Che significa che due task sono tra loro indipendenti? Cosa significa Least Upper Bound?

Deadline Monotonic

Confronto tra Rate Monotonic ed EDF

Utilizzazione di Liu e Layland

Domini funzionali, architetture zonali (disegno)

MAC:

- CSMA-BA, Polling Server

LIN (esempio applicazione: chiusura portiera auto)

CAN

- Bit Stuffing del CAN
- Problematiche del CAN per gestire il RT
- Error Frame
- CAN-FD
- TT-CAN
- Confronto CAN vs Flexray
- Confronto CAN XL e 10Baes T1 S

Flexray

MOST : Confronto MOST vs AVB

Ethernet

- 10BASE T1S
- Switched Ethernet (Perchè Ethernet Switched non può gestire i vincoli Real Time?)
- Cosa significa Overflow nelle code?
- AVB : Cosa significa che un traffico subisce "shaping"?
- Cosa significa che qualcosa non segue gli standard e che problematiche implica? Come viene gestito il traffico in TSN?
- 802.1TSN
- 802.1Q : 802.1Qbv
- In Ethernet automotive che velocità abbiamo in questo momento?

Industrial Ethernet

- Approcci RTE
- Fieldbus
- Performance Indicator, cosa sono e dove li abbiamo visti?
- Confronto Profinet/ Powerlink
- POWERLINK
- ETHERCAT (operazioni e indirizzamento)

Wireless industrial: 802.11 ambito industriale, Flexware

Cosa è uno shaper

Mappe concettuali

1. ARCHITETTURA LIN

Cluster LIN: Master/Slave

- Un solo Master
- Fino a 16 Slave



- Sul bus LIN vengono scambiati dei frame composti da una parte di **Intestazione (Header)** e una di **Risposta (Response)**.
- Ogni frame viene avviato dal Master.
- Il Master controlla tutta la comunicazione sul bus LIN.
- Normalmente il Master interroga (poll) ciascun Slave all'interno di un ciclo continuo trasmettendo un Header.
- I dati viaggiano sul bus in modo che tutti gli Slave vengano a conoscenza del nuovo dato aggiornato.
- Qualsiasi nodo interessato può leggere il frame (modello publisher/subscriber).

TASK SLAVE INTERNO DEL MASTER

- Un task slave separato, presente nel Master, riceve autonomamente tutti i dati pubblicati sul bus e risponde come se fosse uno Slave indipendente.
- Per trasmettere i byte di dati, il Master aggiorna la risposta del proprio task slave interno con i valori desiderati, poi pubblica l'Header appropriato e il task slave interno trasmette il payload.

2. SCHEDULE TABLE (DEL MASTER)

- Il Master segue una tabella di pianificazione (schedule table), definita nel LDF.
- Ogni voce della tabella è chiamata **slot di frame (frame slot)**.
- La tabella elenca gli Header da inviare e i ritardi tra di essi (interframe space).

ESEMPIO DI SCHEDULE TABLE

Ordine	ID (PID)	Delay (Interframe Space)
1	0x10	10 ms
2	0x21	20 ms
3	0x05	100 ms
...	...	...

L'interframe delay (slot time) può essere diverso per ogni entry della tabella e stabilisce quando il Master deve inviare il prossimo Header.

3. COMUNICAZIONE: CHI FA COSA?



1. HEADER (broadcast dal Master)
Il Master invia in modalità broadcast un Header contenente l'ID dei dati da spedire (data-based addressing). Tutti gli Slave ricevono l'Header.



2. RESPONSE (da un solo nodo publisher)
Il nodo che possiede quell'ID (publisher) invia la Response con i dati (payload) e il checksum. Il nodo sorgente dei dati può essere sia il Master (tramite il suo task slave interno) sia uno Slave.



3. RICEZIONE (publisher/subscriber)
Qualsiasi nodo interessato può leggere il frame trasmesso sul bus (modello publisher/subscriber).



FRAME EVENT-TRIGGERED

- Il Master invia un Header di Event-Triggered Frame.
- Più Slave che hanno dati aggiornati possono rispondere contemporaneamente → collisione.
- Il Master rileva la collisione e poi interroga gli Slave uno alla volta tramite Unconditional Frames per ottenere i dati.

PROTOCOLLO LIN

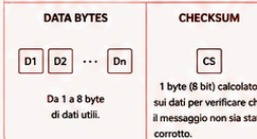
Local Interconnect Network

4. FRAME LIN: STRUTTURA

HEADER (sempre inviato dal Master)



RESPONSE (inviata dallo Slave che possiede i dati)



TIPICI DI CHECKSUM

1 CLASSIC CHECKSUM

(LIN 1.3 e precedenti)
Calcolato basandosi solo sui byte dei dati.

2 ENHANCED CHECKSUM

(LIN 2.0 e successivi)
Calcolato includendo sia i byte dei dati sia il campo PID (protezione maggiore contro gli errori).

6. FRAME: TIPI DI ID (PID)

- ID da 0 a 59 → frame incondizionati (Unconditional Frames): usati per comunicazioni cicliche o diagnostiche.
- ID da 60 a 61 → frame di richiesta diagnostica da parte del Master (ID 60) e risposta diagnostica (ID 61).
- ID = 62 → user-defined frames: possono trasportare qualsiasi tipo di informazione.
- ID = 63 → reserved frame: non deve essere utilizzato in un cluster LIN 2.0.

7. LIN DESCRIPTION FILE (LDF)

File di configurazione che descrive completamente la rete LIN. Contiene:

- Baud rate della rete (es. 19.2 Kbps)
- Definizione dei nodi (Master e Slave)
- Elenco dei frame (ID, lunghezza, publisher)
- Definizione dei segnali nei frame
- Schedule Tables del Master
- Configurazione diagnostica
- Node Attributes (NAD, Supplier ID, Function ID, ...)

Tutti i nodi devono usare lo stesso baud rate definito nell'LDF.

8. GESTIONE DELL'ENERGIA

- Per ottimizzare il consumo energetico, LIN offre servizi per mettere i nodi in modalità di sospensione (sleep mode) e per risvegliarli.

GO-TO-SLEEP COMMAND

Il Master invia un frame diagnostico (ID = 60) con il primo byte di dati = 0 → tutti gli Slave entrano in modalità sleep.

SOSPENSIONE AUTOMATICA

Gli Slave entrano automaticamente in modalità sleep se il bus rimane inattivo per più di 4 s.

WAKEUP (RISVEGLIO)

- Può essere avviato da un nodo qualsiasi (Master o Slave).
- Nella LIN 2.0 il risveglio è richiesto forzando il bus in **stato dominante** per 250 µs - 5 ms.
- Ogni Slave deve essere pronto a elaborare l'Header entro 100 ms.
- Anche il Master deve rilevare il wakeup e iniziare a inviare gli Header quando gli Slave sono pronti (entro 100 ms - 150 ms) dopo la richiesta.

9. LIMITI DEL LIN

- Bassa velocità: massimo 20 Kbps.
- Metodo di rilevamento degli errori di base.
- Non è **fault-tolerant**: non viene utilizzato nelle funzioni **safety critical**.

IN SINTESI

Il bus LIN è semplice, economico e adatto a comunicazioni a bassa velocità in ambito automobilistico (comfort, body electronics, ecc.). Completa il CAN: i cluster LIN sono normalmente interconnessi tra loro tramite una dorsale CAN.

LEGENDA:



Broadcast



Publisher



Subscriber



Tempo/Delay



Energia



Diagnostica



Attenzione

1. COS'È IL CAN

- Protocollo di comunicazione seriale multi-master per applicazioni automotive e industriali.
- Standard ISO 11898 (fisico) e ISO 11898-1 (data link layer).
- Comunicazione broadcast: un messaggio trasmesso è ricevuto da tutti i nodi.
- Non esiste un master: qualsiasi nodo può trasmettere quando ha dati disponibili.
- Elevata affidabilità grazie a: arbitraggio non distruttivo, rilevamento e gestione degli errori.

VANTAGGI

- ✓ Robusto al rumore elettromagnetico
- ✓ Affidabile e con controllo errori
- ✓ Priorità dei messaggi basata sull'ID
- ✓ Basso costo e alta flessibilità

2. LIVELLO FISICO E TOPOLOGIA

- Bus differenziale (ISO 11898-2)



- Terminazione: 120 Ω alle due estremità del bus.
- Topologia: bus lineare (a tronco) con derivazioni brevi (stub).
- Velocità massime tipiche:

Velocità	Lunghezza bus
1 Mbps	fino a 40 m
500 kbps	fino a 100 m
250 kbps	fino a 250 m
125 kbps	fino a 500 m
50 kbps	fino a 1 km

- Livelli logici (dominante/recessivo):

Stato	CAN_H	CAN_L	Differenziale (V)
Dominante (0)	≈ 3.5 V	≈ 1.5 V	≥ 1.5 V
Recessivo (1)	≈ 2.5 V	≈ 2.5 V	≈ 0 V

3. STRUTTURA DEL FRAME CAN (Data Frame)

SOF	Arbitration Field	Control Field	Data Field	CRC Field	ACK Field	EOF
1 bit	11 o 29 bit	6 bit	0-8 byte	16 bit	2 bit	7 bit

- SOF (Start Of Frame): 1 bit dominante, inizio del frame.
- Arbitration Field: ID del messaggio (11 bit in Standard Frame, 29 bit in Extended Frame) + bit RTR, IDE, r0.
- Control Field: DLC (Data Length Code, 4 bit) + bit di riserva.
- Data Field: da 0 a 8 byte di dati.
- CRC Field: 15 bit CRC + 1 bit CRC delimiter (recessivo).
- ACK Field: 1 bit ACK slot (dominante se almeno un ricevitore ha ricevuto correttamente) + 1 bit ACK delimiter (recessivo).
- EOF (End Of Frame): 7 bit recessivi.

TIPI DI FRAME

- Data Frame: trasporta i dati.
- Remote Frame (RTR): richiede dati (poco usato).
- Error Frame: generato in caso di errore.
- Overload Frame: per ritardare l'inizio di un nuovo frame.

4. ARBITRAGGIO NON DISTRUTTIVO

- Se più nodi trasmettono simultaneamente, l'arbitraggio avviene bit per bit sull'ID (dominante = 0, recessivo = 1).
- Il messaggio con ID numericamente più basso (più bit dominanti) ha la priorità più alta.
- Un nodo che trasmette recessivo e legge dominante sul bus perde l'arbitraggio e smette di trasmettere, senza generare errori.

ESEMPIO (ID a 11 bit)

Nodo A (ID)	Nodo B (ID)	Bus	Vincitore
0	0	0	A e B
0	1	0	B perde
1	-	-	A vince

L'arbitraggio avviene durante l'Arbitration Field.

5. RILEVAMENTO DEGLI ERRORI

I nodi monitorano continuamente il bus e rilevano:

- ✓ Bit Error: il nodo trasmette un bit e legge un valore diverso (eccetto in arbitraggio e ACK slot).
- ✓ Stuff Error: più di 5 bit consecutivi uguali.
- ✓ CRC Error: il CRC calcolato non coincide.
- ✓ Form Error: violazione del formato del frame (es. bit non recessivi dove previsti).
- ✓ ACK Error: nessun nodo invia un ACK dominante nell'ACK slot.

- Se viene rilevato un errore:
- il nodo trasmette un Error Flag
 - il frame corrente viene annullato
 - il messaggio verrà ritrasmesso automaticamente

CAN

Controller Area Network



6. GESTIONE DEGLI ERRORI

Contatori di errore per ogni nodo

TEC
(Transmit Error Counter)
Errori commessi in trasmissione

REC
(Receive Error Counter)
Errori rilevati in ricezione

Regole di incremento (valori tipici)

Errori in trasmissione (il nodo è responsabile)
TEC +8

Errori in ricezione (il nodo non è responsabile)
REC +1

Stati del nodo

Stato	Condizione	Comportamento
Error Active	TEC < 128 e REC < 128	Trasmette Active Error Flag (6 bit dominanti)
Error Passive	128 ≤ TEC < 256 o 128 ≤ REC < 256	Trasmette Passive Error Flag (6 bit recessivi)
Bus Off	TEC ≥ 256	Non trasmette più. Disconnesso dal bus. Può rientrare solo dopo 128 occorrenze di 11 bit recessivi (reset) o reset del dispositivo.

Error Flag

- Active Error Flag: 6 bit dominanti. Forza tutti i nodi a rilevare l'errore.
- Passive Error Flag: 6 bit recessivi. Potrebbe non essere rilevato dagli altri nodi.

La gestione errori permette l'auto-recupero della rete e l'isolamento dei nodi guasti.

10. VANTAGGI E LIMITI

Vantaggi

- ✓ Comunicazione multi-master senza master centrale
- ✓ Arbitraggio con priorità basata su ID
- ✓ Rilevamento e gestione avanzata degli errori
- ✓ Robustezza al rumore
- ✓ Scalabile e flessibile
- ✓ Larghezza di banda elevata rispetto a LIN

Limiti

- ✗ Lunghezza del bus limitata (dipende dalla velocità)
- ✗ Complessità maggiore rispetto a LIN
- ✗ Carico del bus legato al numero di messaggi e priorità

8. BIT TIMING

- La comunicazione è sincrona (basata su clock).
- Ogni bit è suddiviso in segmenti:



- I segmenti sono configurabili per adattarsi alla rete (lunghezza bus, velocità, ritardi dei transceiver).
- Una corretta configurazione garantisce campionamento stabile e comunicazione affidabile.

9. APPLICAZIONI TIPICHE

- Automotive (motore, ABS, airbag, comfort, infotainment)
- Industria (automazione, robotica, controllo processi)
- Veicoli commerciali e agricoli
- Sistemi medicali, avionica, marine

- ★ Adatto ad ambienti rumorosi e critici dove affidabilità e tempo reale sono fondamentali.

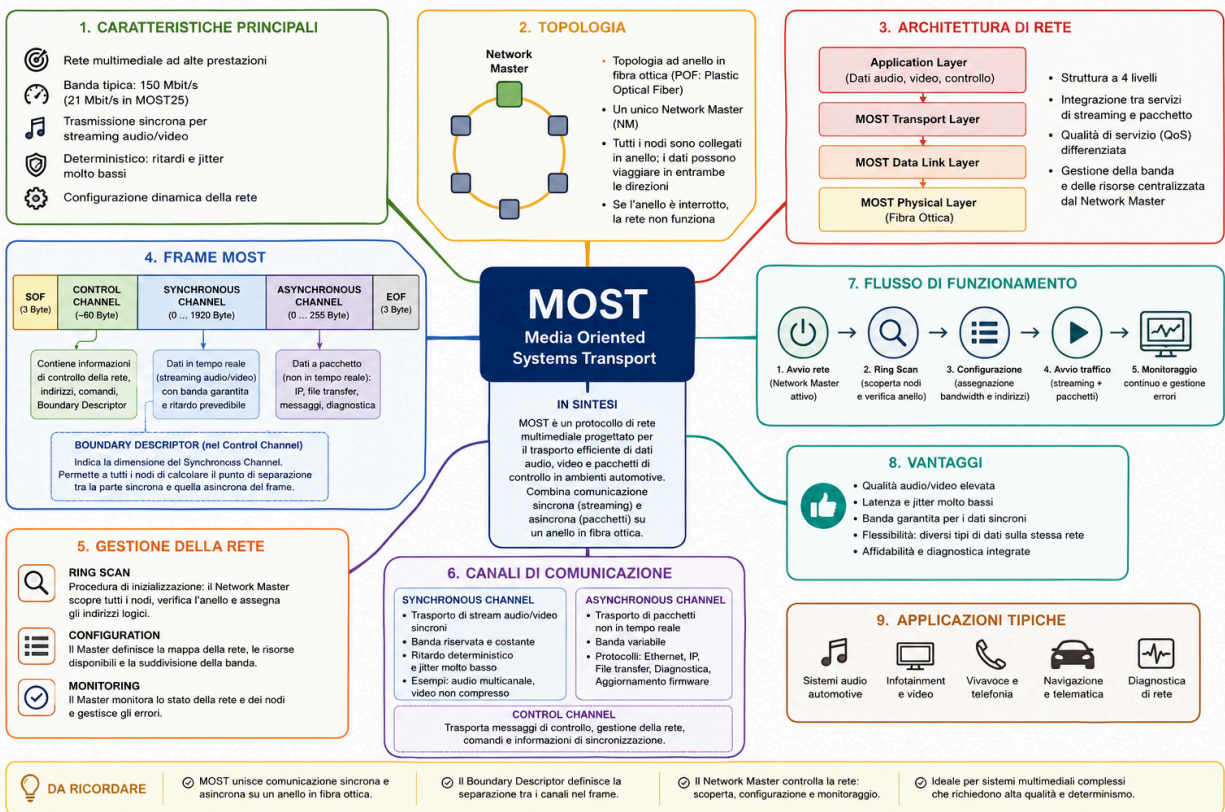
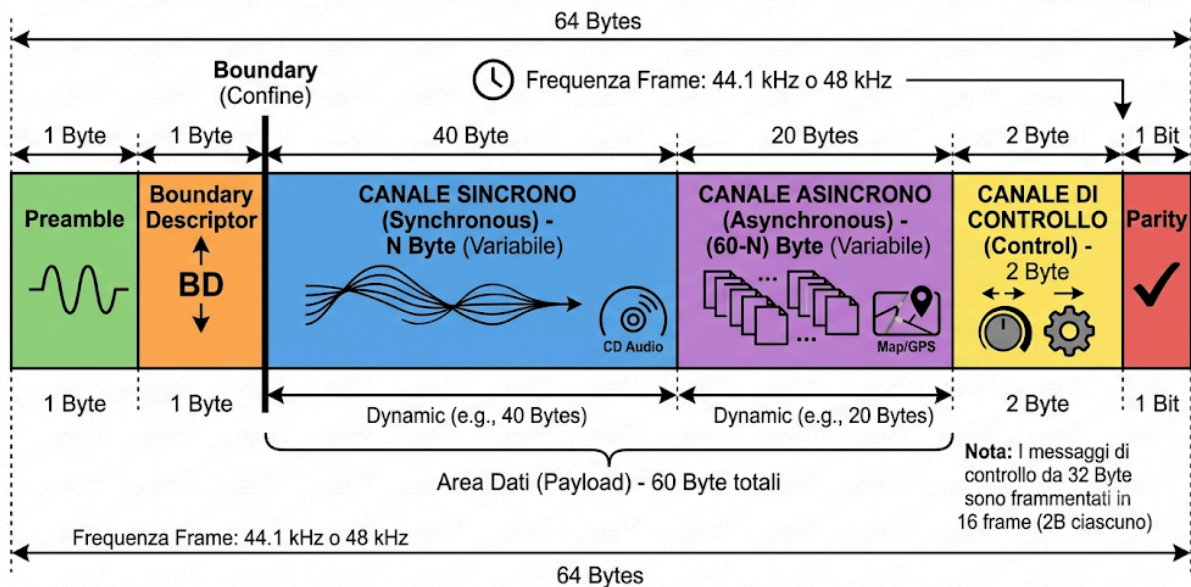
LEGENDA

- Dominante (0): bit logico 0, forza il bus in stato dominante
- Recessivo (1): bit logico 1, rilascia il bus (stato recessivo)
- Un messaggio con più bit dominanti ha priorità più alta.
- Simboli**
- Bus differenziale
- Resistenza di terminazione
- Nodo CAN
- Connessione

IN SINTESI

CAN è un protocollo robusto, affidabile e distribuito. Ogni nodo può trasmettere quando ha dati disponibili. L'arbitraggio garantisce che il messaggio più importante venga trasmesso per primo, mentre il sistema di rilevamento e gestione degli errori assicura l'integrità dei dati e l'auto-recupero della rete.

STRUTTURA DEL FRAME MOSYT (MOST25) - 64 BYTE (512 BIT)



? Domande di ripasso

Scheduling e sistemi real-time

1. Che cosa si intende per sistema hard real-time e soft real-time?
 - In un sistema hard real time se la il pacchetto viene consegnato dopo la deadline non solo non ha valore ma può causare danni al sistema. Un Sistema soft real tiem (come lo streaming dati) il messaggio continua ad avere valore dopo la deadline ma decresce, non causa danni ma causa il peggioramento della qualità del servizio.
2. Qual è la differenza tra sistemi time-driven ed event-driven?
 - Time driven sono azionati dal tempo, periodici, mentre quelli event driven sono azionati dagli eventi, non è possibile prevedere quando arriveranno.
3. Definisci arrival time, computation time, response time e finishing time.
 - Arrival time, tempo di arrivo in coda pronti, computation time il tempo che risiede nella cpu, response time: finishing time - arrival time. finishing time l'istante in cui finisce.
4. Cosa significa che due task sono indipendenti?
 - Che non hanno vincoli di accesso al mezzo, o accesso a risorse condivise, non hanno vincoli di precedenza ecc.
5. Che cos'è la lateness? E la maximum lateness?
 - Lateness: $f_i - d_i$. La lateness massima è il massimo valore di lateness tra due messaggi qualsiasi?
6. In cosa consiste il Timeline Scheduling?
 - Consiste nello schedulare i task periodici attraverso l'utilizzo di slot di tempo fissi , in modo tale da rispettare ogni deadline ma anche gli istanti di attivazione. Il tempo viene diviso in slot temporali fissi, la durata dei singoli slot viene calcolata con il massimo comune divisore tra i periodi dei task. Gli slot fanno parte di un macro periodo, chiamato Hyper Period, che si ripete ciclicamente e che viene determinato dal minimo comune multiplo tra i periodi dei task. Questo tipo di schedulazione garantisce determinismo assoluto ma scarsa flessibilità.
7. Perché FIFO non è adatto ai sistemi real-time?
 - Schedulare in ordine di arrivo significa non tener in conto in alcun modo le deadline, inoltre senza preemption non possiamo in alcun modo sapere quanto tempo il task potrebbe aspettare in coda, magari bloccato da un task best effort di grandi dimensioni.
8. Perché Round Robin non garantisce il rispetto delle deadline?
 - Non ci interessa la fairness.
9. Descrivi l'algoritmo Earliest Deadline First (EDF).
 - L'argoritmo Earliest Deadline first schedula in base alle deadline assolute crescenti.
 - Notazione di graham $1 | \text{preempt} | \text{Lateness max}$
 - I task non sono sincroni, possono arrivare in qualsiasi momento.
 - È ottimo se è preemptive:
10. EDF utilizza deadline assolute o relative? Assoluta.
11. Perché EDF è considerato un algoritmo dinamico? Perché la priorità viene assegnata in base alla deadline assoluta, che dipende dall'istante di attivazione , cioè l'arrivo in coda ($d_i = r_i + D_i$). L'arrivo in coda può variare.
12. In cosa consiste il Rate Monotonic (RM)?

- Il rate monotonic consiste nell'assegnare la priorità in modo inversamente proporzionale al periodo.
- 13. Perché RM è uno scheduler statico?
 - La priorità viene assegnata una volta, perché i task hanno periodo fisso.
- 14. RM è preemptive?
 - Sì. è ottimale tra
- 15. Confronta EDF e RM.
 - RM è ottimale tra gli algoritmi a priorità fissa, RM ha una condizione sufficiente di schedulabilità cioè se Least Upper Bound è minore di $0.69 + \ln(n)$. Mentre EDF è ottimale in assoluto basta che il task set abbia least upper bound minore o uguale a 1.
- 16. Cos'è il Least Upper Bound di Liu e Layland?
 - Il Least Upper Bound di un algoritmo è il minimo Upper Bound, il minimo fattore di utilizzo calcolato tra tutti i task set che saturano il processore.
- 17. Quando un task set è schedulabile?
 - In RM condizione sufficiente di schedulabilità è l'upper bound di liu e layland.
 - In EDF basta che l'upper bound sia minore di 1.

MAC e Data Link

- 18. Quali sono le principali tecniche di accesso al mezzo?
- 19. Confronta CSMA, CSMA/CD e CSMA/BA.
 - CSMA, un nodo trasmette quando ha qualcosa da trasmettere, prima sente se il canale è libero, se è libero comincia a trasmettere, se nel frattempo ha cominciato a trasmettere un altro nodo avviene una collisione, distruttiva. Flessibile, non precidibile.
 - CSMA/CD, collision detection, se si verifica una collisione il nodo torna in ascolto e aspetta per un tempo random, all'interno di una finestra temporale, il meccanismo è exponential back off. Collisioni comunque distruttive.
 - CSMA/BA, utilizzato nel CAN, collisioni non distruttive, il nodo che ha priorità maggiore non si accorge nemmeno della collisione, perché vince l'arbitration bit a bit.
 - CSMA/CA: i nodi che vogliono trasmettere aspettano un tempo fisso più un tempo casuale. Riduce le collisioni ma non è privo di collisioni perché è asincrono: i nodi aspettano tempi diversi ma non partono dallo stesso tempo o potrebbero pescare lo stesso tempo casuale.
 - CSMA/CA sincrono: praticamente il meccanismo delle transmission opportunity.
- 20. Come funziona un protocollo Master/Slave?
- 21. Come funziona il Token Passing?
- 22. Quali vantaggi offre il TDMA?

LIN

23. Perché è nato il protocollo LIN?

- LIN nasce per gestire le comunicazioni nei dispositivi presenti nelle periferie della rete. Bassa potenza, sleep mode, ridondanza del master. LIN utilizza 20KBPS , un payload da 1 a 8 byte, ridotto. Messaggi di comando.

24. Descrivi l'architettura Master/Slave del LIN.

- La rete LIN è composta da unità chiamate LIN Cluster: ogni cluster è formato da un master LIN e i nodi slave collegati al bus multidrop. Possono essere collegati un massimo di 16 slave LIN con un bus di massimo 40 metri.

25. Com'è composta una frame LIN?

- È composta da un Header, che viene mandato dal nodo master, e da una response, mandata dai nodi slave.
- L'header è composto da: Break, Sync , PID. Break segnala la fine del silenzio sulla rete e l'inizio di una nuova trasmissione, sono 13 bit dominanti succeduti da un 1 recessivo.
- Sync bit 1 e 0 che si alternano per permettere la sincronizzazione.
- I protected ID consiste nell'id di 6 bit più 2 bit di parità. Gli ID vanno da 0 a 63 e rappresentano il tipo di dato contenuto nel payload. 0..59 frame dati, 60 - 61 frame di diagnostica. 62 frame utente e 63 utilizzi futuri

26. Qual è la differenza tra Header e Response?

- L'header la manda il master, anzi il master task dato che il master ha un task master che interroga gli slave e uno slave task che a sua volta produce dati.

27. Che ruolo ha il Protected Identifier (PID)?

- Identificare il tipo di messaggio contenuto nel payload.

28. Perché il LIN utilizza una Schedule Table?

- La schedulazione LIN è fatta offline, il task master deve solo leggere il LDF e mandare l'header. Ogni nodo, anche lo slave task del master,

29. Cosa contiene il LIN Description File (LDF)?

- Il LDF contiene gli header da mandare e il ritardo tra invii consecutivi.

30. Perché il baud rate è presente nell'LDF?

-

31. Qual è il ruolo dell'Interframe Delay?

32. Come funzionano le Unconditional Frame?

- Le UF sono frame che contengono l'ID del dato richiesto e vengono mandate dal master.

33. Come funzionano le Event Triggered Frame?

- Le frame event triggered vengono mandate dal master. Il master inserisce l'ID della frame ET e la manda nel bus, i nodi rispondono se i loro dati sono stati aggiornati. In questo modo se nessun nodo ha dati aggiornati il MAster manda una sola frame piuttosto che fare un ciclo di interrogazioni completo. Invece se un nodo ha aggiornato i suoi dati risponderà solo quel nodo, se rispondono tanti nodi tanto vale fare un polling completo. Perché nelle Event Triggered Frame possono verificarsi collisioni? Se tanti nodi hanno aggiornato i propri dati.

34. Come vengono risolte le collisioni nelle Event Triggered Frame?

35. A cosa servono le Sporadic Frame?

- I task sporadici servono a fare in modo che il master task non chieda sempre i dati allo slave task. Quando arriva la frame sporadica nello scheduling il master controlla se lo slave task ha nuovi dati, se ha nuovi dati li manda uno ad uno sulla rete.
36. Perché le Sporadic Frame sono utilizzate solo dal Master?
- Servono a controllare se lo slave task del master ha nuovi dati.
37. Differenza tra checksum classico ed enhanced checksum.
- Il check sum classico viene calcolato solo sul payload mentre il checksum enhanced anche sul pid.
38. Come funziona il meccanismo Sleep/Wakeup?
- I nodi possono essere forzati ad addormentarsi o si addormentano se sentono 4 secondi di silenzio sul bus.
39. Quali sono vantaggi e limiti del LIN?

CAN

41. Perché il CAN utilizza un arbitraggio non distruttivo?
- L'arbitraggio non distruttivo CSMA BA permette di gestire le collisioni in modo deterministico grazie alla priorità che viene definita grazie agli ID.
42. Qual è il ruolo dell'Identifier?
- In CAN, come anche nel LIN, l'indirizzamento è di tipo source addressing, o data-based addressing, che consiste nell'identificare il messaggio in base al tipo di dato che esso trasporta all'interno del suo payload. In CAN 2.0A sono 11 bit di ID mentre in CAN 2.0B abbiamo altri 18 bit di estensione per l'id che quindi diventa 29 bit. L'ID quindi identifica il tipo di dato contenuto nel payload.
43. Perché l'ID identifica il messaggio e non il nodo?
- L'indirizzamento è basato sul contenuto, però se quel tipo di dato lo può produrre solo un determinato nodo stiamo identificando quel nodo.
 - In CAN l'indirizzamento è a oggetti: a ogni oggetto, quindi a ciascun tipo di informazione viene assegnato un id che mappa la priorità dell'oggetto. Questo tipo di indirizzamento determina anche un modello di tipo producer consumer: cioè i nodi mandano tutti i messaggi in broadcast e i singoli nodi intercettano tutti i messaggi e salvano quelli che sono di loro interesse (microcontrollore).
44. Come funziona l'arbitraggio bit per bit?
- In CAN se due nodi cominciano a trasmettere simultaneamente durante la fase di arbitraggio vincerà il messaggio con priorità maggiore perché i nodi in arbitraggio trasmettono e monitorano il canale allo stesso tempo. Quindi se un nodo sta trasmettendo un bit recessivo, 1, mentre sul canale sente che c'è uno stato dominante, 0, significa che c'è un nodo che sta trasmettendo un messaggio con priorità maggiore rispetto al suo, di conseguenza interrompe la trasmissione, ritorna in ascolto e aspetta che il canale torni libero prima di riprovare a trasmettere. In questo modo solo il nodo a priorità maggiore si accorge della collisione, mentre quello a priorità maggiore continua a trasmettere indisturbato.
45. Che cos'è il Bit Stuffing?

- Il Bit Stuffing serve a mantenere la sincronizzazione tra i nodi. Poiché il CAN utilizza una codifica Non Return To Zero (NRZ) e non avendo una linea di clock dedicata, se ci sono diversi bit consecutivi uguali i nodi potrebbero perdere la sincronizzazione, perché essa viene fatta grazie alle transizioni tra i bit. Quindi ogni 5 bit consecutivi uguali i nodi can inseriscono un bit diverso per mantenere la sincronizzazione. La sinc è necessaria in fase di bit e di arbitration.

46. Quando viene inserito uno Stuff Bit?

- Gli Stuff Bit vengono inseriti dalla SOF fino al CRC, le restanti no perché ci sono delle parti della frame can che devono restare uguali.

47. Perché il Bit Stuffing migliora la sincronizzazione?

48. Parti della FRAME CAN:

- Start Of Frame: dominante, permette la sincronizzazione dopo l'eof e intermission recessivi.
- ID: 11 bit che identificano il contenuto.
- RTR: Remote Transmission request, serve per capire se la frame è dati o di richiesta dati. Se la frame è dati allora è dominante, perché così in arbitration la frame dati vince sempre sulla frame richiesta dati, se è recessivo significa che la frame è di richiesta dati e serve per richiedere un particolare tipo di dato. Grazie a questo meccanismo CAN può essere utilizzato anche come rete MAster Slave, anche se nasce come CSMABA.
- IDE: ID extention, se è recessivo indica che l'id continua nell'estensione di CAN 2.0B.
- R0: bit riservato a usi futuri, sempre dominante.
- DLC, data length code: serve a indicare la quantità di dati contenuta nel payload.
- Payload: in CAN da 0 a 8 byte.
- CRC, CRC delimiter, sempre recessivo il delimiter.
- ACK, ack delimiter
- EOF, 7 bit recessivi,
- Intermission minimo 3 bit recessivi

49. Che cos'è un Bit Error?

-

50. Perché durante arbitraggio e ACK non viene rilevato il Bit Error?

51. Quali possono essere le cause di un Bit Error?

52. Descrivi i diversi tipi di errore nel CAN.

- In Can ci sono 5 tipi di errori diversi: ACK error , durante la fase di ack il nodo trasmittente lascia il bus in stato recessivo e aspetta che ci sia un nodo che portando il bus allo stato dominante gli confermi che la ricezione sia avvenuta correttamente. Se il bus rimane recessivo per un certo tempo il nodo trasmittente manda un messaggio di errore.
- Errore di bit stuffing, violazione del bit stuffing
- Errore di formato, ci sono dei valori delle frame che devono essere sempre gli stessi.
- Bit Error: quando un nodo sente sul bus un valore diverso da quello che ha trasmesso, questo errore non avviene in ACK e in Arbitration.
- CRC error: quando controllando il crc risultano degli errori.

53. Differenza tra Active Error Flag e Passive Error Flag.

- La Error Frame può essere di due tipi: Active o Passive in base allo stato in cui si trova un Nodo. Error Active significa che la frame è composta da 6 bit dominanti seguita da 8 bit recessivi. Il nodo Error Active è un nodo sano che presenta i contatori TEC e Rec minori o uguali di 127. Se un nodo è error passive significa che il suo tec o rec sono maggiori di 127 , quindi ha generato troppi errori. Bus off se tec è maggiore di 127.
54. Perché l'incremento del TEC e del REC è asimmetrico?
- Gli errori di trasmissione comportano l'aumento di 8 punti del tec, quelli in ricezione aumentano il rec di 1 punto. Questo perché si tende a isolare l'errore del nodo trasmettente: se un nodo manda frame costantemente corrotte non si può incolpare allo stesso modo i nodi che le ricevono.
 - Mentre per ogni ricezione o trasmissione corretta i contatori vengono decrementati di un punto.
55. Come funziona il Fault Confinement?
- Il nodo error passive manda frame di errore solamente recessive, in questo modo non può distruggere il traffico nella rete, ma rimane sincronizzato, mentre un nodo bus off è scollegato logicamente dalla rete, non può né mandare né ricevere nulla, problema del bubbling idiot.
56. Quando un nodo diventa Error Passive?
- REC e TEC maggiori di 127.
57. Quando entra nello stato Bus-Off?
- Tec maggiore di 255.
58. Cos'è una L_DATA primitive?
- La primitiva L_DATA, appartenente al livello LLC, serve a mandare una frame in broadcast,
59. Cos'è una L_REMOTE primitive?
- Serve a mandare una frame di richiesta dati per richiedere un oggetto specifico.
60. A cosa serve una Remote Frame?
61. Perché nelle reti moderne CAN FD la Remote Frame non viene più utilizzata?
- CAN FD applica il bit rate switch, ma sarebbe inutile utilizzare il bit rate switch per un payload vuoto della frame richiesta dati. Inoltre l'id del dato richiesto può essere inserito nel payload per richiedere il dato.

CAN FD – CAN XL – TTCAN

61. Perché è nato CAN FD?
- CAN FD è nato per rendere il CAN più efficiente, supportare payload (leggermente) maggiori, e velocità di trasmissione più elevate.
62. Quali differenze ci sono tra CAN classico e CAN FD?
- RTR viene sostituito da RRS sempre dominante, perché le frame RTR non sono supportate su CAN FD.
 - Il bit R0 viene cambiato e diventa FD form , per indicare che la frame è CAN FD appunto diventa recessivo. I nodi che sono CAN legacy generano un messaggio di errore.
 - Inoltre CAN FD introduce 3 campi aggiuntivi: il primo è il reserved bit un altro bit riservato agli sviluppi futuri, dominante, potrebbe diventare recessivo in futuro. BRS: bit

rate switch se è dominante viene mandato alla velocità normale di arbitraggio 1 Mbps, se è recessivo significa che la restante parte della frame fino al CRC può essere mandata con bit rate più elevato, limitato dalla rete.

- ESI: serve per capire se il nodo che lo sta mandando è error active o error passive.
- Altre differenze sono il DLC , i 4 byte vengono mappati per indicare gli intervalli di dimensioni alle quali appartiene il payload.
- Il CRC è leggermente diverso , più robusto per supportare la rilevazione degli errori per payload più lunghi. In CAN classico il CRC è 15 bit nel CAN FD è 17 per payload fino a 16 byte e 21 bit per campi superiori ai 16 byte. Il CRC inoltre subisce il bit stuffing statico, ogni 4 byte. Il CRC ha anche lo stuff count per calcolare il numero di bit inseriti durante lo stuffing dinamico.

63. Perché CAN FD utilizza due velocità di trasmissione?

- Perché in CAN la velocità di trasmissione è limitata dalla sincronizzazione: dato che in arbitration e ack i nodi hanno bisogno di essere sincronizzati , il tempo di bit deve essere due volte maggiore del ritardo di propagazione, perché i nodi devono poter scrivere e leggere il bus allo stesso tempo, di conseguenza con un bus di 40 metri la velocità deve essere massimo di 1Mbps. Ma dato che una volta vinta l'arbitration e prima che ci sia bisogno dell'ACK vi è sempre un solo nodo che trasmette, quindi non c'è più bisogno della sincronizzazione. Quindi partendo dal BRS al CRC delimita il bit rate viene aumentato, rispettando le limitazioni della rete e dei transeiver (dall'ESI - CRC Delimiter). Le velocità dati possono raggiungere anche i 2,5, 3, 5 Mbps ma utilizzando transeiver speciali come i SIC , anche 8 mbps.

64. Perché CAN XL è stato introdotto?

- Per introdurre Payload maggiori , ma anche per permettere il tunnelling ethernet.

65. Perché CAN XL utilizza due CRC differenti?

- CAN XL utilizza due CRC differenti perché il primo CRC , il Preface CRC viene calcolato dalla parte iniziale dell'header fino a prima del payload, in questo modo se la frame presenta

66. Differenza tra Preface CRC e Frame CRC.

67. Un nodo CAN FD può interpretare una frame CAN XL?

- No I nodi CAN FD non possono interpretare le frame CAN XL,

68. Quali vantaggi introduce TTCAN rispetto al CAN classico?

69. Cosa sono i TX Trigger?

70. Cosa sono gli RX Trigger?

71. Perché TTCAN è considerato deterministico?

FlexRay

72. Perché FlexRay è stato sviluppato?

73. Come è suddiviso il ciclo di comunicazione di FlexRay?

74. Differenza tra segmento statico e dinamico.

75. Quali vantaggi offre FlexRay rispetto al CAN?

76. Confronta CAN e FlexRay.

MOST

- 77. Perché è nato il protocollo MOST?
 - 78. Quali tipi di dati trasporta MOST?
 - 79. Qual è la topologia della rete MOST?
 - 80. Come funziona il Ring Scan?
 - 81. Qual è il ruolo del Network Master?
 - 82. Come è strutturata una frame MOST?
 - 83. Qual è la differenza tra canale sincrono, asincrono e di controllo?
 - 84. Che cos'è il Boundary Descriptor?
 - 85. Come fa il Boundary Descriptor a separare la parte sincrona da quella asincrona?
 - 86. Quali sono i principali limiti del MOST?
 - 87. Perché Automotive Ethernet tende a sostituire MOST?
-

Automotive Ethernet

- 88. Perché Ethernet è stata introdotta nelle automobili?
 - 89. Perché lo Switched Ethernet tradizionale non è deterministico?
 - 90. Che cos'è un overflow nelle code?
 - 91. Che cos'è uno shaper?
 - 92. Perché lo shaper viene applicato alle code di uscita?
 - 93. Cos'è IEEE 802.1Q?
 - 94. Che cos'è una VLAN?
 - 95. Quali vantaggi offre lo Switched Ethernet rispetto al bus condiviso?
-

10BASE-T1S

- 96. Perché è nato 10BASE-T1S?
- 97. In quale standard IEEE è definito?
- 98. Quali differenze esistono tra 10BASE-T1S e 10BASE-T1L?
- 99. Perché 10BASE-T1S utilizza il PLCA?
- 100. Come funziona il PLCA?
- 101. Qual è il ruolo del BEACON?
- 102. Che cos'è una Transmit Opportunity?
- 103. Cosa succede se un nodo non ha dati da trasmettere?

- 104. Quando viene inviato il COMMIT?
 - 105. Come funziona la Burst Mode?
 - 106. Qual è il ruolo dell'ESDBR?
-

AVB

- 107. Perché è stato introdotto Audio Video Bridging (AVB)?
 - 108. Quali problemi risolve rispetto allo Switched Ethernet tradizionale?
 - 109. Quali standard compongono AVB?
 - 110. Che cos'è lo SRP (Stream Reservation Protocol)?
 - 111. Chi sono il Talker e il Listener?
 - 112. Che cos'è un flusso SR?
 - 113. Come avviene la prenotazione della banda?
 - 114. Quali informazioni contiene un messaggio Talker Advertise?
 - 115. Che cos'è un messaggio Listener Ready?
 - 116. Perché gli switch aggregano le registrazioni dei Listener?
 - 117. Qual è il ruolo della Forwarding Table?
 - 118. Perché la banda riservata alle classi SR è limitata (ad esempio al 75%)?
 - 119. Perché AVB garantisce la latenza ma non il determinismo assoluto?
 - 120. Come funziona il Credit-Based Shaper (CBS)?
 - 121. Che cosa rappresentano **idleSlope**, **sendSlope**, **hiCredit** e **loCredit**?
 - 122. Perché il CBS evita i burst di traffico?
-

IEEE 802.1AS

- 123. Qual è lo scopo dello standard IEEE 802.1AS?
- 124. Che cos'è una stazione time-aware?
- 125. Chi è il Grand Master?
- 126. Come viene eletto il Grand Master?
- 127. In cosa consiste il Best Master Clock Algorithm (BMCA)?
- 128. Che cos'è il synchronization spanning tree?
- 129. Perché è necessario misurare il link delay?
- 130. Quali vantaggi offre una sincronizzazione con accuratezza inferiore a 1 μ s?

Queste 130 domande coprono in modo pressoché completo il programma dall'inizio fino a **AVB** e **IEEE 802.1AS**, con un livello di approfondimento molto simile a quello di un esame orale.

Domande:

Timeline Scheduling (cycling scheduling)

Definizione di Taskset Schedulabile e Test di schedulabilità

Che significa che due task sono tra loro indipendenti? Cosa significa Least Upper Bound?

Deadline Monotonic

Confronto tra Rate Monotonic ed EDF

Utilizzazione di Liu e Layland

Domini funzionali, architetture zonali (disegno)

MAC:

- CSMA-BA, Polling Server

LIN (esempio applicazione: chiusura portiera auto)

CAN

- Bit Stuffing del CAN
- Problematiche del CAN per gestire il RT
- Error Frame
- CAN-FD
- TT-CAN
- Confronto CAN vs Flexray
- Confronto CAN XL e 10Baes T1 S

Flexray

MOST : Confronto MOST vs AVB

Ethernet

- 10BASE T1S
- Switched Ethernet (Perchè Ethernet Switched non può gestire i vincoli Real Time?)
- Cosa significa Overflow nelle code?
- AVB : Cosa significa che un traffico subisce "shaping"?
- Cosa significa che qualcosa non segue gli standard e che problematiche implica? Come viene gestito il traffico in TSN?
- 802.1TSN
- 802.1Q : 802.1Qbv
- In Ethernet automotive che velocità abbiamo in questo momento?

Industrial Ethernet

- Approcci RTE
- Fieldbus
- Performance Indicator, cosa sono e dove li abbiamo visti?
- Confronto Profinet/ Powerlink
- POWERLINK
- ETHERCAT (operazioni e indirizzamento)

Wireless industrial: 802.11 ambito industriale, Flexware

Cosa è uno shaper

